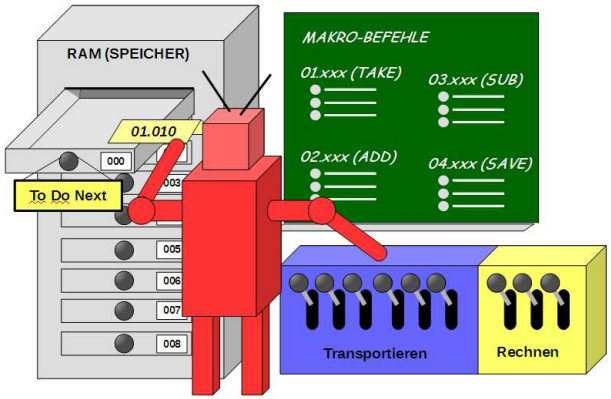


Rechnerarchitektur

Matthias Meyer

Ausgabe Sommersemester 2026
Unterlagen werden in der Lehrveranstaltung ergänzt

Alle Rechte vorbehalten
Weiterverbreitung und Vervielfältigung nur mit Genehmigung
Nutzung nur im Zusammenhang mit der Lehrveranstaltung

Rechnerarchitektur

Matthias Meyer

Kontakt

Dipl.-Ing. Matthias Meyer

Universität Stuttgart
Institut für Kommunikationsnetze und Rechnersysteme (IKR)
Pfaffenwaldring 47
70569 Stuttgart

matthias.meyer@ikr.uni-stuttgart.de

+49 711 685 67975

Quelle für Bild auf Titelseite: www.inf-schule.de

Rechnerarchitektur

Matthias Meyer

Inhalt

Kapitel 1 Kombinatorische Netzwerke 8

1.1 Boolesche Algebra 9

1.2 Logische Grundschaltungen 12

1.3 Beispiele 20

1.4 Darstellungsformen 24

1.5 Minimierungsverfahren 42

Kapitel 2 Sequenzielle Netzwerke 51

2.1 Flipflop-Speicherelemente 52

2.2 Beispiele 65

2.3 Modellierung 71

2.4 Entwurfsverfahren 92

Kapitel 3 Rechenwerke 118

3.1 Zahlensysteme und Zahlendarstellungen 119

3.2 Addier- und Subtrahierwerke 153

3.3 Arithmetisch-Logische Einheit (ALU) 170

3.4 Multiplizier- und Dividierwerke 178

Kapitel 4 Entwurf eines Minimalprozessors 227

4.1 Mikroprozessor 228

4.2 Entwurfsebenen 251

4.3 Programmiermodell (Instruction Set Architecture) 264

4.4 Register-Transfer-Ebene 283

4.5 Implementierung 309

Stand 20. März 2026, Änderungen vorbehalten

Ausblick: Rechnerarchitektur 2 (Wahlfach im Sommersemester)

Inhalt

Kapitel 0 Entwurf eines Minimalprozessors

0.1 Mikroprozessor

0.2 Entwurfsebenen

0.3 Programmiermodell (Instruction Set Architecture)

0.4 Register-Transfer-Ebene

0.5 Implementierung

Kapitel 1 CISC-Prozessoren

1.1 Mikroprogrammierung

1.2 Mikroprogrammierter Minimalprozessor

1.3 IKR CISC (ISA und Programmbeispiele)

1.4 IKR CISC (Implementierung)

Kapitel 2 Speicherhierarchie

2.1 Einführung

2.2 Cache

2.3 Virtueller Speicher

Kapitel 3 RISC-Prozessoren

3.1 Pipelining

3.2 Design von RISC-Befehlssätzen

3.3 IKR RISC II (ISA)

3.4 Pipeline Hazards

3.5 IKR RISC II (Implementierung)

3.6 RISC-V

Kapitel 4 Superskalare Prozessoren

4.1 Sprungvorhersage

4.2 Befehlsabhängigkeiten

4.3 Statisches Scheduling

4.4 Dynamisches Scheduling

4.5 Spekulation

Änderungen vorbehalten

Rechnerarchitektur

Matthias Meyer

Kapitel 1 Kombinatorische Netzwerke

Inhaltsübersicht

1.1 Boolesche Algebra

1.2 Logische Grundschaltungen

1.3 Beispiele

1.4 Darstellungsformen

1.5 Minimierungsverfahren

1.1 Boolesche Algebra



Quelle: MacTutor History of Mathematics Archive

George Boole, 1815 – 1864

"An Investigation of The Laws of Thought On Which Are Founded the Mathematical Theories of Logic and Probabilities" (1854)

Inhaltsübersicht

1.1.1 Axiome der Booleschen Algebra

1.1.2 Sätze der Booleschen Algebra (Auswahl)

1.1.1 Axiome der Booleschen Algebra (Grundsatz ohne Warum zu fragen)

ABLE (?)

Axiom 1: Kommutativgesetz

$A \cdot B = B \cdot A$	(1a)	$A + B = B + A$	(1b)

Axiom 2: Distributivgesetz

$A \cdot (B + C) = A \cdot B + A \cdot C$	(2a)	$A + B \cdot C = (A + B) \cdot (A + C)$	(2b)

Axiom 3: Neutrales Element

$A \cdot 1 = A$	(3a)	$A + 0 = A$	(3b)

Axiom 4: Inverses Element

$A \cdot \overline{A} = 0$	(4a)	$A + \overline{A} = 1$	(4b)

kleinste möglichkeit Menge $M_2 = \{0,1\}$

Welche Operation verbirgt sich hinter am "·"-operator

A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

Fa

(3a) $A+0=A$

(3a),(1a) $A\cdot 1=A$

(3a)

$0 \cdot 0 = ?$

$A \cdot A \overset{1a}{=} (A\cdot A)+0 \overset{4a}{=} (A\cdot A)+(A\cdot\bar{A}) \overset{3a}{=} A\cdot(A+\bar{A}) \overset{4b}{=} A\cdot 1 \overset{1a}{=} A$ Fa

"·" logisches UND auch "∧" (Idempotenzgesetz)

Welche Operation verbirgt sich über denn "+"-Operator in Akizme

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

3b

3b, 1b

3b

Fb

$A+A \overset{3a}{=} (A+A)\cdot 1 \overset{4b}{=} (A+A)\cdot(A+\bar{A}) \overset{A+B\ A+C}{=} A+(A\cdot\bar{A}) \overset{4a}{=} A+0 \overset{1b}{=} A$ Fb

"+" logisches ODER auch "∨" (inklusives ODER)

A	$\bar{A}$	(4a) $A\cdot\bar{A}=0$	(4b) $A+\bar{A}=1$
	0	~~0~~	~~0~~
0	1	0	1
1	0	0	1
	1	~~1~~ Weil es 0 sein soll	~~1~~

↑ WT des UND

A	$\bar{A}$
0	1
1	0

"¬" logischer NICHT

Boolisches Algebra für $M_3 = \{0, 1, 😐\}$

Inverses Element zu 😐

$A = 😐$	$\bar{A}$	(4a) $A \cdot \bar{A} = 0$	(4b) $A + \bar{A} = 1$
	0	$😐 \cdot 0 = 0$	$😐 + 0 = 1$ (3b)
😐	1	$😐 \cdot 1 = 0$ (3a)	$😐 + 1 = 1$
	😐	$😐 \cdot 😐 = 0$ (5a)	$😐 + 😐 = 1$ (5b)

keine Boolische Algebra möglich für $M = M_3$

Boolisches Algebra für $M_4 = \{0, 1, 😐, ♡\}$

A	$\bar{A}$
0	1
1	0
😐	♡
♡	😐

·	0	1	😐	♡
0	0	0	0	0
1	0	1	😐	♡
😐	0	😐	😐	0
♡	0	♡	0	♡

n-neutrales Element

$A \cdot 0 = ?$

↳ $A \cdot 0 \overset{6a}{=} A \cdot (A \cdot \bar{A}) \overset{5a}{=} A \cdot \bar{A} \overset{6a}{=} 0$ ↲

$A \cdot (A \cdot \bar{A}) \neq (A \cdot A) \cdot \bar{A}$

(Assoziativgesetz, nicht bewiesen)

→ Beweis bezüglich Dominanz

$A \cdot 0 \overset{3b}{=} A \cdot 0 + 0 \overset{4a}{=} A \cdot 0 + A \cdot \bar{A} \overset{2a}{=} A \cdot (0 + \bar{A}) \overset{3b}{=} A \cdot \bar{A} \overset{4a}{=} 0$ 6a

1.1.2 Sätze der Booleschen Algebra (Auswahl)

Idempotenzgesetz

$A \cdot A = A$	(5a)	$A + A = A$	(5b)

Dominanzgesetz

$A \cdot 0 = 0$	(6a)	$A + 1 = 1$	(6b)

Absorptionsgesetz

$A \cdot (A + B) = A$	(7a)	$A + (A \cdot B) = A$	(7b)

Assoziativgesetz

$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	(8a)	$A + (B + C) = (A + B) + C$	(8b)

Doppelte Negation

$\overline{\overline{A}} = A$ (9)

De Morgans Gesetz

$\overline{A \cdot B} = \overline{A} + \overline{B}$	(10a)	$\overline{A + B} = \overline{A} \cdot \overline{B}$	(10b)

7b) $A + AB \overset{3A}{=} A \cdot 1 + A \cdot B \overset{2a}{=} A(A+B) \overset{4A}{=} A \cdot 1 = A$

7a)

A	B	A·B	A+AB
0	0	0	0
0	1	0	0
1	0	1	1
1	1	1	1

1.2 Logische Grundschaltungen

Die Schaltalgebra ist eine Anwendung der Booleschen Algebra zur Beschreibung der Funktion kombinatorischer Netzwerke in Form schaltalgebraischer Funktionen. Die Menge M, auf die die Operatoren der Schaltalgebra angewendet werden können, umfasst nur die Werte 1 und 0 bzw. "true" und "false" (zweiwertige Boolesche Algebra). Die vom Axiomensystem der Booleschen Algebra definierten Grundoperationen werden in der Schaltalgebra als logisches UND, als logisches ODER und als logisches NICHT interpretiert.

Die Bausteine zur Realisierung dieser und davon abgeleiteter Operationen werden als logische Grundschaltungen, als binäre Verknüpfungsglieder oder als (logische) Gatter bezeichnet.

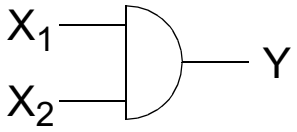
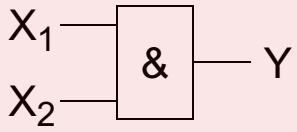
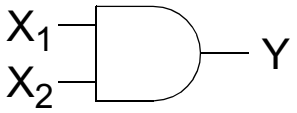
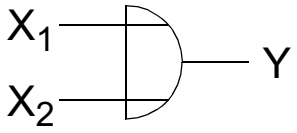
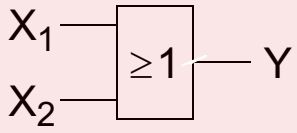
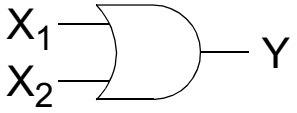
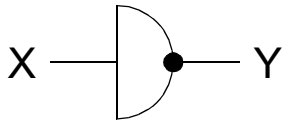
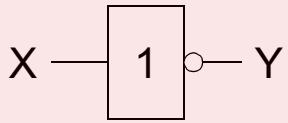
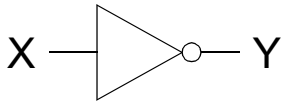
Inhaltsübersicht

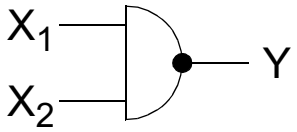
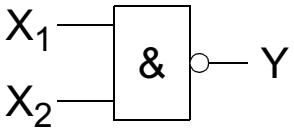
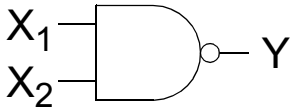
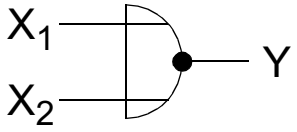
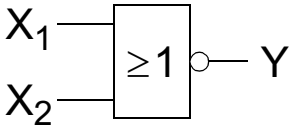
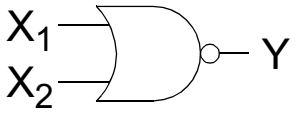
1.2.1 Schaltzeichen

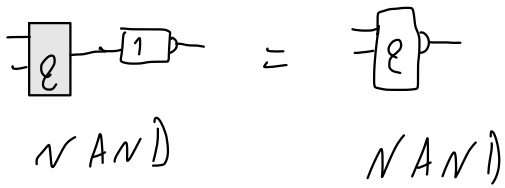
1.2.2 Technische Realisierungsformen

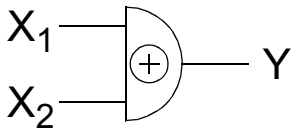
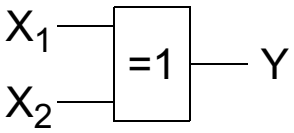
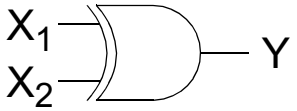
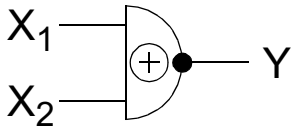
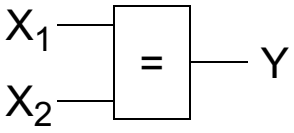
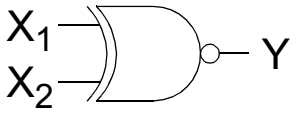
1.2.1 Schaltzeichen

Schalten nur das an

Operation	Wahrheitstabelle	DIN (alt)	Schaltzeichen DIN (neu)	ANSI
UND (AND)	X_2 X_1 | Y 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1			
ODER (OR)	X_2 X_1 | Y 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1			
NICHT (NOT)	X | Y 0 | 1 1 | 0			

Operation	Wahrheitstabelle			Schaltzeichen DIN (alt)	Schaltzeichen DIN (neu)	Schaltzeichen ANSI
NICHT UND (NAND)	X_2	X_1	Y			
	0	0	1			
	0	1	1			
	1	0	1			
	1	1	0			
NICHT ODER (NOR)	X_2	X_1	Y			
	0	0	1			
	0	1	0			
	1	0	0			
	1	1	0			

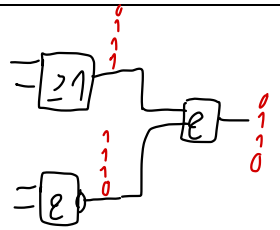


Operation	Wahrheitstabelle	Schaltzeichen DIN (alt)	DIN (neu)	ANSI
Exklusives ODER, Antivalenz (EXOR, XOR)	X_2 X_1 Y 0 0 0 0 1 1 1 0 1 1 1 0			
Exkl. NICHT ODER, Äquivalenz (EXNOR, XNOR)	X_2 X_1 Y 0 0 1 0 1 0 1 0 0 1 1 1			

logic-ly

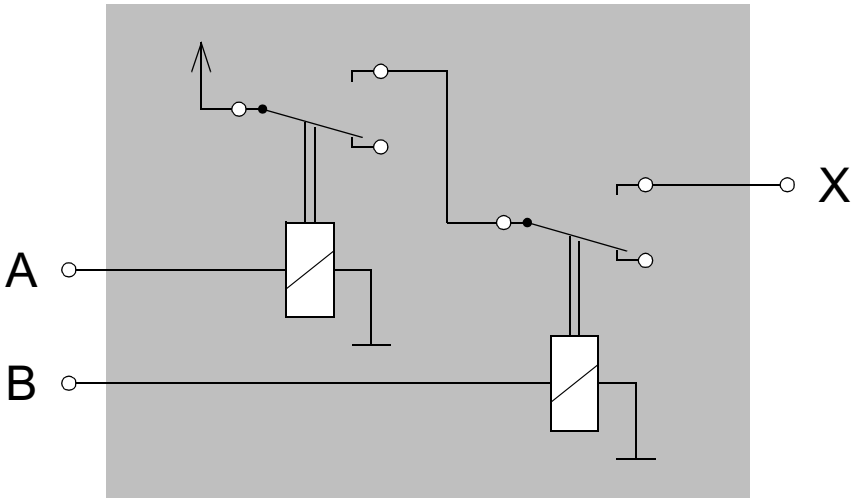
A	B	
0	0	0
0	1	1
1	0	1
1	1	0

$\bar{A}B$, $A\bar{B}$ $\Rightarrow$ $\bar{A}B + A\bar{B}$

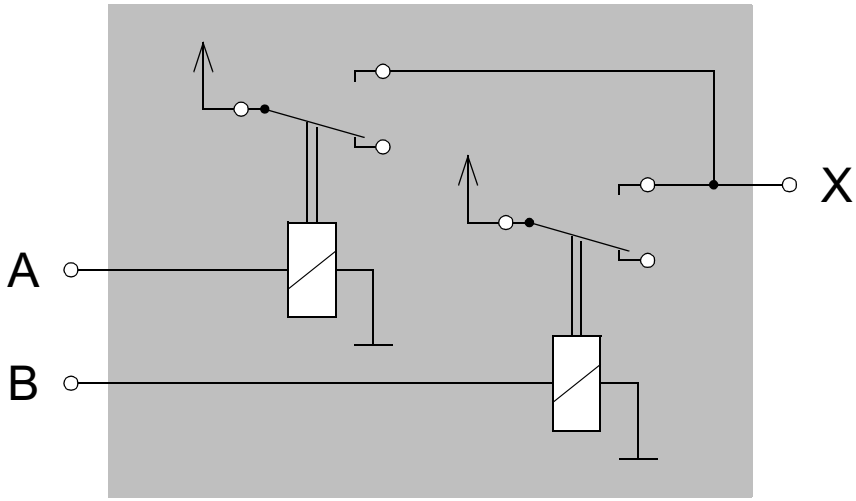


1.2.2 Technische Realisierungsformen

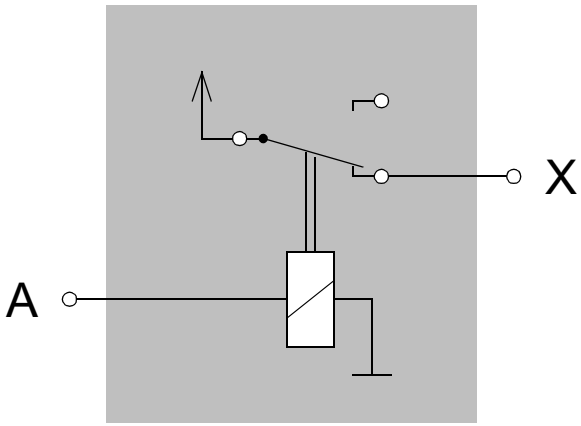
I. Relais-Technik ("Schalt"-Algebra)



AND



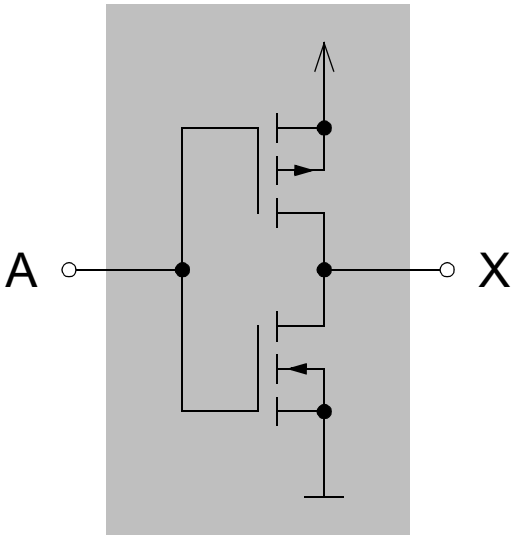
OR



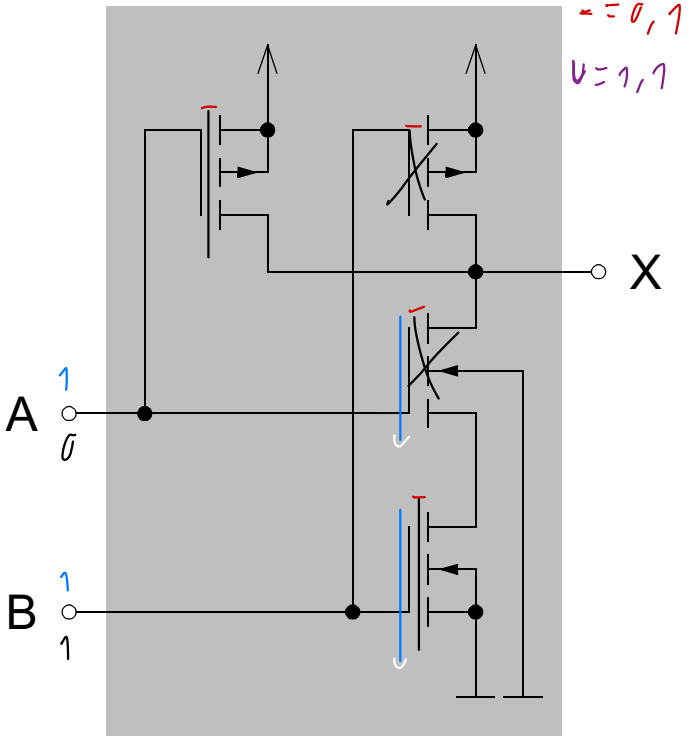
NOT

II. CMOS-Technik

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

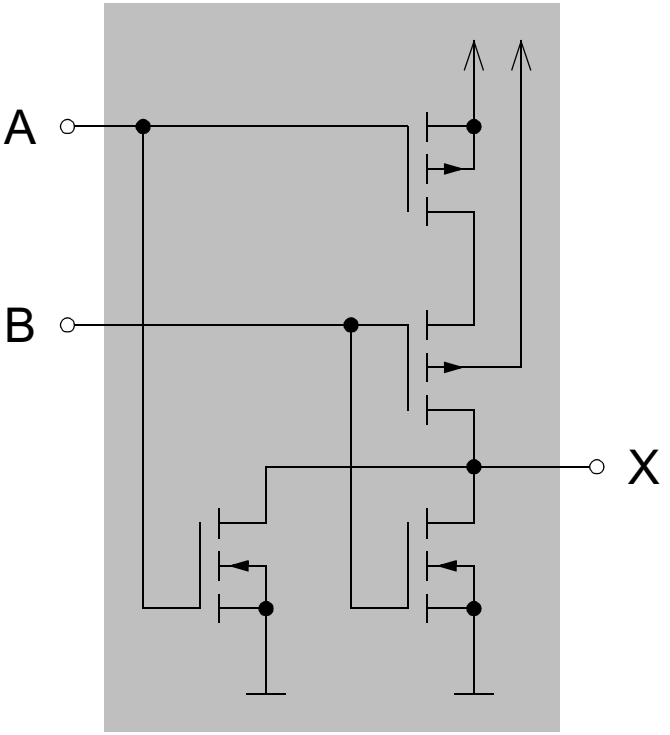


NOT



NAND

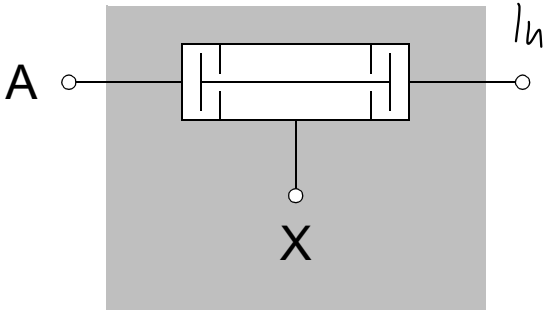
1 = leitet
X = geschlossen



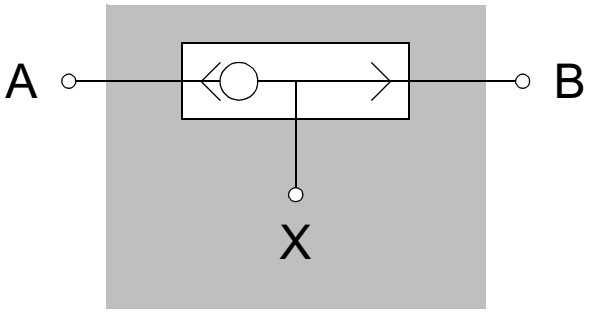
NOR

III. Pneumatik

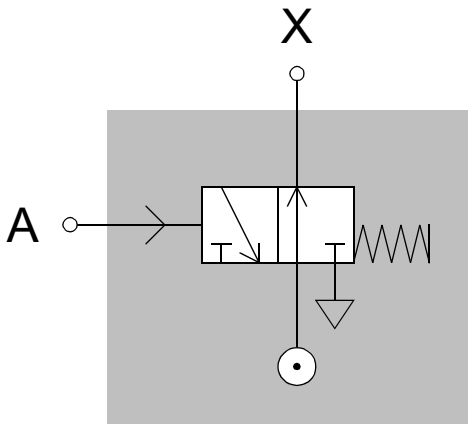
A und B muss gleich viel Luft durch kommen, das er aufgeht



Zweidruckventil
("AND")



Wechseldruckventil, Doppelrückschlagventil
("OR")



Druckgesteuertes Ventil
("NOT")

IV. Weitere Realisierungsformen

- Röhren-Technik
- DTL-Technik
- TTL-Technik
- ECL-Technik
- Optik ...?
- ...?

1.3 Beispiele

Inhaltsübersicht

1.3.1 Multiplexer

1.3.2 Halbaddierer

1.3.3 Paritätsschaltung

1.3.1 Multiplexer

$S=0$ $S=1$

$y=x_0$ $y=x_1$

Wahrheitstabelle

S	X_0	X_1	Y
0	0	0	0
0	0	1	0
0	1	0	(1)
0	1	1	(1)
1	0	0	0
1	0	1	(1)
1	1	0	0
1	1	1	(1)

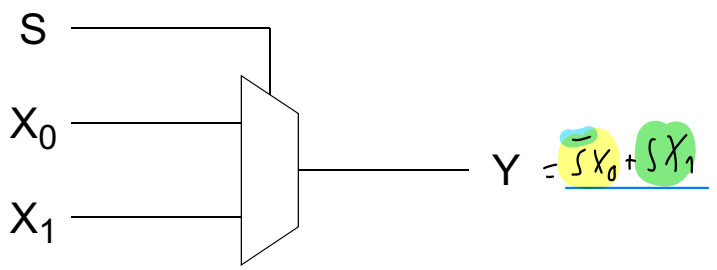
DNF: $y = \bar{S}X_0\bar{X_1} + \bar{S}X_0X_1 + S\bar{X_0}X_1 + SX_0X_1$

010 011 101 111

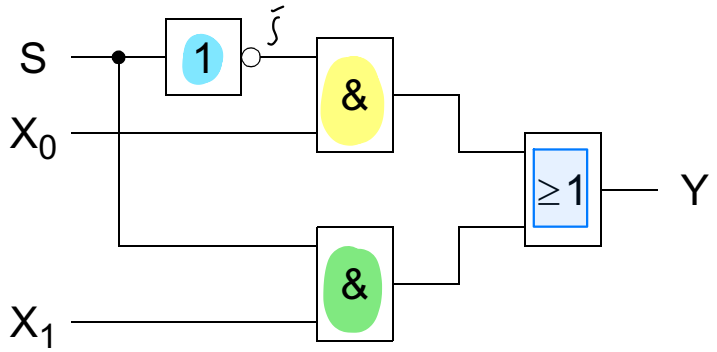
KNF: $y = (S+X_0+X_1)(S+X_0+\bar{X_1})(\bar{S}+X_0+X_1)(\bar{S}+\bar{X_0}+X_1)$

000 001 100 110

Schaltzeichen



Schaltung

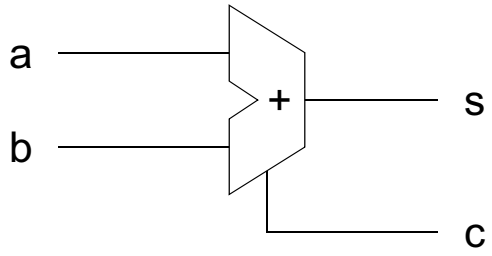


1.3.2 Halbaddierer

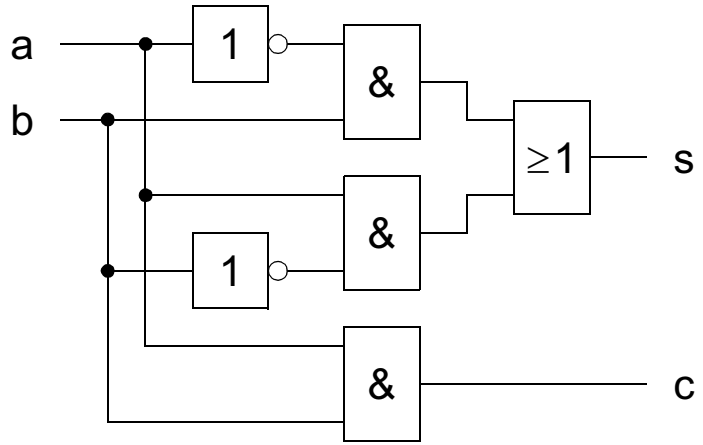
Wahrheitstabelle

a	b	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

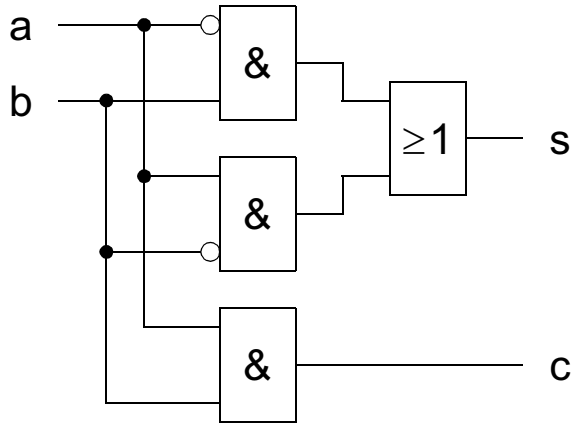
Schaltzeichen



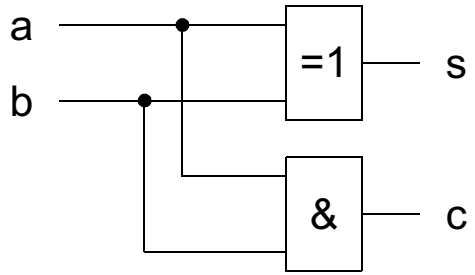
Schaltung



mit invertierenden Eingängen

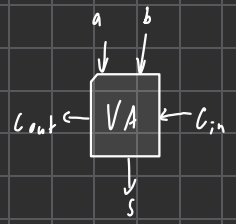


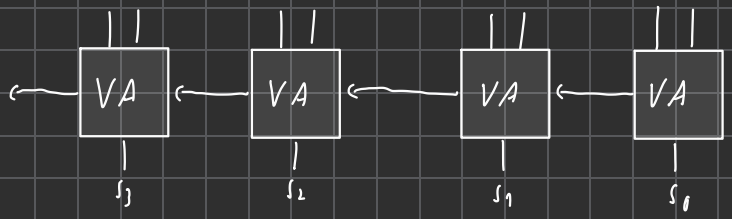
mit XOR-Gatter



Volladdierer

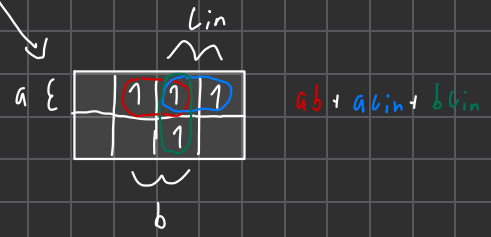
a	b	c_{in}	$2^1=2$ c_{out}	$2^0=1$ s	
0	0	0	0	0	
0	0	1	0	1	
0	1	0	0	1	
0	1	1	1	0	1+1=0 übertrage 1
1	0	0	0	1	
1	0	1	1	0	
1	1	0	1	0	1+1=0 übertrage 1
1	1	1	1	1	





DNF für $c_{out} = \bar{a}bc_{in} + a\bar{b}c_{in} + abc_{in} + abc_{in}$

$KNF = (abc_{in}) + (ab\bar{c}_{in}) + (a\bar{b}c_{in}) + (\bar{a}bc_{in})$



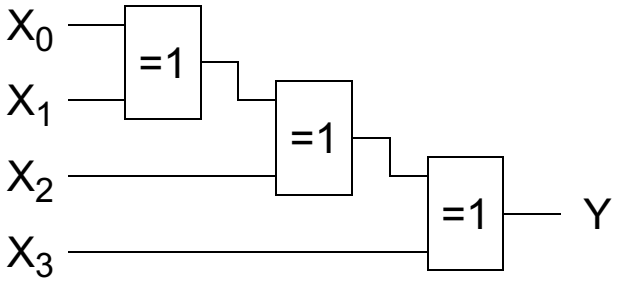
DNF für $s = \bar{a}\bar{b}c_{in} \vee \bar{a}b\bar{c}_{in} + a\bar{b}\bar{c}_{in} + abc_{in}$

1.3.3 Paritätsschaltung

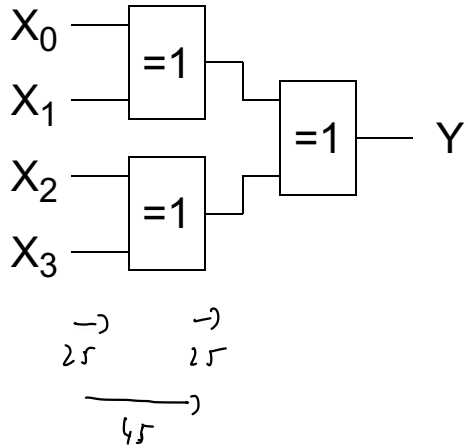
Wahrheitstabelle

X_3	X_2	X_1	X_0	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

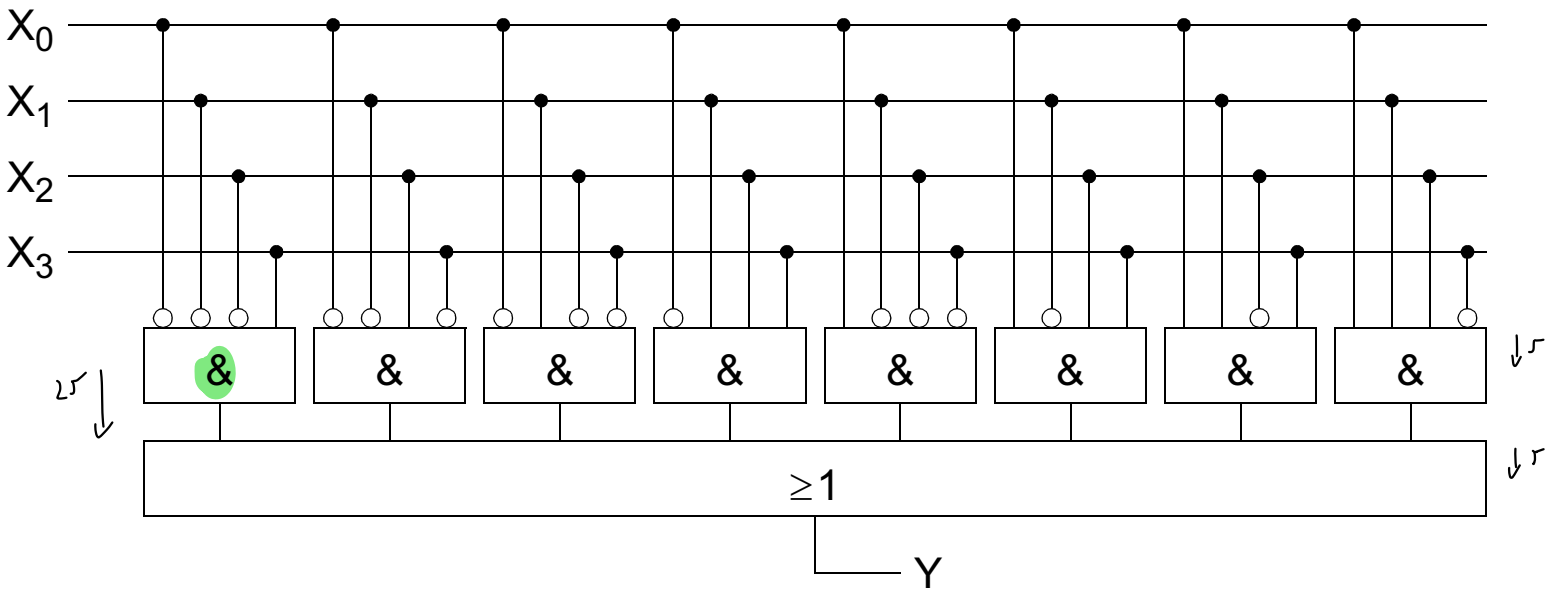
Schaltungsvariante 1



Schaltungsvariante 2



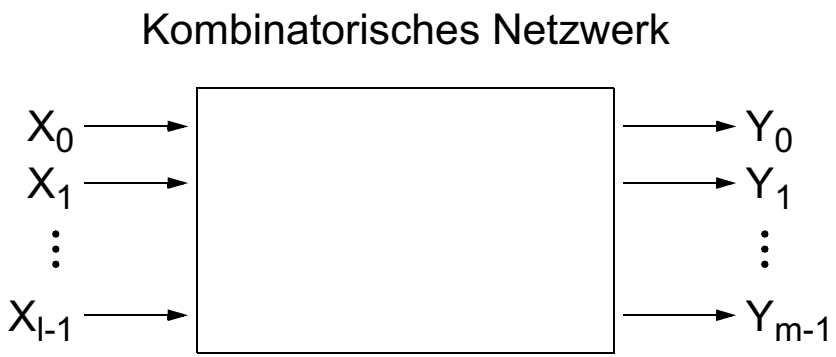
Schaltungsvariante 3



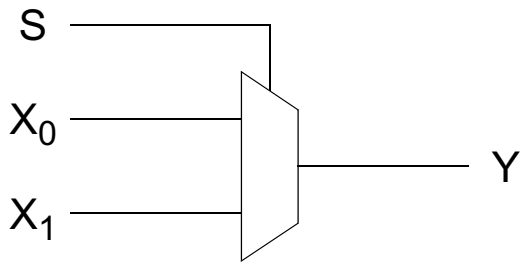
1.4 Darstellungsformen

Definition: Kombinatorisches Netzwerk

Rückkopplungsfreie digitale Schaltung ohne Speicherelemente zur Realisierung schaltalgebraischer Funktionen.



Beispiel: Multiplexer



Inhaltsübersicht

1.4.1 Schaltalgebraische Gleichung

1.4.2 Schaltplan

1.4.3 Wahrheitstabelle

1.4.4 Mengendarstellung (Venn-Diagramm)

1.4.5 Karnaugh-Diagramm

1.4.6 Normalformen

1.4.7 Hardware-Beschreibungssprache (Beispiel: VHDL)

1.4.1 Schaltalgebraische Gleichung

Allgemein

Vektorschreibweise

$\underline{Y} = \underline{f}(\underline{X})$ mit $\underline{X} = X_0, X_1, X_2, \ldots, X_{l-1}$

$\underline{Y} = Y_0, Y_1, Y_2, \ldots, Y_{m-1}$ $l, m \geq 1$

skalare Schreibweise

$Y_0 = f_0(X_0, X_1, X_2, \ldots, X_{l-1})$

$Y_1 = f_1(X_0, X_1, X_2, \ldots, X_{l-1})$

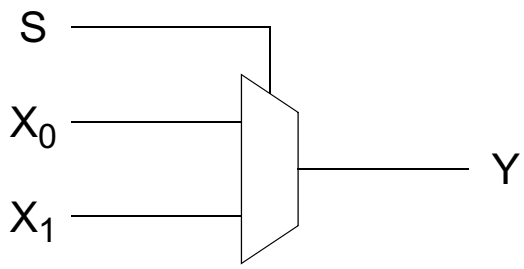
$Y_2 = f_2(X_0, X_1, X_2, \ldots, X_{l-1})$

...

$Y_{m-1} = f_{m-1}(X_0, X_1, X_2, \ldots, X_{l-1})$

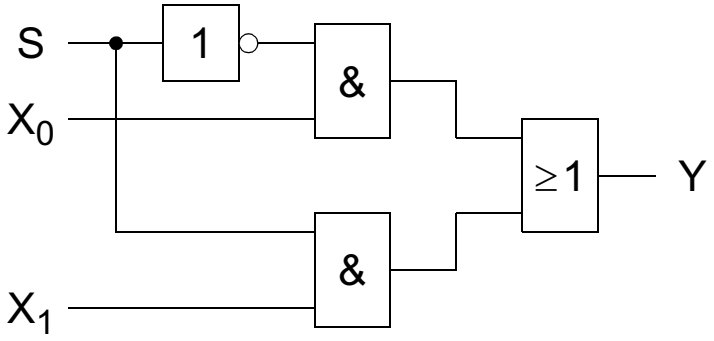
Beispiel

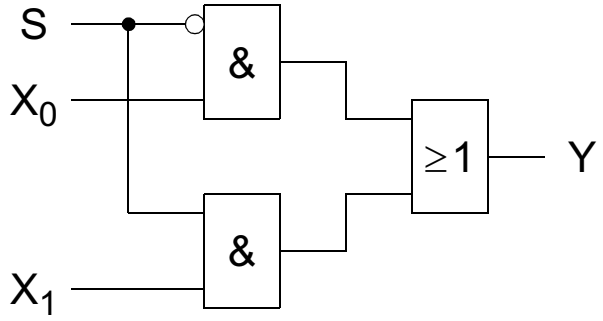
$Y = (\overline{S} \cdot X_0) + (S \cdot X_1) = \overline{S} \cdot X_0 + S \cdot X_1 = \overline{S}X_0 + SX_1$



1.4.2 Schaltplan

Beispiel





Alternativ: Verwendung von Gattern mit invertierenden Eingängen

1.4.3 Wahrheitstabelle

Allgemein für l = 3, m = 1

X_2	X_1	X_0	Y
0	0	0	a_0
0	0	1	a_1
0	1	0	a_2
0	1	1	a_3
1	0	0	a_4
1	0	1	a_5
1	1	0	a_6
1	1	1	a_7

Beispiel

S	X_0	X_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

1.4.4 Mengendarstellung (Venn-Diagramm)

Schaltalgebraische Variable X darstellen als Menge

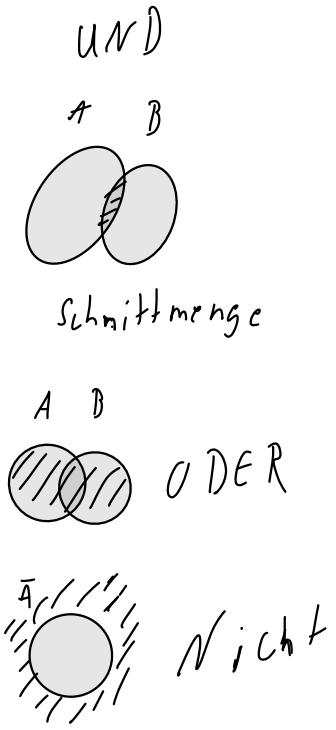
X = 1: in Menge enthalten

X = 0: nicht in Menge enthalten

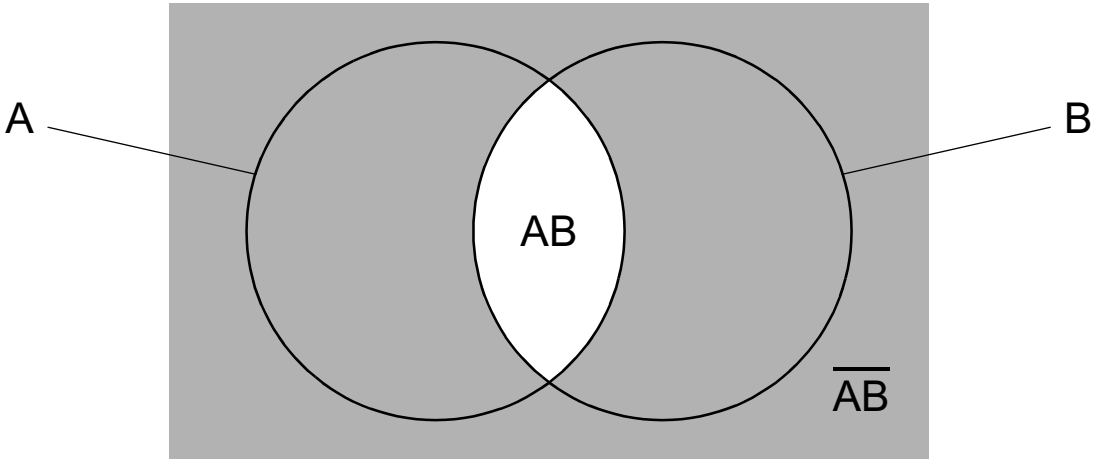
Logisches UND: Schnittmenge

Logisches ODER: Vereinigungsmenge

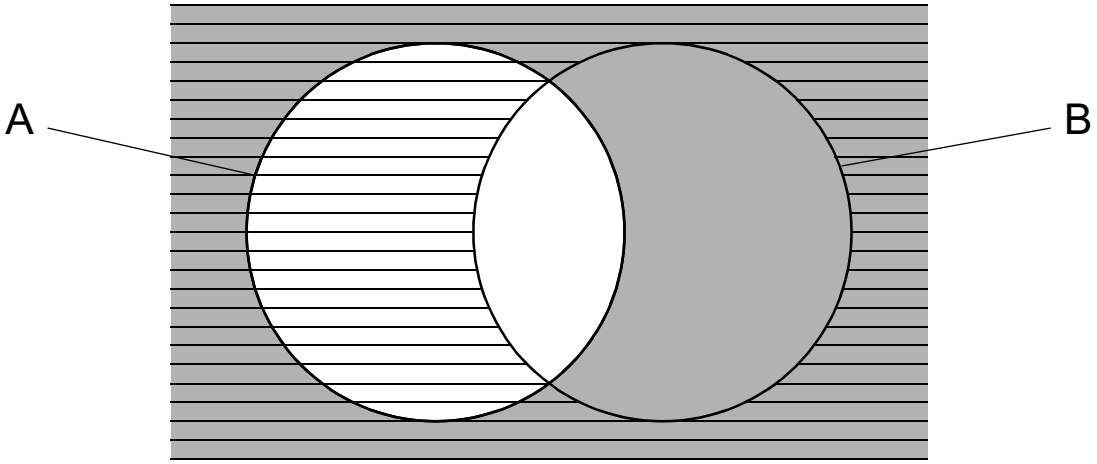
Logisches NICHT: Komplementärmenge



Beispiel 1: Veranschaulichung von De Morgans Gesetz

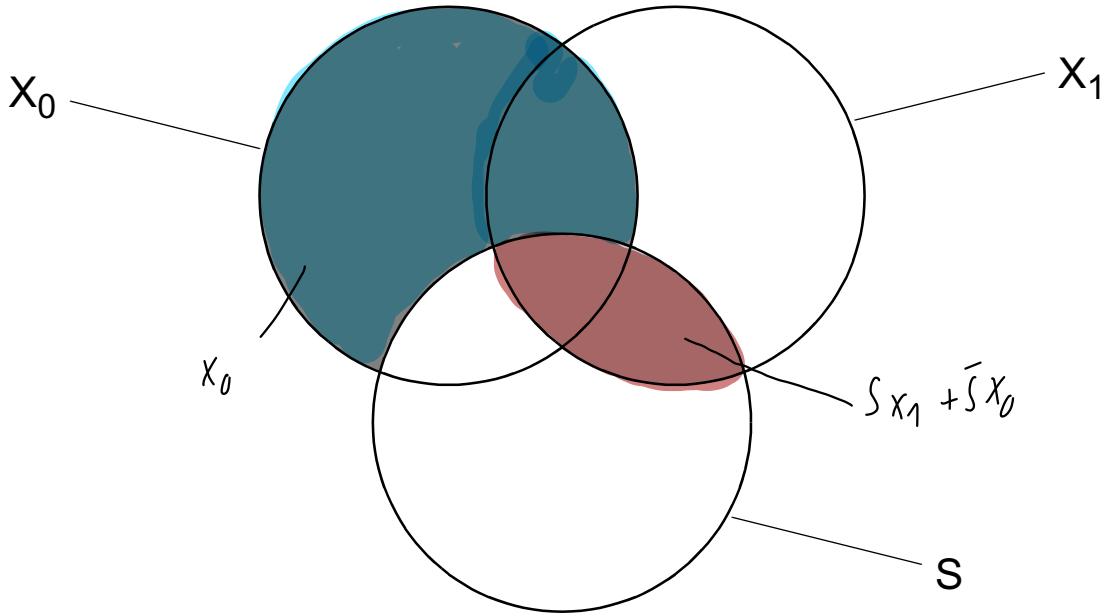


$\overline{AB}$ (Alles was grau ist)



$\overline{A}$ $\overline{B}$ } $\bar{A}+\bar{B}$

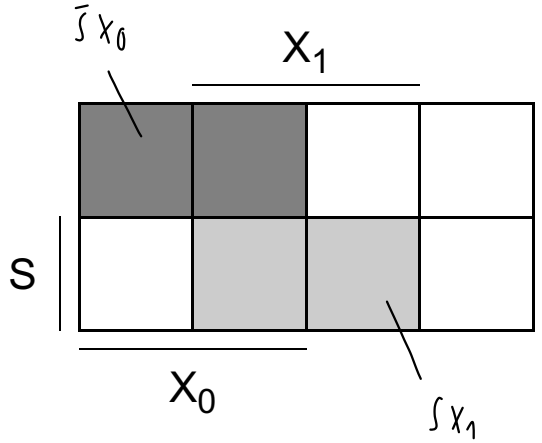
Beispiel 2: Multiplexer



1.4.5 Karnaugh-Diagramm

Systematischere Variante eines Venn-Diagramms

Beispiel



1.4.6 Normalformen

Definitionen

Minterm: Term, der alle Eingangsvariablen enthält und nur für eine Eingangskombination "1" wird.

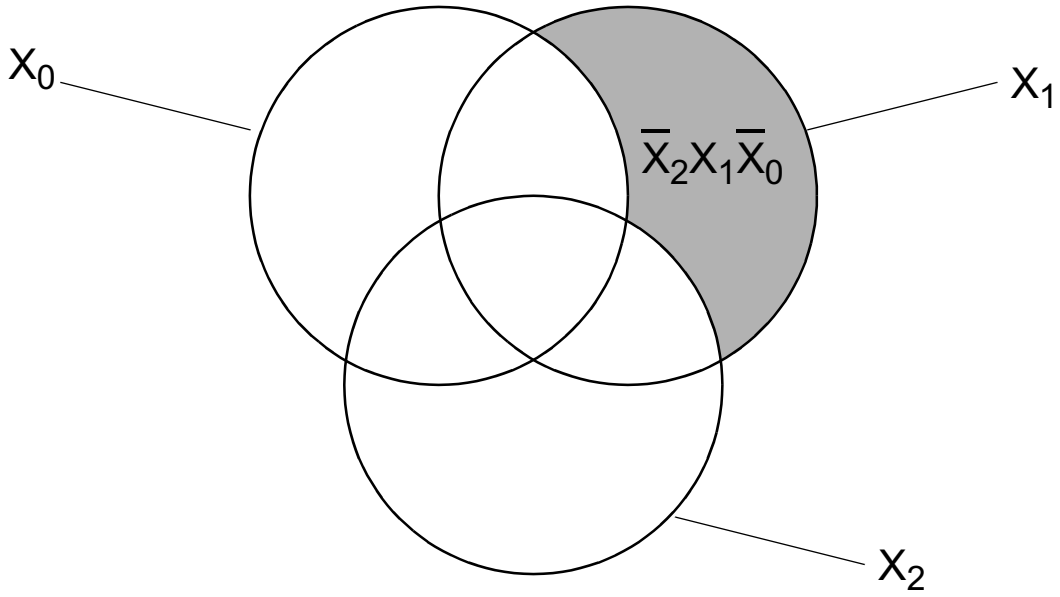
Maxterm: Term, der alle Eingangsvariablen enthält und nur für eine Eingangskombination "0" wird.

Beispiel (l = 3, m = 1): $Y = f(X_0, X_1, X_2)$

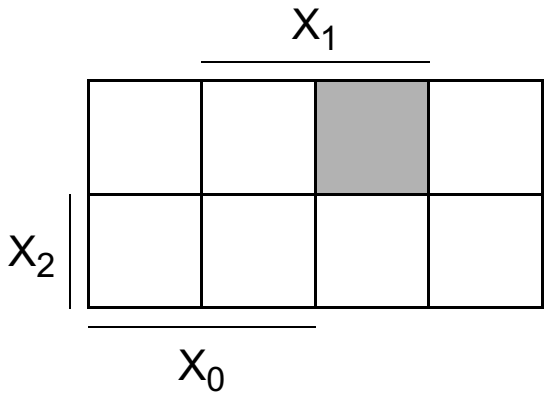
wo 0 steht immer – drauf bei Min

X_2	X_1	X_0	Y	Minterm	Maxterm
0	0	0	a_0	$\overline{X}_2\,\overline{X}_1\,\overline{X}_0$	$X_2 + X_1 + X_0$
0	0	1	a_1	$\overline{X}_2\,\overline{X}_1\,X_0$	$X_2 + X_1 + \overline{X}_0$
0	1	0	a_2	$\overline{X}_2\,X_1\,\overline{X}_0$	$X_2 + \overline{X}_1 + X_0$
0	1	1	a_3	$\overline{X}_2\,X_1\,X_0$	$X_2 + \overline{X}_1 + \overline{X}_0$
1	0	0	a_4	$X_2\,\overline{X}_1\,\overline{X}_0$	$\overline{X}_2 + X_1 + X_0$
1	0	1	a_5	$X_2\,\overline{X}_1\,X_0$	$\overline{X}_2 + X_1 + \overline{X}_0$
1	1	0	a_6	$X_2\,X_1\,\overline{X}_0$	$\overline{X}_2 + \overline{X}_1 + X_0$
1	1	1	a_7	$X_2\,X_1\,X_0$	$\overline{X}_2 + \overline{X}_1 + \overline{X}_0$

Beispiel: Illustration des Minterms $\overline{X}_2X_1\overline{X}_0$ mit Hilfe eines Venn- und eines Karnaugh-Diagramms

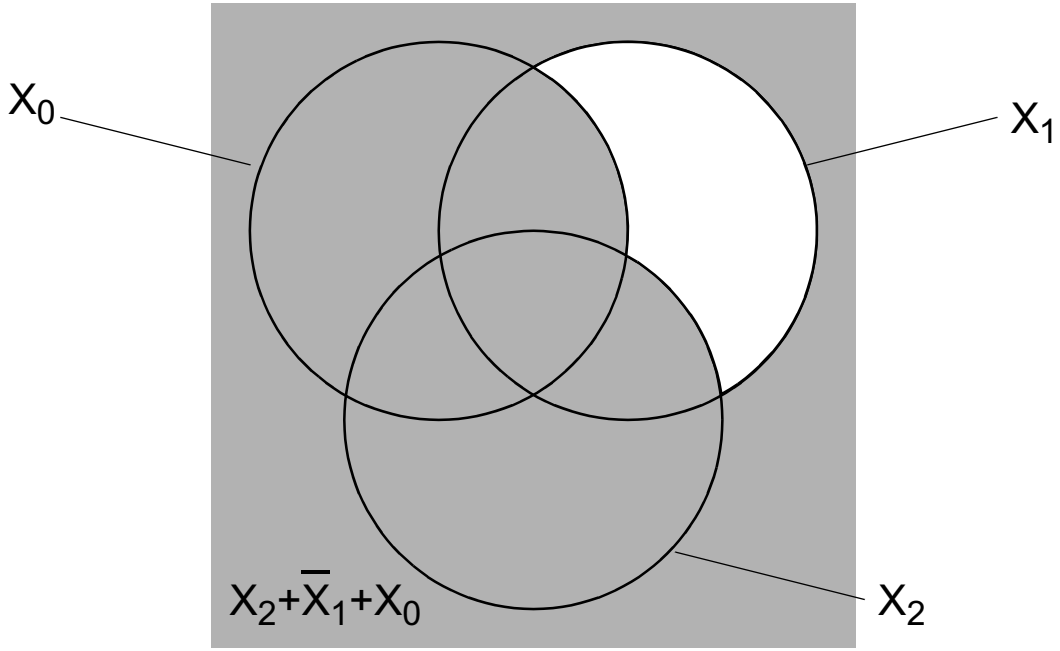


Venn-Diagramm

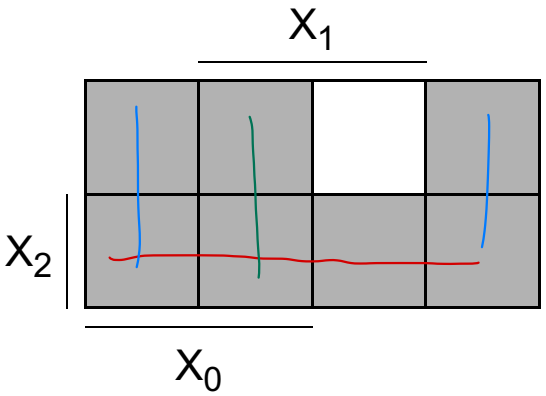


Karnaugh-Diagramm

Beispiel: Illustration des Maxterms $X_2+\overline{X}_1+X_0$ mit Hilfe eines Venn- und eines Karnaugh-Diagramms



Venn-Diagramm



Karnaugh-Diagramm

I. Disjunktive Normalform (DNF)

Disjunktive Verknüpfung (ODER) aller Minterme, für die der Funktionswert $a_i = 1$ ist.

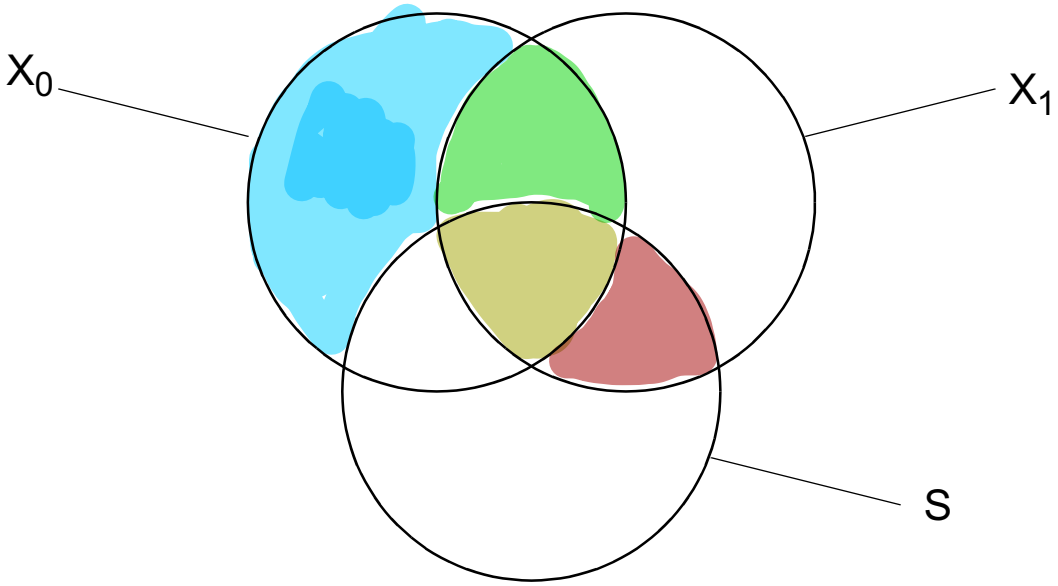
Allgemein (I = 3, m = 1)

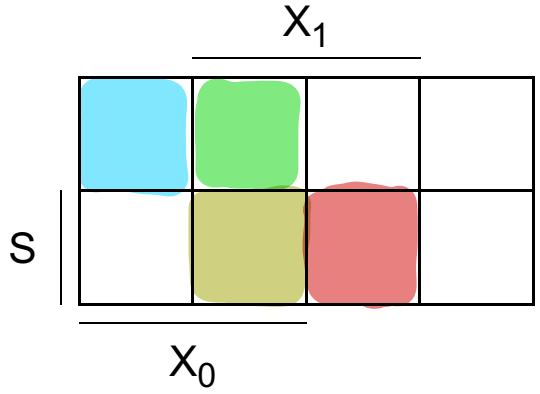
$$Y = f(X_2, X_1, X_0) = a_0 \overline{X}_2\overline{X}_1\overline{X}_0 + a_1 \overline{X}_2\overline{X}_1X_0 + a_2 \overline{X}_2X_1\overline{X}_0 + \ldots + a_6 X_2X_1\overline{X}_0 + a_7 X_2X_1X_0$$

Angewandt auf Beispiel

$$Y = \overline{S}X_0\overline{X}_1 + \overline{S}X_0X_1 + S\overline{X}_0X_1 + SX_0X_1$$

Venn-Diagramm, Karnaugh-Diagramm: **Vereinigungsmenge** der Minterme, für die $a_i = 1$ gilt





II. Konjunktive Normalform (KNF)

Konjunktive Verknüpfung (UND) aller Maxterme, für die der Funktionswert $a_i = 0$ ist.

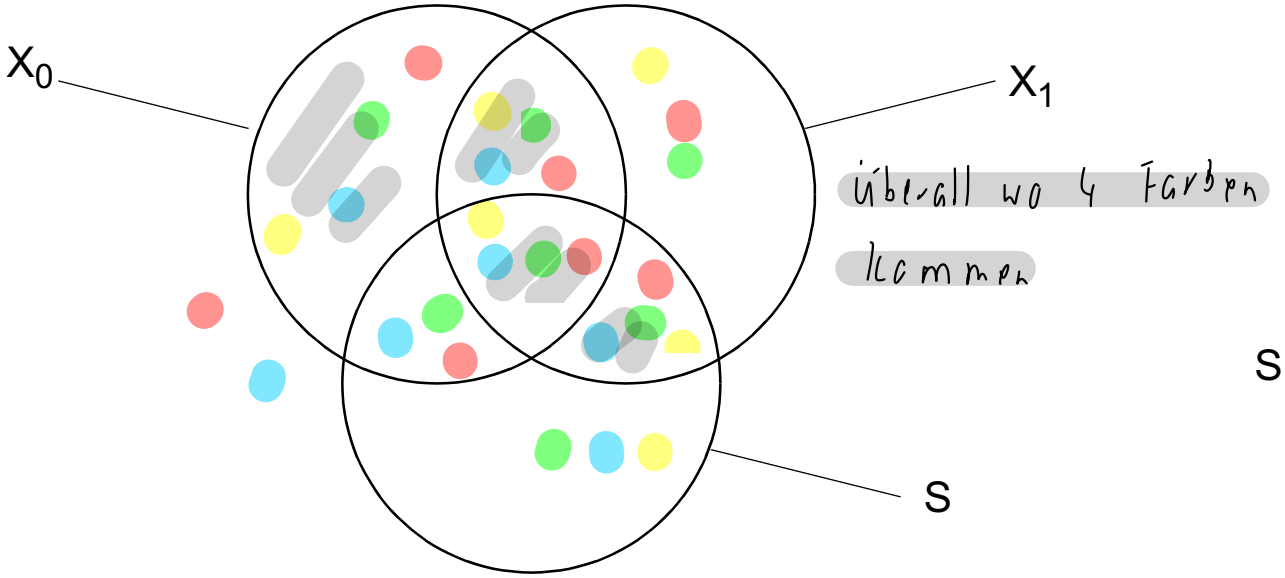
Allgemein (I = 3, m = 1)

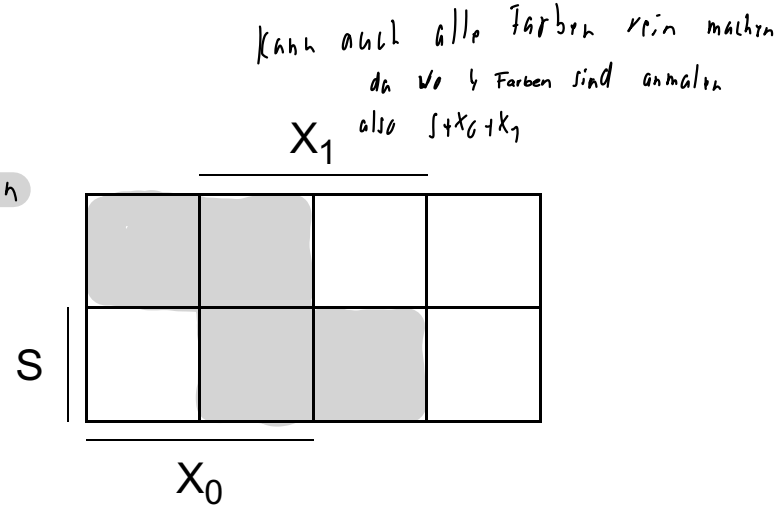
$$Y = f(X_2, X_1, X_0) = (a_0 + X_2+X_1+X_0)(a_1 + X_2+X_1+\overline{X}_0)(a_2 + X_2+\overline{X}_1+X_0) \dots (a_6 + \overline{X}_2+\overline{X}_1+X_0)(a_7 + \overline{X}_2+\overline{X}_1+\overline{X}_0)$$

Angewandt auf Beispiel

$$Y = (S + X_0 + X_1) \cdot (S + X_0 + \overline{X}_1) \cdot (\overline{S} + X_0 + X_1) \cdot (\overline{S} + \overline{X}_0 + X_1)$$

Venn-Diagramm, Karnaugh-Diagramm: **Schnittmenge** der Maxterme, für die $a_i = 0$ gilt



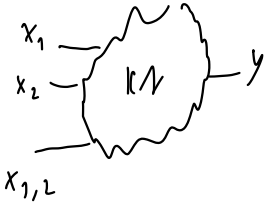


III. Shannon-Zerlegung

$$Y = f(X_0, X_1, X_2, \dots, X_{l-1}) = X_i \cdot f(X_0, \dots, X_{i-1}, 1, X_{i+1}, \dots, X_{l-1}) + \overline{X}_i \cdot f(X_0, \dots, X_{i-1}, 0, X_{i+1}, \dots, X_{l-1})$$

$$= X_i \cdot f(X_i = 1) + \overline{X}_i \cdot f(X_i = 0)$$

Beweis

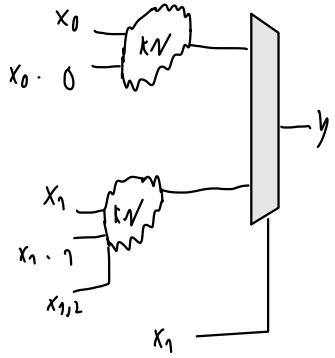


Einsetzen von $X_i = 1$: $Y = f(X_i = 1) = 1 \cdot f(X_i = 1) + 0 \cdot f(X_i = 0)$

Einsetzen von $X_i = 0$: $Y = f(X_i = 0) = 0 \cdot f(X_i = 1) + 1 \cdot f(X_i = 0)$

Beispiel

$Y = A(BC + \overline{B})$



Entwickeln nach A: $Y = A \cdot (BC + \overline{B}) + \overline{A} \cdot 0$

Entwickeln nach B: $Y = A \cdot B \cdot (C + 0) + A \cdot \overline{B} \cdot (0 + 1)$

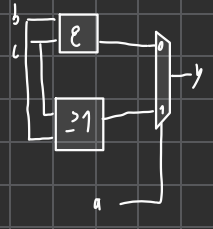
Entwickeln nach C: $Y = A \cdot B \cdot (C \cdot 1 + \overline{C} \cdot 0) + A \cdot \overline{B} \cdot (C \cdot 1 + \overline{C} \cdot 1)$

$= A \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot \overline{B} \cdot \overline{C}$ (Disjunktive Normalform!)

$y = ab + ac + bc$

Entwickeln nach a

$y = \bar{a}\,(y(a=0)) + a\,(y(a=1)) = \bar{a}\,(bc) + a\,(b+c)$



Entwickeln nach b

$y = \bar{a}\,(\bar{b}\,(0) + b\,(c)) + a\,(\bar{b}\,(c) + b\,(1))$

Entwickeln nach c

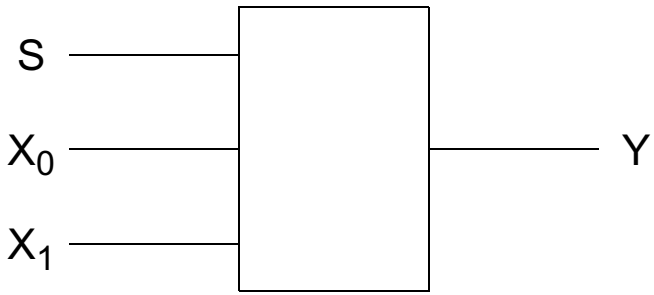
$y = \bar{a}\,(\bar{b}\,(\bar{c}\,0 + c\,0) + b\,(\bar{c}\,0 + c\,1)) + a\,(\bar{b}\,(\bar{c}\,0 + c\,1) + b\,(\bar{c}\,1 + c\,1))$

$y = 0\,\bar{a}\bar{b}\bar{c} + 0\,\bar{a}\bar{b}c + 0\,\bar{a}b\bar{c} + 1\,\bar{a}bc + 0\,a\bar{b}\bar{c} + 1\,a\bar{b}c + 1\,ab\bar{c} + 1\,abc$

1.4.7 Hardware-Beschreibungssprache (Beispiel: VHDL)

I. Beschreibung einer Schnittstelle

```
entity mux is
  port(
    s: in std_logic;
    x0, x1: in std_logic;
    y: out std_logic
  );
end entity mux;
```



II. Beschreibung der Implementierung

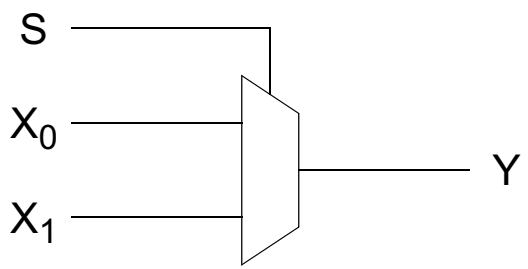
Möglichkeit 1: Verhaltensbeschreibung mit Hilfe schaltalgebraischer Gleichungen

```
architecture behav_equation of mux is
begin
  y <= (not s and x0) or (s and x1);
end architecture behav_equation;
```

$$Y = \overline{S}X_0 + SX_1$$

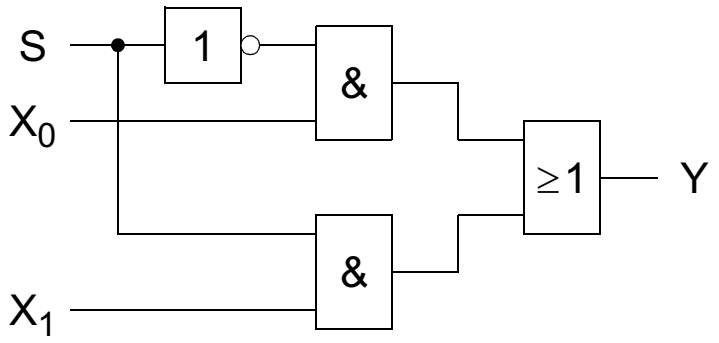
Möglichkeit 2: Verhaltensbeschreibung mit Hilfe sequenzieller Anweisungen

```
architecture behav_if of mux is
begin
  process(s, x0, x1) is
  begin
    if s = '0' then
      y <= x0;
    else
      y <= x1;
    end if;
  end process;
end architecture behav_if;
```



Möglichkeit 3: Strukturbeschreibung

```
architecture struct of mux is
  signal s_n, a0, a1: std_logic;
begin
  u1: entity notgate port map(s, s_n);
  u2: entity andgate port map(s_n, x0, a0);
  u3: entity andgate port map(s, x1, a1);
  u4: entity orgate port map(a0, a1, y);
end architecture struct;
```



1.5 Minimierungsverfahren

1.5.1 Grundprinzip

Axiome

$A \cdot B + A \cdot \overline{B} = A(B + \overline{B}) = A \cdot 1 = A$ (2a), (4a), (3a)

analog

$(A + B) \cdot (A + \overline{B}) = A + B\overline{B} = A + 0 = A$ (2b), (4a), (3b)

Beispiel

$ABC + AB\overline{C} + \overline{A}BC + A\overline{B}\,\overline{C} = AB + \overline{A}BC + A\overline{B}\,\overline{C} =$

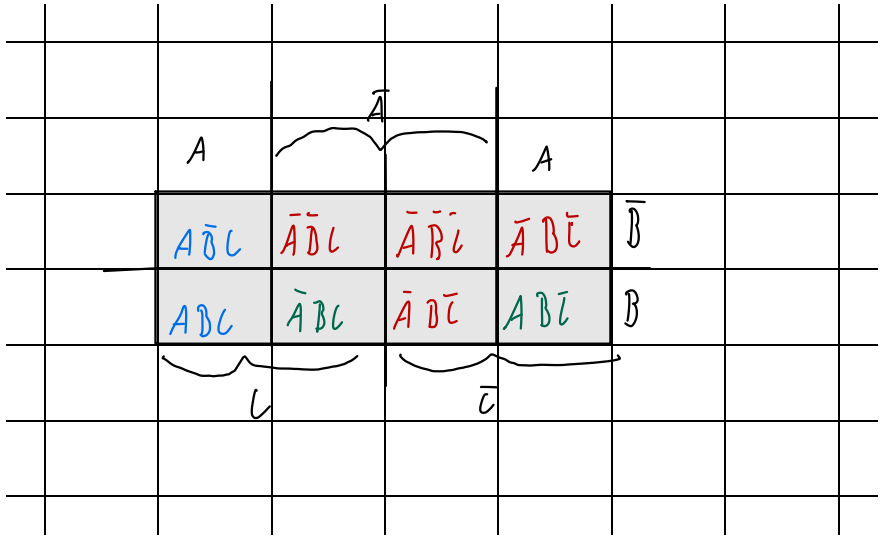
Fazit: Schaltalgebraische Umformungen führen nicht notwendigerweise zum minimalen Ausdruck.

1.5.2 Karnaugh-Diagramm

Beispiel: Disjunktive Normalform (z. B. aus Wahrheitstabelle) für n = 3 Variable A, B, C

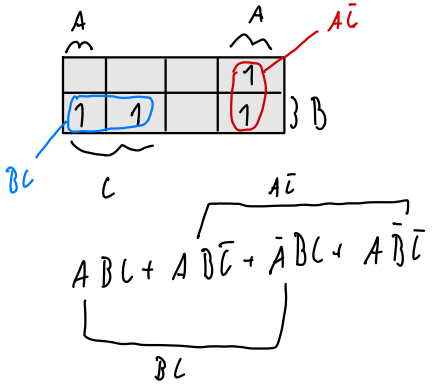
Mögliche Minterme: $\overline{A}\overline{B}\overline{C}$, $\overline{A}\overline{B}C$, $\overline{A}B\overline{C}$, $\overline{A}BC$, $A\overline{B}\overline{C}$, $A\overline{B}C$, $AB\overline{C}$, ABC

Idee: Anordnung der Minterme so, dass sich benachbarte Terme nur in einer Variablen unterscheiden.



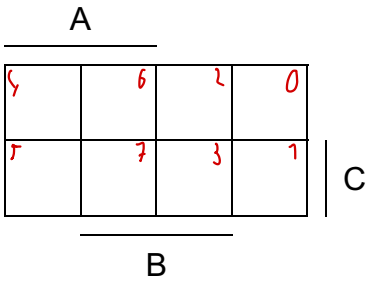
Muss immer Rechteck sein

$A\bar{B}C$ und ABC könnte man recht und links schreiben



Karnaugh-Diagramm für 3 Variablen

Zeile	A	B	C	Minterm
0	0	0	0	$\overline{A}\,\overline{B}\,\overline{C}$
1	0	0	1	$\overline{A}\,\overline{B}\,C$
2	0	1	0	$\overline{A}\,B\,\overline{C}$
3	0	1	1	$\overline{A}\,B\,C$
4	1	0	0	$A\,\overline{B}\,\overline{C}$
5	1	0	1	$A\,\overline{B}\,C$
6	1	1	0	$A\,B\,\overline{C}$
7	1	1	1	$A\,B\,C$

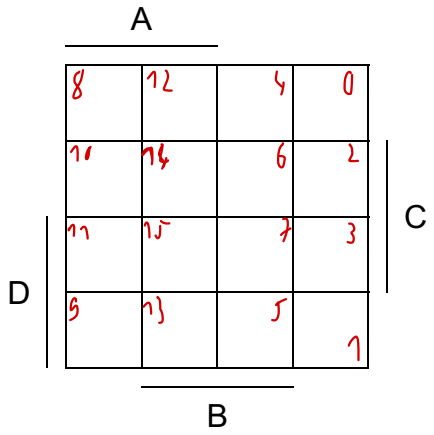


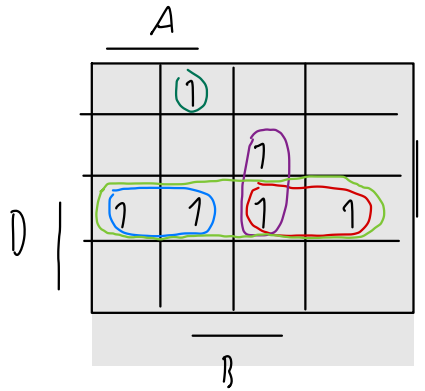
- 1 hälfte immer ungerade z.b 1,3,7,5
- Das unterschiedet sich in der Formel also was alles gerollt ist
- Wenn ich 0, 1 und 2 habe dann ist es ab da symmetrisch

$A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D} + A\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}CD$

Karnaugh-Diagramm für 4 Variablen

Zeile	A	B	C	D	Minterm
0	0	0	0	0	$\overline{A}\,\overline{B}\,\overline{C}\,\overline{D}$
1	0	0	0	1	$\overline{A}\,\overline{B}\,\overline{C}\,D$
2	0	0	1	0	$\overline{A}\,\overline{B}\,C\,\overline{D}$
3	0	0	1	1	$\overline{A}\,\overline{B}\,C\,D$
4	0	1	0	0	$\overline{A}\,B\,\overline{C}\,\overline{D}$
...					
8	1	0	0	0	$A\,\overline{B}\,\overline{C}\,\overline{D}$
...					





$A\bar{C}D + \bar{A}\bar{C}D + \bar{A}BC$

CD

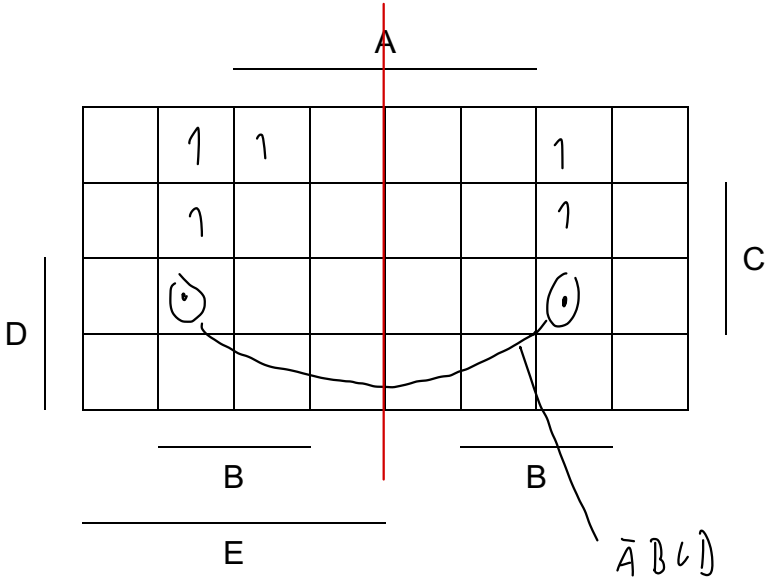
$A\bar{B}\bar{C}D + \bar{A}BC + CD$

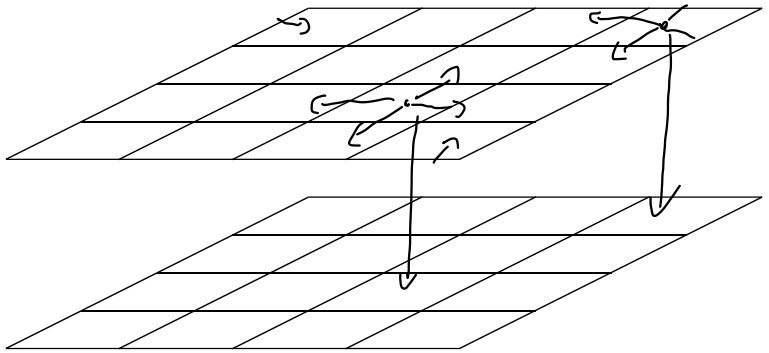
(Bei möglichst großen)

unten nur ungerade zahlen, oben nur gerade zahlen

links alle zahlen größer als 8

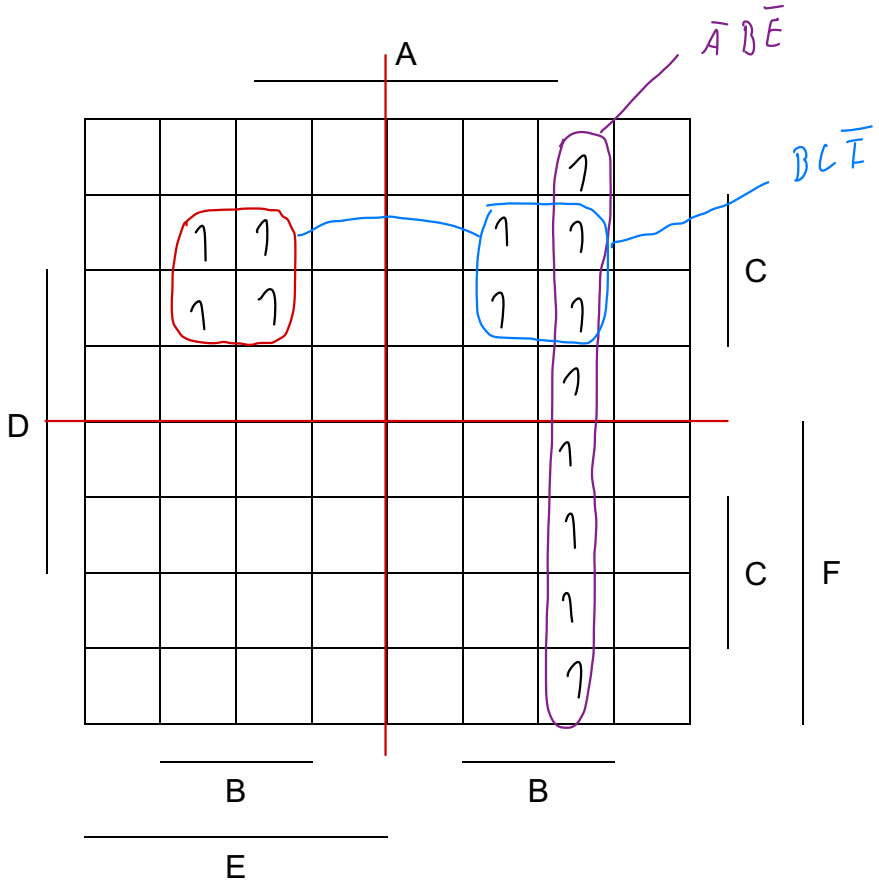
Karnaugh-Diagramm für 5 Variablen

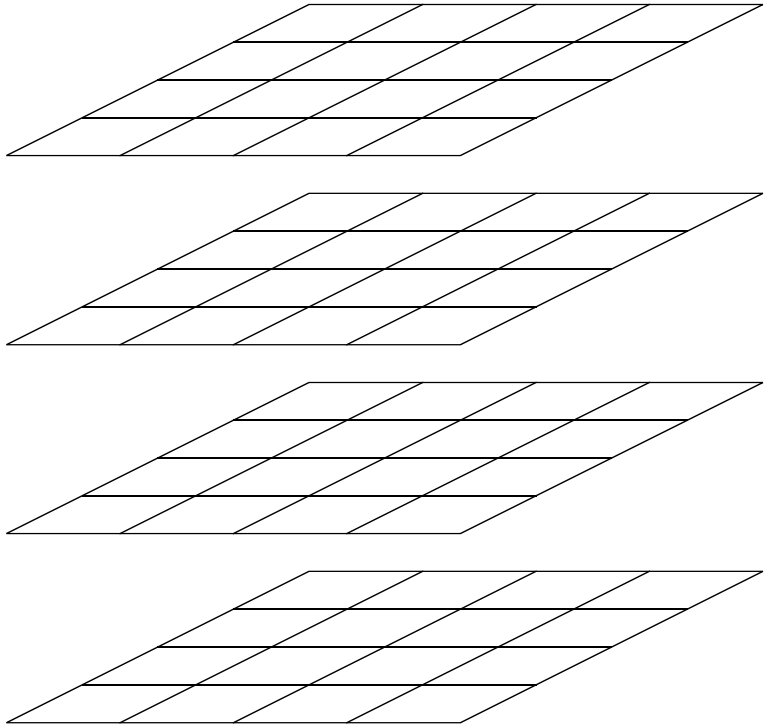




Das könnte kommen

Karnaugh-Diagramm für 6 Variablen

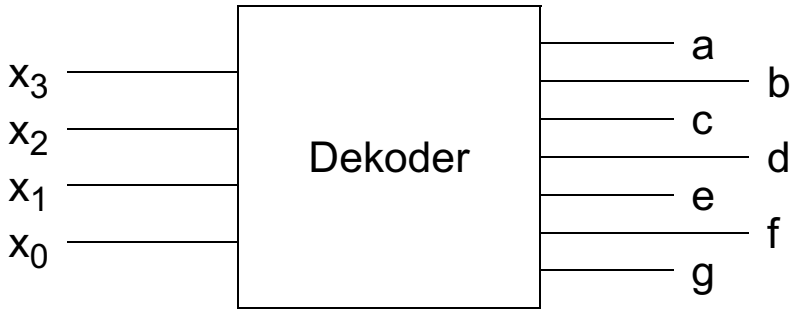


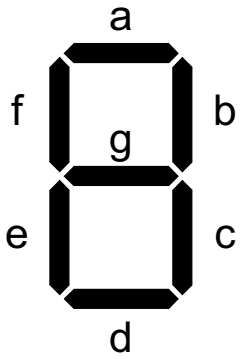


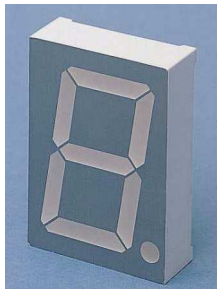
Nicht Prüfung relevant

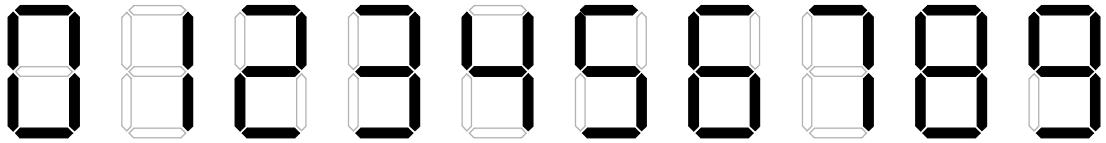
Beispiel

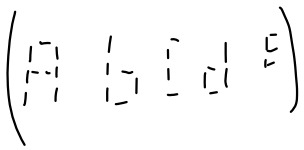
Dekoder für 7-Segment-Anzeige











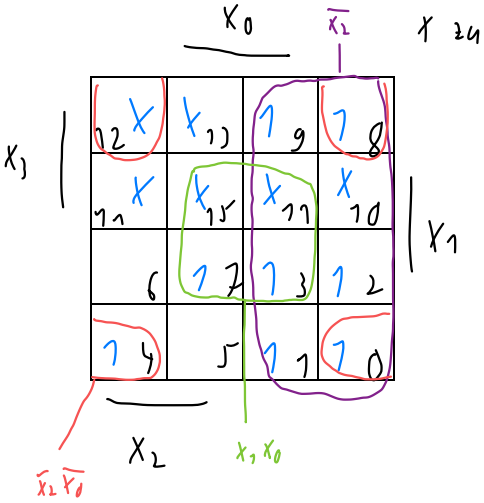
Wahrheitstabelle

	BCD-Ziffer				Segmente						
	x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
10	1	0	1	0	X	X	X	X	X	X	X
11	1	0	1	1	X	X	X	X	X	X	X
12	1	1	0	0	X	X	X	X	X	X	X
13	1	1	0	1	X	X	X	X	X	X	X
14	1	1	1	0	X	X	X	X	X	X	X
15	1	1	1	1	X	X	X	X	X	X	X

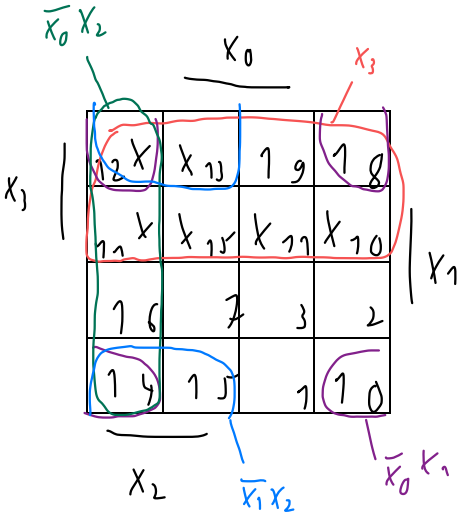
Segment $b = \bar{x}_2 + \bar{x}_1\bar{x}_0 + x_1x_0$

Wenn ich große Schlaufen machen kann, kann ich X zu 1 machen sonst 0

- b muss 0 sein
- Schlaufe immer 2 potenz



$f = x_3 + \bar{x}_0x_2 + \bar{x}_1x_2 + \bar{x}_0x_1$



dont care

1.5.3 Rechnergestützte Verfahren

Beispiel: Verfahren nach Quine McCluskey Nicht Prüfung relevant

1. Schritt: Bestimmung der Primterme

- Ausgangspunkt: Funktion $Y = f(X_0, X_1, ..., X_{n-1})$ in Disjunktiver Normalform bzw. Wahrheitstabelle
- systematisch alle Paare von Mintermen prüfen, für die der Funktionswert "1" ist
  - Paar wenn möglich durch Anwendung von $AX + A\overline{X} = A$ zusammenfassen
  - es entstehen Terme mit n−1 Variablen
- systematisch alle Paare entstandener Terme mit n−1 Variablen prüfen und ggf. zusammenfassen
- systematisch alle Paare entstandener Terme mit n−2 Variablen prüfen und ggf. zusammenfassen
- weiter, bis keine weiteren Zusammenfassungen mehr möglich

Optimierung: Um den Rechenaufwand zu reduzieren, teilt man die Terme in Klassen ein, wobei eine Klasse K_i Terme mit i nicht-negierten Variablen enthält. Dann müssen in den folgenden Schritten nur noch Paare betrachtet werden, deren Elemente aus benachbarten Klassen stammen.

Karnaugh-Diagramm: Primterme = Schlaufen maximaler Größe.

2. Schritt: Auswahl der wesentlichen Primterme

Gibt es Minterme, die nur in einem Primterm enthalten sind (nur von einem Primterm abgedeckt werden), so muss dieser Primterm auf jeden Fall in die minimierte Funktion aufgenommen werden. Man nennt diese Primterme deshalb wesentliche Primterme.

Karnaugh-Diagramm: Ein wesentlicher Primterm ist eine Schlaufe maximaler Größe, die mindestens eine "1" enthält, die ausschließlich in dieser Schlaufe enthalten ist.

3. Schritt: Restmatrix

Die nicht von wesentlichen Primtermen abgedeckten Minterme müssen durch die nicht-wesentlichen Primterme abgedeckt werden. Dabei gibt es in der Regel mehrere Möglichkeiten, die man mit Hilfe einer sogenannten Restmatrix bestimmt. Man spricht von Optionen zur Restüberdeckung.

4. Schritt: Minimierte Funktion

Minimierte Schaltfunktion setzt sich zusammen aus den wesentlichen Primtermen und der Restüberdeckung durch nicht wesentliche Primterme.

Rechnerarchitektur

Matthias Meyer

Kapitel 2 Sequenzielle Netzwerke

Inhaltsübersicht

2.1 Flipflop-Speicherelemente

2.2 Beispiele

2.3 Modellierung

2.4 Entwurfsverfahren

2.1 Flipflop-Speicherelemente

Inhaltsübersicht

2.1.1 Grund-Flipflop

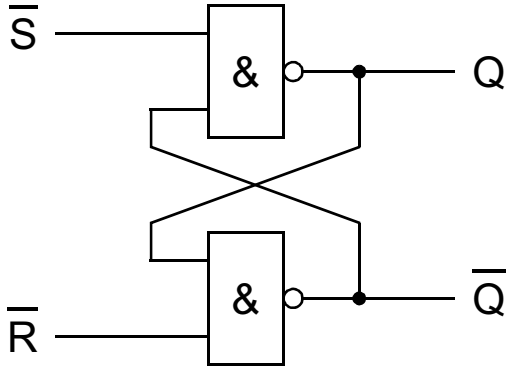
2.1.2 Taktzustandsgesteuertes Flipflop (Latch)

2.1.3 Master-Slave-Flipflop

2.1.4 Taktflankengesteuertes Flipflop

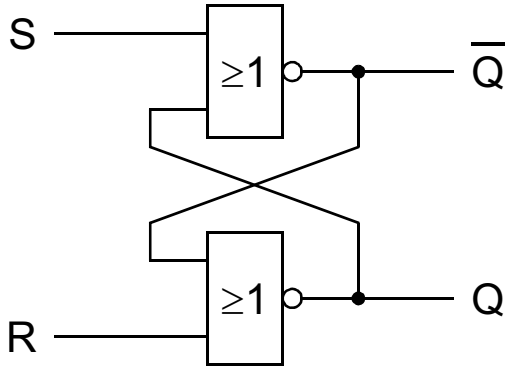
2.1.5 Charakteristische Schaltfunktion von Flipflops

2.1.1 Grund-Flipflop



aus rückgekoppelten NAND-Gattern

(low-aktive Setz- und Rücksetzeingänge)



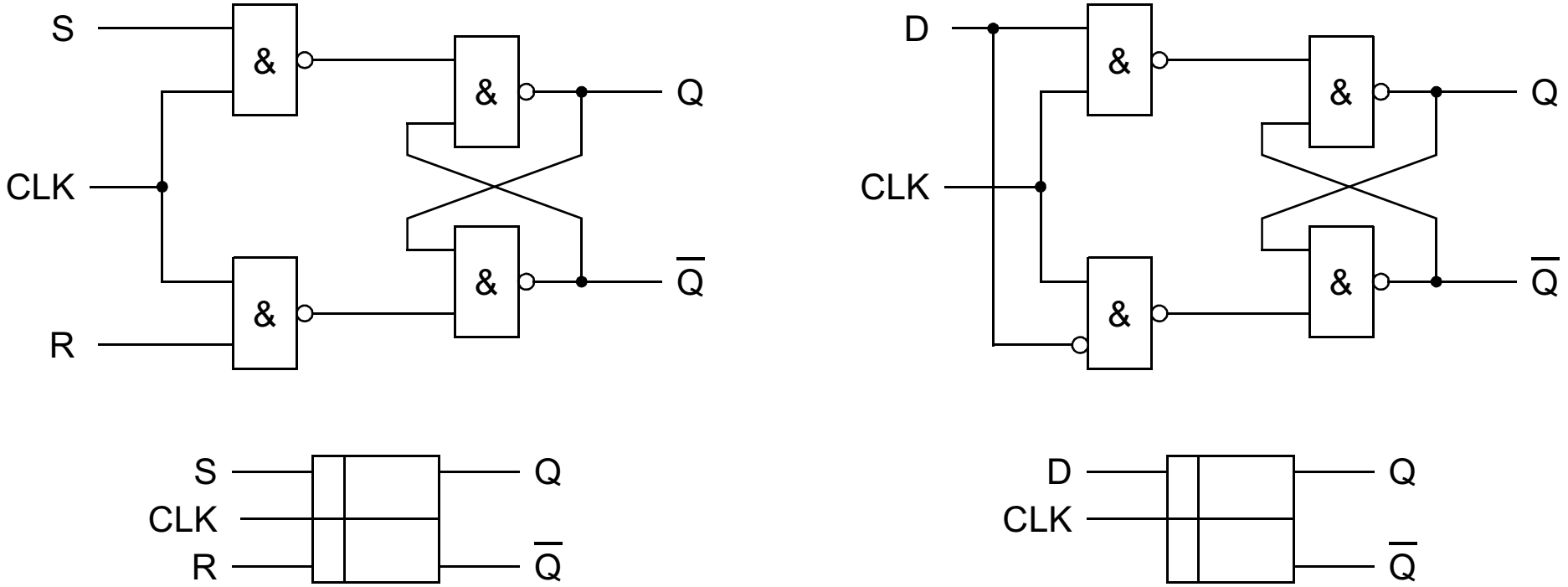
aus rückgekoppelten NOR-Gattern

(high-aktive Setz- und Rücksetzeingänge)

Merkmale

- zwei stabile Zustände Q = 0, $\overline{Q}$ = 1 und Q = 1, $\overline{Q}$ = 0
- Zustandsänderung durch (kurze) Setz- oder Rücksetzimpulse
- asynchron: Setz- und Rücksetzimpulse werden jederzeit ausgewertet
- $\overline{S}$ = $\overline{R}$ = 0 bzw. S = R = 1 irreguläre Eingangskombination, weil dann Q = $\overline{Q}$ = 1 bzw. Q = $\overline{Q}$ = 0

2.1.2 Taktzustandsgesteuertes Flipflop (Latch)



(a) SR-Latch

(b) D-Latch

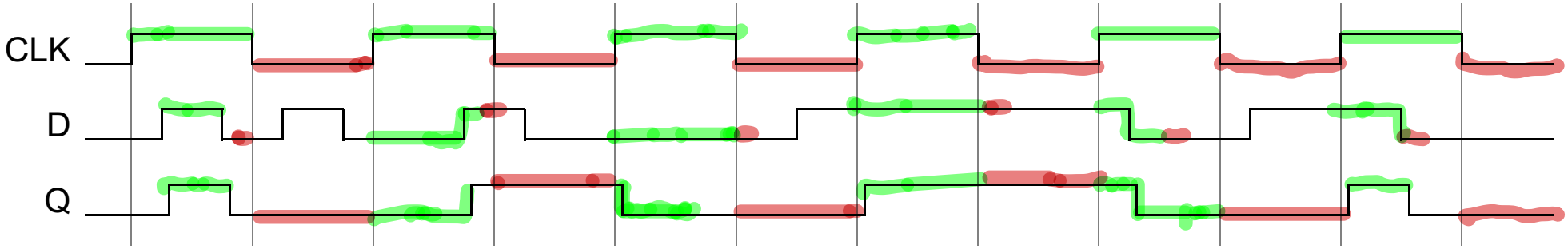
Merkmale

- besteht aus Ansteuerschaltung und Grundflipflop (GFF)
- Ansteuerschaltung sorgt dafür, dass die Eingangssignale nur während CLK = 1 auf das GFF wirken

D-Latch

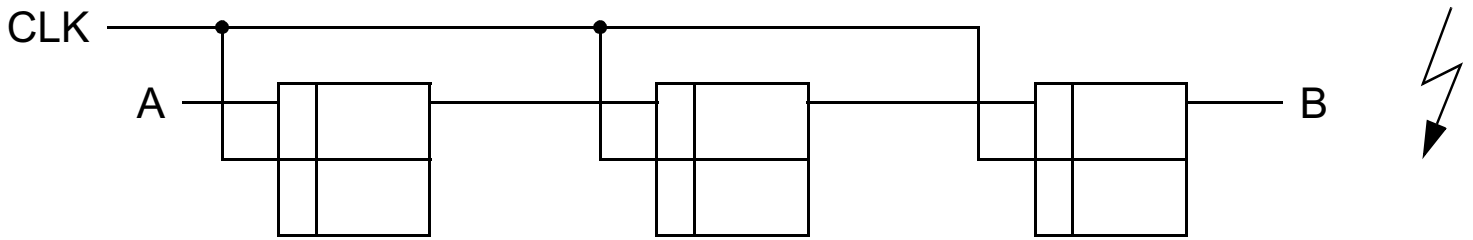
- CLK = 1: Latch ist transparent, d.h. D erscheint direkt am Ausgang
- CLK = 0: Latch ist verriegelt und speichert den letzten Wert von D vor der fallenden Flanke von CLK

Signal-Zeit-Diagramm für das D-Latch

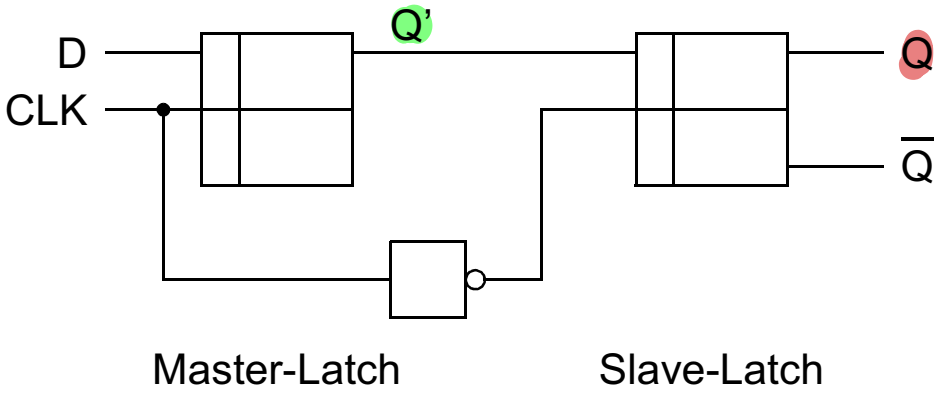


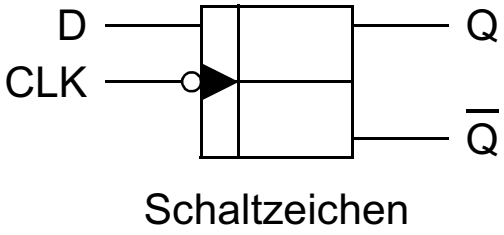
Latches können **nicht** zum Aufbau synchroner Steuerwerke (Schieberegister, Zähler, allg.: sequenzielle Netzwerke) verwendet werden!

Beispiel

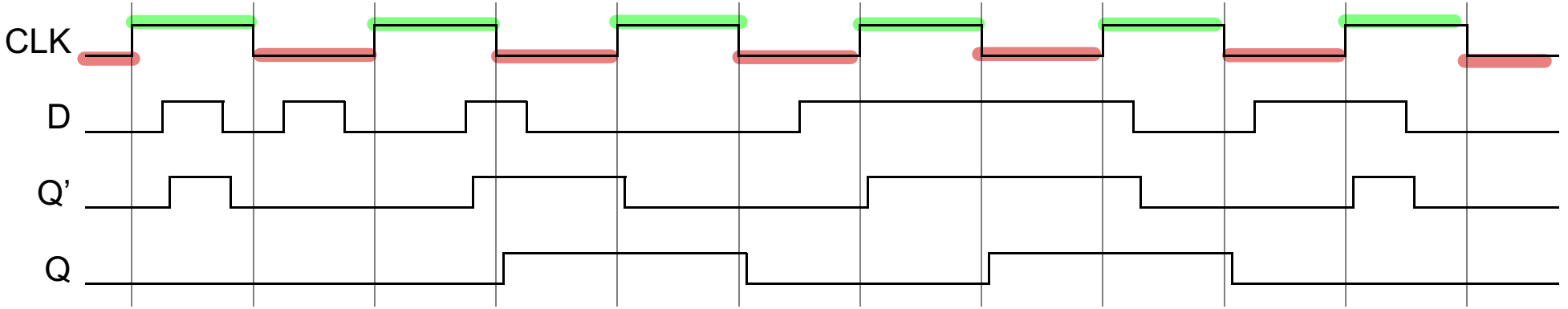


2.1.3 Master-Slave-Flipflop





Signal-Zeit-Diagramm für das Master-Slave-FF



Merkmale

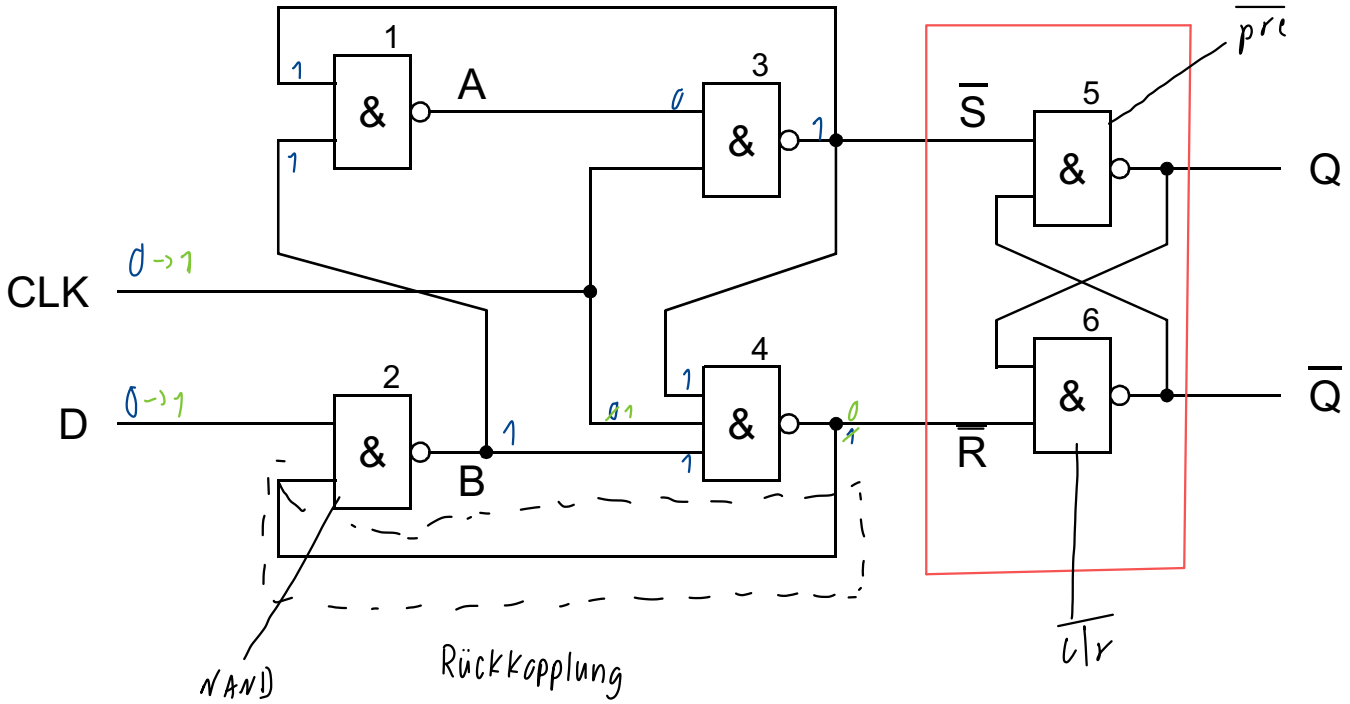
- Flipflop übernimmt Eingang D zur fallenden Taktflanke
- Aufbau allgemeiner synchroner Schaltwerke möglich!

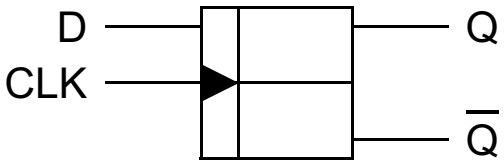
2.1.4 Taktflankengesteuertes Flipflop

Merkmale

- Eingänge werden tatsächlich nur zum Zeitpunkt der aktiven Taktflanke ausgewertet (üblicherweise positive Taktflanke)
- Einsatzmöglichkeit wie Master-Slave-FF

Realisierungsmöglichkeit: Rückkopplungen zur Eingangsverriegelung





Schaltzeichen

Analyse

- CLK = 0: $\overline{S} = \overline{R} = 1$, d.h. GFF behält Zustand unabhängig von D bei
- Übergang von CLK = 0 auf CLK = 1
  - D = 0 $\rightarrow$ B = 1, A = 0 $\rightarrow$ $\overline{R} = 0$, $\overline{S} = 1$ $\rightarrow$ GFF wird zurückgesetzt

    durch Rückkopplung von $\overline{R}$ auf Gatter 2 bleibt B = 1 und damit $\overline{R} = 0$ (unabhängig von D!)

    wegen B = 1 und $\overline{S} = 1$ bleibt A = 0 und damit $\overline{S} = 1$ (unabhängig von D!)
  - D = 1 $\rightarrow$ B = 0, A = 1 $\rightarrow$ $\overline{R} = 1$, $\overline{S} = 0$ $\rightarrow$ GFF wird gesetzt

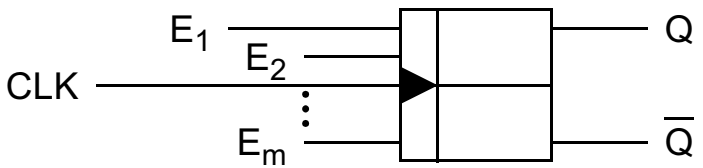
    durch Rückkopplung von $\overline{S}$ auf Gatter 1 bleibt A = 1 und damit $\overline{S} = 0$ (unabhängig von D!)

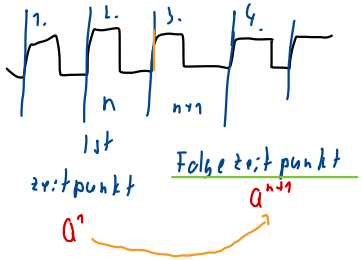
    durch Rückkopplung von $\overline{S}$ auf Gatter 4 bleibt $\overline{R} = 1$ (unabhängig von D!)

2.1.5 Charakteristische Schaltfunktion von Flipflops

Flipflops lassen sich nach der Art ihrer Eingangssignale klassifizieren und danach, wie diese Eingangssignale das Folgeschaltverhalten des Flipflops beeinflussen. Dabei werden nachfolgend nur Flipflops betrachtet, die die Eingangssignale zu einem definierten Zeitpunkt (aktive Taktflanke) auswerten und die sich damit zum Aufbau allgemeiner synchroner Steuerwerke eignen (Master-Slave-FF, taktflankengesteuertes FF). Sie behalten während einer Taktperiode n ihren Zustand Q^n unabhängig von ihren Eingangssignalen bei.

Alle diese Flipflops gehen in Folge einer aktiven Taktflanke und in Abhängigkeit vom Wert ihrer Eingangssignale und ihrem bisherigen Zustand Q^n in einen Folgezustand $Q^{n+1} = f(E_1^n, E_2^n, \ldots E_m^n, Q^n)$ über. Diese Gleichung beschreibt also das Folgeschaltverhalten eines Flipflops und wird deshalb als charakteristische Schaltfunktion des jeweiligen Flipflops bezeichnet.





D-Flip Flop

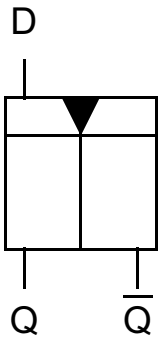
$Q^{n+1} = D^n$

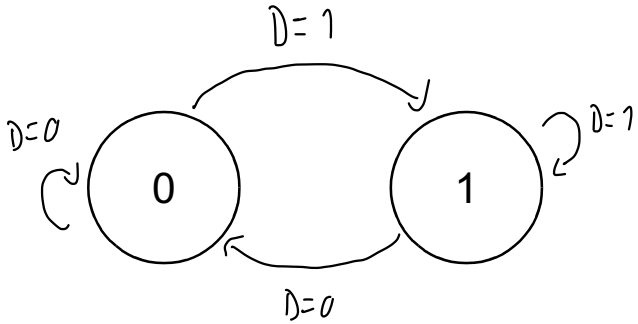
$E_1, E_2, \ldots E_m$	Eingangssignale
Q^n	Zustand des Flipflops im Taktintervall n
Q^{n+1}	Zustand des Flipflops im Folgetaktintervall n + 1
$Q^{n+1} = f(E_1^n, E_2^n, \ldots E_m^n, Q^n)$	charakteristische Schaltfunktion

Die charakteristische Schaltfunktion von Flipflops lässt sich mit Hilfe von Zustands-Übergangsdiagrammen, Wertetafeln, Karnaugh-Diagrammen und mit Hilfe schaltalgebraischer Gleichungen beschreiben.

Beispiele

D-Flipflop





Zustands-Übergangsdiagramm

D^n	Q^{n+1}
0	0
1	1

Wertetafel

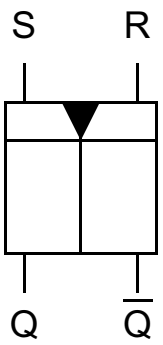
$Q^{n+1} = D^n$

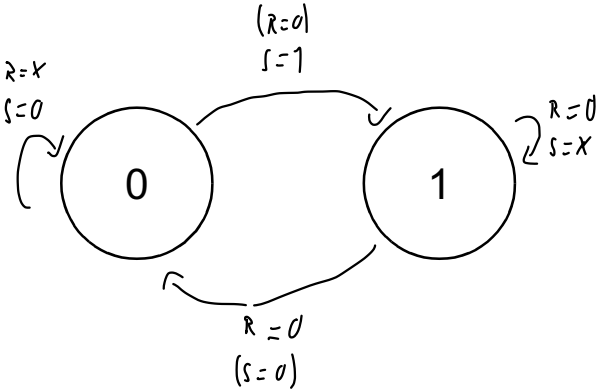
char. Schaltfunktion

SR-Flipflop

Ist es Prüfungsrelevant?

Nebenbedingung $S \cdot R = 0$



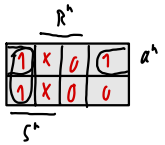


S^n	R^n	Q^{n+1}
0	0	Q^n
0	1	0
1	0	1
1	1	X

$Q^{n+1} = (S + \bar{R}\,Q)$

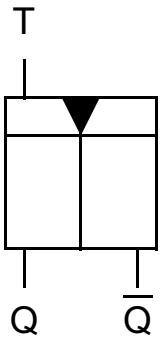
Zustands-Übergangsdiagramm | Wertetafel | char. Schaltfunktion

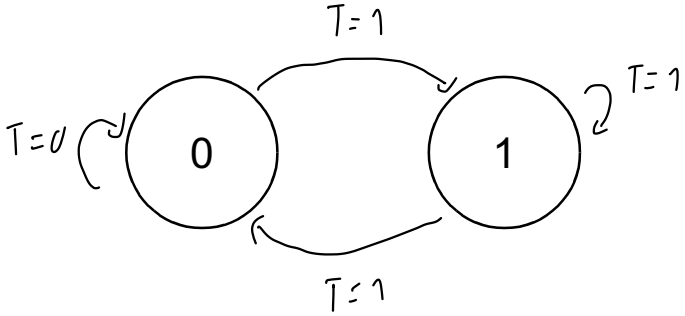
S^n	R^n	Q^n	Q^{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X



$S^n + Q^n \overline{R^n}$

T-Flipflop (Toggle-Flipflop)





Zustands-Übergangsdiagramm

T^n	Q^{n+1}
0	Q^n
1	$\overline{Q^n}$

Wertetafel

$Q^{n+1} = \overline{T^n} \cdot Q^n + T^n \cdot \overline{Q^n}$

$T=0 \quad Q^{n+1} = Q^n$

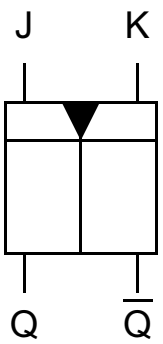
$T=1 \quad Q^{n+1} = \overline{Q^n}$

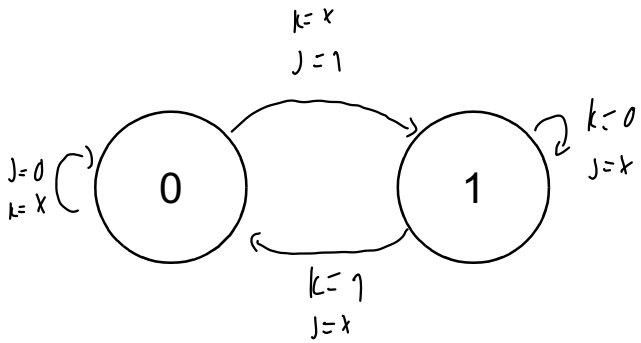
char. Schaltfunktion

$(0\,Q + 0 \cdot \bar{Q})^n = Q^n$

$(1\,Q + 1 \cdot \bar{Q})^n = \overline{Q^n}$

JK-Flipflop



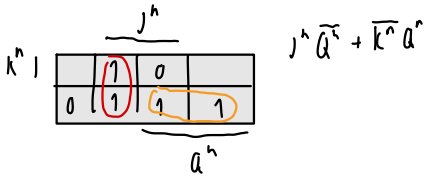


J^n	K^n	Q^{n+1}
0	0	Q^n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

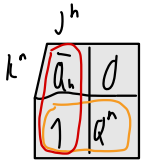
(S) (R)

$Q^{n+1} = (K Q + J \bar{Q})^n$

Zustands-Übergangsdiagramm



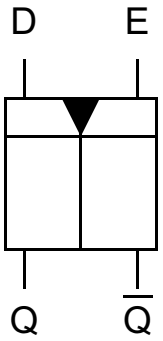
Wertetafel

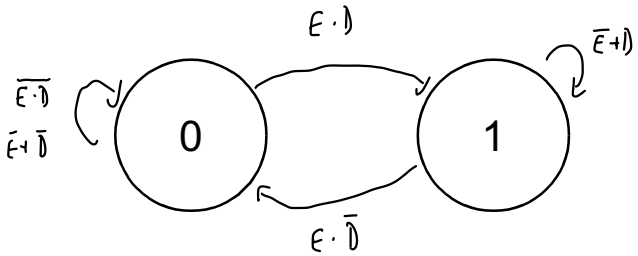


char. Schaltfunktion

$J^n \bar{Q}^n + K^n Q^n$

Enable-D-Flipflop (D-Flipflop mit Freigabeeingang)



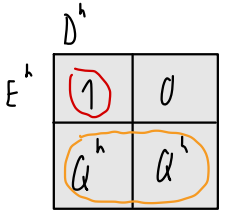


D^n	E^n	Q^{n+1}
0	0	Q^n
0	1	0
1	0	Q^n
1	1	1

$Q^{n+1} = \left(\overline{E}\,Q + E\,D\right)^n$

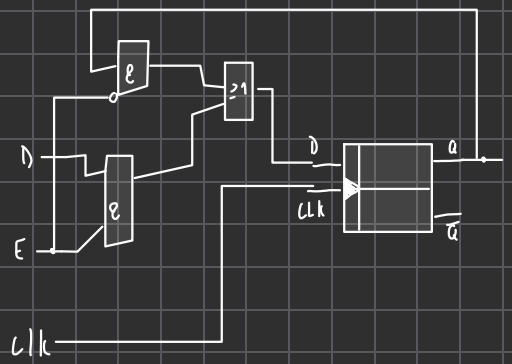
Zustands-Übergangsdiagramm

Wertetafel



char. Schaltfunktion

$E^n D^n + \overline{E}^n Q^n$



$(\bar{E}Q + ED)^n$

+ = ODER

oben = NOT

• = UND

2.2 Beispiele

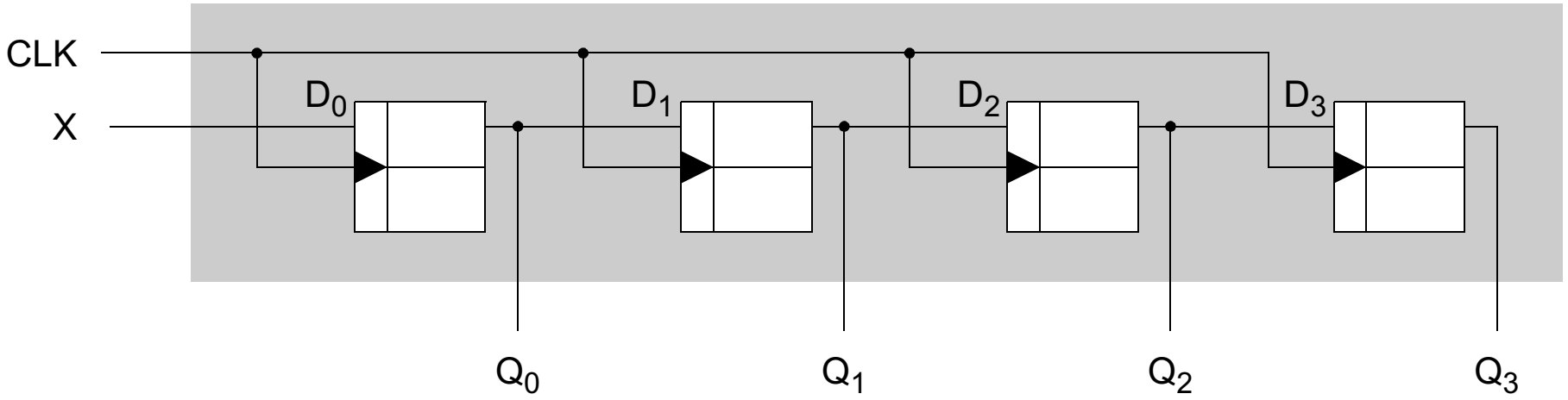
Inhaltsübersicht

2.2.1 Schieberegister

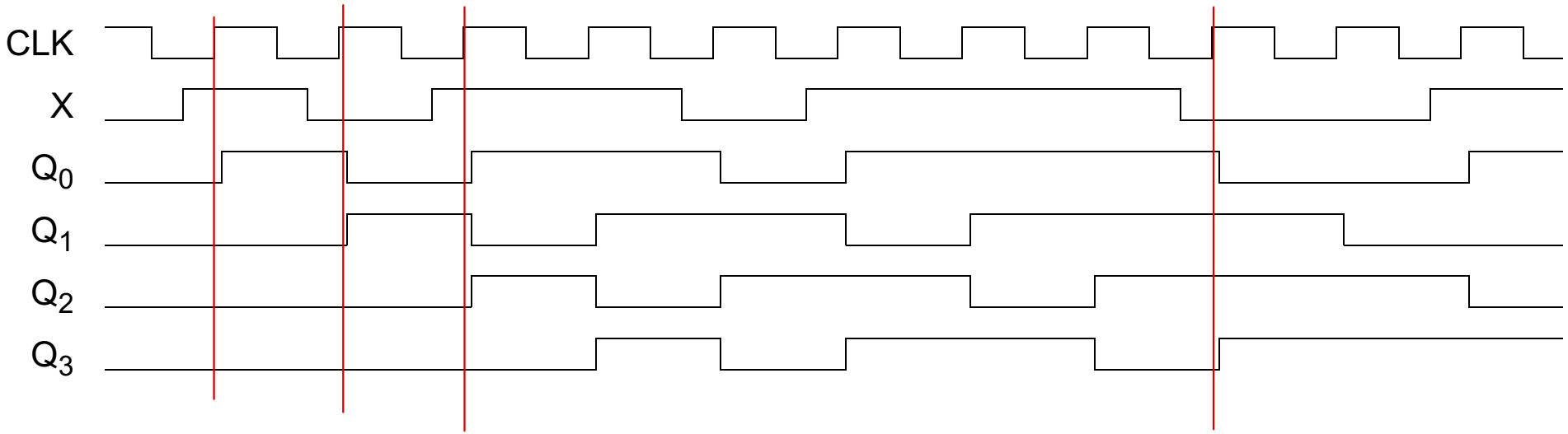
2.2.2 Synchroner Binärzähler

2.2.1 Schieberegister

Schaltplan



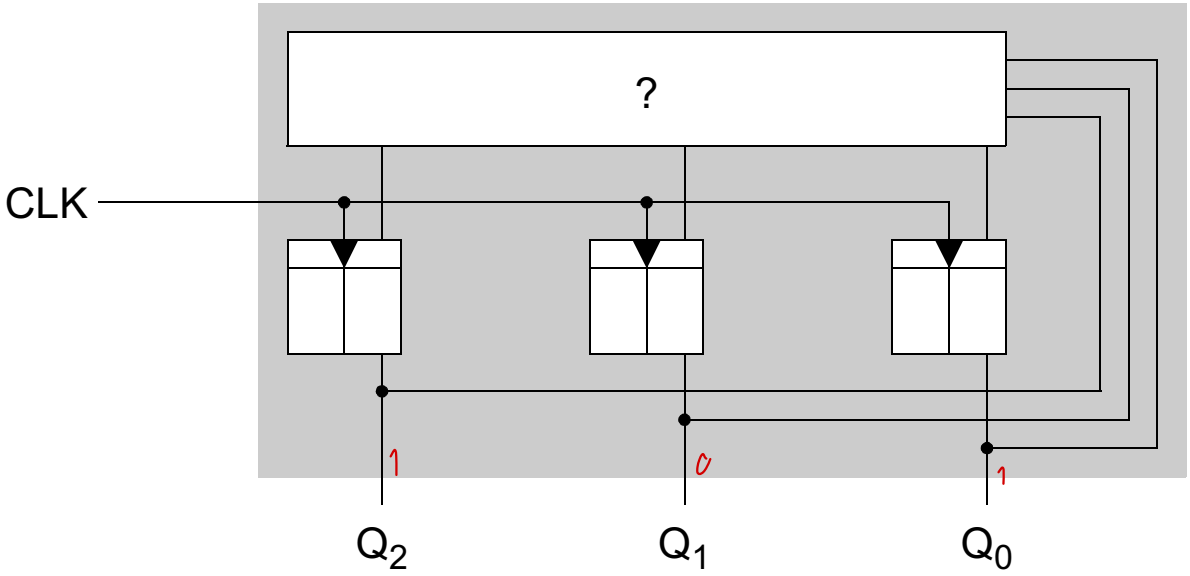
Signal-Zeitdiagramm



2.2.2 Synchroner Binärzähler

Beispiel: Zähler mit 3 Bit (Q_2, Q_1, Q_0), Zählfolge 000, 001, 010, 011, 100, 101, 110, 111, 000, 001, ...

Synchron: Alle Flipflops schalten gleichzeitig, d.h. werden an dasselbe Taktsignal angeschlossen.



Signal-Zeitdiagramm

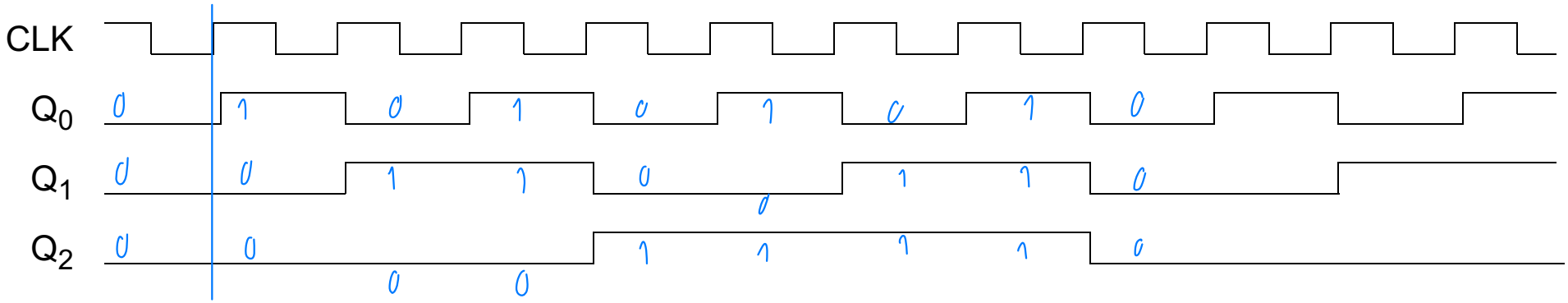


Tabelle zur Beschreibung des Folgeschaltverhaltens $\underline{Q}^{n+1} = \underline{f}(\underline{Q}^{n})$

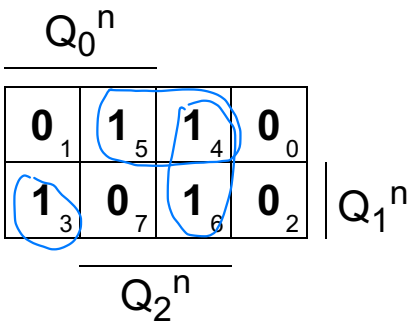
$Q_i^{n+1} = D_i^n$

$i = 0, 1, 2$

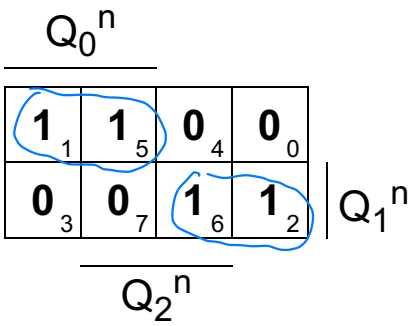
	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	1	0	0	1	1
3	0	1	1	1	0	0
4	1	0	0	1	0	1
5	1	0	1	1	1	0
6	1	1	0	1	1	1
7	1	1	1	0	0	0
				D_2^n	D_1^n	D_0^n

Problem: Bestimmung der Funktionen $Q_i^{n+1} = f_i(Q_2^n, Q_1^n, Q_0^n)$, i = 0, 1, 2 ("Problemfunktionen")

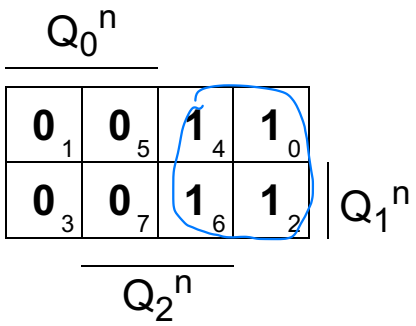
Karnaugh-Diagramme



Q_2^{n+1}



Q_1^{n+1}



Q_0^{n+1}

$$Q_2^{n+1} = (\overline{Q}_2 Q_1 Q_0 + Q_2 \overline{Q}_1 + Q_2 \overline{Q}_0)^n$$

$$Q_1^{n+1} = (\overline{Q}_1 Q_0 + Q_1 \overline{Q}_0)^n$$

$$Q_0^{n+1} = \overline{Q}_0^n$$

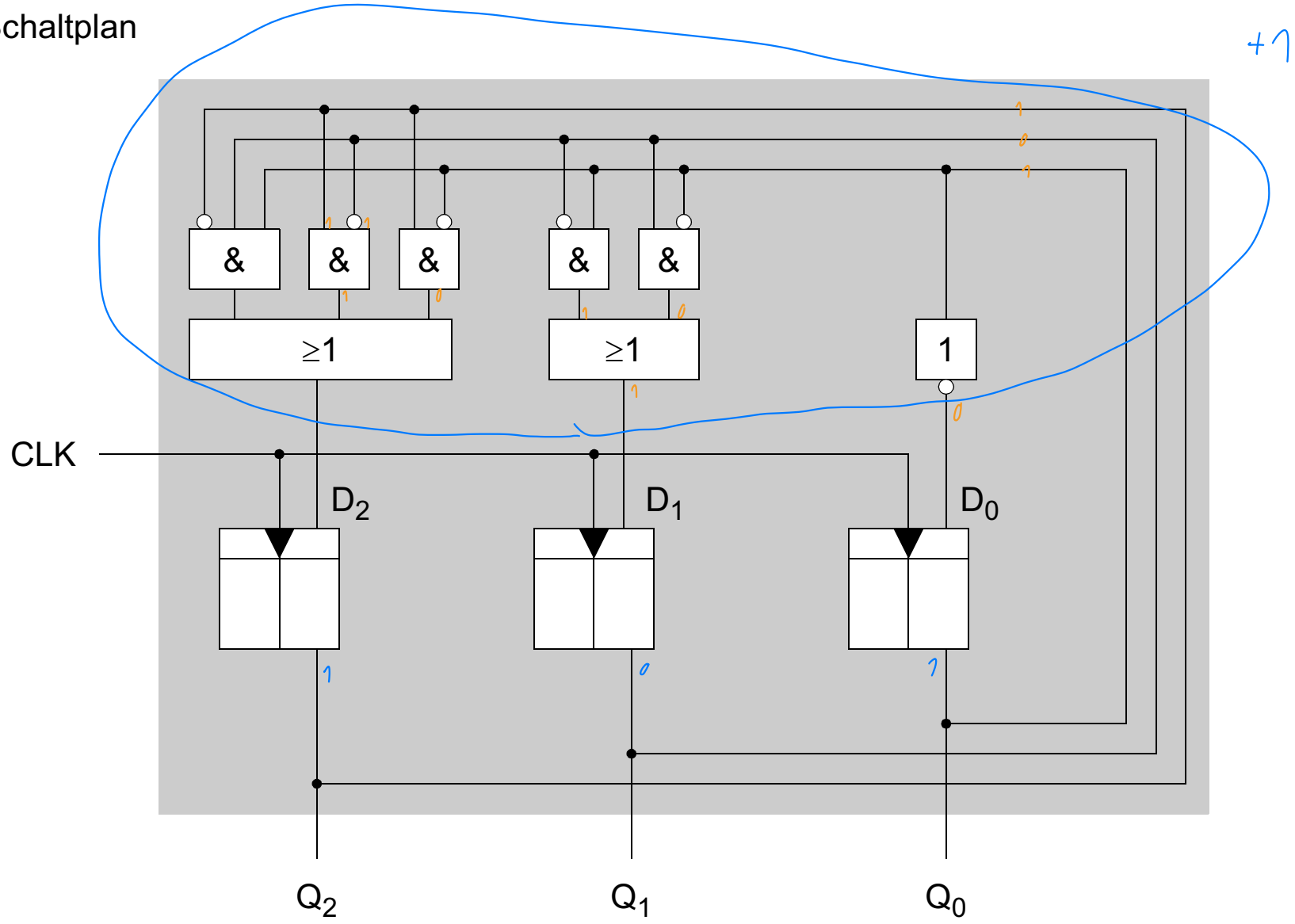
Mit charakteristischer Schaltfunktion des D-Flipflops $Q^{n+1} = D^n$

$$Q_2^{n+1} = (\overline{Q}_2 Q_1 Q_0 + Q_2 \overline{Q}_1 + Q_2 \overline{Q}_0)^n = D_2^n$$

$$Q_1^{n+1} = (\overline{Q}_1 Q_0 + Q_1 \overline{Q}_0)^n = D_1^n$$

$$Q_0^{n+1} = (\overline{Q}_0)^n = D_0^n$$

Schaltplan



2.3 Modellierung

Inhaltsübersicht

2.3.1 Definitionen

2.3.2 Mealy-Automat

2.3.3 Moore-Automat

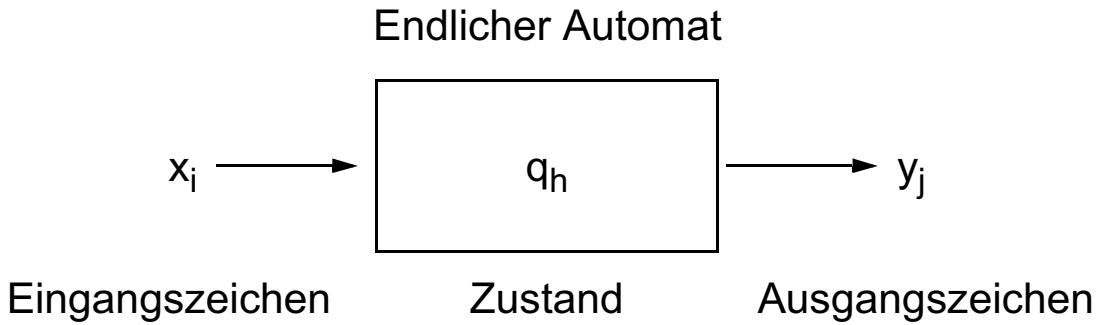
2.3.4 Äquivalenz von Automaten

2.3.5 Beispiel 1

2.3.6 Beispiel 2 ("Morsedetektor")

2.3.1 Definitionen

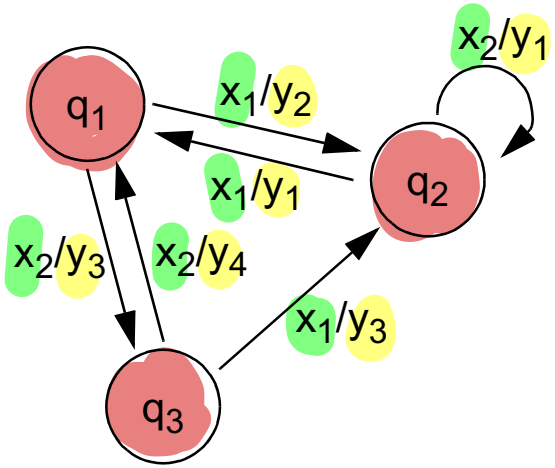
Endlicher Automat, Finite State Machine (FSM): Mathematisches Modell für eine Vorrichtung mit einer endlichen Anzahl interner Zustände, die eine Folge von Eingangszeichen einliest, verarbeitet und dabei eine Folge von Ausgangszeichen erzeugt.



Eingangsmenge	$\underline{M}_x = \{x_1, x_2, ..., x_{xmax}\}$	Eingangszeichen	$x_i \in \underline{M}_x$
Zustandsmenge	$\underline{M}_q = \{q_1, q_2, ..., q_{qmax}\}$	Zustand	$q_h \in \underline{M}_q$
Ausgangsmenge	$\underline{M}_y = \{y_1, y_2, ..., y_{ymax}\}$	Ausgangszeichen	$y_j \in \underline{M}_y$
Übergangsfunktion	f: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_q$		
Ausgangsfunktion	g: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_y$	(Mealy-Automat)	
	g: $\underline{M}_q \rightarrow \underline{M}_y$	(Moore-Automat)	

2.3.2 Mealy-Automat

Zustandsübergangsdiagramm (Mealy-Graph)



Zustandsübergangstabelle

Ist-zustand	Folge-zustand		Ausgangs-zeichen	
	x_1	x_2	x_1	x_2
q_1	q_2	q_3	y_2	y_3
q_2	q_1	q_2	y_1	y_1
q_3	q_2	q_1	y_3	y_4

$\underline{M}_x = \{x_1, x_2\}$, $\underline{M}_q = \{q_1, q_2, q_3\}$, $\underline{M}_y = \{y_1, y_2, y_3, y_4\}$

f: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_q$

g: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_y$

Eingangsfolge	?	x_1	x_1	x_2	x_1	x_2	x_1	x_1
Zustandsfolge	q_1	q_2	q_1	q_3	q_2	q_2	q_1	q_2
Ausgangsfolge	?	y_2	y_1	y_3	y_3	y_1	y_1	y_2

Dsp: Erkenne Sequenz "bab"

$M_x = \{a, b\}$

$M_y = \{ja, nein\}$

J N

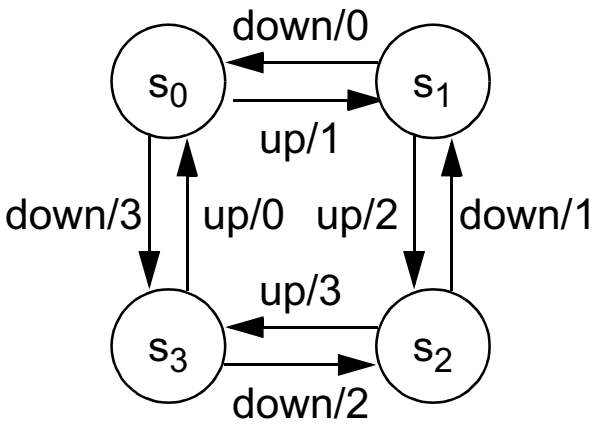
a b b a a b a b a a b a b b a

n n n n n n J n n n J n J n n

Zustandsübergang

Beispiel: Auf-/Abwärtszähler

Mealy-Graph



Zustandsübergangstabelle

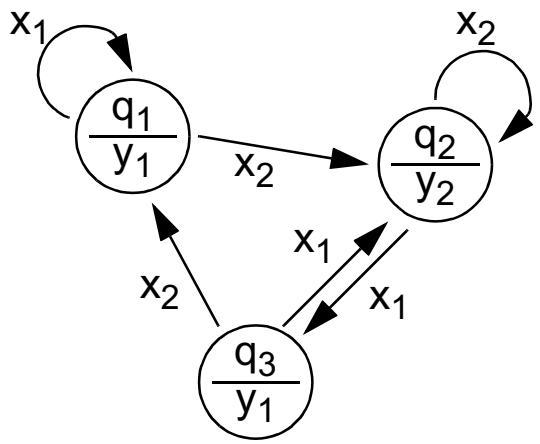
Ist-zustand	Folge-zustand		Ausgangs-zeichen	
	up	down	up	down
s_0	s_1	s_3	1	3
s_1	s_2	s_0	2	0
s_2	s_3	s_1	3	1
s_3	s_0	s_2	0	2

$\underline{M}_x = \{up, down\}$, $\underline{M}_q = \{s_0, s_1, s_2, s_3\}$, $\underline{M}_y = \{0, 1, 2, 3\}$

Eingangsfolge	?	up	up	down	up	up	up	down
Zustandsfolge	s_0	s_1	s_2	s_1	s_2	s_3	s_0	s_3
Ausgangsfolge	?	1	2	1	2	3	0	3

2.3.3 Moore-Automat

Zustandsübergangsdiagramm (Moore-Graph)



Zustandsübergangstabelle

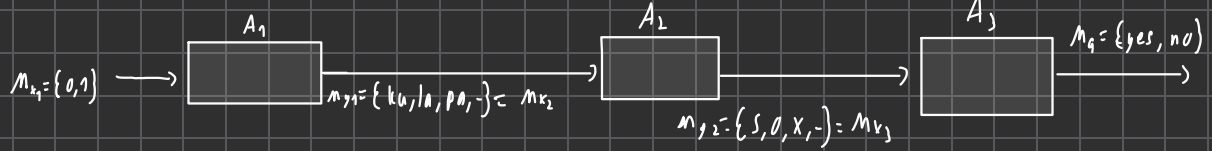
Ist-zustand	Folge-zustand		Ausgangs-zeichen
	x_1	x_2	
q_1	q_1	q_2	y_1
q_2	q_3	q_2	y_2
q_3	q_2	q_1	y_1

$\underline{M}_x = \{x_1, x_2\}$, $\underline{M}_q = \{q_1, q_2, q_3\}$, $\underline{M}_y = \{y_1, y_2\}$

f: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_q$

g: $\underline{M}_q \rightarrow \underline{M}_y$

Eingangsfolge	?	x_1	x_2	x_2	x_1	x_1	x_1	x_2
Zustandsfolge	q_1	q_1	q_2	q_2	q_3	q_2	q_3	q_1
Ausgangsfolge	y_1	y_1	y_2	y_2	y_1	y_2	y_1	y_1



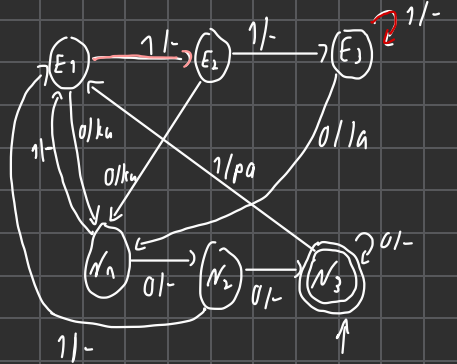


kurz "1" ≤ Takte

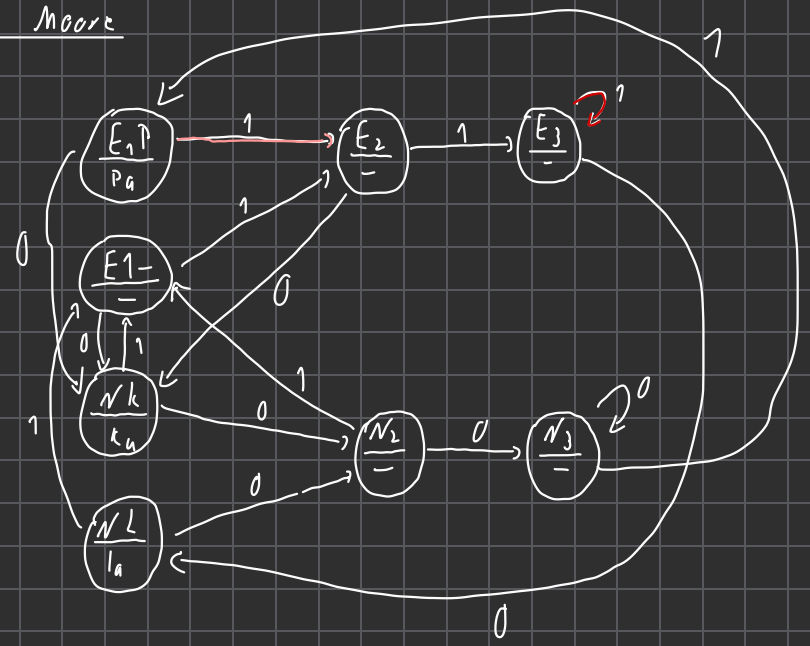
lang "0" > Takte

Pause "1" > Takte

A_1 als Mealy



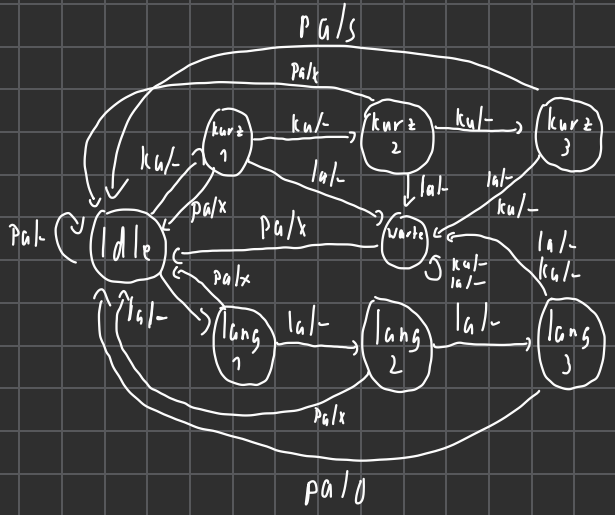
A_2 als Moore



Einfach schauen wo das obige überall hingeht und aufschreiben.

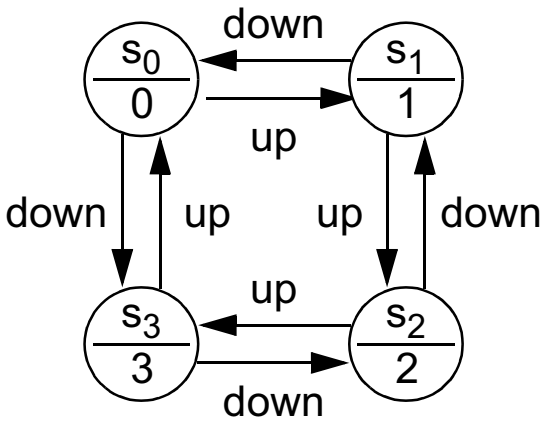
Seite 50 ist bei Schönky

A_2 als Mealy



Beispiel: Auf-/Abwärtszähler

Moore-Graph



Zustandsübergangstabelle

Ist-zustand	Folge-zustand		Ausgangs-zeichen
	up	down	
s_0	s_1	s_3	0
s_1	s_2	s_0	1
s_2	s_3	s_1	2
s_3	s_0	s_2	3

$\underline{M}_x = \{up, down\}$, $\underline{M}_q = \{s_0, s_1, s_2, s_3\}$, $\underline{M}_y = \{0, 1, 2, 3\}$

Eingangsfolge	?	up	up	down	up	up	up	down
Zustandsfolge	s_0	s_1	s_2	s_1	s_2	s_3	s_0	s_3
Ausgangsfolge	0	1	2	1	2	3	0	3

2.3.4 Äquivalenz von Automaten

I. Definitionen

Zwei Automaten A_1 und A_2 sind äquivalent, wenn sie aufgrund ihres äußeren Verhaltens nicht unterschieden werden können, d.h. wenn sie auf beliebige und beliebig lange Folgen von Eingangszeichen identische Folgen von Ausgangszeichen erzeugen.

Äquivalenz von Zuständen

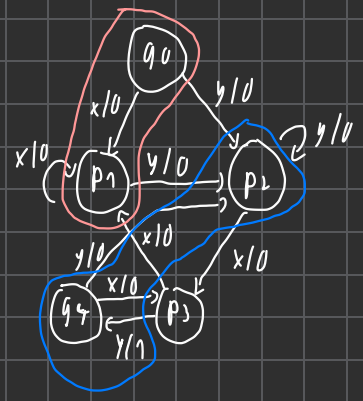
Zwei Zustände q_i und q_j sind **k-äquivalent**, wenn ein Automat sowohl mit dem Ausgangszustand q_i als auch mit dem Ausgangszustand q_j für eine beliebige **Eingangsfolge der Länge k** identische Ausgangsfolgen erzeugt.

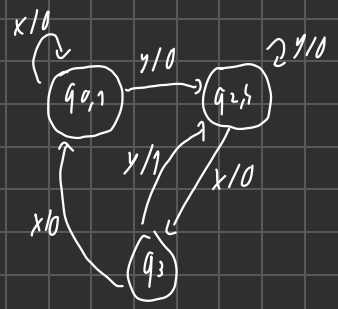
Zwei Zustände q_i und q_j sind **äquivalent**, wenn ein Automat sowohl mit dem Ausgangszustand q_i als auch mit dem Ausgangszustand q_j für eine beliebige **Eingangsfolge beliebiger Länge** identische Ausgangsfolgen erzeugt.

Folgerungen

1-äquivalente Zustände, die für jedes Eingangszeichen jeweils in denselben Folgezustand übergehen, sind äquivalent. (Der Umkehrschluss gilt nicht.)

Äquivalente Zustände können zusammengefasst werden, da sie aufgrund des äußeren Verhaltens des Automaten nicht unterscheidbar sind.





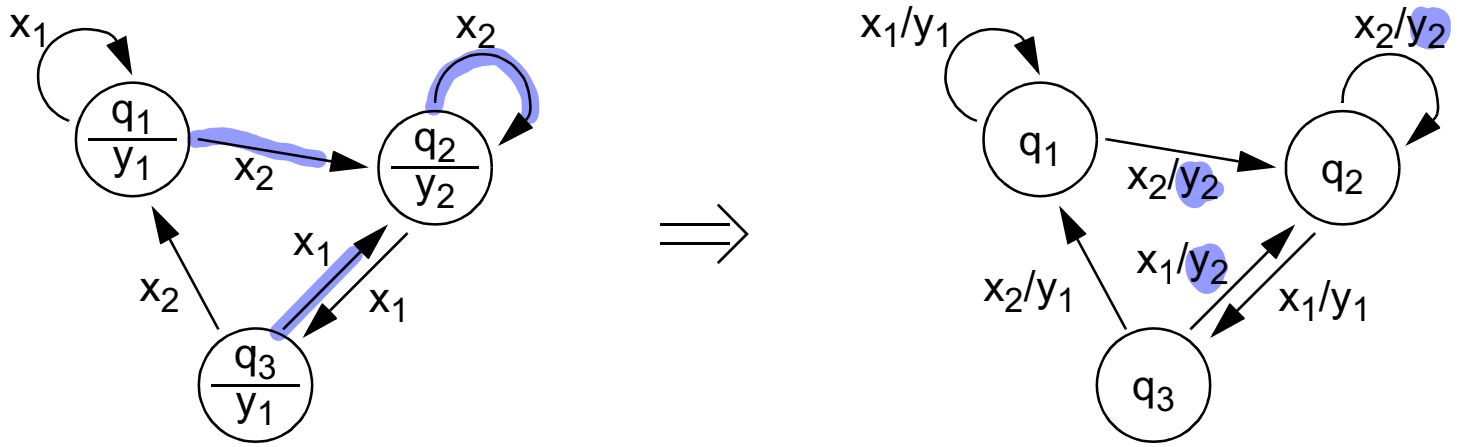
$M_x = \{x, y\}$

$M_y = \{0, 1\}$

$M_q = \{q_0, q_1, q_2, q_3, q_4\}$

II. Umwandlung Moore nach Mealy

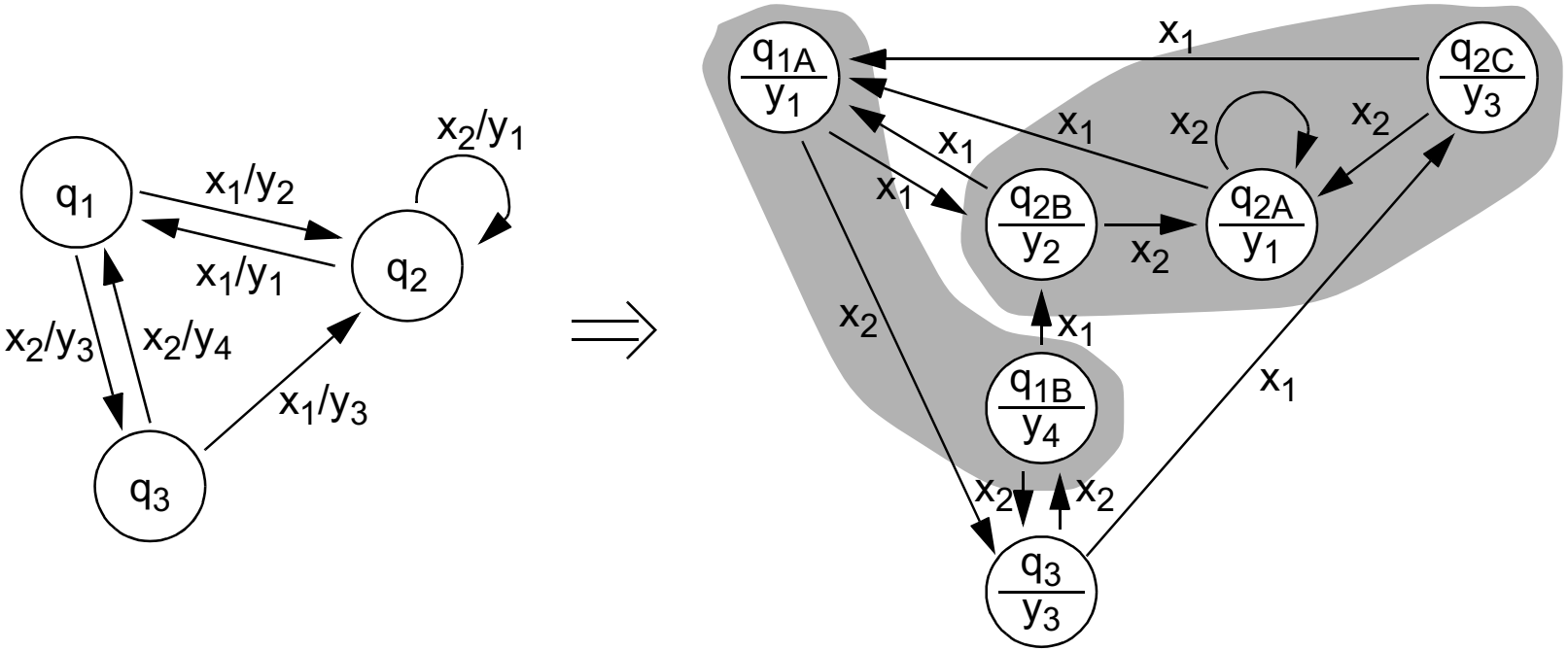
Immer direkt möglich, da der Moore-Automat ein Spezialfall des Mealy-Automaten ist.



Eingangsfolge	?	x_1	x_2	x_2	x_1	x_1	x_1	x_2
Zustandsfolge	q_1	q_1	q_2	q_2	q_3	q_2	q_3	q_1
Ausgangsfolge	y_1, ?	y_1	y_2	y_2	y_1	y_2	y_1	y_1

III. Umwandlung Mealy nach Moore

Wenn verschiedene Übergänge in einen Zustand verschiedene Ausgangszeichen erzeugen, dann muss dieser Zustand aufgespaltet werden.



Eingangsfolge	?	x_1	x_1	x_2	x_1	x_2	x_1	x_1
Zustandsfolge	q_1, q_{1A}	q_2, q_{2B}	q_1, q_{1A}	q_3	q_2, q_{2C}	q_2, q_{2A}	q_1, q_{1A}	q_2, q_{2B}
Ausgangsfolge	?, y_1	y_2	y_1	y_3	y_3	y_1	y_1	y_2

2.3.5 Beispiel 1

Automat, der in einer Folge aus den Zeichen "a" und "b" die Sequenz "bab" erkennt., z.B.

Eingangssequenz	a	b	a	b	b	a	b	a	b	a	a	b
Ausgangssequenz	no	no	no	yes	no	no	yes	no	yes	no	no	no

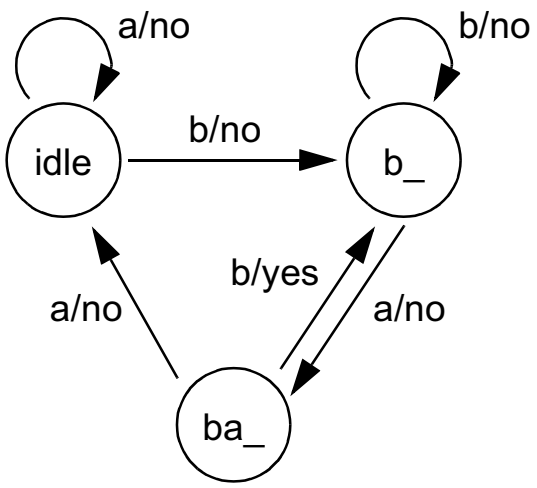
Eingangsmenge $\underline{M}_x = \{a, b\}$

Ausgangsmenge $\underline{M}_y = \{no, yes\}$

Zustandsmenge $\underline{M}_q = ?$

Realisierung als Mealy-Automat

Zustandsübergangsdiagramm
(Mealy-Graph)



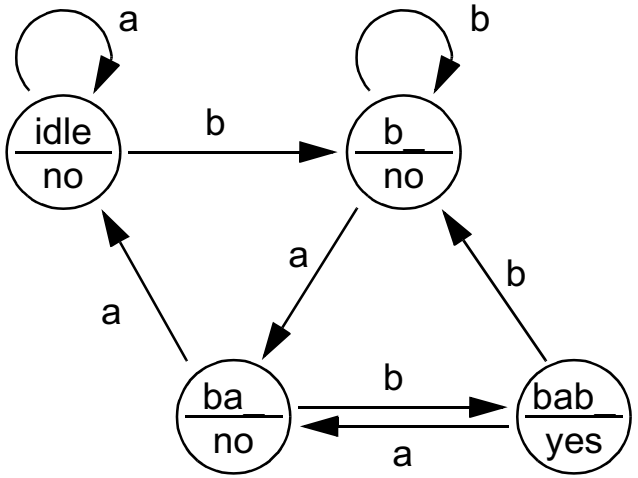
Zustandsübergangstabelle

Ist-zustand	Folge-zustand		Ausgangs-zeichen	
	a	b	a	b
idle	idle	b_	no	no
b_	ba_	b_	no	no
ba_	idle	b_	no	yes

$\underline{M}_x = \{a, b\}$, $\underline{M}_y = \{no, yes\}$, $\underline{M}_q = \{idle, b_, ba_\}$

Realisierung als Moore-Automat

Zustandsübergangsdiagramm
(Moore-Graph)



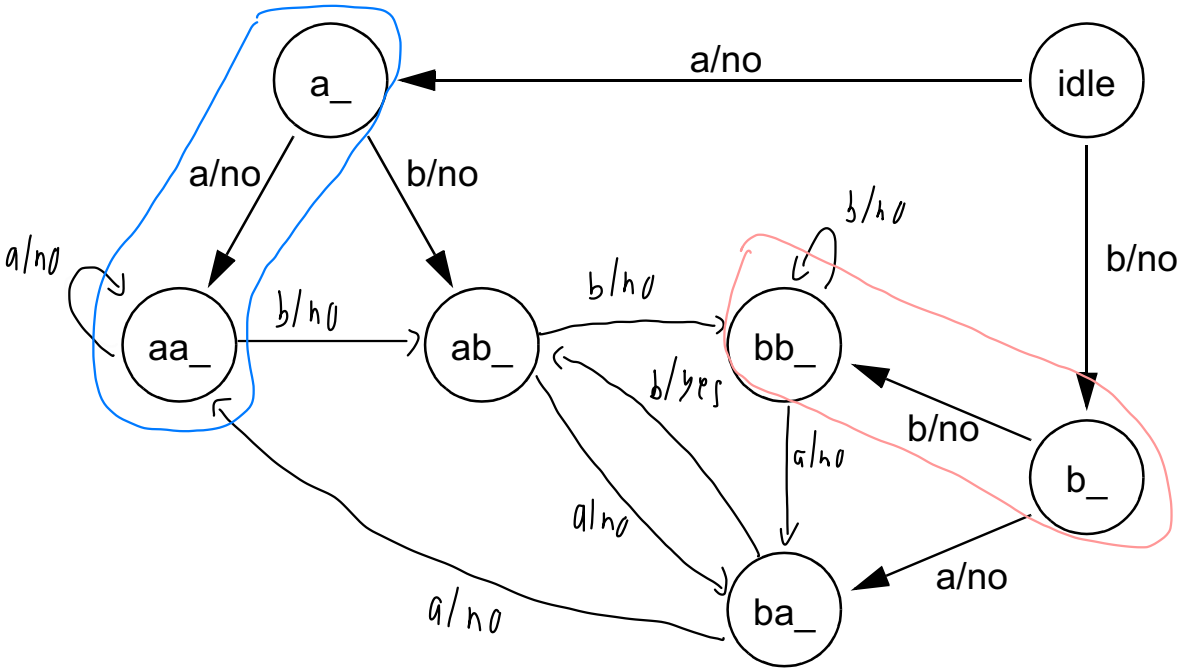
Zustandsübergangstabelle

Ist-zustand	Folge-zustand		Ausgangs-zeichen
	a	b	
idle	idle	b_	no
b_	ba_	b_	no
ba_	idle	bab_	no
bab_	ba_	b_	yes

$\underline{M}_x = \{a, b\}$, $\underline{M}_y = \{no, yes\}$, $\underline{M}_q = \{idle, b_, ba_, bab_\}$

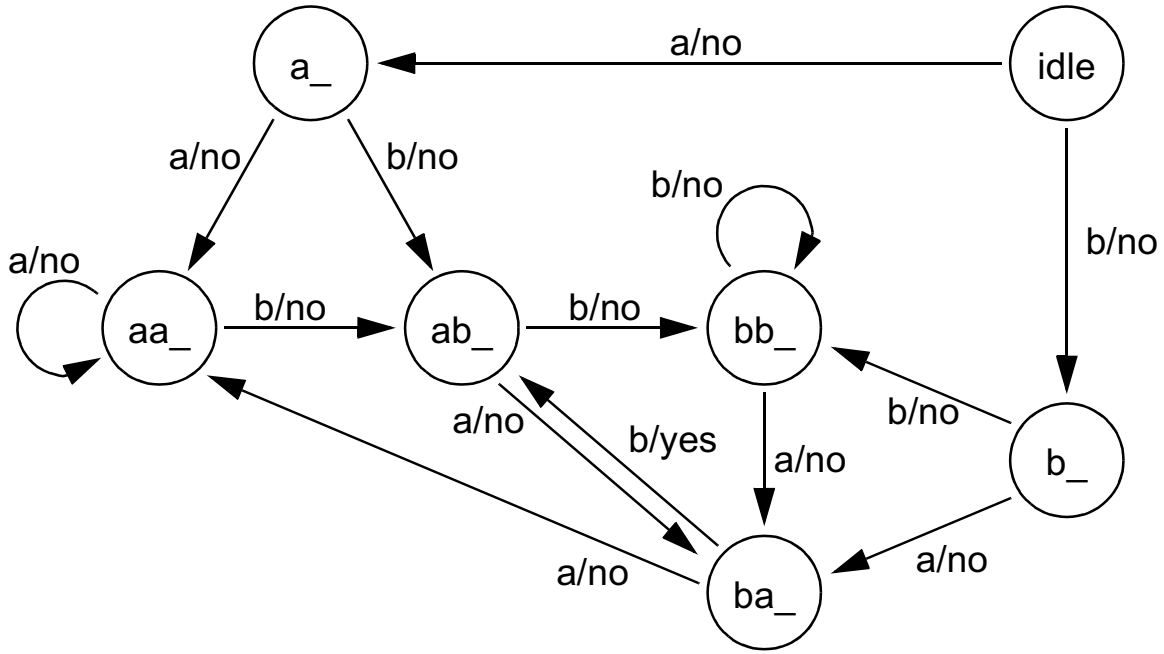
Realisierung als Mealy-Automat: Alternativer Ansatz

Ansatz: Baum



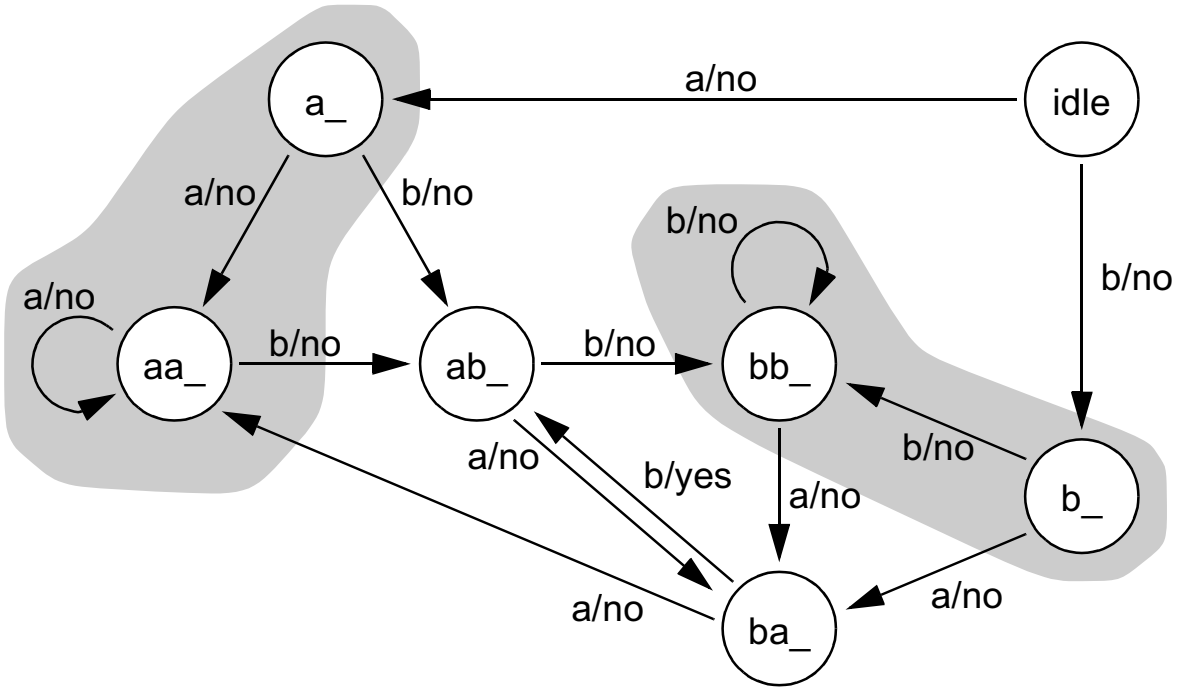
Realisierung als Mealy-Automat: Alternativer Ansatz

Vervollständigung des Baums



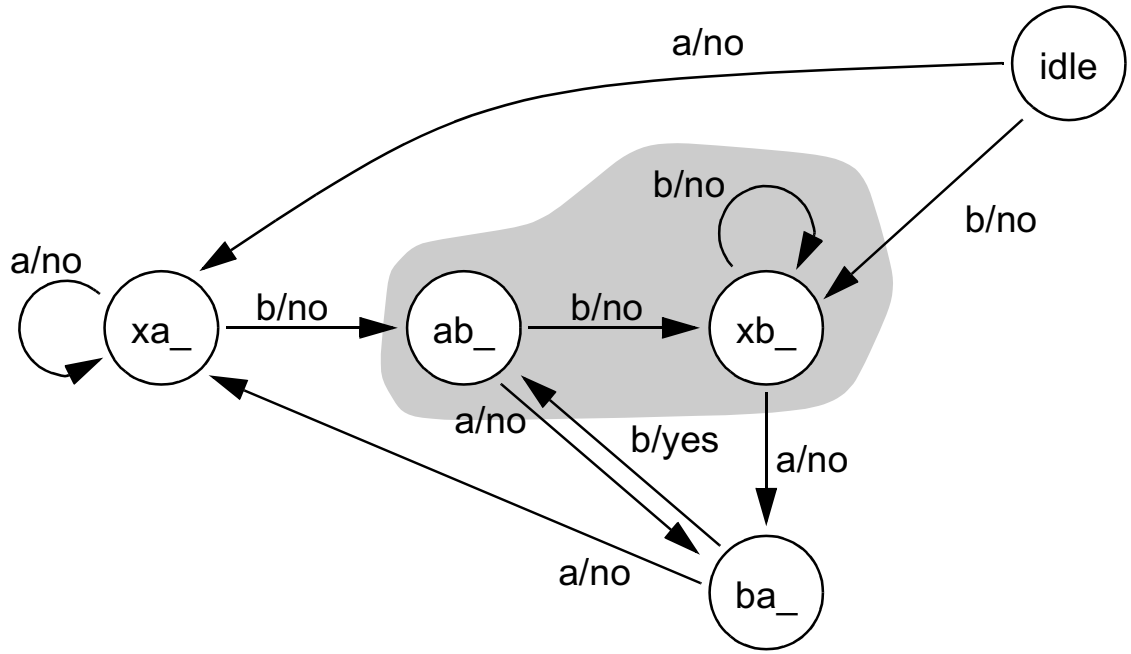
Realisierung als Mealy-Automat: Alternativer Ansatz

Minimierung des Graphen (1)



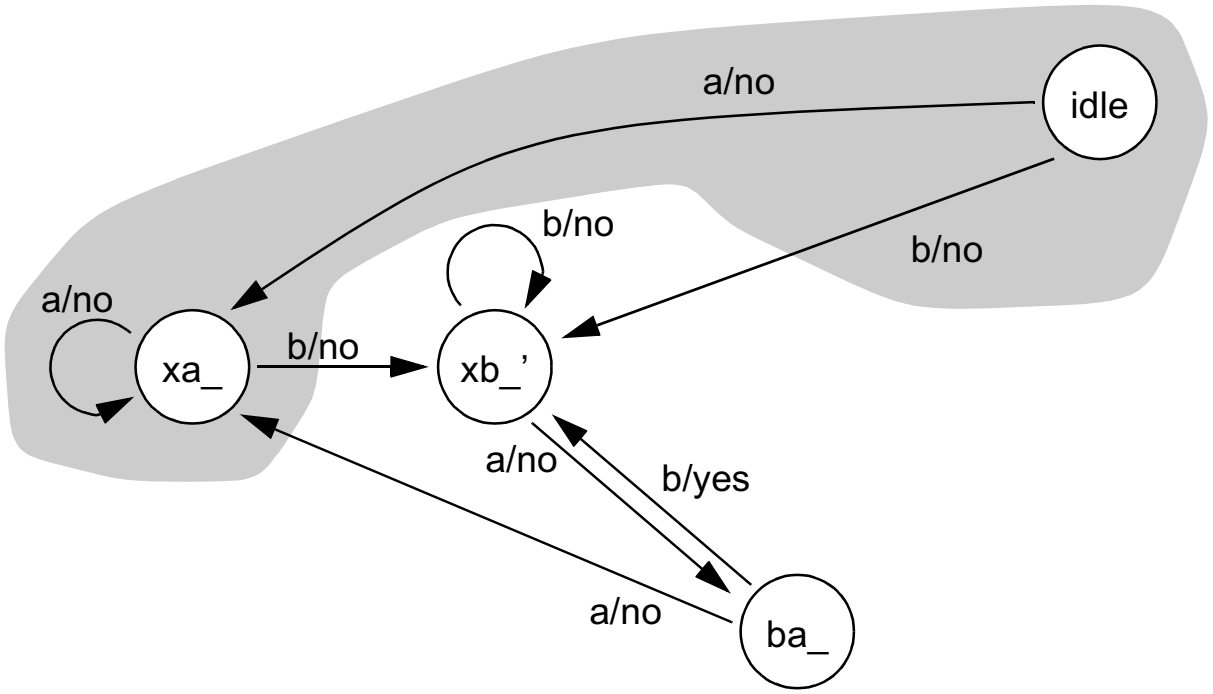
Realisierung als Mealy-Automat: Alternativer Ansatz

Minimierung des Graphen (2)



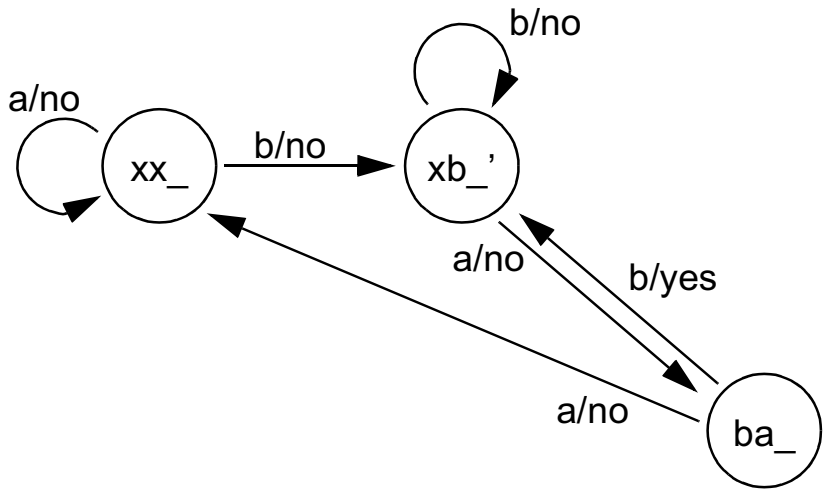
Realisierung als Mealy-Automat: Alternativer Ansatz

Minimierung des Graphen (3)



Realisierung als Mealy-Automat: Alternativer Ansatz

Minimimaler Graph (vergleiche erste Lösung)



2.3.6 Beispiel 2 ("Morsedetektor")

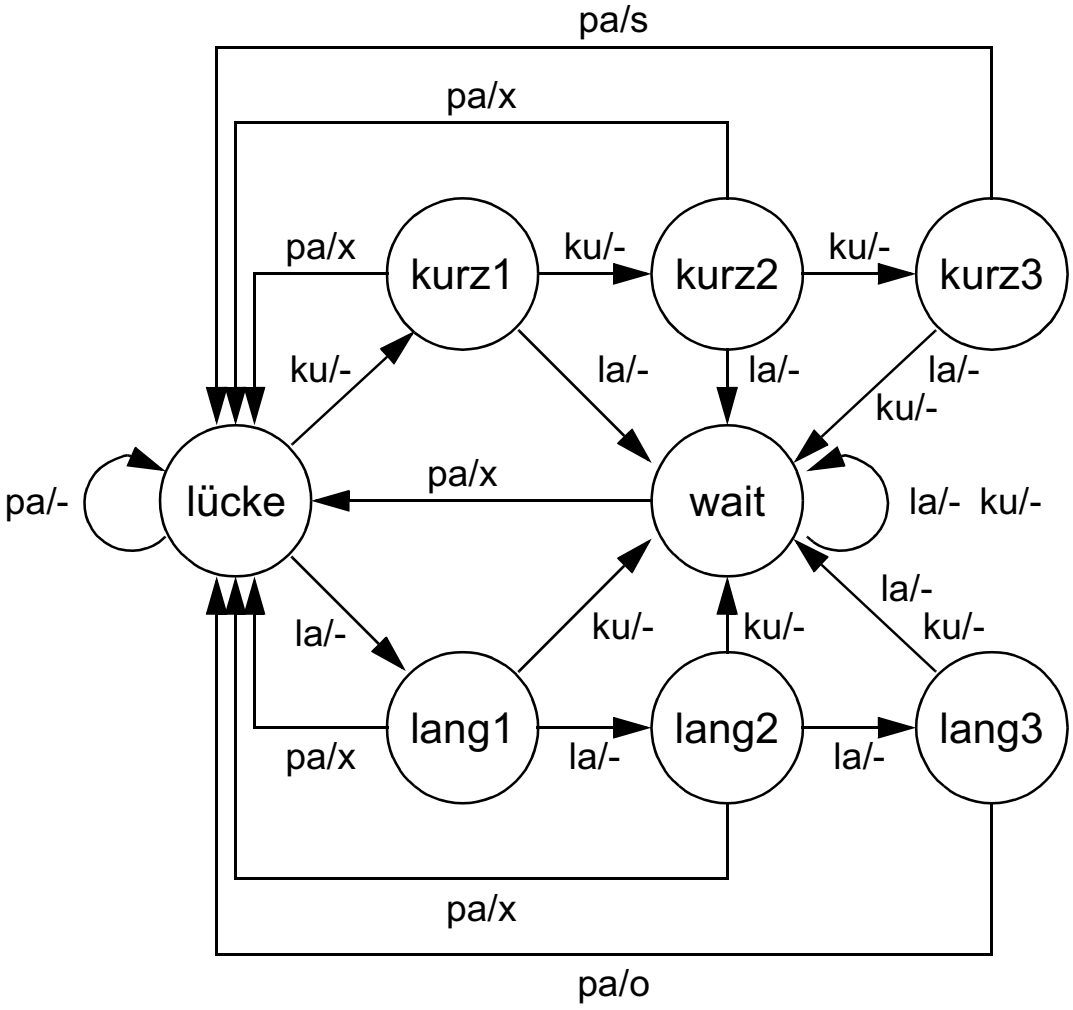
Aufgabe: Erkenne die Zeichen ’s’ und ’o’ in einer Folge von Morsezeichen.

Eingangsmenge $\underline{M}_x$ = {kurz, lang, pause, "nichts"} = {ku, la, pa, -}

Ausgangsmenge $\underline{M}_y$ = {’s’, ’o’, "anderes Zeichen", "nichts"} = {s, o, x, - }

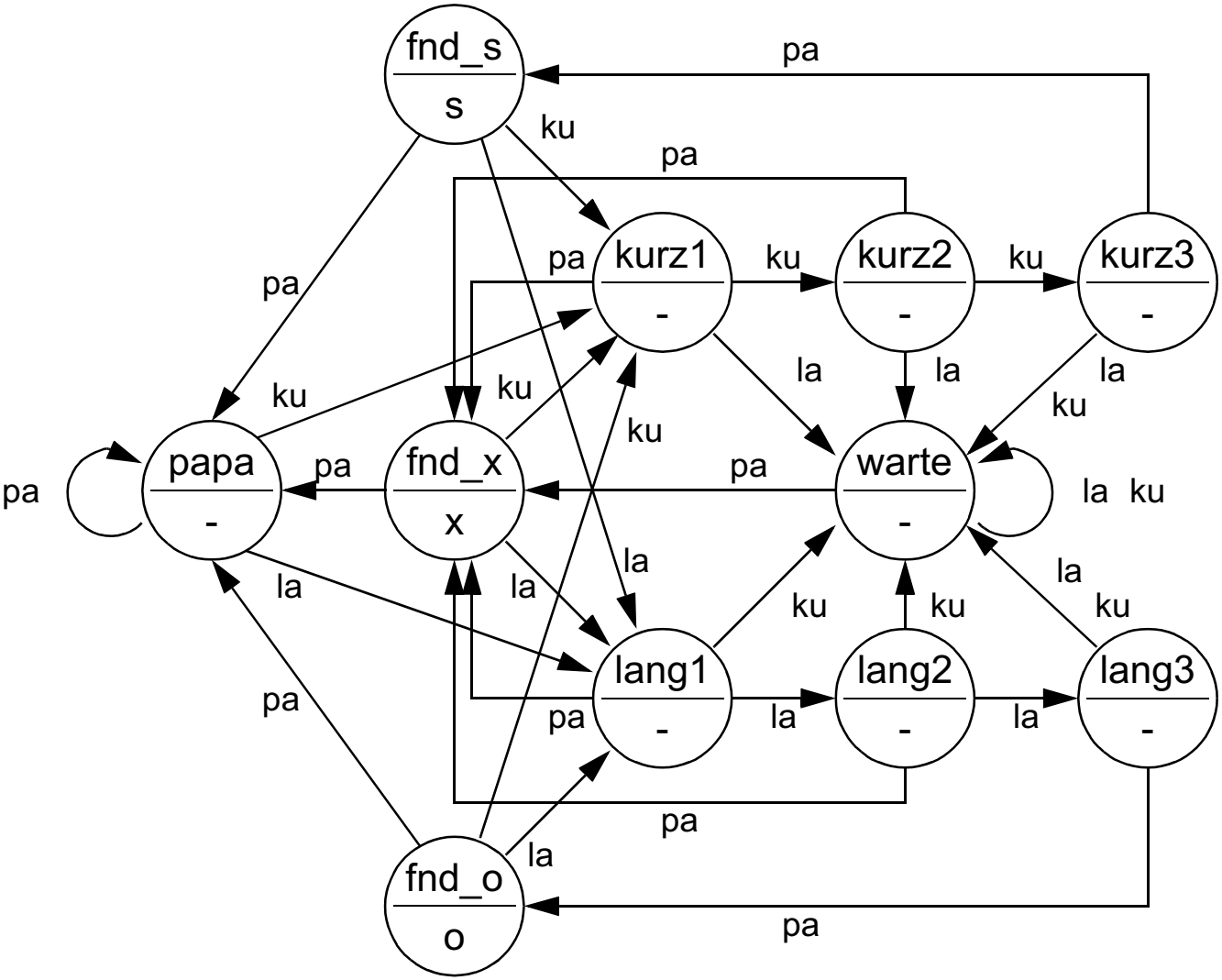


Mealy-Graph



Beim Eingangszeichen "-" bleibt der Automat im jeweiligen Zustand und erzeugt das Ausgangszeichen "-"

Moore-Graph



Beim Eingangszeichen "-" bleibt der Automat im jeweiligen Zustand. Ausnahme: In den Zuständen *fnd_s*, *fnd_x* und *fnd_o* geht der Automat beim Eingangszeichen "-" in den Zustand *papa* über.

2.4 Entwurfsverfahren

Inhaltsübersicht

2.4.1 Definitionen

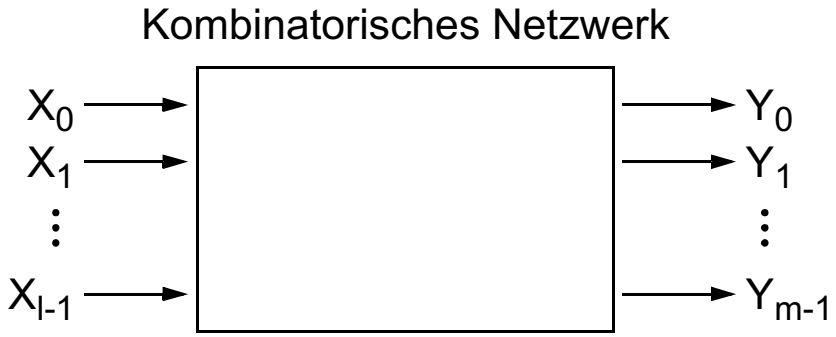
2.4.2 Sequenzielle Netzwerke als schaltungstechnische Realisierung endlicher Automaten

2.4.3 Das Entwurfsverfahren anhand eines Beispiels

2.4.1 Definitionen

I. Kombinatorisches Netzwerk *(siehe Kapitel 1)*

Rückkopplungsfreie digitale Schaltung ohne Speicherelemente zur Realisierung schaltalgebraischer Funktionen.



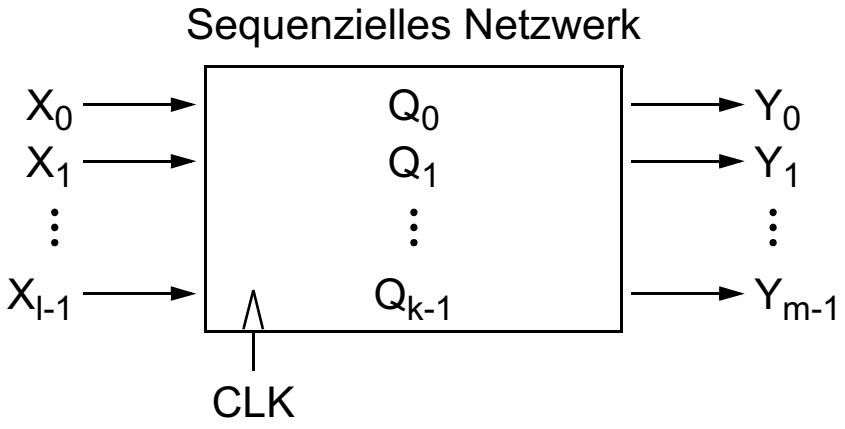
Beispiele für kombinatorische Netzwerke

- Datentore, Multiplexer
- kombinatorische Schiebe- und Rotierwerke
- arithmetische Elemente (Addierer, Subtrahierer, ALU, einschrittige Multiplizierwerke, ...)
- Decoder und Encoder (Codeumsetzer)

II. Sequenzielles Netzwerk

Digitale Schaltung mit Speicherelementen.

Die Ausgänge eines sequenziellen Netzwerks zu einem gegebenen Zeitpunkt hängen nicht nur vom Zustand der Eingänge zu diesem Zeitpunkt ab, sondern auch vom Zustand der Eingänge zu möglicherweise beliebig vielen vergangenen Zeitpunkten.



Eingänge	Speicherelemente	Ausgänge
$X_0, X_1, ..., X_{l-1}$	$Q_0, Q_1, ..., Q_{k-1}$	$Y_0, Y_1, ..., Y_{m-1}$

Beispiele für sequenzielle Netzwerke

- Register, Schieberegister, Zähler
- Steuerwerke

2.4.2 Sequenzielle Netzwerke als schaltungstechnische Realisierung endlicher Automaten

Binäre Codierung von Eingangszeichen und Ausgangszeichen als Boolesche Vektoren

Codierung der Eingangszeichen $x_i \in \underline{M}_x$: Eingangsvektor $\underline{X} = (X_0, X_1, ..., X_{l-1})$

Codierung der Ausgangszeichen $y_j \in \underline{M}_y$: Ausgangsvektor $\underline{Y} = (Y_0, Y_1, ..., Y_{m-1})$

Binäre Codierung der Zustände als Boolesche Vektoren

Codierung der Zustände $q_h \in \underline{M}_q$: Zustandsvektor $\underline{Q} = (Q_0, Q_1, ..., Q_{k-1})$

Schaltungstechnische Realisierung der Übergangsfunktion

f: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_q$ als $\underline{Q}^{n+1} = \underline{f}(\underline{X}^n, \underline{Q}^n)$

Substitution $\underline{Q}^{n+1} = \underline{D}^n$ $\rightarrow$ $\underline{D}^n = \underline{f}(\underline{X}^n, \underline{Q}^n)$

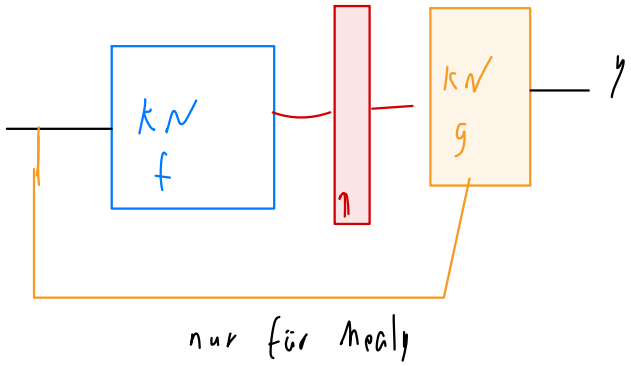
$\underline{Q}^{n+1} = \underline{D}^n$ reines Speicherwerk (Satz aus k D-Flipflops)

$\underline{D}^n = \underline{f}(\underline{X}^n, \underline{Q}^n)$ rein kombinatorisches Netzwerk

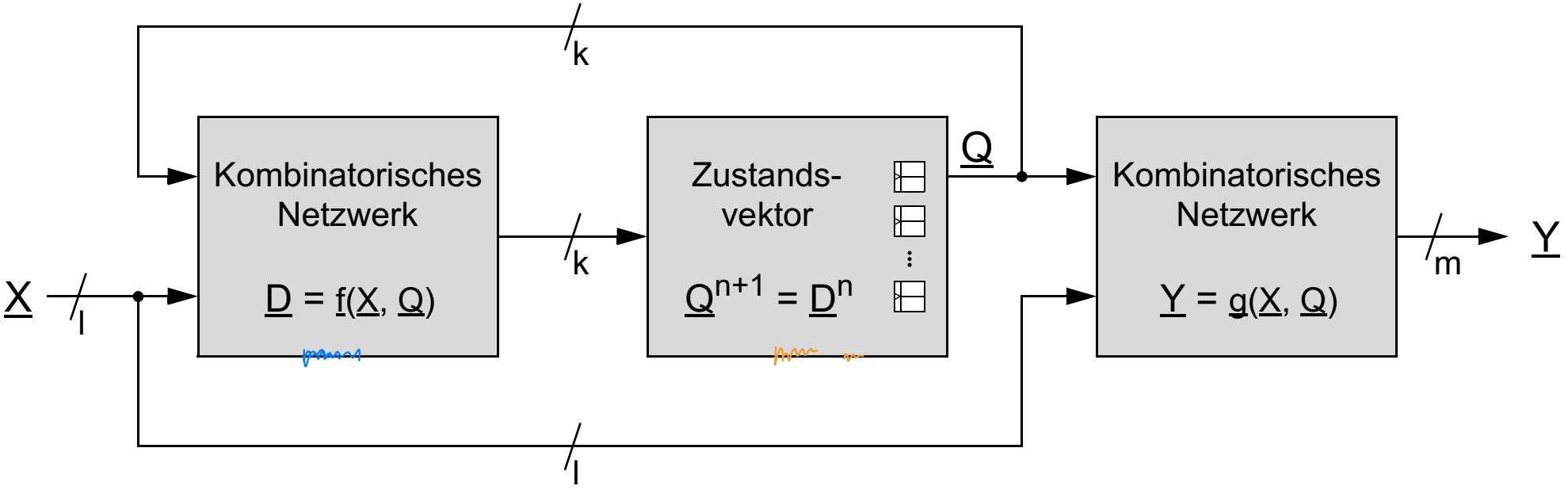
Schaltungstechnische Realisierung der Ausgangsfunktion

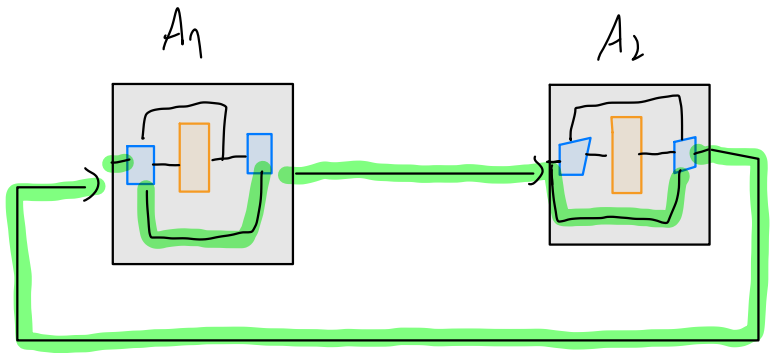
g: $\underline{M}_x \times \underline{M}_q \rightarrow \underline{M}_y$ als $\underline{Y}^n = \underline{g}(\underline{X}^n, \underline{Q}^n)$ (Mealy) ... rein kombinatorisches Netzwerk

g: $\underline{M}_q \rightarrow \underline{M}_y$ als $\underline{Y}^n = \underline{g}(\underline{Q}^n)$ (Moore) ... rein kombinatorisches Netzwerk

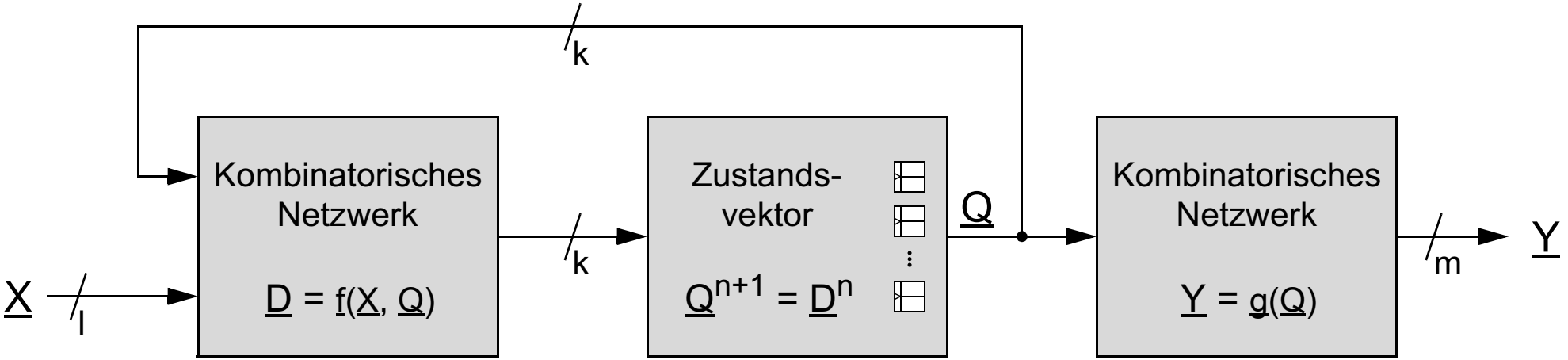


I. Mealy-Automat



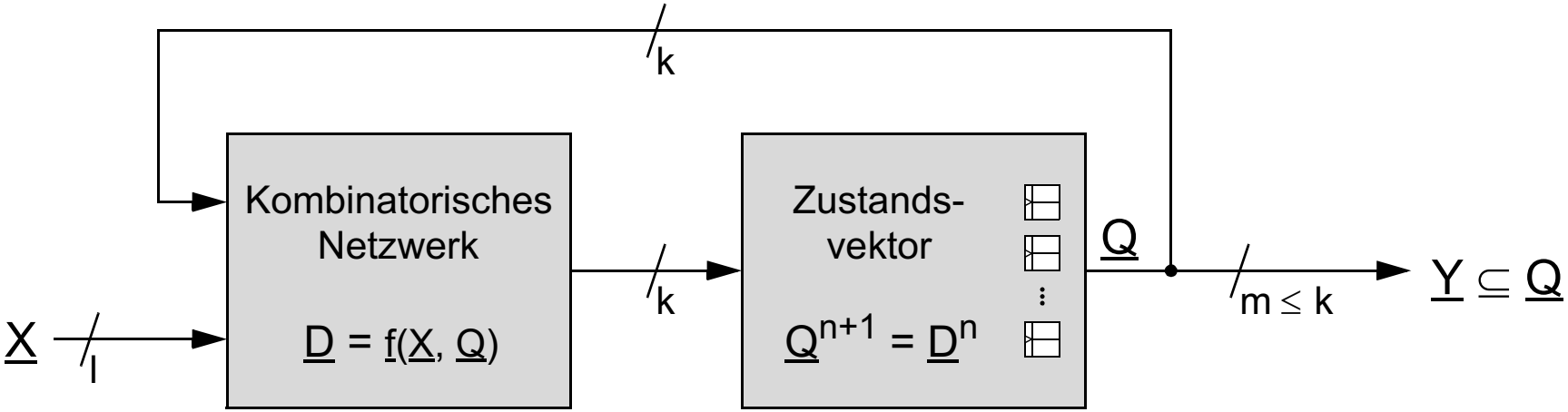


II. Moore-Automat



Unterschied zum Mealy-Automat: Ausgangsvektor hängt nur vom Zustandsvektor ab

III. Medwedew-Automat

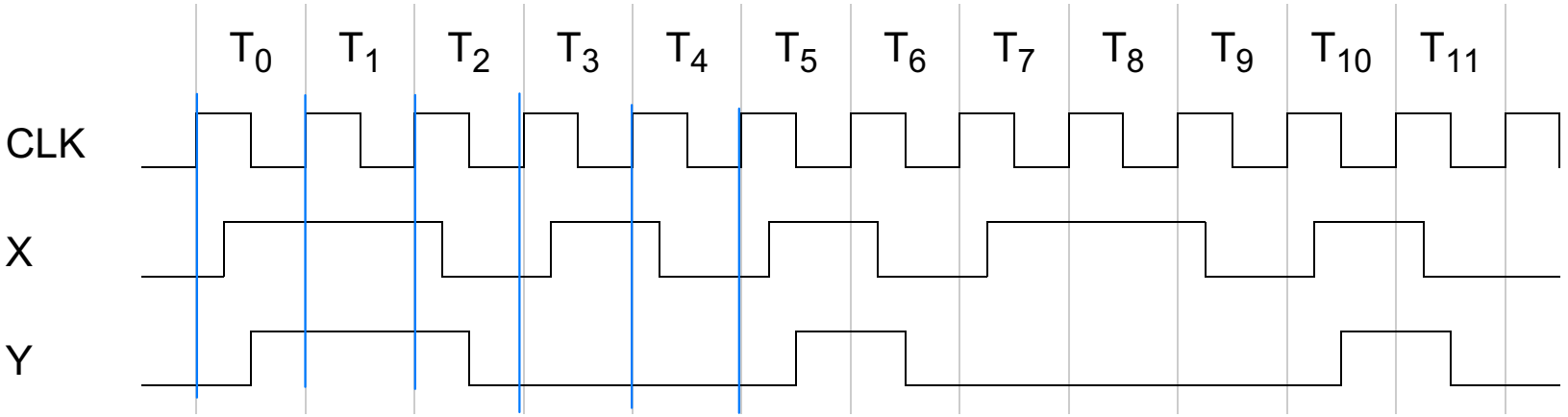


Unterschiede zum Moore-Automat

- Ausgangsvektor ist Teilvektor des Zustandsvektors
- deshalb entsprechende Codierung der Zustände erforderlich

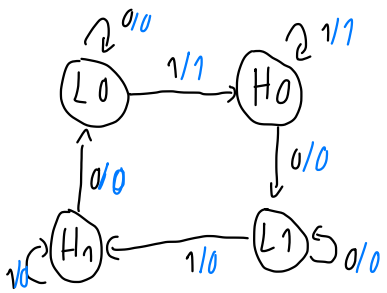
2.4.3 Das Entwurfsverfahren anhand eines Beispiels

Es ist ein Mealy-Automat in Form eines synchronen sequenziellen Netzwerks zu realisieren, bei dem (nur) jeder zweite Impuls am Eingang X am Ausgang Y erscheint. Das Eingangssignal X sei synchron, d. h. es ändert sich ausschließlich infolge der positiven Flanke des Taktes CLK.



More Automat nicht weil es sich verzögert Mealy ist richtig





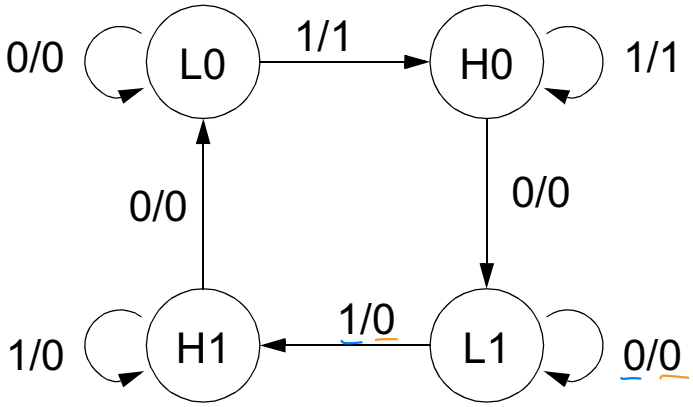
Übersicht

1. Entwicklung des Zustandsübergangsdiagramms
2. Zustandsreduzierung (optional)
3. Codierung der Eingangszeichen, der Ausgangszeichen und der Zustände
4. Aufstellen einer erweiterten Zustandsübergangstabelle
5. Wahl der Speicherelemente: Spezialfall D-Flipflops
6. Aufstellen der Problemfunktionen (Spezialfall D-Flipflops)
7. Bestimmung der Eingangsfunktionen (Spezialfall D-Flipflops)
8. Bestimmung der Ausgangsfunktionen (unabhängig vom Flipflop-Typ)
9. Schaltplan (mit D-Flipflops)

5. Wahl der Speicherelemente (allgemeiner Fall)
6. Aufstellen der Problemfunktionen (allgemeiner Fall)
7. Bestimmung der Eingangsfunktionen für T-, SR- und JK-Flipflops
9. Schaltplan (mit JK-Flipflops)

1. Entwicklung des Zustandsübergangsdiagramms

Hier: Mealy-Graph



H_1 L_1 äquivalent

L_0 H_0 auch

Entsprechende Zustandsübergangstabelle

Ist-zustand q_i^n	Folgezustand q_i^{n+1}		Ausgang Y	
	X = 0	X = 1	X = 0	X = 1
L0	L0	H0	0	1
H0	L1	H0	0	1
L1	L1	H1	0	0
H1	L0	H1	0	0

2. Zustandsreduzierung (optional)

Gegebenenfalls Zustandsreduzierung durchführen (siehe Kapitel 2.3).

Hinweise

- selten möglich
- Ausnahme: Mustererkennungsautomaten

Im vorliegenden Beispiel nicht möglich.

3. Codierung der Eingangszeichen, der Ausgangszeichen und der Zustände

Codierung der Eingangszeichen

im vorliegenden Beispiel bereits 0 oder 1, Codierung nicht nötig.

Codierung der Ausgangszeichen

im vorliegenden Beispiel bereits 0 oder 1, Codierung nicht nötig.

Codierung der Zustände

N = 4 Zustände, k = ld N = ld 4 = 2 Flipflops Q_1, Q_0 erforderlich

Beispielcodierung

q_i	Codierung	
	Q_1	Q_0
L0	0	0
H0	0	1
L1	1	0
H1	1	1

4. Aufstellen einer erweiterten Zustandsübergangstabelle

Mealy Tabelle

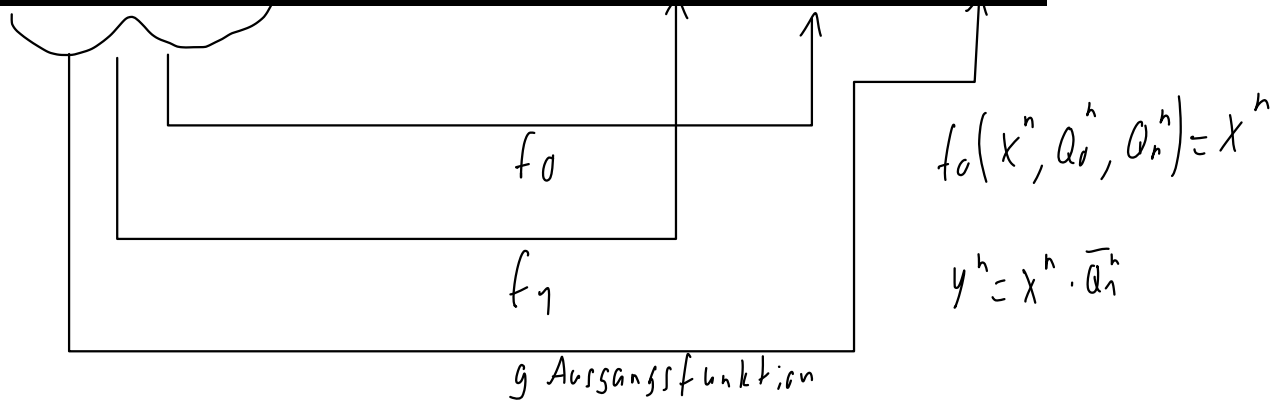
Erweiterte Zustandsübergangstabelle = Zustandsübergangstabelle + Codierung

Zeile	Istzustand			Eing.	Folgezustand			Ausg.	
	q^n	Q_1^n	Q_0^n	X^n	q^{n+1}	Q_1^{n+1}	Q_0^{n+1}	Y^n	
0	L0	0	0	0	L0	0	0	0	0
1				1	H0	0	1	1	0
2	H0	0	1	0	L1	1	0	0	
3				1	H0	0	1	1	
4	L1	1	0	0	L1	1	0	0	
5				1	H1	1	1	0	
6	H1	1	1	0	L0	0	0	0	
7				1	H1	1	1	0	

Bei mealy ist Ausg wie Eing also gleich.

---

moore darf nicht 0,1 sein sondern muss wie q_1^n sein



$f_0(x^n, Q_0^n, Q_1^n) = x^n$

$y^n = x^n \cdot \overline{q_1^n}$

5. Wahl der Speicherelemente: Spezialfall D-Flipflops

Charakteristische Schaltfunktion $Q^{n+1} = D^n$

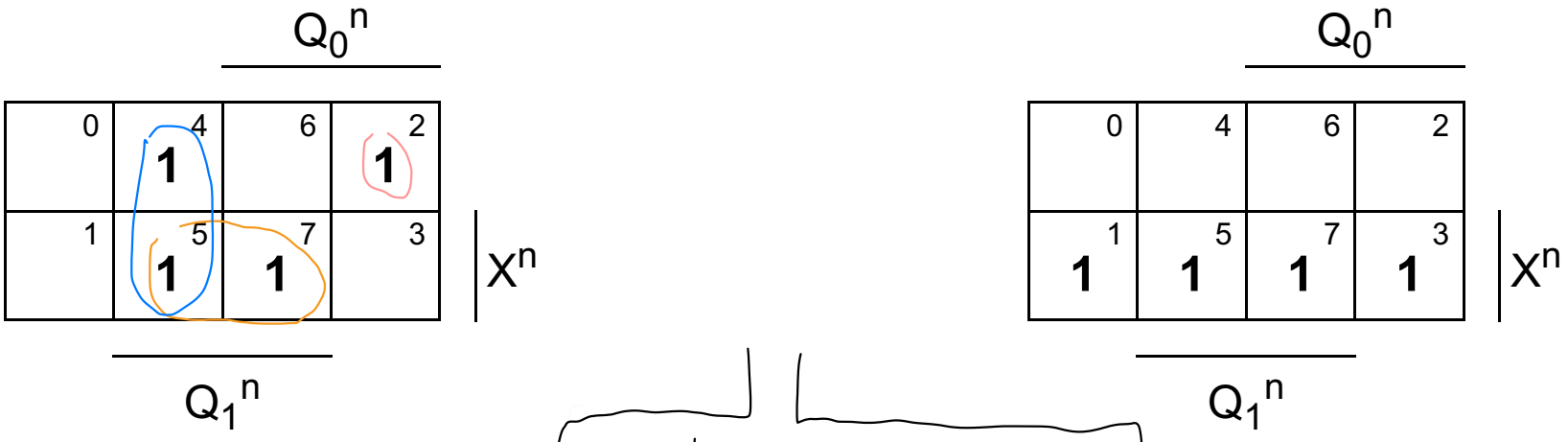
6. Aufstellen der Problemfunktionen (Spezialfall D-Flipflops)

Problemfunktion gibt an, wie sich ein Flipflop im gegebenen Kontext verhalten soll.

Allgemein $Q_i^{n+1} = f_i(X_0, X_1, ..., X_{l-1}, Q_0, Q_1, ..., Q_{k-1})^n$

Im vorliegenden Beispiel $Q_0^{n+1} = f_0(X, Q_0, Q_1)^n$

$Q_1^{n+1} = f_1(X, Q_0, Q_1)^n$



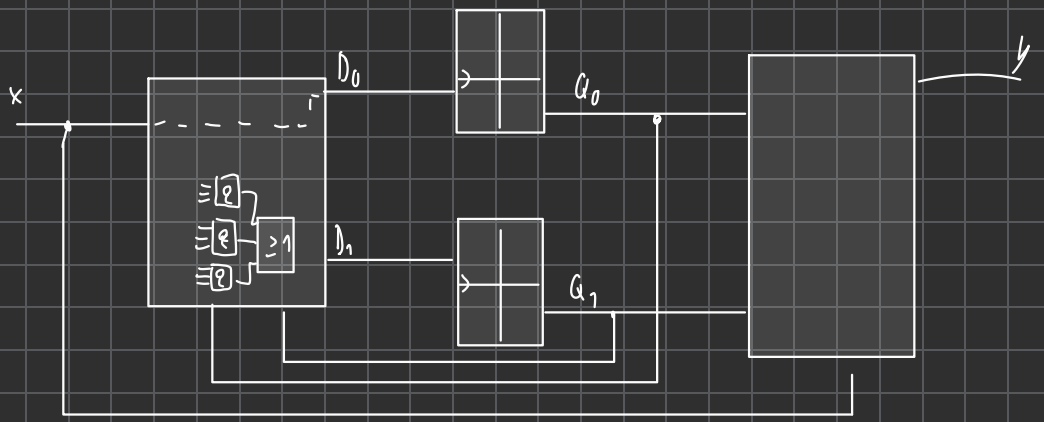
gleichsetzen

$Q_1^{n+1} = (XQ_1 + Q_1\overline{Q}_0 + \overline{X}\overline{Q}_1Q_0)^n = D_1^n$

$Q_1^{n+1} = D_1^n$

$Q_0^{n+1} = X^n = D_0^n$ ← Problem funk

$Q_0^{n+1} = D_0^n$ ← Char. Schaltfunk


x
D_0
Q_0
y
D_1
Q_1
≥1

7. Bestimmung der Eingangsfunktionen (Spezialfall D-Flipflops)

Die Eingangsfunktionen geben an, wie die Eingänge der Flipflops im vorliegenden Kontext beschaltet werden müssen, im Falle von D-Flipflops also, was an die D-Eingänge angeschlossen werden muss.

Eingangsfunktion D_i = ?

Bei der Verwendung von D-Flipflops gilt wegen $Q_i^{n+1} = D_i^n$

$$D_i^n = Q_i^{n+1} = f_i(X_0, X_1, ..., X_{l-1}, Q_0, Q_1, ..., Q_{k-1})^n$$

und damit

$$D_i = f_i(X_0, X_1, ..., X_{l-1}, Q_0, Q_1, ..., Q_{k-1}),$$

das heißt

Eingangsfunktion = Problemfunktion

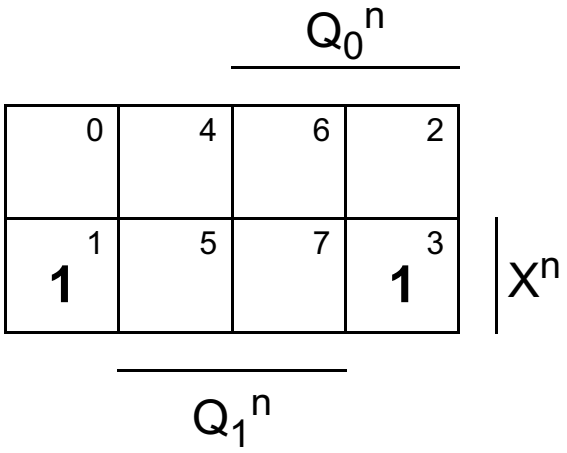
Im vorliegenden Beispiel

$$D_1 = XQ_1 + Q_1\overline{Q}_0 + \overline{X}\overline{Q}_1Q_0$$

$$D_0 = X$$

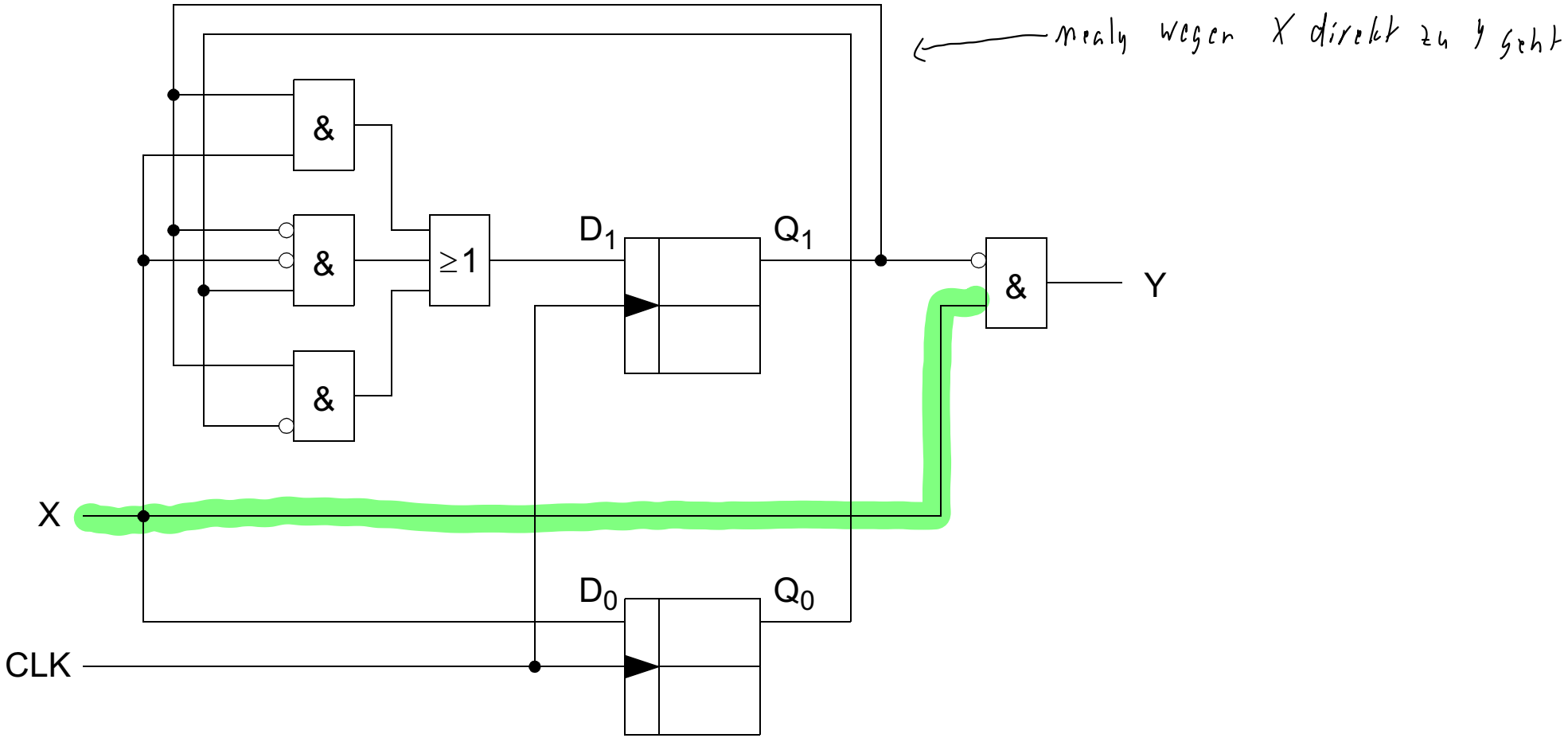
8. Bestimmung der Ausgangsfunktionen (unabhängig vom Flipflop-Typ)

Karnaugh-Diagramm für Y



$Y = X\overline{Q}_1$

9. Schaltplan (mit D-Flipflops)



5. Wahl der Speicherelemente (allgemeiner Fall)

Flipflop bestimmt durch charakteristische Schaltfunktion

D-FF $Q^{n+1} = D^n$

T-FF $Q^{n+1} = (\overline{T}Q + T\overline{Q})^n$

SR-FF $Q^{n+1} = (\overline{R}Q + S)^n, \quad (SR)^n = 0$

JK-FF $Q^{n+1} = (\overline{K}Q + J\overline{Q})^n$

6. Aufstellen der Problemfunktionen (allgemeiner Fall)

Problemfunktion gibt an, wie sich ein Flipflop im gegebenen Kontext verhalten soll.

Falls nicht D-Flipflops verwendet werden, ist es hilfreich die Problemfunktion in der folgenden standardisierten Form anzugeben:

Allgemein

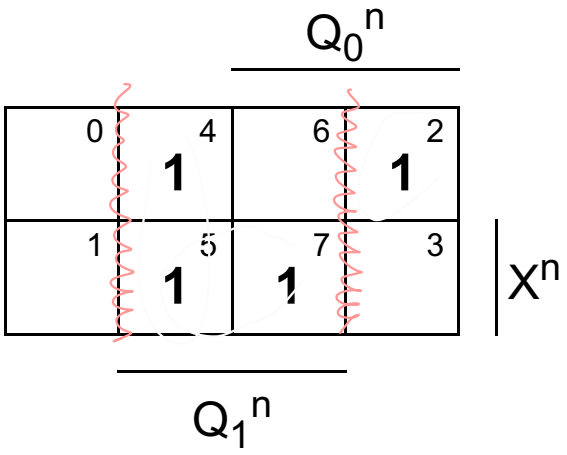
$$Q_i^{n+1} = (f_{i1}Q_i + f_{i2}\overline{Q}_i)^n$$

Im vorliegenden Beispiel

$$Q_0^{n+1} = (f_{01}Q_0 + f_{02}\overline{Q}_0)^n$$

$$Q_1^{n+1} = (f_{11}Q_1 + f_{12}\overline{Q}_1)^n$$

Karnaugh-Diagramm für Q_1^{n+1}

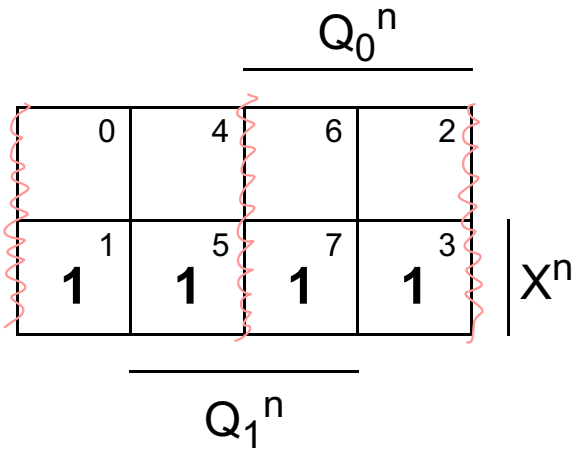


$Q_1^{n+1} = (XQ_1 + Q_1\overline{Q}_0 + \overline{X}\overline{Q}_1Q_0)^n$ *minimal*

$= ((X + \overline{Q}_0)Q_1 + (\overline{X}Q_0)\overline{Q}_1)^n$ *standardisiert*

f_{11} f_{12}

Karnaugh-Diagramm für Q_0^{n+1}



$Q_0^{n+1} = (X)^n$ *minimal*

$= ((X)Q_0 + (X)\overline{Q}_0)^n$ *standardisiert*

f_{01} f_{02}

7. Bestimmung der Eingangsfunktionen für T-, SR- und JK-Flipflops

Charakteristische Schaltfunktionenen

T-FF $Q_i^{n+1} = (\overline{T}_i Q_i + T_i \overline{Q}_i)^n$

SR-FF $Q_i^{n+1} = (\overline{R}_i Q_i + S_i)^n, \quad (S_i R_i)^n = 0$

JK-FF $Q_i^{n+1} = (\overline{K}_i Q_i + J_i \overline{Q}_i)^n$

Problemfunktionen

$Q_i^{n+1} = (f_{i1} Q_i + f_{i2} \overline{Q}_i)^n$

für JK-FF

$$\left(T_1 Q_n + J_i \overline{Q}_i\right)^n = \left(f_{i1} Q_i + \overline{f}_{i2} \overline{Q}_i\right)^n$$

Eingangsfunktionen

$T_i = ?$

$S_i = ?, R_i = ?$

$J_i = ?, K_i = ?$

$J_i = f_{i2}$

$K_i = f_{i1}$

Aufgabe: Lösung der schaltalgebraischen Gleichung, die durch Gleichsetzen von Problemfunktion und charakteristischer Schaltfunktion entsteht:

T-FF	$f_{i1}Q_i + f_{i2}\overline{Q}_i$	=	$\overline{T}_iQ_i + T_i\overline{Q}_i$	
SR-FF	$f_{i1}Q_i + f_{i2}\overline{Q}_i$	=	$\overline{R}_iQ_i + S_i,$	$S_iR_i = 0$
JK-FF	$f_{i1}Q_i + f_{i2}\overline{Q}_i$	=	$\overline{K}_iQ_i + J_i\overline{Q}_i$	

Lösung (ohne Herleitung)

T-FF	$T_i = \bar{f}_{i1}Q_i + f_{i2}\overline{Q}_i$	
SR-FF	$S_i = f_{i2}\overline{Q}_i$	$R_i = \bar{f}_{i1}Q_i$
JK-FF	$J_i = f_{i2}$	$K_i = \bar{f}_{i1}$

Beispiel, realisiert mit JK-FF

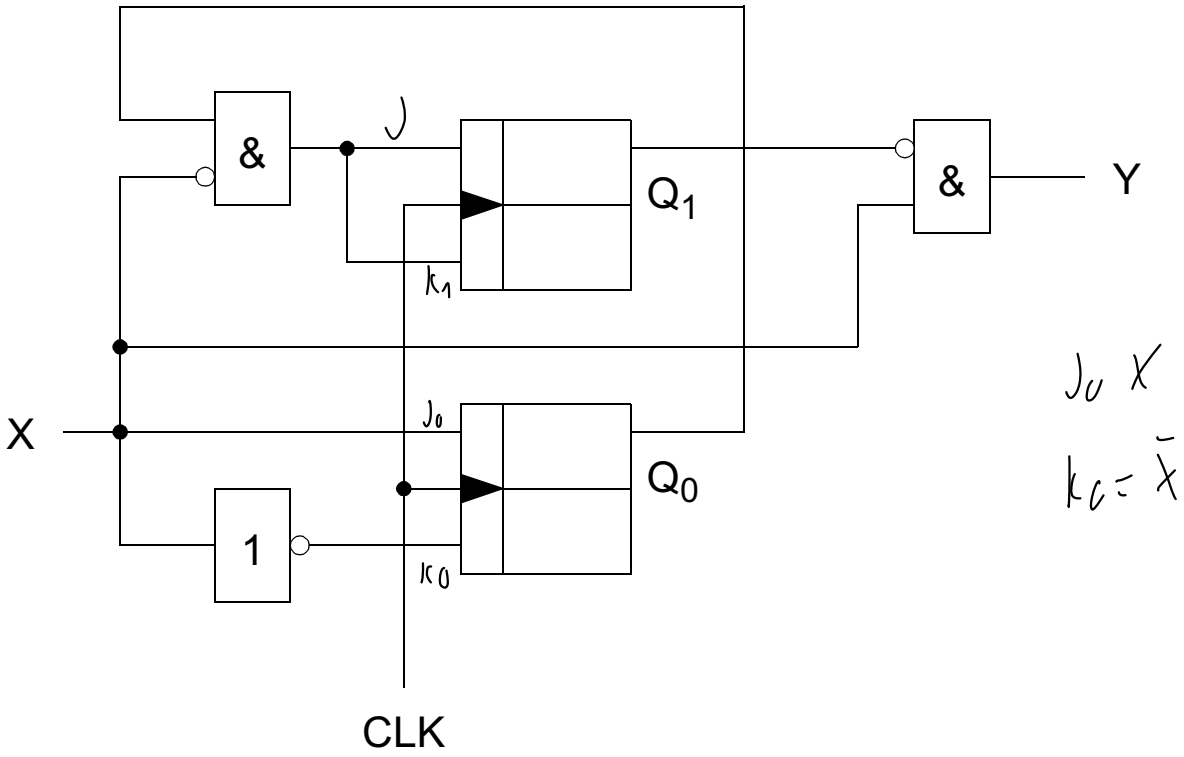
$J_1 = \overline{X} Q_0$

$\overline{K}_1 = X + \overline{Q}_0 \rightarrow K_1 = \overline{X + \overline{Q}_0} = \overline{X} \cdot Q_0$

$J_0 = X$

$\overline{K}_0 = X$

9. Schaltplan (mit JK-Flipflops)



$J_1 = Q_0 \cdot \overline{X}$

$k_1 = a_0 \overline{x}$

$J_0 \; X$

$k_0 = \overline{X}$

Rechnerarchitektur

Matthias Meyer

Kapitel 3 Rechenwerke

Inhaltsübersicht

3.1 Zahlensysteme und Zahlendarstellungen

3.2 Addier- und Subtrahierwerke

3.3 Arithmetisch-Logische Einheit (ALU)

3.4 Multiplizier- und Dividierwerke

3.1 Zahlensysteme und Zahlendarstellungen

In Rechnersystemen müssen Informationen stets mit einer endlichen Anzahl von Worten mit jeweils endlicher Stellenzahl dargestellt werden. Bei der Verarbeitung von ganzen und gebrochenen Zahlen stellt sich dabei das Problem, dass Mengen mit unendlich vielen Elementen wie z. B. die Menge der natürlichen, der rationalen oder der reellen Zahlen auf endliche Mengen abgebildet werden müssen. Als Folge davon muss man Einschränkungen bezüglich des darstellbaren Zahlenbereichs und bezüglich der erzielbaren Genauigkeit in Kauf nehmen.

Inhaltsübersicht

3.1.1 Stellenwertsysteme

3.1.2 Komplementdarstellungen für negative Zahlen

3.1.3 Zweierkomplementarithmetik

3.1.4 Festkommazahlen

3.1.5 Fließkommazahlen

3.1.1 Stellenwertsysteme

$1701 = 1 \cdot 10^3 + 7 \cdot 10^2 + 0 \cdot 10^1 + 1 \cdot 10^0$

Basis
Ziffern

I. Prinzip

Die meisten heute zur Darstellung von ganzen vorzeichenlosen Zahlen benutzten Systeme sind Stellenwertsysteme. Dabei wird eine Zahl als n-stellige Ziffernreihe

$$a_{n-1} \quad a_{n-2} \quad \ldots \quad a_1 \quad a_0$$

dargestellt, wobei jeder Position i mit $0 \leq i < n$ ein Stellenwert zugeordnet ist. Bei Stellenwertsystemen mit einheitlicher Basis z ist der Stellenwert der i-ten Stelle von rechts z^i, d. h. der Stellenwert der einzelnen Ziffern einer Zahl erhöht sich von rechts nach links fortschreitend nach Potenzen der Basis z. Der Wert einer Zahl N_z zur Basis z ergibt sich aus der Summe der nach den Stellenwerten gewichteten Ziffern:

$$N_z = a_{n-1} \cdot z^{n-1} + a_{n-2} \cdot z^{n-2} + \ldots + a_1 \cdot z^1 + a_0 \cdot z^0 = \sum_{i=0}^{n-1} a_i \cdot z^i$$

mit a_i ganzzahlige Koeffizienten (Ziffern), $a_i \in \{0, 1, \ldots, z-1\}$

z ganzzahlige Basis, $z \geq 2$

n Stellenzahl

N_z Wert der n-stelligen Zahl zur Basis z

Zahlensystems, die keine Stellenwertsysteme sind:
- Römische Zahlen: I II III IV
- Strichlisten: 𝍸 III
- Finger
- Gray-Code 0 0000
  1 0 001
  2 0 011
  3 0 010

II. Beispiele

Dezimalsystem

Basis $z = 10$

Koeffizienten $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Beispiel $N_{10} = 1701 = \mathbf{1} \cdot 10^3 + \mathbf{7} \cdot 10^2 + \mathbf{0} \cdot 10^1 + \mathbf{1} \cdot 10^0$

Dualsystem (Binärsystem)

Basis $z = 2$

Koeffizienten $a_i \in \{0, 1\}$

Beispiel $N_2 = 1101001 = \mathbf{1} \cdot 2^6 + \mathbf{1} \cdot 2^5 + \mathbf{0} \cdot 2^4 + \mathbf{1} \cdot 2^3 + \mathbf{0} \cdot 2^2 + \mathbf{0} \cdot 2^1 + \mathbf{1} \cdot 2^0$

gleichwertige Dezimalzahl: $64 + 32 + 8 + 1 = 105$

Oktalsystem

Basis $z = 8$

Koeffizienten $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7\}$

Beispiel $N_8 = 1701 = \mathbf{1} \cdot 8^3 + \mathbf{7} \cdot 8^2 + \mathbf{0} \cdot 8^1 + \mathbf{1} \cdot 8^0$

gleichwertige Dezimalzahl: $512 + 7 \cdot 64 + 1 = 961$

Hexadezimalsystem

Basis $z = 16$

Koeffizienten $a_i \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\}$

A B C D E F

Damit jeder der möglichen Koeffizienten durch ein Zeichen dargestellt werden kann, verwendet man neben den Ziffern von 0 bis 9 die Buchstaben von A bis F. Dabei gilt A = 10, B = 11, C = 12, D = 13, E = 14 und F = 15.

Beispiel $N_{16} = A7CF = \mathbf{10} \cdot 16^3 + \mathbf{7} \cdot 16^2 + \mathbf{12} \cdot 16^1 + \mathbf{15} \cdot 16^0$

gleichwertige Dezimalzahl: $10 \cdot 4096 + 7 \cdot 256 + 12 \cdot 16 + 15 = 42959$

100 dezim = 64 hex = 01100100 binär

0x64 (C, C++, Java, ...)

$64 (Assembler)

64_{16}

hex dez

25 37

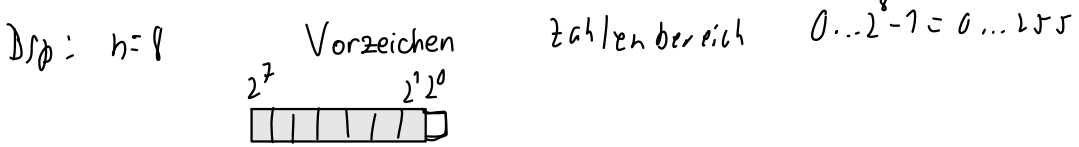
+AB 171

D0 208

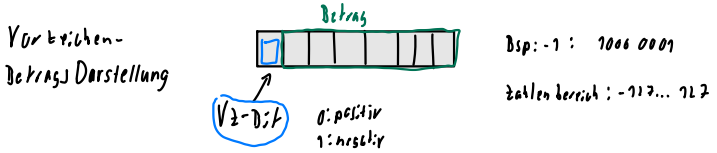
3.1.2 Komplementdarstellungen für negative Zahlen

Es ist allgemein üblich, negative Zahlen durch ein vorangestelltes Minus-Zeichen zu kennzeichnen. Diese Art der Darstellung nennt man **"Vorzeichen/Betrags - Darstellung"**, weil Vorzeichen und Betrag unabhängig voneinander dargestellt werden. Diese Darstellung ist für die Verarbeitung in Rechensystemen nicht besonders gut geeignet, weil Vorzeichen und Betrag separat gespeichert und behandelt werden müssen.

Aus diesem Grund verwendet man in Rechensystemen für vorzeichenbehaftete Zahlen in der Regel **Komplementdarstellungen**. Dabei führt man die Subtraktion (d.h. die Addition einer negativen Zahl) auf die Addition eines positiven Komplements zurück. Die größte praktische Bedeutung hat dabei das z-Komplement für z = 2 (Zweierkomplement).







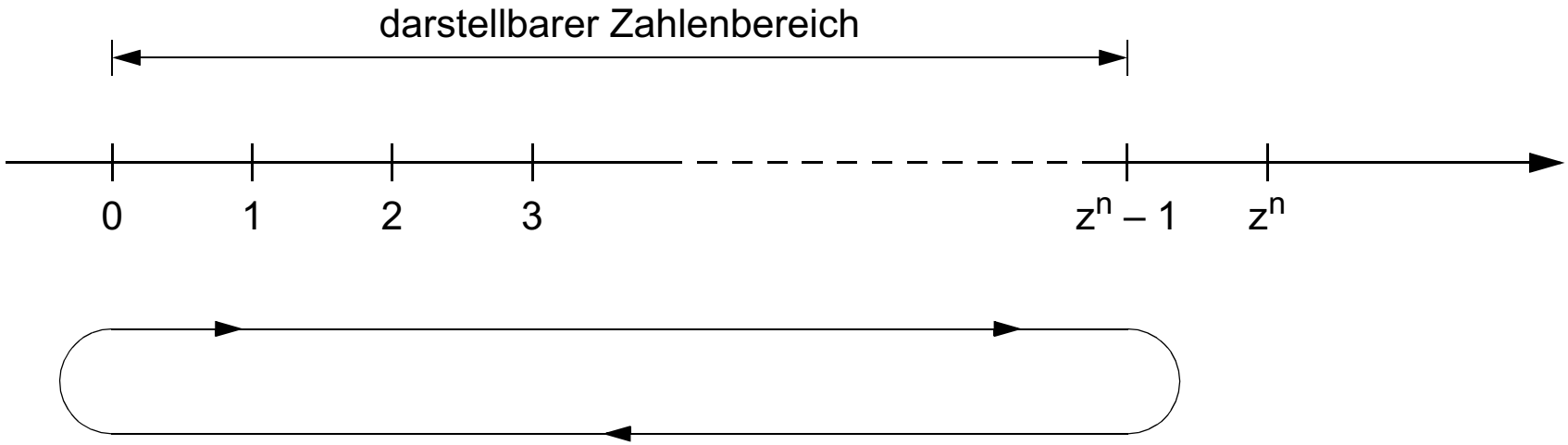
Nachteile der VZ-Betragsdarstellung

- Doppelte Darstellung der 0
- Separate VZ-Rechnung u. Betragsrechnung
- verschiedenen Rechenwerte für vorzeichenlose und vorzeichen Zahlen

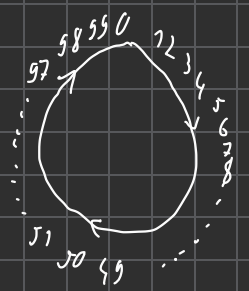
I. Motivation für Komplementdarstellungen

In Rechensystemen muss man zur Darstellung von ganzen Zahlen stets mit **begrenzten Stellenzahlen** arbeiten. Daraus folgt, dass es bei Stellenwertsystemen mit einheitlicher Basis z in Abhängigkeit von der Stellenzahl n eine größtmögliche darstellbare Zahl $z^n - 1$ gibt. Erhöht man diese Zahl um eins, so erhält man einen Übertrag in die nicht vorhandene Stelle n + 1. Das n-stellige Ergebnis selbst hat den Wert Null. Man spricht von einer Addition „Modulo z^n“, da sich das Ergebnis einer Addition aus dem Rest der eigentlichen Summe nach einer Division durch z^n ergibt.

Wegen der Addition „Modulo z^n“ kann man sich die darstellbaren Zahlen **zyklisch angeordnet** denken:



Bsp: z=10 n:2



Es ist deshalb möglich, ausgehend von einer beliebigen darstellbaren Zahl A jede beliebige andere darstellbare Zahl B zu erhalten, indem man zu A eine geeignete positive Zahl X addiert. Da B nicht notwendigerweise größer sein muss als A, kann auch die Addition negativer Zahlen (d. h. die Subtraktion) durch die Addition einer geeigneten positiven Zahl realisiert werden. Bei dieser positiven Zahl handelt es sich um ein **Komplement** der zu subtrahierenden Zahl.

II. Das z-Komplement

Das z-Komplement K_z einer positiven Zahl $|N_z|$ „ergänzt“ diese zur ersten nicht mehr darstellbaren positiven Zahl z^n (oder besser zur Anzahl der insgesamt darstellbaren Zahlen), d.h. es gilt

$$|N_z| + K_z = z^n$$

Daraus lässt sich die Bildungsvorschrift für das z-Komplement ableiten:

$$K_z = z^n - |N_z|$$

mit

K_z	z-Komplement von $\|N_z\|$
z	Basis
n	Stellenzahl
$\|N_z\|$	Betrag der zu komplementierenden Zahl (Basis z),

$$|N_z| = a_{n-1} \cdot z^{n-1} + a_{n-2} \cdot z^{n-2} + \ldots + a_1 \cdot z^1 + a_0 \cdot z^0 = \sum_{i=0}^{n-1} a_i \cdot z^i \quad \text{mit } a_i \in \{0, 1, ..., z-1\}$$

$z^n - 1$ ist die größte positive Zahl zur Basis z, die mit n Stellen darstellbar ist. Deshalb gilt

$$z^n - 1 = (z-1)\cdot z^{n-1} + (z-1)\cdot z^{n-2} + \ldots + (z-1)\cdot z^1 + (z-1)\cdot z^0 = \sum_{i=0}^{n-1} (z-1)\cdot z^i$$

Damit ergibt sich

$$K_z = z^n - |N_z| = \sum_{i=0}^{n-1} (z-1)\cdot z^i - \sum_{i=0}^{n-1} a_i \cdot z^i = \sum_{i=0}^{n-1} (z-1-a_i)\cdot z^i$$

$$z^n - 1 = z^n - z^{n-1} + z^{n-1} - 1$$
$$= (z-1)\,z^{n-1} + z^{n-1} - z^{n-2} + z^{n-2} - 1 = (z-1)\,z^{n-1} + (z-1)\,z^{n-2}$$

Folgerung

Die Komplementbildung kann erfolgen, indem man zunächst stellenweise jede Ziffer a_i von der größtmöglichen Ziffer z – 1 subtrahiert, und dann zur so entstandenen Zahl eins addiert.

Beispiele

Zehnerkomplement (z = 10)	n = 2,	$\lvert N_{10}\rvert = \lvert -23\rvert$,	$K_{10} = 77$
Zweierkomplement (z = 2)	n = 8,	$\lvert N_2\rvert = \lvert 00010111\rvert$,	$K_2 = 11101001$

Hinweis: Die Rückkomplementierung erfolgt nach der gleichen Vorschrift wie die Komplementbildung.

$$K_z = z^n - \lvert N_z\rvert$$

$$\lvert N_z\rvert = z^n - K_z$$

Bsp: $\sqrt{-6}$ $n=4$ $z=2$

$$\begin{array}{r} 0101 \\ -\ 0110 \\ \hline 1111 \end{array} \quad ! \quad = -1$$

III. Realisierung der Subtraktion durch die Addition des z-Komplements

Voraussetzung: Die Anzahl der zur Darstellung verwendeten Stellen ist auf n begrenzt, d.h. alle Additionen erfolgen „Modulo z^n“.

Vorschrift: Positive Zahlen werden unverändert übernommen, negative Zahlen werden im z-Komplement dargestellt.

Fall I: $\mathbf{A - B \geq 0}$ (keine Komplementbildung für Differenz erforderlich)

$$A - B = A + (z^n - |B|) \quad \text{... mod } z^n$$

$$A - B = A - B + z^n \quad \text{... mod } z^n$$

$$0 = z^n \quad \text{... mod } z^n \quad \text{qed}$$

Fall II: $\mathbf{A - B < 0}$ (Komplementbildung für Differenz erforderlich)

$$z^n - |A - B| = A + (z^n - |B|) \quad \text{... mod } z^n$$

$$z^n - (-(A - B)) = A - B + z^n \quad \text{... mod } z^n$$

$$z^n + A - B = A - B + z^n \quad \text{... mod } z^n \quad \text{qed}$$

3.1.3 Zweierkomplementarithmetik

I. Vorzeichenbehaftet versus vorzeichenlos

In der Zweierkomplementarithmetik werden **Zahlen mit gesetztem höchstwertigem Bit** nach Vereinbarung als **negative Zahlen**, d.h. als Komplemente ihres Betrags interpretiert. Das höchstwertige Bit wird deshalb auch **Vorzeichenbit** genannt. Durch diese Vereinbarung erhält man eine nahezu symmetrische Aufteilung des Wertebereichs in positive und negative Zahlen.

Ein großer Vorteil der Zweierkomplementdarstellung für negative Zahlen ist, dass vorzeichenlose und vorzeichenbehaftete Zahlen einheitlich behandelt werden können. So sind keine speziellen Addier- oder Subtrahierwerke für vorzeichenbehaftete Zahlen notwendig, und die Unterscheidung vorzeichenbehafteter und vorzeichenloser Zahlen wird zur reinen Interpretationssache (siehe Beispiel).

Beispiel für n = 8

```
  00110011
+ 11001001
------------
  11111100
```

$z=2,\ n=8$

vorzeichenlose Interpretation (unsigned int)

51_{10}

201_{10}

252_{10}

vorzeichenbehaftete Interpretation (int)

$+51$
-55
-4

alle Stellen invertieren

```
11001001
00110110
00110111
```

vorzeichenlose Interpretation (Wertebereich $0 \leq x \leq 255$)

```
  51
+ 201
--------
  252
```

vorzeichenbehaftete Interpretation (Wertebereich $-128 \leq x \leq 127$)

```
  51
– 55
--------
– 4
```

$1\,0\,1\,0\,1\,0\,1\,0$

negativ

Zer-Komplement der Betrags

alle stellen invertieren → $0\,1\,0\,1\,0\,1\,0\,1$ $+1$ → $0\,1\,0\,1\,0\,1\,1\,0$

$= 86_{16}$

-86

$-85 = 55_{16} = 0\,1\,0\,1\,0\,1\,0\,1$

alle stellen invertieren → $1\,0\,1\,0\,1\,0\,1\,0$ → $1\,0\,1\,0\,1\,0\,1\,1$

-85

$n=5$

$0 \dots 31$

$19 + 6 = 25$

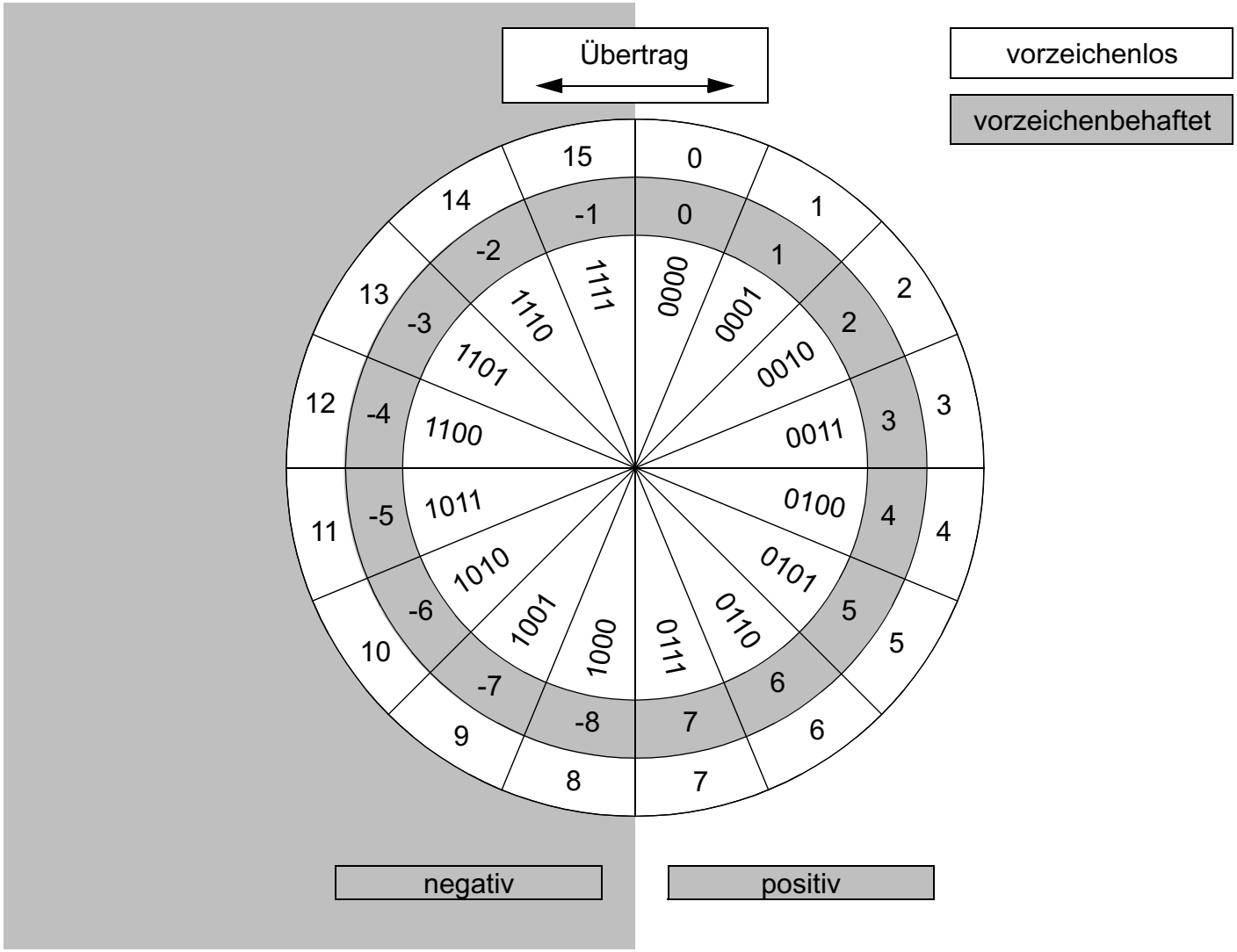
$-16 \dots +15$

$-13 + 6 = -7$

$$\begin{array}{r} {\scriptstyle 16}\quad {\scriptstyle 1\ 1} \\ 1\,0\,0\,1\,1 \\ +\ 0\,1\,1\,1\,0 \\ \hline 1\,1\,0\,0\,1 \end{array}$$

$$\begin{array}{l} 1\,0\,0\,1\,1_{(2} \\ 0\,1\,1\,0\,0_{(2}{}^{+1} \\ 0\,1\,1\,0\,1 \\ \hline 1\,1\,0\,0\,1 \\ 0\,0\,1\,1\,0_{(2}{}^{+1} \\ 0\,0\,1\,1\,1_{(2} \end{array}$$

II. Der Zahlenkreis



Die Subtraktion „A – B“ lässt sich am Zahlenkreis veranschaulichen, indem man von der Zahl A aus B Schritte entgegen des Uhrzeigersinns wandert. Bei der entsprechenden Addition des Zweierkomplements erreicht man das Ziel „A – B“ auf einem Weg im Uhrzeigersinn. Dabei **ergänzen** sich die beiden Wege zum Vollkreis, denn bei der Anzahl der Schritte im Uhrzeigersinn handelt es sich um das **Komplement** der entsprechenden Anzahl Schritte entgegen des Uhrzeigersinns.

III. Überschreitungen des Wertebereichs

Bei der vorzeichenlosen Interpretation von n-stelligen Dualzahlen steht ein Wertebereich von

$$0 \leq x \leq 2^n - 1$$

zur Verfügung. Überschreitungen des darstellbaren Wertebereichs werden durch einen **Übertrag C** = c_n aus der höchstwertigen Stelle n – 1 angezeigt.

Beispiel für n = 4

```
      1011    (11)
    + 0111    (7)
    ----------
(1)   0010    (2)
```

→ Ergebnis falsch wegen Übertrag C = c_4 = 1

Bei der vorzeichenbehafteten Interpretation von n-stelligen Dualzahlen (Zweierkomplementarithmetik) steht ein Wertebereich von

$$-2^{n-1} \leq x \leq 2^{n-1} - 1$$

Bsp. n=5 Bit −8...+7 n=8 −128...+127

zur Verfügung. Überschreitungen des darstellbaren Wertebereichs werden durch einen **Überlauf V** angezeigt. Ein Überlauf entsteht in jedem der vier nachfolgend aufgeführten Fälle:

positiv	+	positiv	=	negativ
negativ	+	negativ	=	positiv
positiv	–	negativ	=	negativ
negativ	–	positiv	=	positiv

Überträge werden in der Zweierkomplementarithmetik ignoriert.

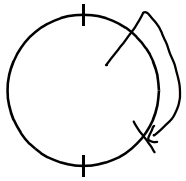
Beispiel für n = 4

```
      0101    (+5)
    + 0111    (+7)
   ----------
0   1100    (–4)
```

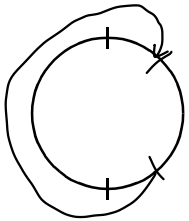
→ Ergebnis falsch wegen Überlauf V = 1

Überlauferkennung bei Additionen

```
  0xxx...xxx
+ 0xxx...xxx
-------------
  0xxx...xxx
```

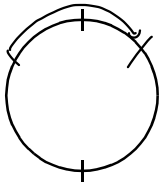


```
  0xxx...xxx
+ 1xxx...xxx
-------------
  0xxx...xxx
```



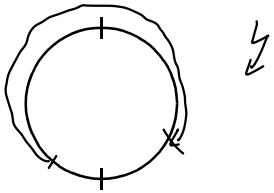
negativ + positiv

```
  1xxx...xxx
+ 0xxx...xxx
-------------
  0xxx...xxx
```

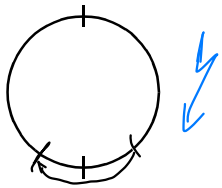


negativ + negativ

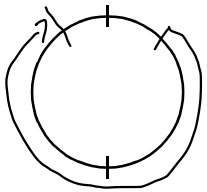
```
  1xxx...xxx
+ 1xxx...xxx
-------------
  0xxx...xxx
```



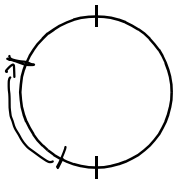
```
  0xxx...xxx
+ 0xxx...xxx
-------------
  1xxx...xxx
```



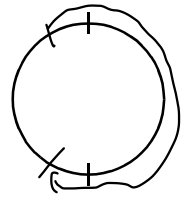
```
  0xxx...xxx
+ 1xxx...xxx
-------------
  1xxx...xxx
```



```
  1xxx...xxx
+ 0xxx...xxx
-------------
  1xxx...xxx
```



```
  1xxx...xxx
+ 1xxx...xxx
-------------
  1xxx...xxx
```



n = 4 Bit

rückkomplementieren

```
  1 0 0 1      -7
+ 1 0 1 1      -5
-------------------
  0 1 0 0      +4
```

```
  1 1
+ 1 1
-------
  0
```

Übertrag 1

n = 5 Bit

```
  1 1 0 0 1      -7
+ 1 1 0 1 1      -5
-------------------
  1 0 1 0 0      -12
```

0 1 0 1 1

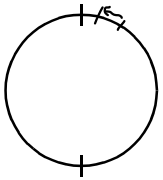
0 1 1 0 0 ≙ 12

V = 0

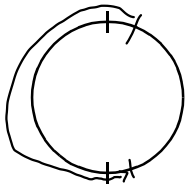
keine Überlauf

Überlauferkennung bei Subtraktionen

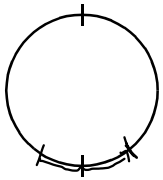
```
  0xxx...xxx
- 0xxx...xxx
-----------
  0xxx...xxx
```



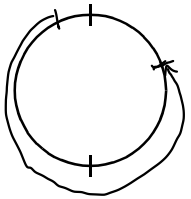
```
  0xxx...xxx
- 1xxx...xxx
-----------
  0xxx...xxx
```



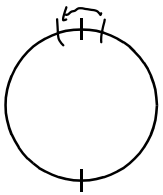
```
  1xxx...xxx
- 0xxx...xxx
-----------
  0xxx...xxx
```



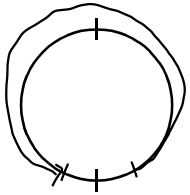
```
  1xxx...xxx
- 1xxx...xxx
-----------
  0xxx...xxx
```



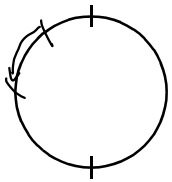
```
  0xxx...xxx
- 0xxx...xxx
-----------
  1xxx...xxx
```



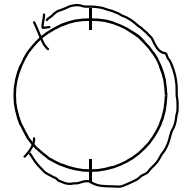
```
  0xxx...xxx
- 1xxx...xxx
-----------
  1xxx...xxx
```



```
  1xxx...xxx
- 0xxx...xxx
-----------
  1xxx...xxx
```



```
  1xxx...xxx
- 1xxx...xxx
-----------
  1xxx...xxx
```



Überlauferkennung: Zusammenfassung

$V = c_{n-1} \oplus c_n$

↑ XOR "Verschieden"

gilt für Addition und Subtraktion

```
 10011        +19
 11110         30
(1)1001        17
```

IV. Geschlossene Darstellung für vorzeichenbehaftete Zahlen

Fall 1: $a \geq 0$, d.h. $a_{n-1} = 0$

$$a = 0 \cdot 2^{n-1} + a_{n-2} \cdot 2^{n-2} + \ldots + a_1 \cdot 2^1 + a_0 \cdot 2^0 = 0 + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

Fall 2: $a < 0$, d.h. $a_{n-1} = 1$, Darstellung von $-a$ im Zweierkomplement

$$-a = \sum_{i=0}^{n-2} (1 - a_i) \cdot 2^i + 1 = \sum_{i=0}^{n-2} 2^i - \sum_{i=0}^{n-2} a_i \cdot 2^i + 1$$

$$a = -\sum_{i=0}^{n-2} 2^i - 1 + \sum_{i=0}^{n-2} a_i \cdot 2^i = -2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

Daraus: geschlossene Darstellung für beide Fälle

$$a = -a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

Folgerung: Vorzeichenbit kann interpretiert werden als Stelle mit **negativem Stellenwert**

V. Vorzeichenrichtige Erweiterung

Ausgehend von der geschlossenen Darstellung für vorzeichenbehaftete Zahlen

$$a = -a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

erhält man

$$a = a_{n-1} \cdot 2^{n-1} - a_{n-1} \cdot 2^{n-1} - a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i$$

0

$-2 \cdot a_{n-1} \cdot 2^{n-1} = a_{n-1} \cdot 2^n$

$$= -a_{n-1} \cdot 2^n + \sum_{i=0}^{n-1} a_i \cdot 2^i$$

Interpretation:

Um die Stellenzahl einer vorzeichenbehafteten Zahl zu erhöhen, muss der Wert des Vorzeichenbits in die neu hinzukommenden Stellen kopiert werden.

3.1.4 Festkommazahlen

Die Festkommadarstellung ist eine logische Fortsetzung des im ersten Abschnitt eingeführten Stellenwertsystems. Für den Spezialfall z = 2 (Basis 2) erhält man

$$X = a_{n-1} \cdot 2^{n-1} + a_{n-2} \cdot 2^{n-2} + \ldots + a_1 \cdot 2^1 + a_0 \cdot 2^0 + a_{-1} \cdot 2^{-1} + a_{-2} \cdot 2^{-2} + \ldots + a_{-m} \cdot 2^{-m}$$

mit

- a_i Koeffizienten, $a_i \in \{0, 1\}$
- X Wert der darzustellenden gebrochenen Zahl
- n Anzahl der Vorkommastellen
- m Anzahl der Nachkommastellen
- s Gesamtanzahl der Stellen, s = n + m

Beispiel

n = 4, m = 4, X = 13.75

$X = \mathbf{1} \cdot 2^3 + \mathbf{1} \cdot 2^2 + \mathbf{0} \cdot 2^1 + \mathbf{1} \cdot 2^0 + \mathbf{1} \cdot 2^{-1} + \mathbf{1} \cdot 2^{-2} + \mathbf{0} \cdot 2^{-3} + \mathbf{0} \cdot 2^{-4}$

$\rightarrow$ X = 1101.1100

3.1.5 Fließkommazahlen

I. Darstellung

Allgemeine Darstellung einer Fließkommazahl im Dualsystem:

$$X = a \cdot 2^e = (1 + f) \cdot 2^e$$

mit

- a normierte Mantisse, $1 \leq a < 2$
- f Fraction, $f = a - 1$
- e Exponent, ganzzahlig
- X Wert der darzustellenden Fließkommazahl

Codierung

- Fraction f codiert als Festkommazahl mit n = 0 Vorkommastellen und m Nachkommastellen
- zum Exponenten e wird ein „Bias“ B (Offset) addiert
- B wird so gewählt, dass e + B immer positiv ist
- Codierung des Exponenten mit Bias als vorzeichenlose Binärzahl mit k Stellen
  (in der Regel wählt man $B = 2^{k-1} - 1$)

Beispiel

k = 4, m = 8, X = 13.75

$B = 2^3 - 1 = 7$

$X = 13.75 = (13.75 / 8) \cdot 8 = 1.71875 \cdot 2^3$

Codierung der Fraction

$f = \mathbf{1} \cdot 2^{-1} + \mathbf{0} \cdot 2^{-2} + \mathbf{1} \cdot 2^{-3} + \mathbf{1} \cdot 2^{-4} + \mathbf{1} \cdot 2^{-5} + \mathbf{0} \cdot 2^{-6} + \mathbf{0} \cdot 2^{-7} + \mathbf{0} \cdot 2^{-8}$

$\rightarrow$ f = 10111000

Codierung des Exponenten mit Bias

$3 + B = 3 + 7 = 10$

$\rightarrow$ e + B = 1010

Genormte Fließkommaformate (IEEE)

Single Precision	(32 bit)	m = 23,	k = 8,	1 bit Vorzeichen Mantisse
Double Precision	(64 bit)	m = 52,	k = 11,	1 bit Vorzeichen Mantisse
Extended Precision	(80 bit)	m = 64,	k = 15,	1 bit Vorzeichen Mantisse

Anmerkungen

- Darstellung mit „normierter Mantisse“: Man führt die Operation $a \cdot 2^e = (a \cdot 2) \cdot 2^{e-1}$ so lange aus, bis die Mantisse zwischen 1 (incl.) und 2 (excl.) liegt. Dann ist das erste Bit der Mantisse (ganzzahliger Anteil) immer „1“ und muss nicht explizit gespeichert werden.
- In der Regel ist die Fließkommadarstellung nicht exakt, da die Mantisse im allgemeinen unendlich lang ist und nach endlich vielen Stellen abgebrochen werden muss

  $\rightarrow$ Rundungsfehler! (Beispiel: Darstellung der Zahl 0.1)
- Addition von Fließkommazahlen (Subtraktion analog):

  $(a_1 \cdot 2^e) + (a_2 \cdot 2^e) = (a_1 + a_2) \cdot 2^e$

  – beide Operanden so umformen, dass sie den gleichen Exponenten besitzen

  – Mantissen addieren bzw. subtrahieren

  – Ergebnis evtl. so umformen, dass Mantisse normiert ist
- Multiplikation von Fließkommazahlen (Division analog):

  $(a_1 \cdot 2^{e1}) \cdot (a_2 \cdot 2^{e2}) = a_1 \cdot a_2 \cdot 2^{e1+e2}$

  – Mantissen multiplizieren und Exponenten addieren

  – Ergebnis evtl. so umformen, dass Mantisse normiert ist

Bsp: 0,1 als Single Prec. Float (k=8, m=23)

$0,1 = \frac{0,1 \cdot 16}{16} = 1,6 \cdot 2^{-4}$

$a = 1,6$

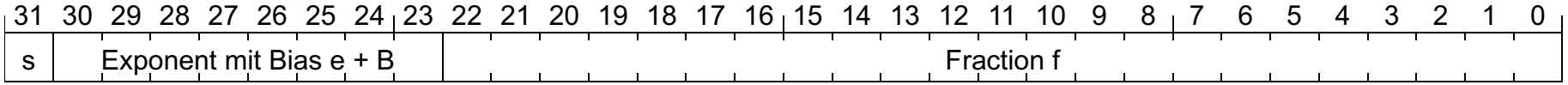
$B = 2^{k-1} - 1 = 127$

$f = a - 1 = 0,6 \cdot 2^{23} \cdot 2^{-23} = 5.033.164,8 \approx 5.033.165 = 4CCCCD$

$e + B = -4 + 127 = 123 = 7B_{16} = 0111\ 1011_2$

II. Beispiele (Single Precision)

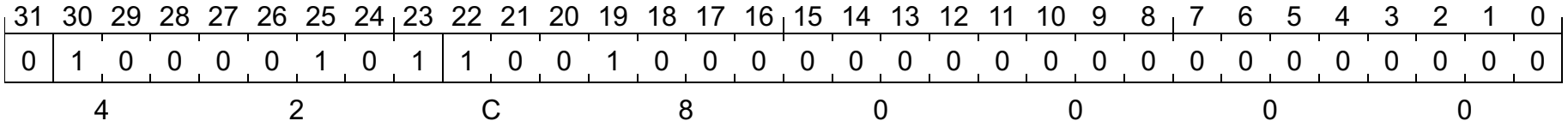
m = 23 Bit für Fraction, k = 8 Bit für Exponent mit Bias, 1 Bit für Vorzeichen, Bias B = $2^{k-1} - 1 = 127$



Beispiel 1: A = 100 = 100 * 2^{-6} * 2^{6} = 100/64 * 2^{6}

Mantisse a = 100/64, Fraction f = 100/64 – 1 = 36/64 = $100100_2/64 = 0.100100_2 = 480000_{16} * 2^{-23}$

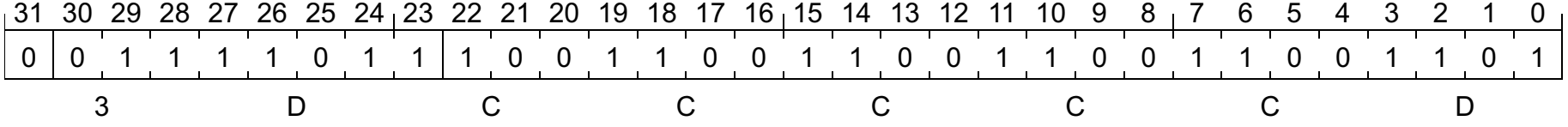
Exponent e = 6, Exponent mit Bias e + B = 6 + 127 = 133 = $85_{16} = 10000101_2$



Beispiel 2: B = 0.1 = 0.1 * 2^{4} * 2^{-4} = 1.6 * 2^{-4}

Mantisse a = 1.6, Fraction f = 1.6 – 1 = 0.6 = $5033164.8/2^{23} \approx 4CCCCD_{16}/2^{23} = 4CCCCD_{16} * 2^{-23}$

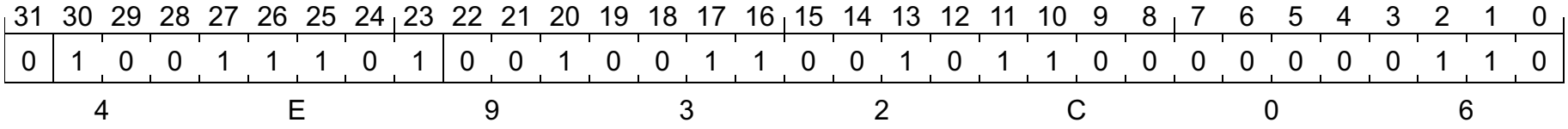
Exponent e = –4, Exponent mit Bias e + B = –4 + 127 = 123 = $7B_{16} = 01111011_2$



Beispiel 3: C = 1234567890 ≈ 1.1497809459 * 2^{30}

Fraction f = 0.1497809459 ≈ 1256453.6/2^{23} ≈ $132C06_{16}/2^{23}$ = $132C06_{16}$ * 2^{-23}

Exponent e = 30, Exponent mit Bias e + B = 30 + 127 = 157 = $9D_{16}$ = 10011101_2



Hinweis zur nachfolgend verwendeten hexadezimalen Darstellung von Mantissen

$a_0.a_{-3}\ a_{-7}\ a_{-11}\ a_{-15}\ a_{-19}\ a_{-23}$ mit $a_0 \in \{0, 1\}$; $a_{-3} \in \{0, 1, ..., 7\}$; $a_{-7}, a_{-11}, a_{-15}, a_{-19}, a_{-23} \in \{0, 1, ..., F\}$

repräsentiere den Wert $a_0 * 2^0 + a_{-3} * 2^{-3} + a_{-7} * 2^{-7} + a_{-11} * 2^{-11} + a_{-15} * 2^{-15} + a_{-19} * 2^{-19} + a_{-23} * 2^{-23}$

Beispiel: $\underline{1.4}80000_{16} = 1 * 2^0 + 4 * 2^{-3} + 8 * 2^{-7} = 1.5625_{10}$ (unterstrichene Stellen entsprechen 4 Bit)

III. Absorption

Beispiel

1234567890 + 100 (Addition stark unterschiedlich großer Zahlen)

$\underline{1.1}32C06_{16} * 2^{30}$ (23 Bit Fraction erw. zu 24 Bit Mantisse)

+ $\underline{1.4}80000_{16} * 2^{6} = \underline{1.4}80000_{16} * 2^{6} * 2^{24} * 2^{-24} = \underline{1.4}80000_{16} * 2^{-24} * 2^{30} = 0.000000\underline{C}8 * 2^{30}$

= $(\underline{1.1}32C06_{16} + 0.000000\underline{C}8_{16}) * 2^{30}$

= $\underline{1.1}32C06C8_{16} * 2^{30} \approx \underline{1.1}32C07_{16} * 2^{30}$ (Rundung auf 24 Bit Mantisse bzw. 23 Bit Fraction)

Signifikante Stellen der kleinen Zahl gehen verloren, d.h. werden von der großen Zahl absorbiert.

1234567890 dargestellt als $\underline{1.1}32C06_{16} * 2^{30} = \underline{9}32C06_{16} * 2^{7} = 1234567936$ (Fehler +46)

100 dargestellt als $\underline{1.4}80000_{16} * 2^{6} = \underline{C}80000 * 2^{-17} = 100$ (Fehler 0)

100 nach Anpassung des Exponenten und Rundung ist $\underline{0.0}00001_{16} * 2^{30} = 1 * 2^{7} = 128$ (Fehler +28)

1234567890 + 100 ermittelt als $\underline{1.1}32C07_{16} * 2^{30} = \underline{9}32C07_{16} * 2^{7} = 1234568064$ (Fehler +74)

1234567990 direkt dargestellt ergibt

$1.1497810390 * 2^{30} = 1.1497810390 * 2^{23} * 2^{7} = 9645062.4 \approx \underline{9}32C06_{16} * 2^{7} = 1234567936$ (Fehler –54)

IV. Auslöschung

Beispiel

1234567890 – 1234567790 (Subtraktion ähnlich großer Zahlen)

$\underline{1.1}32C06_{16} * 2^{30}$

– $\underline{1.1}32C05_{16} * 2^{30}$

= $(\underline{1.1}32C06_{16} - \underline{1.1}32C05_{16}) * 2^{30}$

= $\underline{0.0}00001_{16} * 2^{30} = 1 * 2^{-23} * 2^{30} = 1 * 2^{7} = 128$

Bei der Subtraktion ähnlich großer Zahlen werden signifikante Stellen der Operanden ausgelöscht.

1234567890 dargestellt als $\underline{1.1}32C06_{16} * 2^{30} = \underline{9}32C06_{16} * 2^{7} = 1234567936$ (Fehler +46)

1234567790 dargestellt als $\underline{1.1}32C05_{16} * 2^{30} = \underline{9}32C05_{16} * 2^{7} = 1234567808$ (Fehler +18)

Differenz dargestellt als $\underline{0.0}00001_{16} * 2^{30} = 1 * 2^{7} = 128$ (Fehler +28)

V. Zusammenfassung

1. Vergleiche Fließkommazahlen nie mit "gleich" oder "ungleich"
2. Vermeide Summen stark unterschiedlich großer Zahlen
3. Vermeide Differenzen ähnlich großer Zahlen

VI. Programmbeispiele in Java

Binärdarstellung von Fließkommazahlen anzeigen (Beispiel float, d.h. Single Precision)

Programmcode

```
float A = 100.0f;           int Ai = 100;
float B = 0.1f;
float C = 1234567890.0f;    int Ci = 1234567890;
float D = 1234567790.0f;    int Di = 1234567790;

System.out.println("A = " + A + " " + Integer.toHexString(Float.floatToIntBits(A)));
System.out.println("B = " + B + " " + Integer.toHexString(Float.floatToIntBits(B)));
System.out.println("C = " + C + " " + Integer.toHexString(Float.floatToIntBits(C)));
System.out.println("D = " + D + " " + Integer.toHexString(Float.floatToIntBits(D)));
```

Ausgabe

```
A = 100.0 42c80000
B = 0.1 3dcccccd
C = 1.23456794E9 4e932c06
D = 1.23456781E9 4e932c05
```

Illustration zu Absorption und Auslöschung

Programmcode

```
System.out.println("Absorption und Auslöschung");
System.out.println("C + A = " + (C + A) + " (korrekt " + (Ci + Ai) +")");
System.out.println("C - D = " + (C - D) + " (korrekt " + (Ci - Di) +")");
System.out.println();
System.out.println("Folge: Assoziativgesetze gelten nicht mehr");
System.out.println("(C + A) - D = " +  ((C + A) - D) );
System.out.println("(C - D) + A = " +  ((C - D) + A) );
```

Ausgabe

```
Absorption und Auslöschung
C + A = 1.23456806E9 (korrekt 1234567990)
C - D = 128.0 (korrekt 100)

Folge: Assoziativgesetze gelten nicht mehr
(C + A) - D = 256.0
(C - D) + A = 228.0
```

Illustration zu "Vergleiche Fließkommazahlen nie mit "gleich" oder "ungleich""

Programmcode

```
if(C + A == 1234567990.0f) {
    System.out.println("equal!");
} else {
    System.out.println("not equal! (but should be equal ...!)");
}

if(C + A == 1234567990.0f + 74.0f) {
    System.out.println("equal! (but should be not equal ...!)");
} else {
    System.out.println("not equal!");
}
```

Ausgabe

```
not equal! (but should be equal ...!)
equal! (but should be not equal ...!)
```

3.2 Addier- und Subtrahierwerke

Inhaltsübersicht

3.2.1 Halbaddierer

3.2.2 Volladdierer

3.2.3 Ripple-Carry-Addierer

3.2.4 Carry-Lookahead-Addierer

3.2.1 Halbaddierer

Einfache arithmetische Schaltung zur Addition zweier einstelliger Dualzahlen (Binärziffern, Bits).

Prinzip: Realisierung arithmetischer Funktionen mit Hilfe von Logikschaltungen durch Interpretation der logischen Wahrheitswerte "true" und "false" als binäre Ziffern "1" und "0".

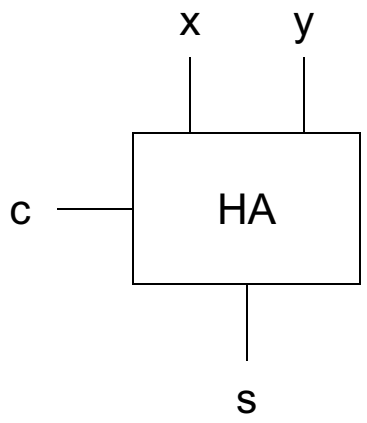
x	y	Summe $_{10}$
0	0	0
0	1	1
1	0	1
1	1	2

Zur Darstellung der Summe sind zwei Binärziffern notwendig

- Summenbit s, Stellenwert $2^0 = 1$
- Übertragsbit (Carry) c, Stellenwert $2^1 = 2$

Arithmetische Gleichung

$$x \cdot 2^0 + y \cdot 2^0 = c \cdot 2^1 + s \cdot 2^0$$



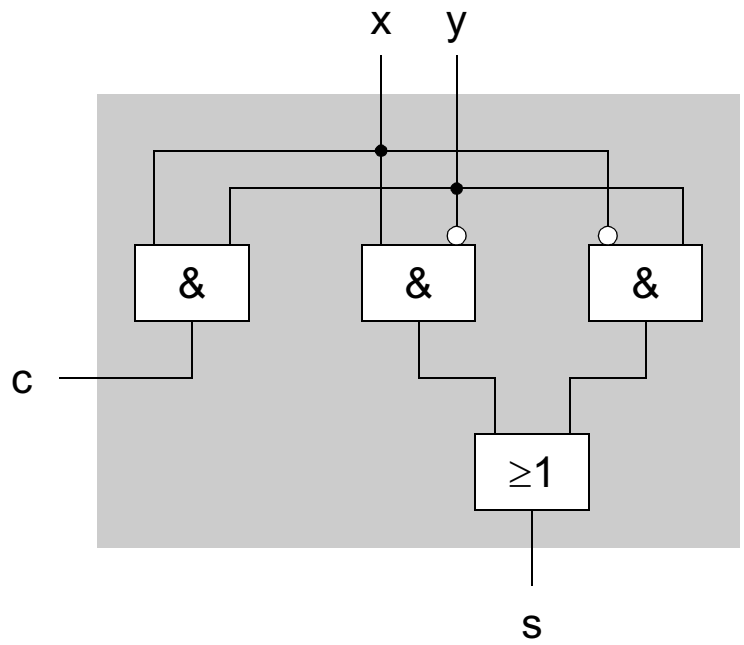
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Logische Gleichungen

$$s = \bar{x} \cdot y + x \cdot \bar{y}$$

$$c = x \cdot y$$

Schaltung



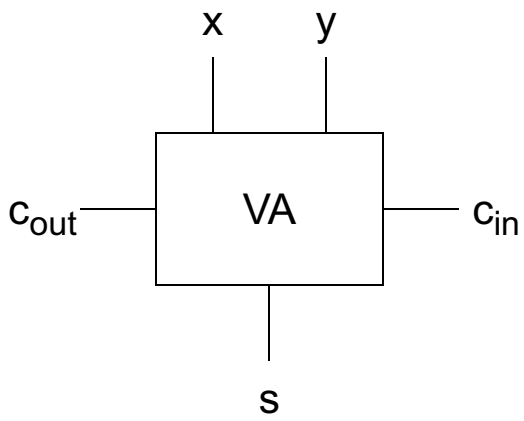
Verzögerungszeit: 2 Gatterlaufzeiten (2τ)

3.2.2 Volladdierer

Zur Addition mehrstelliger Dualzahlen ist es für jede Stelle notwendig, neben den Eingangsbits x und y einen möglicherweise in der nächst niederwertigen Stelle entstandenen Übertrag c_{in} mit zu berücksichtigen.

Beispiel

```
  1011
+ 0011
------
```



x	y	c_{in}	Summe $_{10}$	c_{out} s
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

Logische Gleichungen

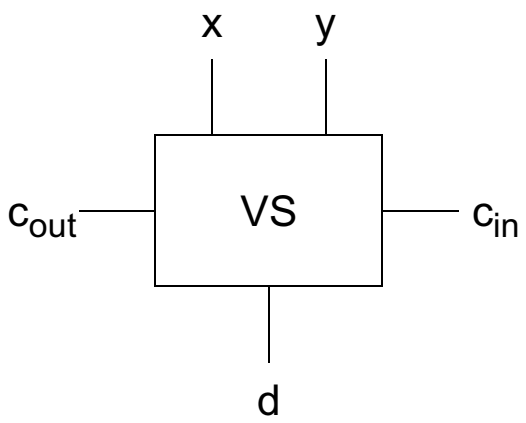
$$s = \overline{x} \cdot \overline{y} \cdot c_{in} + \overline{x} \cdot y \cdot \overline{c_{in}} + x \cdot \overline{y} \cdot \overline{c_{in}} + x \cdot y \cdot c_{in}$$

$$c_{out} = y \cdot c_{in} + x \cdot c_{in} + x \cdot y$$

Analog: Vollsubtrahierer

Beispiel

```
   1010
 - 0011
 ------
```



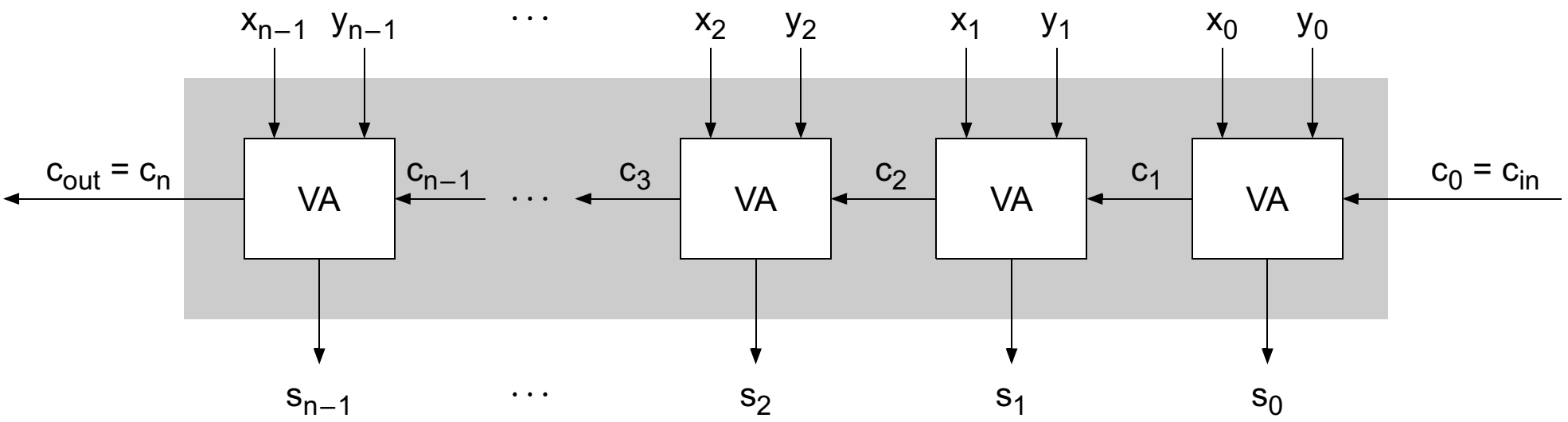
x	y	c_{in}	c_{out}, x	$(y + c_{in})_2$	c_{out} d
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Logische Gleichungen

$$d = \overline{x} \cdot \overline{y} \cdot c_{in} + \overline{x} \cdot y \cdot \overline{c_{in}} + x \cdot \overline{y} \cdot \overline{c_{in}} + x \cdot y \cdot c_{in}$$

$$c_{out} = y \cdot c_{in} + \overline{x} \cdot c_{in} + \overline{x} \cdot y$$

3.2.3 Ripple-Carry-Addierer



Funktionsprinzip

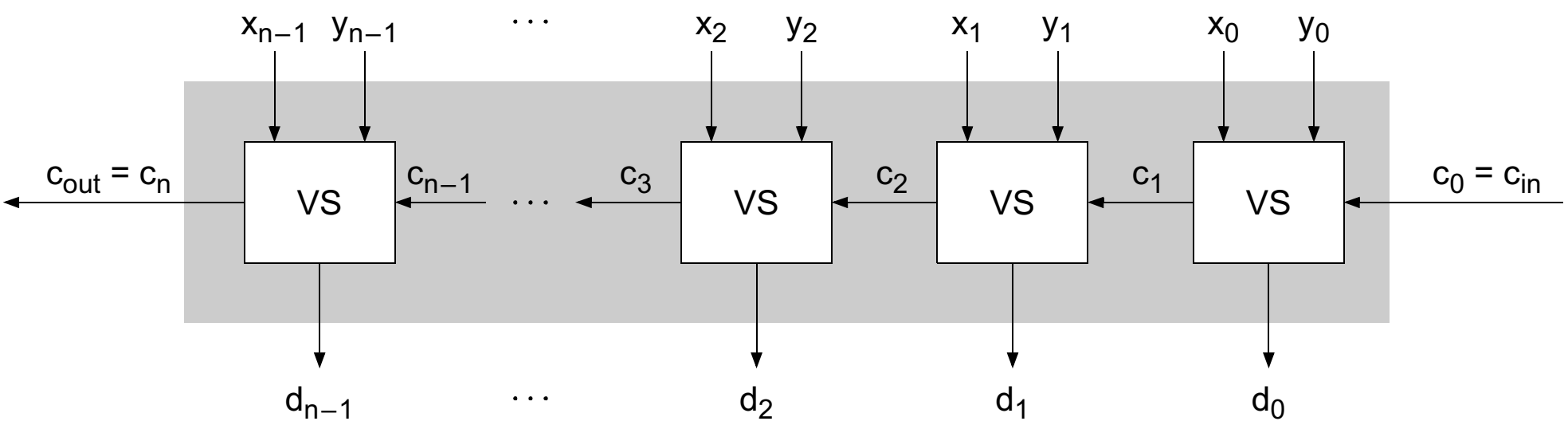
- paralleles (gleichzeitiges) Addieren aller n Stellen
- sukzessive Übertragungskorrektur, im schlimmsten Falle (n – 1) mal

Additionszeit t_{add}

$t_{add,min} = 2\tau$

$t_{add,max} = 2\tau + (n - 1) \cdot 2\,\tau = n \cdot 2\tau$

Analog: Ripple-Carry-Subtrahierer



3.2.4 Carry-Lookahead-Addierer

Idee: Carry nicht von vorausgehendem Volladdierer übernehmen, sondern lokal berechnen

x_i	y_i	c_i	c_{i+1}	s_i	
0	0	0	0	0	
0	0	1	0	1	
0	1	0	0	1	
0	1	1	1	0	$c_{i+1} = c_i$ (*Carry Propagate*)
1	0	0	0	1	
1	0	1	1	0	
1	1	0	1	0	
1	1	1	1	1	$c_{i+1} = 1$ unabhängig von c_i (*Carry Generate*)

Carry Propagate $c_{i+1} = c_i$, wenn $P'_i = x_i \oplus y_i = 1$

Carry Generate $c_{i+1} = 1$, wenn $G_i = x_i \cdot y_i = 1$

Damit $c_{i+1} = G_i + P'_i c_i$

$= x_i y_i + (x_i \oplus y_i) c_i$

$= x_i y_i + (x_i \overline{y_i} + \overline{x_i} y_i) c_i$

$= x_i y_i + x_i \overline{y_i} c_i + x_i y_i c_i + \overline{x_i} y_i c_i + x_i y_i c_i$

$= x_i y_i + x_i c_i + y_i c_i$

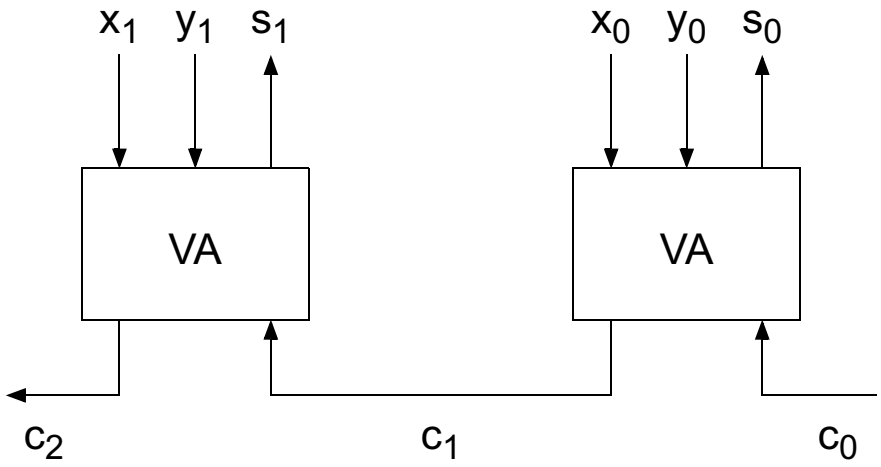
$= x_i y_i + (x_i + y_i) c_i$

$= G_i + P_i c_i$ mit $P_i = x_i + y_i$

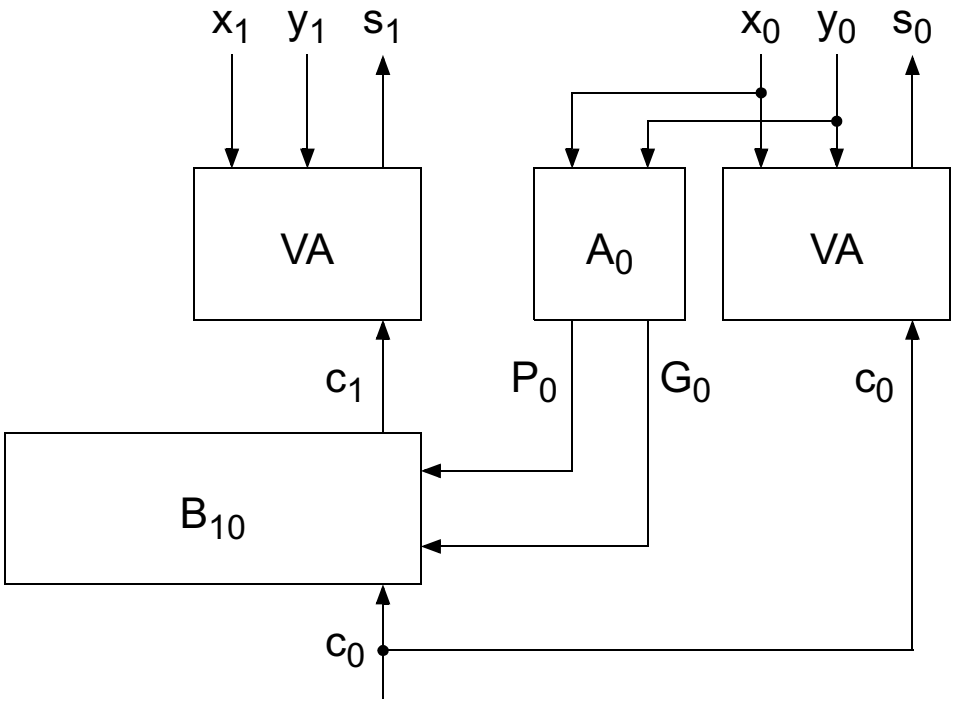
Das aufwändige "Exklusiv-Oder" in P'_i kann also durch ein einfaches "Oder" ersetzt werden:

$c_{i+1} = G_i + P_i c_i$ mit $G_i = x_i y_i$ und $P_i = x_i + y_i$

Ausgangspunkt: Ripple-Carry Addierer für n = 2 Stellen



Carry-Kette unterbrechen, c_1 aus c_0, P_0 und G_0 in B_{10} lokal berechnen



B_{10} realisiert Funktion

$$c_1 = G_0 + P_0 c_0$$

B_{10} um Ausgänge P_{10} und G_{10} ergänzen, um 2-Bit-Addierer zu 4-Bit-Addierer erweitern zu können:

$P_{10} = 1$, wenn $c_2 = c_0$

$G_{10} = 1$, wenn $c_2 = 1$ unabhängig von c_0

P_{10} wird genau dann aktiv, wenn Stelle 0 und Stelle 1 den Übertrag c_0 propagieren:

$$P_{10} = P_1 P_0$$

G_{10} wird aktiv, wenn Stelle 1 unabhängig von c_1 einen Übertrag erzeugt oder Stelle 0 unabhängig von c_0 einen Übertrag erzeugt und Stelle 1 diesen Übertrag propagiert:

$$G_{10} = G_1 + P_1 G_0$$

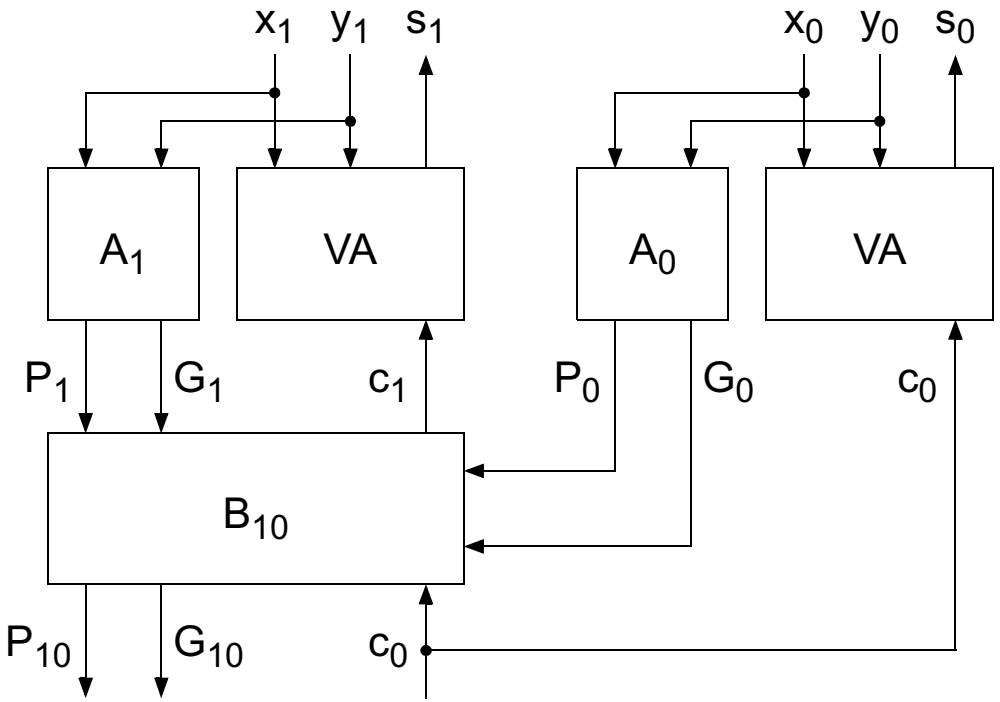
B_{10} realisiert also Funktionen

$$c_1 = G_0 + P_0 c_0$$

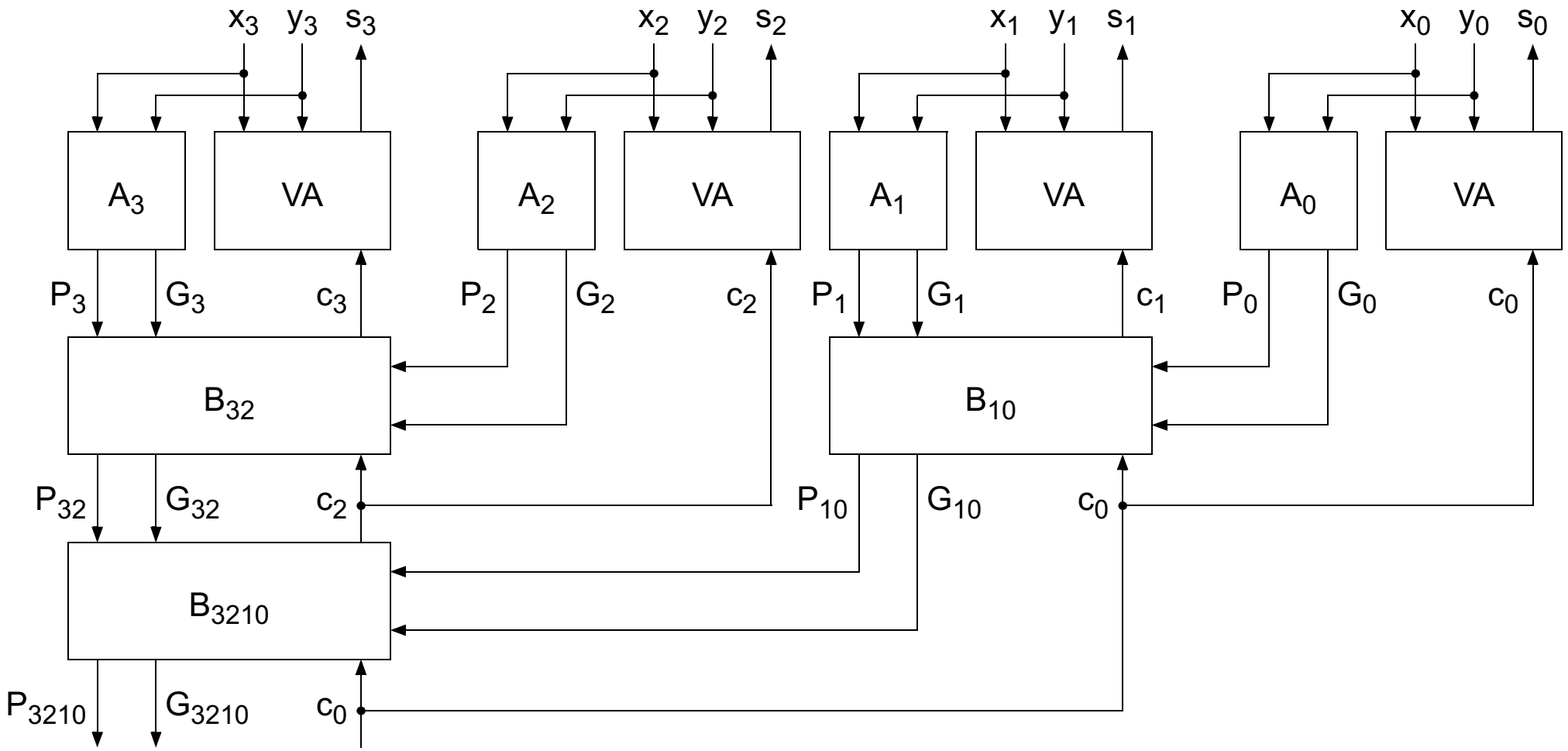
$$P_{10} = P_1 P_0$$

$$G_{10} = G_1 + P_1 G_0$$

Resultierende Schaltung



Erweiterung auf 4 Bit



B_{3210} realisiert Funktionen

$c_2 = G_{10} + P_{10}c_0$

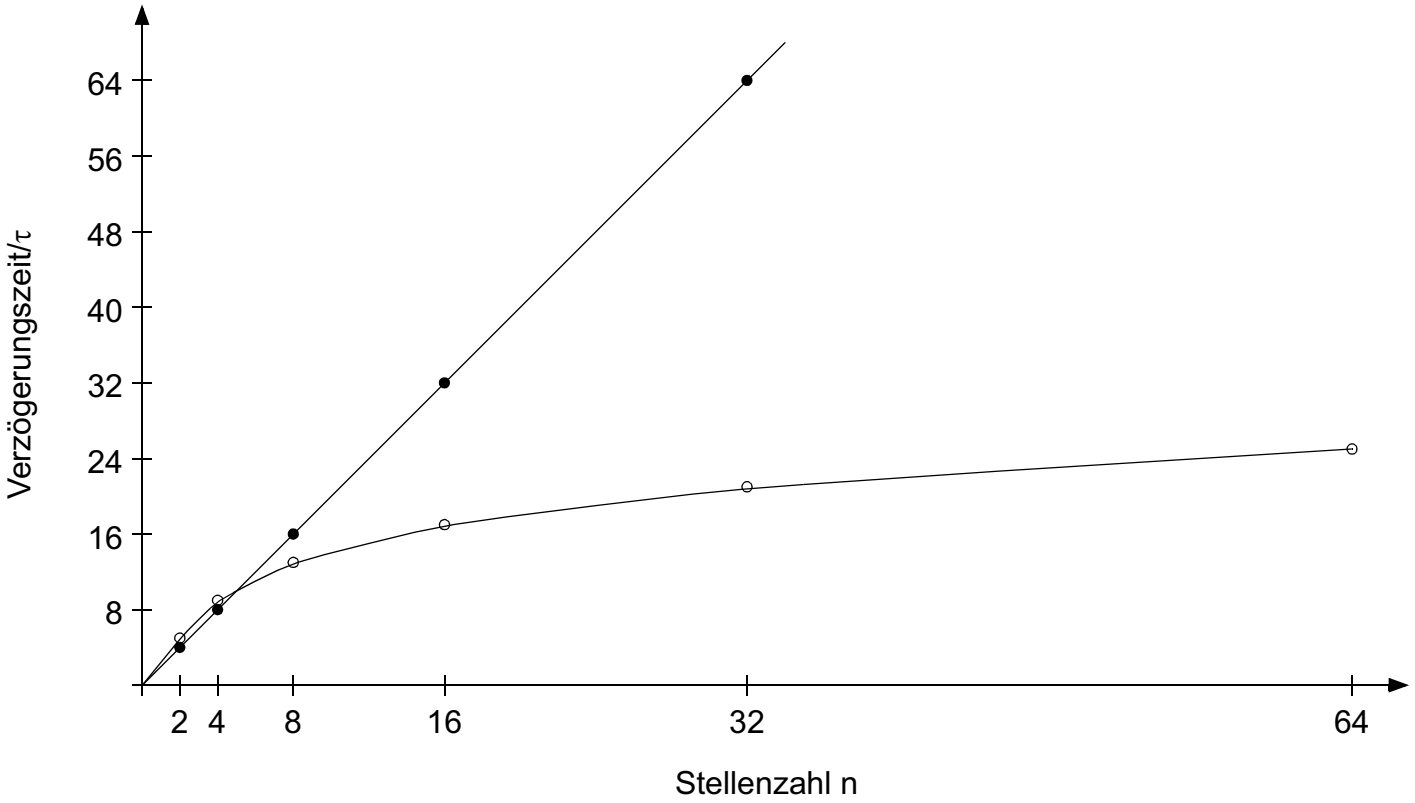
$P_{3210} = P_{32}P_{10}$

$G_{3210} = G_{32} + P_{32}G_{10}$

Identisch mit den entsprechenden Funktionen in B_{10} bzw. B_{32}!

Analyse

Stellenzahl	Ripple-Carry	Carry-Lookahead
2	4τ	
4	8τ	
8	16τ	
16	32τ	
...	...	
n	$2n\tau$	



Verzögerungszeit Ripple-Carry-Addierer wächst linear mit Stellenzahl n

Verzögerungszeit Carry-Lookahead-Addierer wächst nur logarithmisch mit Stellenzahl n

Aber: Hardware-Aufwand für Carry-Lookahead-Addierer höher!

3.3 Arithmetisch-Logische Einheit (ALU)

Inhaltsübersicht

3.3.1 Arithmetische Einheit

3.3.2 Arithmetisch-Logische Einheit (ALU)

3.3.1 Arithmetische Einheit

Einstellige Logikfunktionen

Einstellige Logikfunktion: Hängt nur von einer Booleschen Variablen ab.

Allgemeine einstellige Logikfunktion $\psi(x)$

x	$\psi(x)$
0	b
1	a

$\psi(x) = b \cdot \overline{x} + a \cdot x$, a, b: boolesche Parameter

$\rightarrow$ vier verschiedene einstellige Logikfunktionen $\psi(x)$ möglich

a = 0, b = 0

x	$\psi(x)$
0	0
1	0

Nullfunktion

a = 0, b = 1

x	$\psi(x)$
0	1
1	0

Inversion

a = 1, b = 0

x	$\psi(x)$
0	0
1	1

Identität

a = 1, b = 1

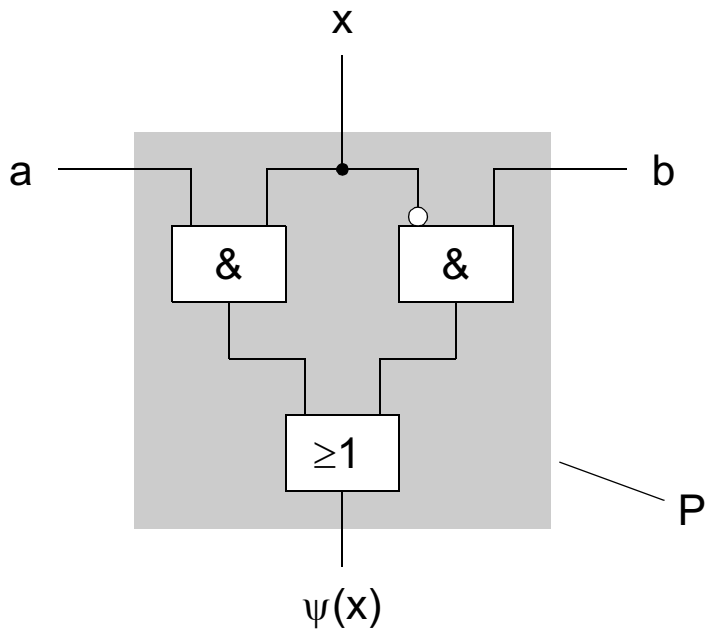
x	$\psi(x)$
0	1
1	1

Einsfunktion

Gesucht: Schaltung, die in Abhängigkeit der Parameter a und b eine der vier möglichen einstelligen Logikfunktionen realisiert (parametrische Schaltung P)

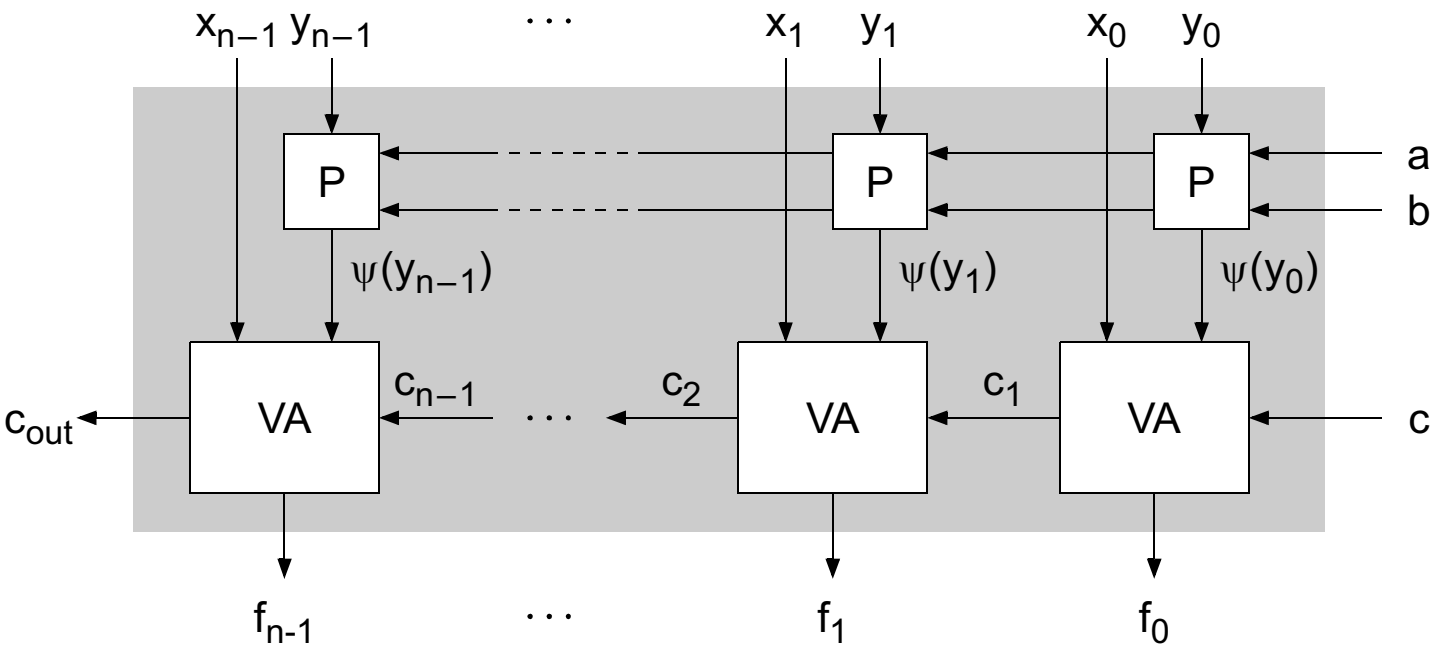
a	b	$\psi(x)$
0	0	0
0	1	$\bar{x}$
1	0	x
1	1	1

$$\psi(x) = \bar{a}\bar{b}x + a\bar{b}x + ab$$
$$= \bar{a}\bar{b}x + a\bar{b}x + abx + ab\bar{x}$$
$$= ax + b\bar{x}$$



Arithmetische Einheit (AU)

Erweiterung des Ripple-Carry-Addierers durch je eine parametrische Schaltung pro y-Eingang



Analyse

a	b	c	$\psi(y_i)$				arithm. Interpretation
0	0	0	0	$x_{n-1} \ldots x_0$	$+ 0 \ldots 0$	$+ 0$	x
0	0	1	0	$x_{n-1} \ldots x_0$	$+ 0 \ldots 0$	$+ 1$	$x + 1$
0	1	0	$\overline{y}_i$	$x_{n-1} \ldots x_0$	$+ \overline{y}_{n-1} \ldots \overline{y}_0$	$+ 0$	$x - y - 1$
0	1	1	$\overline{y}_i$	$x_{n-1} \ldots x_0$	$+ \overline{y}_{n-1} \ldots \overline{y}_0$	$+ 1$	$x - y$
1	0	0	y_i	$x_{n-1} \ldots x_0$	$+ y_{n-1} \ldots y_0$	$+ 0$	$x + y$
1	0	1	y_i	$x_{n-1} \ldots x_0$	$+ y_{n-1} \ldots y_0$	$+ 1$	$x + y + 1$
1	1	0	1	$x_{n-1} \ldots x_0$	$+ 1 \ldots 1$	$+ 0$	$x - 1$
1	1	1	1	$x_{n-1} \ldots x_0$	$+ 1 \ldots 1$	$+ 1$	x

Erläuterungen

$x_{n-1} \ldots x_0$	Binärdarstellung der Zahl x
$y_{n-1} \ldots y_0$	Binärdarstellung der Zahl y
$\overline{y}_{n-1} \ldots \overline{y}_0$	Einerkomplement von y, entspricht $-y - 1$
$\overline{y}_{n-1} \ldots \overline{y}_0 + 1$	Zweierkomplement von y, entspricht $-y$

→ Arithmetische Einheit realisiert die arithm. Funktionen Addition, Subtraktion, Inkrement, Dekrement

3.3.2 Arithmetisch-Logische Einheit (ALU)

Weitere Steuerleitung

d = 1: Arithmetische Funktionen (Addition, Subtraktion, Inkrement, Dekrement)

d = 0: Logische Funktionen (bitweises UND, ODER, EXKLUSIV-ODER, NICHT)

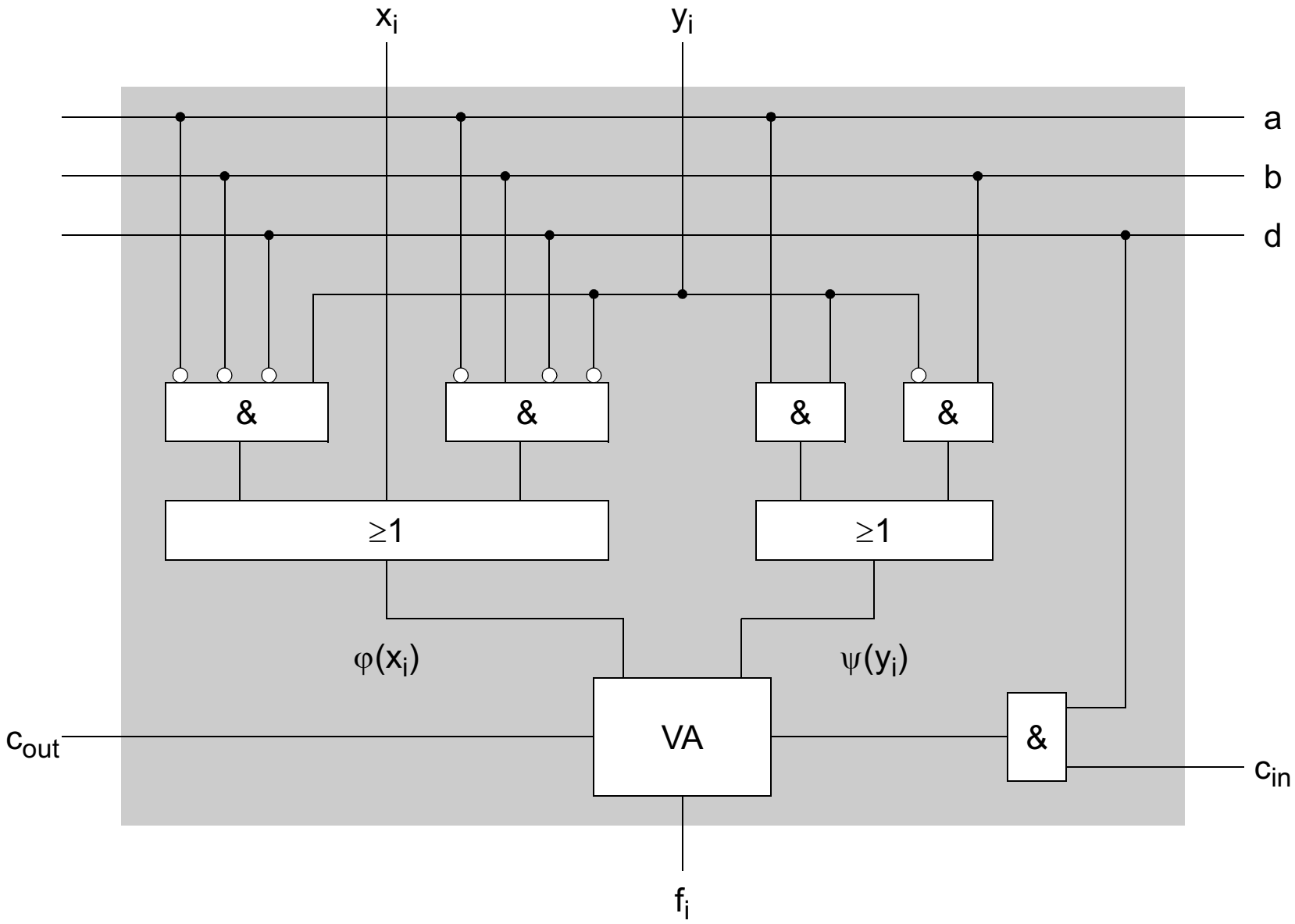
Erweiterungen der Arithmetischen Einheit um

– eine geeignete parametrische Schaltung $\varphi(x)$ vor dem x-Eingang

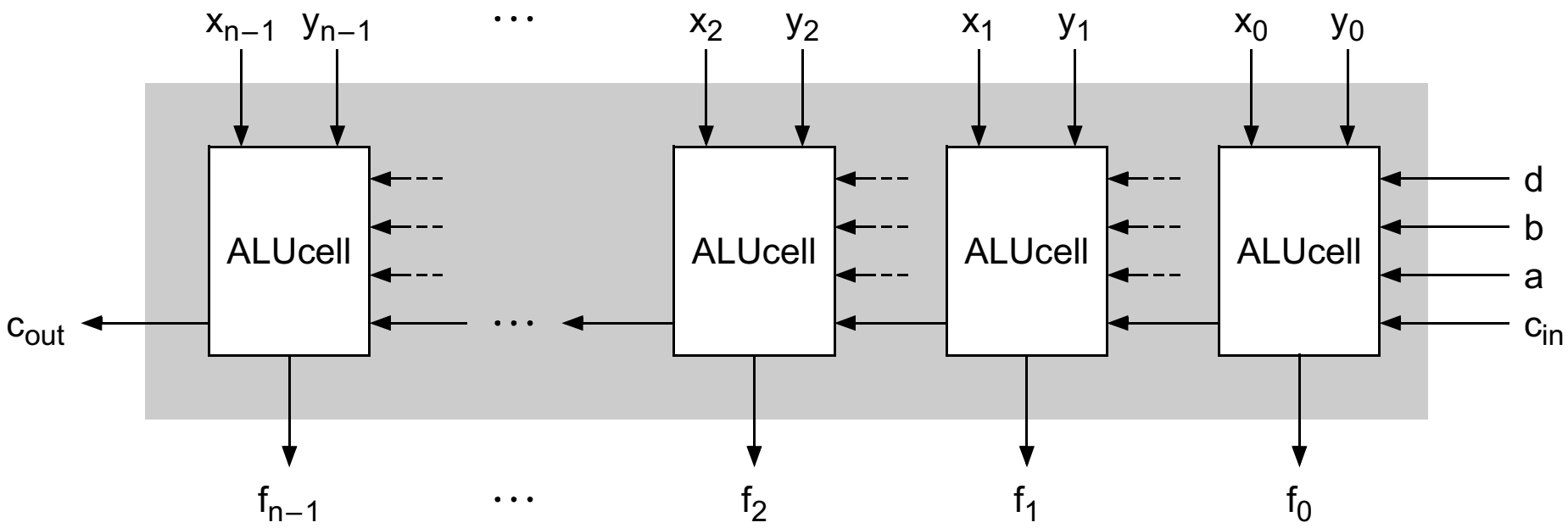
– ein UND-Gatter zur Unterbrechung der Carry-Kette bei logischen Funktionen (d = 0)

	d	b	a	Funktion
logische Funktionen	0	0	0	x OR y
	0	0	1	x XOR y
	0	1	0	x AND y
	0	1	1	NOT x
arithmetische Funktionen	1	0	0	x + c
	1	0	1	x + y + c
	1	1	0	$x - y - \overline{c}$
	1	1	1	$x - \overline{c}$

Schaltung einer ALU-Zelle (für ein Bit, ohne Herleitung)



ALU für n-stellige Operanden



3.4 Multiplizier- und Dividierwerke

Inhaltsübersicht

3.4.1 Multiplizierwerke

I. Vorzeichenlose Multiplikation

II. N-schrittiges Multiplizierwerk für vorzeichenlose Zahlen

III. Einschrittiges Parallel-Multiplizierwerk für vorzeichenlose Zahlen

IV. Vorzeichenbehaftete Multiplikation

V. N-schrittiges Multiplizierwerk für vorzeichenbehaftete Zahlen

VI. Einschrittiges Parallel-Multiplizierwerk für vorzeichenbehaftete Zahlen

3.4.2 Dividierwerke

I. Vorzeichenlose Division

II. N-schrittiges Dividierwerk für vorzeichenlose Zahlen

III. Vorzeichenbehaftete Division

IV. N-schrittiges Dividierwerk für vorzeichenbehaftete Zahlen

3.4.1 Multiplizierwerke

Dank des Stellenwertprinzips kann die Multiplikation mehrstelliger Operanden auf die Addition entsprechend der Stellenwertigkeit verschobener Teilprodukte zurückgeführt werden, die aus dem Multiplikanden und den einzelnen Stellen des Multiplikators gebildet werden.

Speziell bei der Multiplikation von Binärzahlen gestaltet sich die Berechnung der Teilprodukte als trivial, da die Multiplikatorstellen nur 0 oder 1 sind.

I. Vorzeichenlose Multiplikation

Beispiel (Dezimalsystem)

Beispiel (Dualsystem)

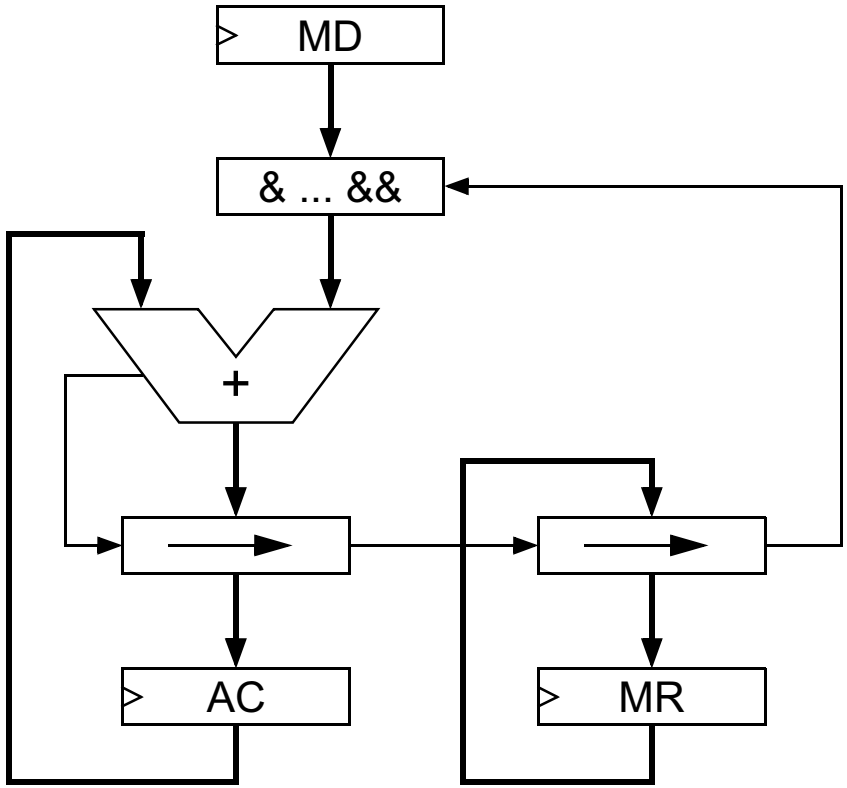
Merkmale

- die stellenrichtige Addition wird durch das entsprechende Linksschieben der Teilprodukte erreicht
- die Stellenzahl des Produktes ergibt sich aus der Summe der Stellenzahl der Faktoren

Die Multiplikation kann auf unterschiedliche Weisen realisiert werden, die sich im Grad der Parallelverarbeitung unterscheiden.

II. *N*-schrittiges Multiplizierwerk für vorzeichenlose Zahlen

Schaltung (auf Register-Transfer-Ebene)



Bezeichnungen

MD	Register für Multiplikand	*AC*	Akkumulator-Register
MR	Register für Multiplikator	*N*	Stellenzahl der Operanden

Funktionsweise

- Inhalt der Register vor Beginn der Multiplikation

  MD Multiplikand

  MR Multiplikator

  AC gelöscht, d.h. 0 ... 0

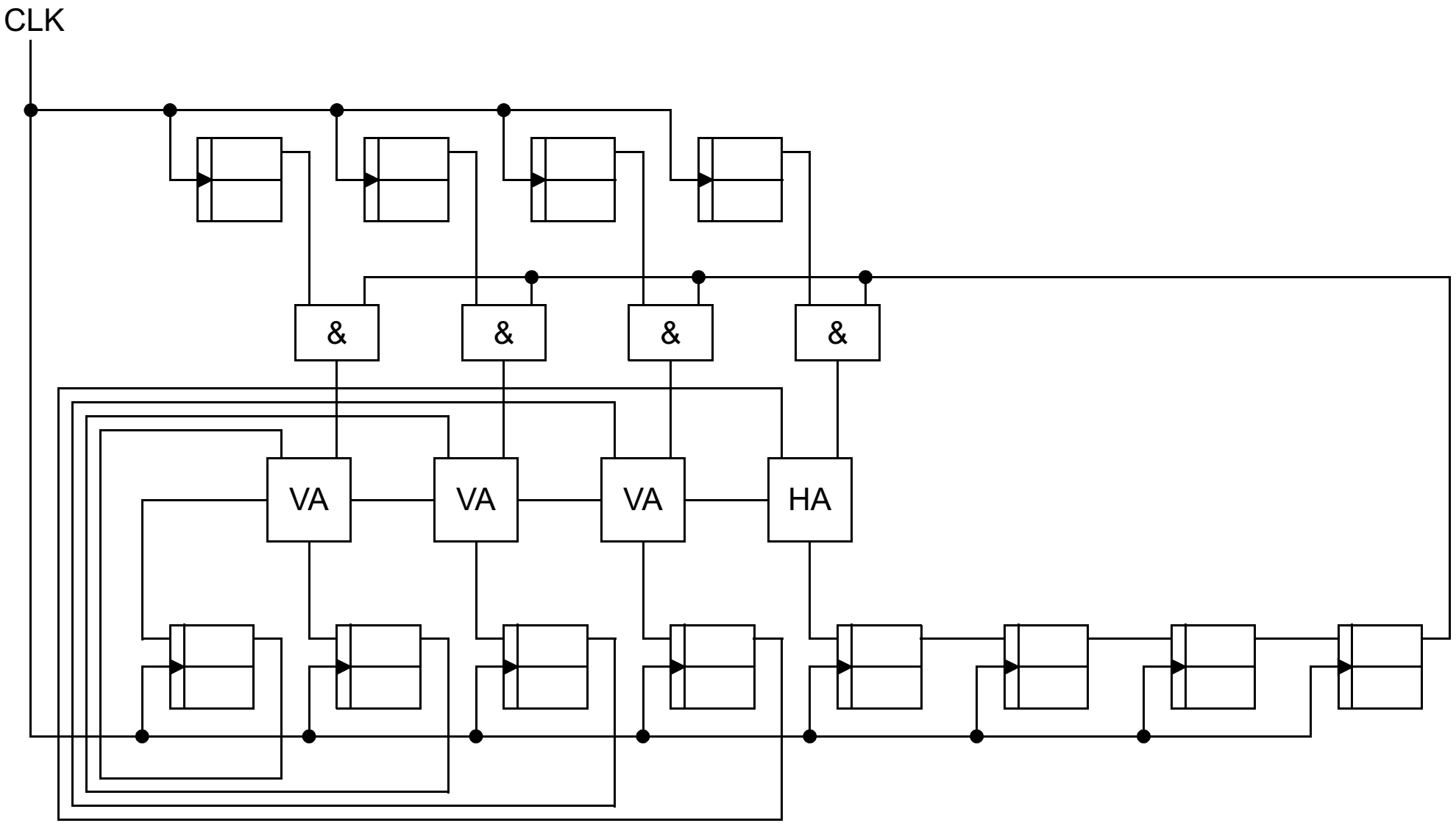
- Multiplikation erfolgt in *N* Schritten bzw. *N* Taktzyklen
- Inhalt der Register nach Ende der Multiplikation

  MD Multiplikand (unverändert)

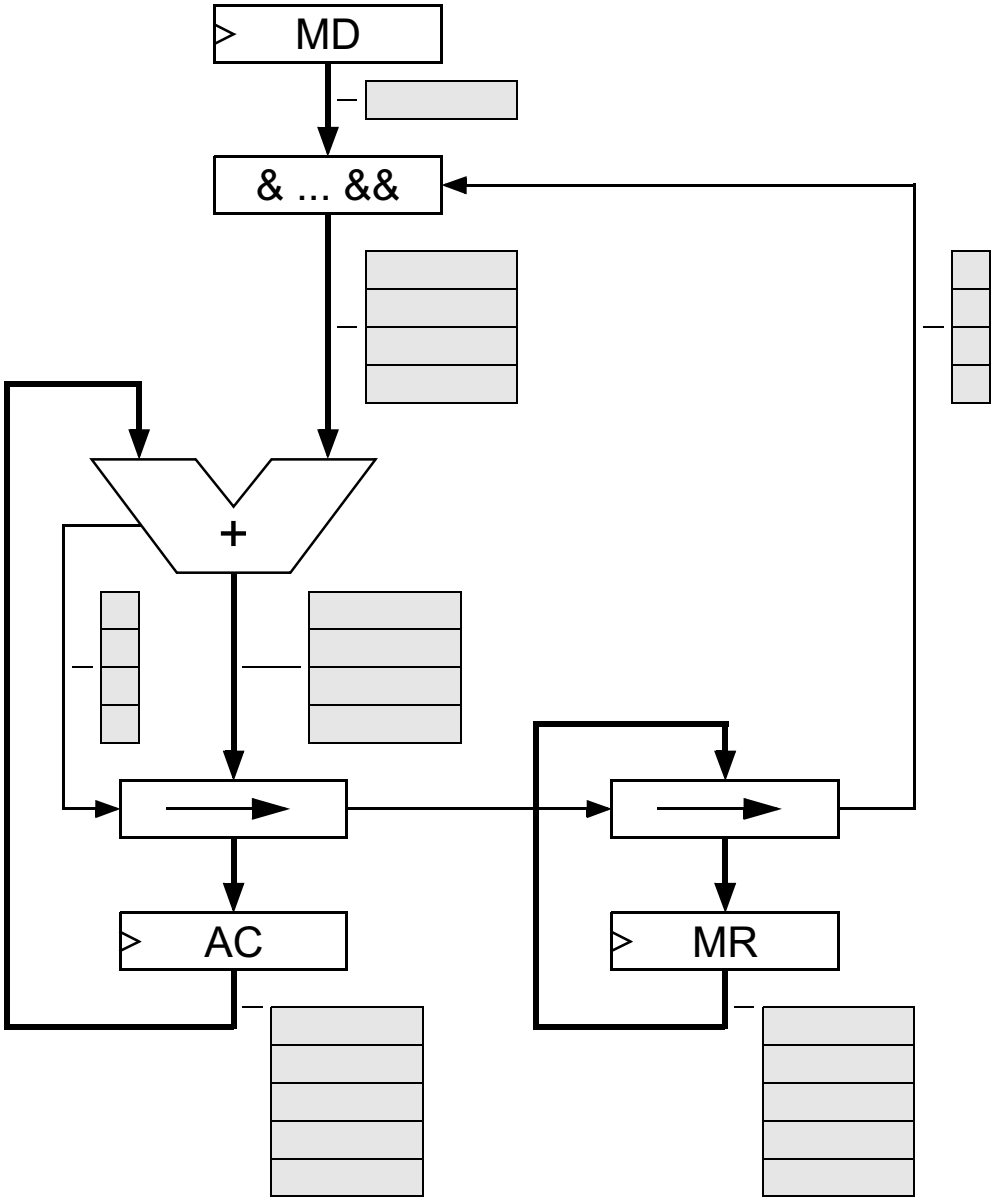
  MR niederwertige Hälfte des 2*N* bit breiten Produktes

  AC höherwertige Hälfte des 2*N* bit breiten Produktes

Entsprechende Schaltung auf Gatter-Ebene (für N = 4)



Beispiel



III. Einschrittiges Parallel-Multiplizierwerk für vorzeichenlose Zahlen

Ziel

Multiplikation realisieren durch rein kombinatorisches Netzwerk

Ansatz

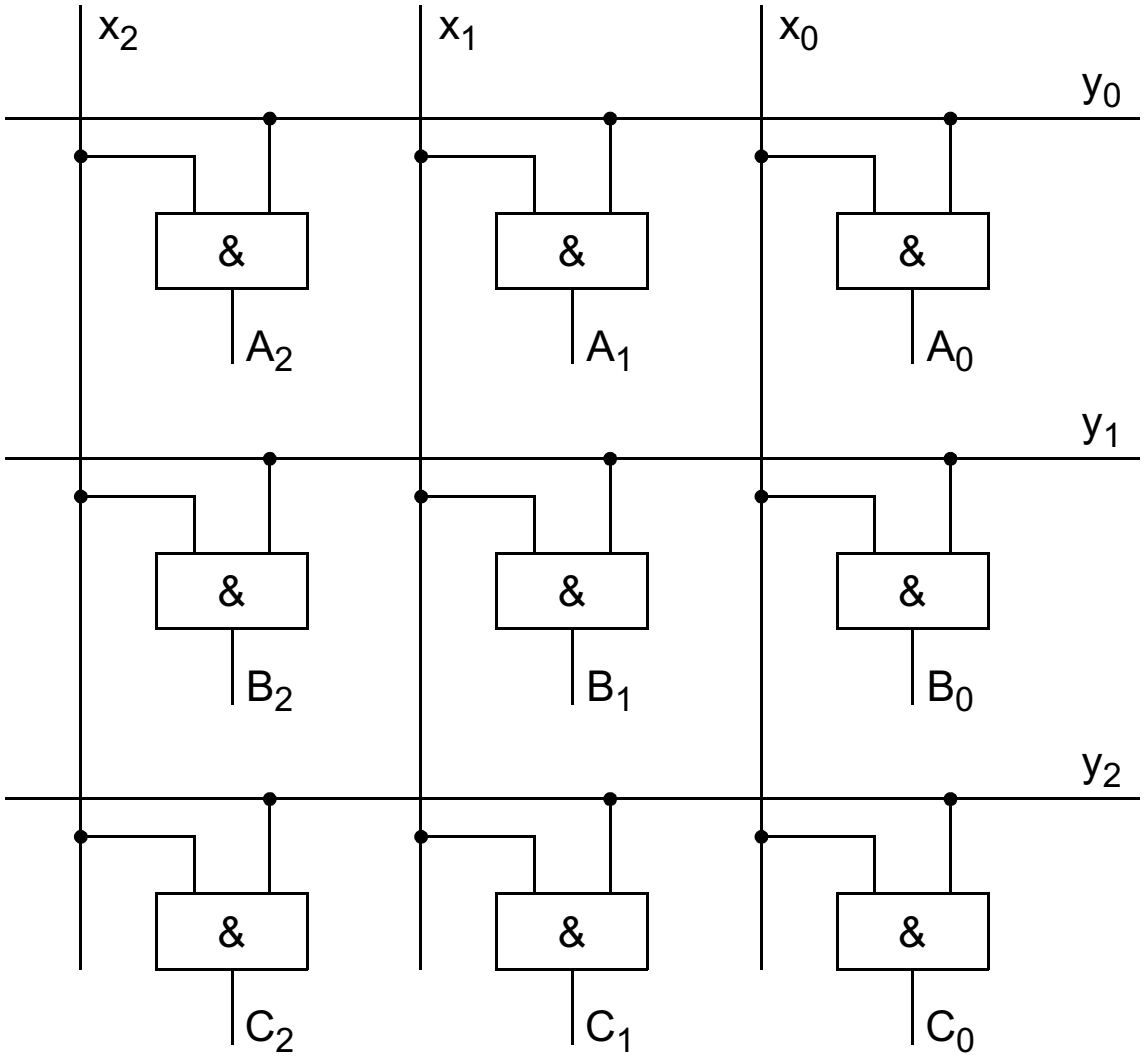
Beispiel: Multiplikation zweier 3-stelliger Binärzahlen $X = x_2\, x_1\, x_0$, $Y = y_2\, y_1\, y_0$

$$
\begin{array}{cccccccc}
x_2 & x_1 & x_0 & \cdot & y_2 & y_1 & y_0 & \\
\hline
 & & & & \underbrace{x_2y_0}_{A_2} & \underbrace{x_1y_0}_{A_1} & \underbrace{x_0y_0}_{A_0} & \\
 & & & \underbrace{x_2y_1}_{B_2} & \underbrace{x_1y_1}_{B_1} & \underbrace{x_0y_1}_{B_0} & & \\
 & & \underbrace{x_2y_2}_{C_2} & \underbrace{x_1y_2}_{C_1} & \underbrace{x_0y_2}_{C_0} & & & \\
\hline
 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0 &
\end{array}
$$

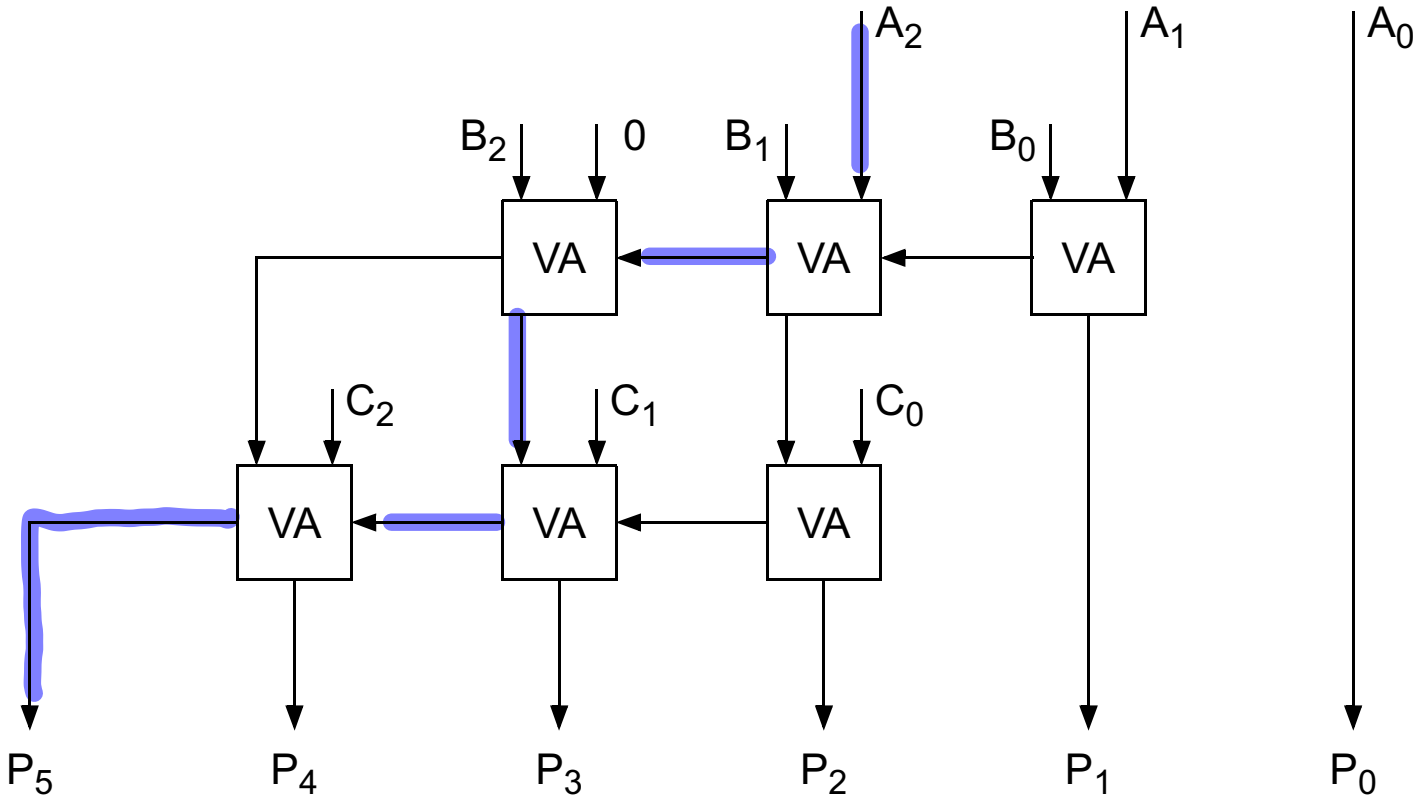
$$
\begin{array}{r}
110 \cdot 101 \\
\hline
110 \\
000 \\
110 \\
\hline
11110
\end{array}
$$

Schaltung

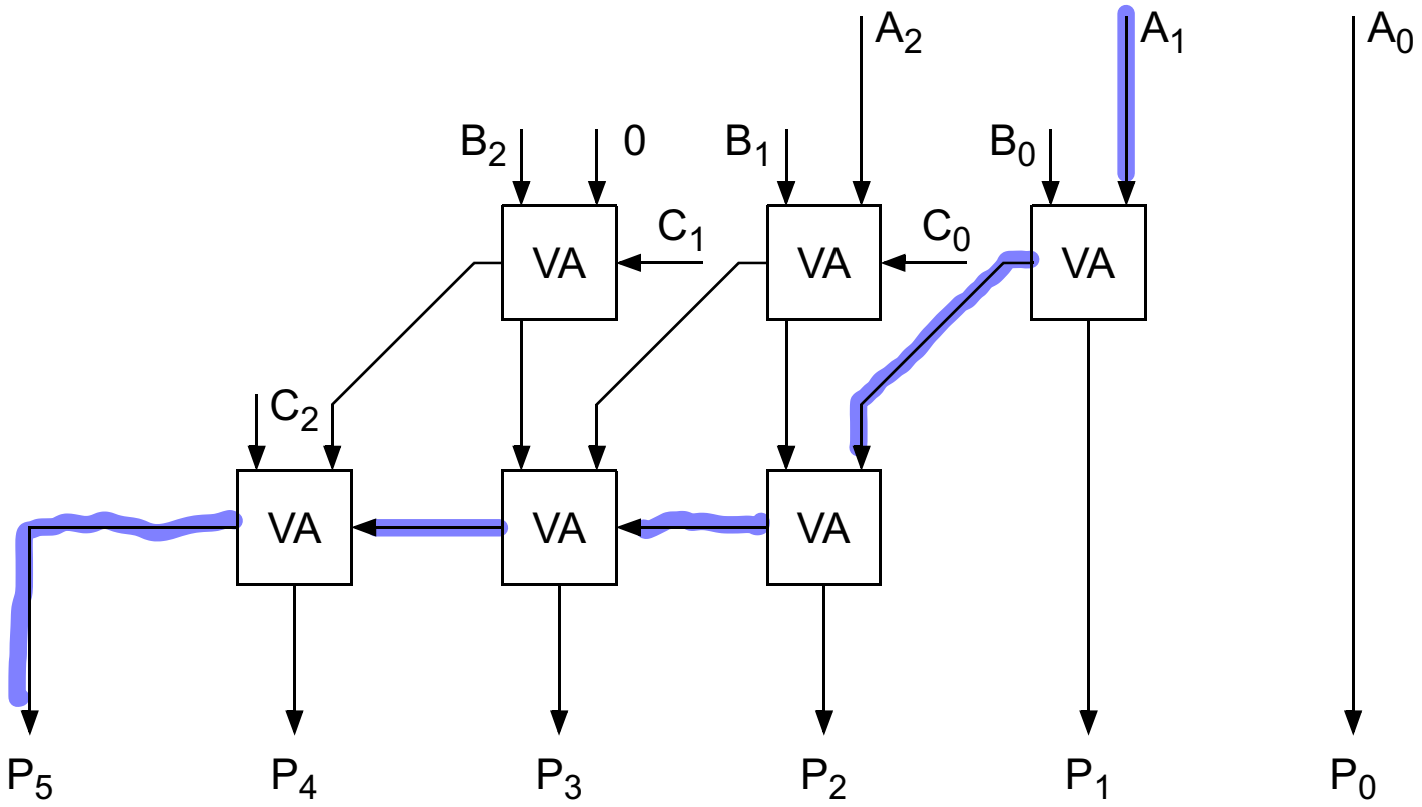
Einzelmultiplikationen



Addition der Teilprodukte



Alternativ: "Carry Save"-Verfahren zur Addition der Teilprodukte



IV. Vorzeichenbehaftete Multiplikation

Die bisher vorgestellten Multiplikationswerke arbeiten nur mit vorzeichenlosen Binärzahlen. Nun werde die vorzeichenbehaftete Multiplikation in Zweierkomplementarithmetik betrachtet.

Ansatz

$$p = b \cdot a = b \cdot \left(-a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} a_i \cdot 2^i \right) = -b \cdot a_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b \cdot a_i \cdot 2^i$$

mit Multiplikand *b* und Multiplikator *a*

Interpretation

- Teilprodukte $b \cdot a_i$ sind vorzeichenbehaftet, müssen vor Addition vorzeichenrichtig erweitert werden
- höchstwertiges Teilprodukt $b \cdot a_{n-1}$ muss negiert werden

Beispiele

```
0 0 1 1 · 0 1 0 1
```

```
1 1 0 1 · 0 1 0 1
```

```
0 0 1 1 · 1 0 1 1
```

```
1 1 0 1 · 1 0 1 1
```

Beispiele

```
  0 0 1 1 · 0 1 0 1
  ---------------
    0 0 0 0 0 0 1 1
    0 0 0 0 0 0 0
    0 0 0 0 1 1
    0 0 0 0 0
  ---------------
    0 0 0 0 1 1 1 1
```

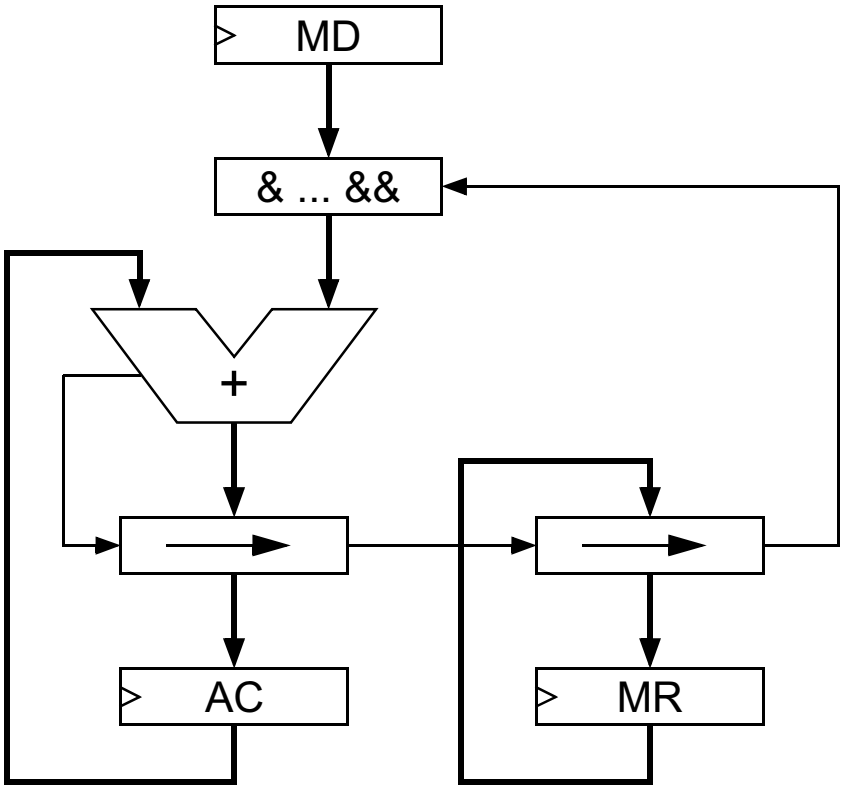
```
  1 1 0 1 · 0 1 0 1
  ---------------
    1 1 1 1 1 1 0 1
    0 0 0 0 0 0 0
    1 1 1 1 0 1
    0 0 0 0 0
  ---------------
    1 1 1 1 0 0 0 1
```

```
  0 0 1 1 · 1 0 1 1
  ---------------
    0 0 0 0 0 0 1 1
    0 0 0 0 0 1 1
    0 0 0 0 0 0
    1 1 1 0 1
  ---------------
    1 1 1 1 0 0 0 1
```

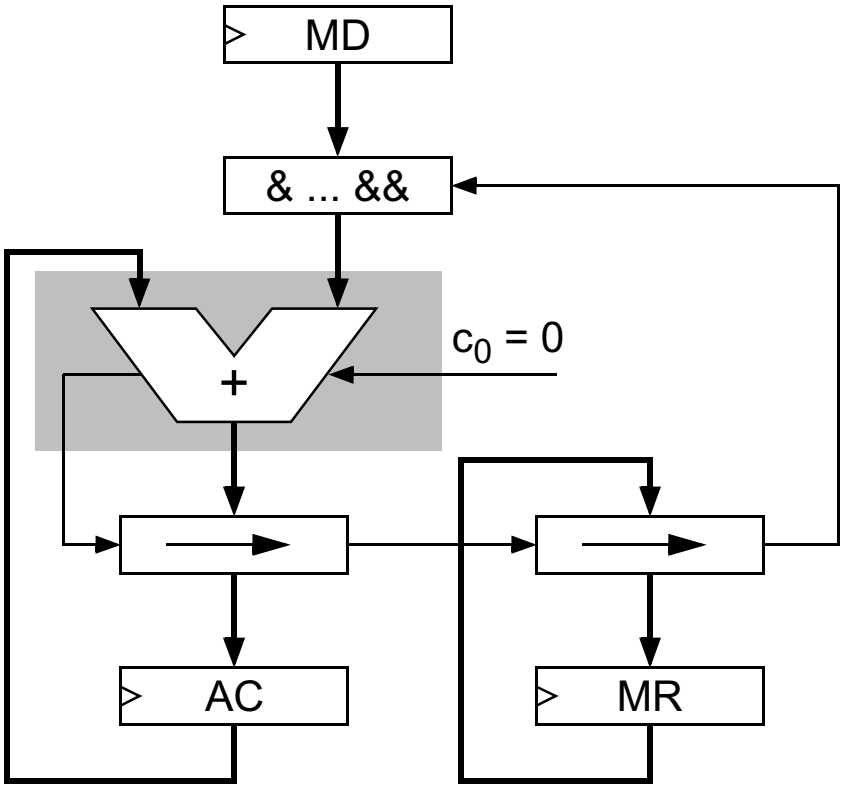
```
  1 1 0 1 · 1 0 1 1
  ---------------
    1 1 1 1 1 1 0 1
    1 1 1 1 1 0 1
    0 0 0 0 0 0
    0 0 0 1 1
  ---------------
    0 0 0 0 1 1 1 1
```

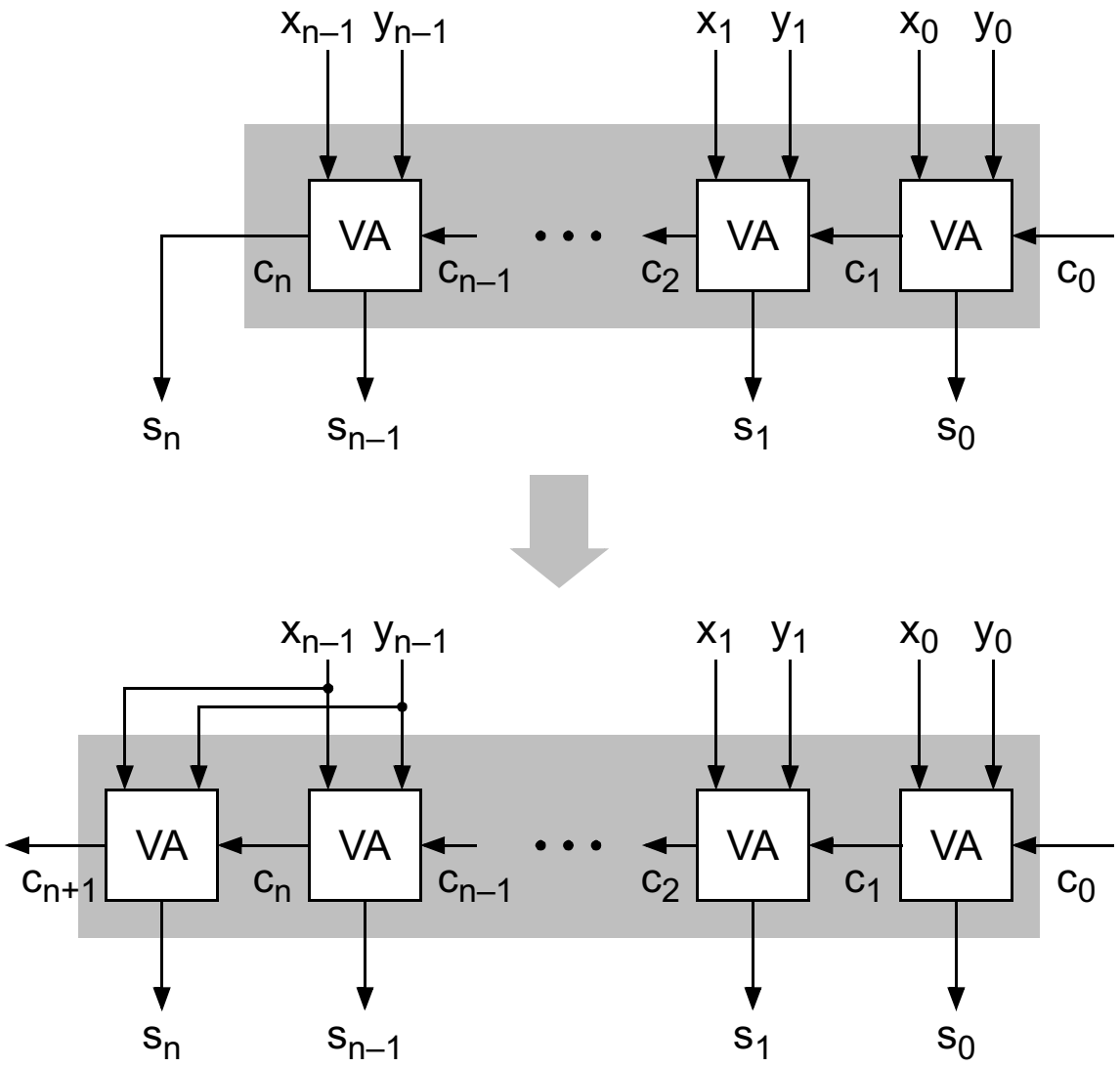
V. *N*-schrittiges Multiplizierwerk für vorzeichenbehaftete Zahlen

Ausgangspunkt: Schaltung für vorzeichenlose Multiplikation

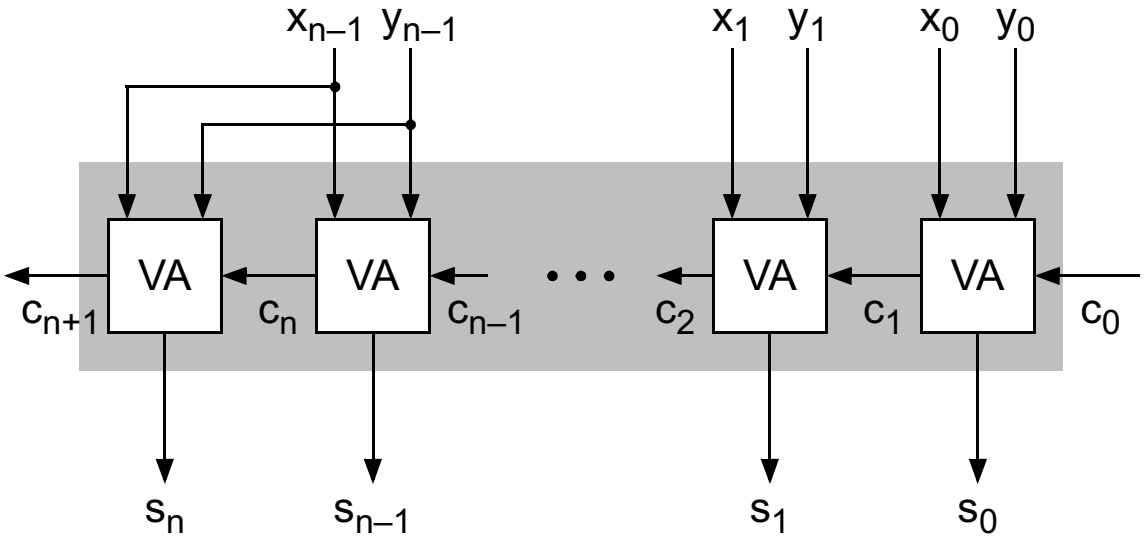


Schritt 1: Vorzeichenrichtige Erweiterung der Teilprodukte





Vereinfachung: Ausgangspunkt



$C = c_{n+1}$ kann ignoriert werden

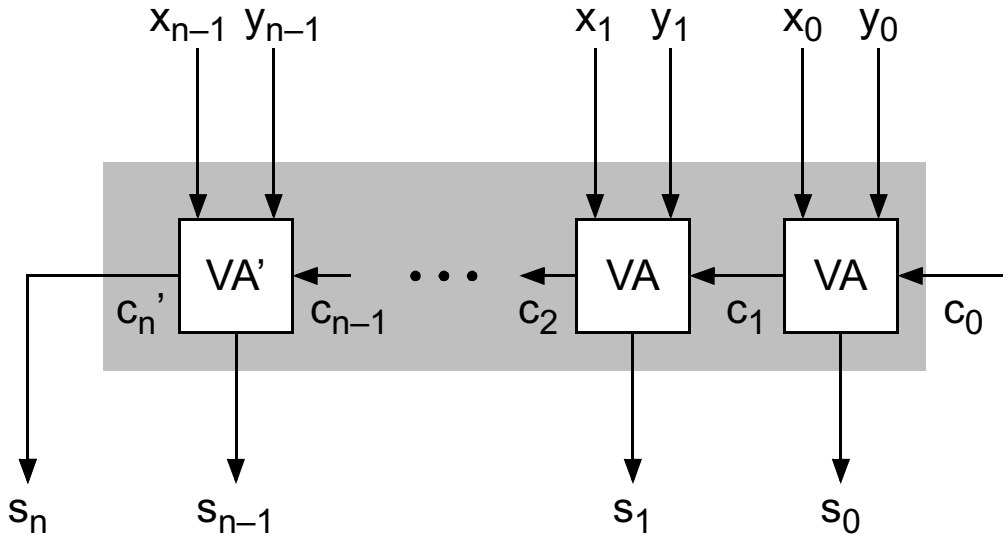
$V = c_{n+1} \oplus c_n$ ist immer Null, es kann kein Überlauf auftreten (ohne Beweis)

$s_n = x_{n-1} \oplus y_{n-1} \oplus c_n$

$c_n = x_{n-1}y_{n-1} + x_{n-1}c_{n-1} + y_{n-1}c_{n-1}$

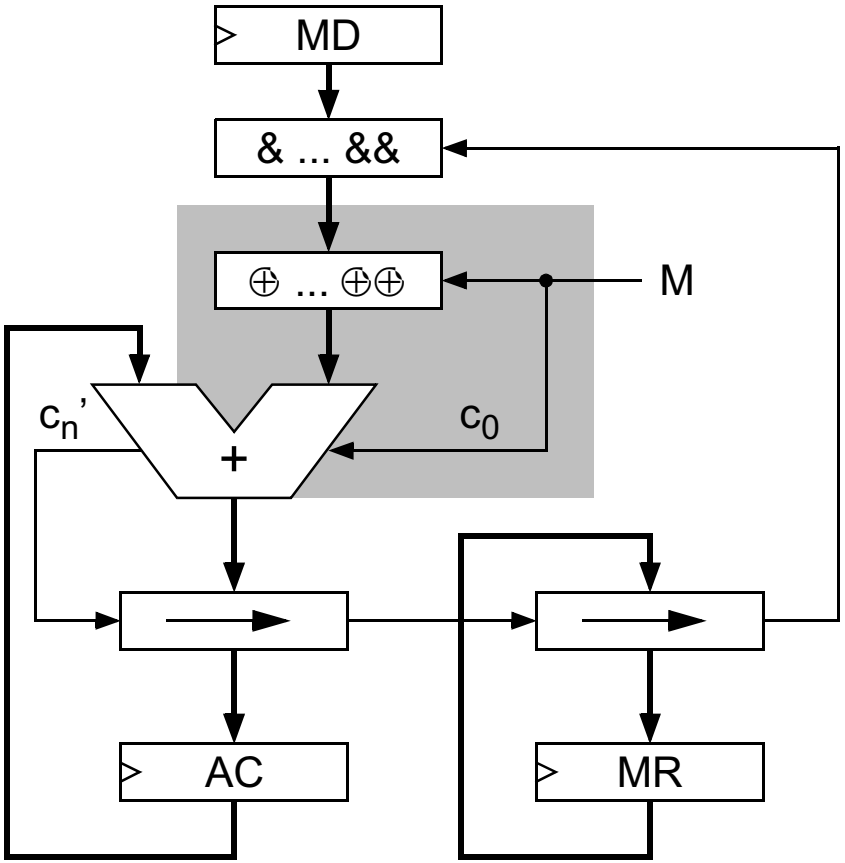
$s_n = x_{n-1} \oplus y_{n-1} \oplus (x_{n-1}y_{n-1} + x_{n-1}c_{n-1} + y_{n-1}c_{n-1}) = x_{n-1}y_{n-1} + x_{n-1}\overline{c}_{n-1} + y_{n-1}\overline{c}_{n-1} = c_n'$ (ohne Beweis)

Vereinfachung: Ergebnis



VA' invertiert c_{n-1} zur Ermittlung von c_n'

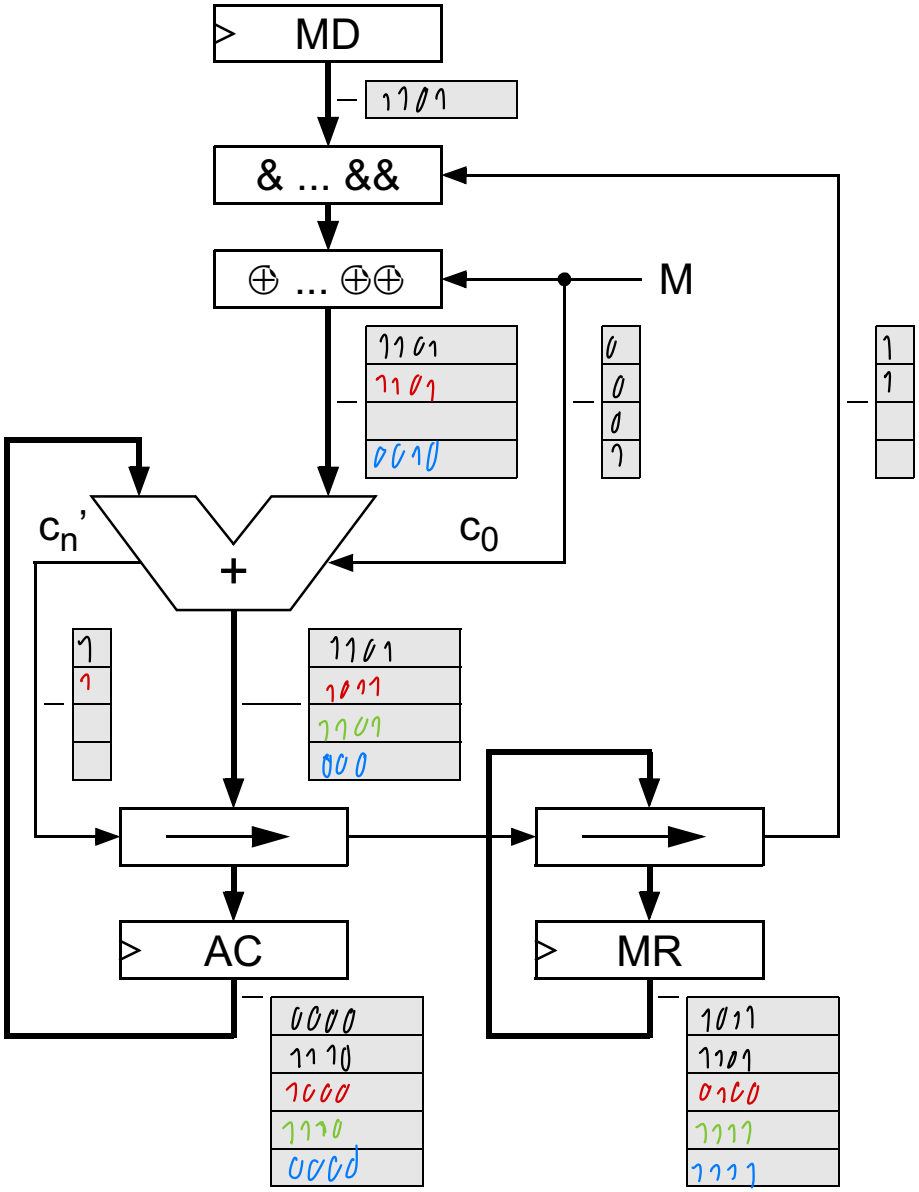
Schritt 2: Negation des höchstwertigen Teilproduktes



Steuerwerk (Zähler) setzt $M = 0$ in den ersten $N - 1$ Takten, $M = 1$ im letzten Takt

- $M = 0$: Addition von *MD* oder 0
- $M = 1$: Subtraktion von *MD* oder 0 (durch Zweierkomplementbildung)

Beispiel



1. Takt:

3. Takt

4. Takt

Nochmal mit Video schauen

VI. Einschrittiges Parallel-Multiplizierwerk für vorzeichenbehaftete Zahlen

Rückblick: vorzeichenlose Multiplikation

	x_2	x_1	x_0	·	y_2	y_1	y_0
+					x_2y_0	x_1y_0	x_0y_0
+				x_2y_1	x_1y_1	x_0y_1	
+			x_2y_2	x_1y_2	x_0y_2		
		P_5	P_4	P_3	P_2	P_1	P_0

Zur besseren Lesbarkeit mit Abkürzungen für die Stellen der Teilprodukte

	x_2	x_1	x_0	·	y_2	y_1	y_0
+					A_2	A_1	A_0
+				B_2	B_1	B_0	
+			C_2	C_1	C_0		
		P_5	P_4	P_3	P_2	P_1	P_0

Vorzeichenbehaftete Multiplikation: Ansatz

	x_2	x_1	x_0	·	y_2	y_1	y_0
+		$\mathbf{A_2}$	$\mathbf{A_2}$	$\mathbf{A_2}$	A_2	A_1	A_0
+		$\mathbf{B_2}$	$\mathbf{B_2}$	B_2	B_1	B_0	
–		$\mathbf{C_2}$	C_2	C_1	C_0		
		P_5	P_4	P_3	P_2	P_1	P_0

Schritt 1: Subtraktion ersetzen durch Addition des Zweierkomplements

	x_2	x_1	x_0	.	y_2	y_1	y_0
+		A_2	A_2	A_2	A_2	A_1	A_0
+		B_2	B_2	B_2	B_1	B_0	
+		$\overline{\mathbf{C_2}}$	$\overline{\mathbf{C_2}}$	$\overline{\mathbf{C_1}}$	$\overline{\mathbf{C_0}}$		
+					**1**		
		P_5	P_4	P_3	P_2	P_1	P_0

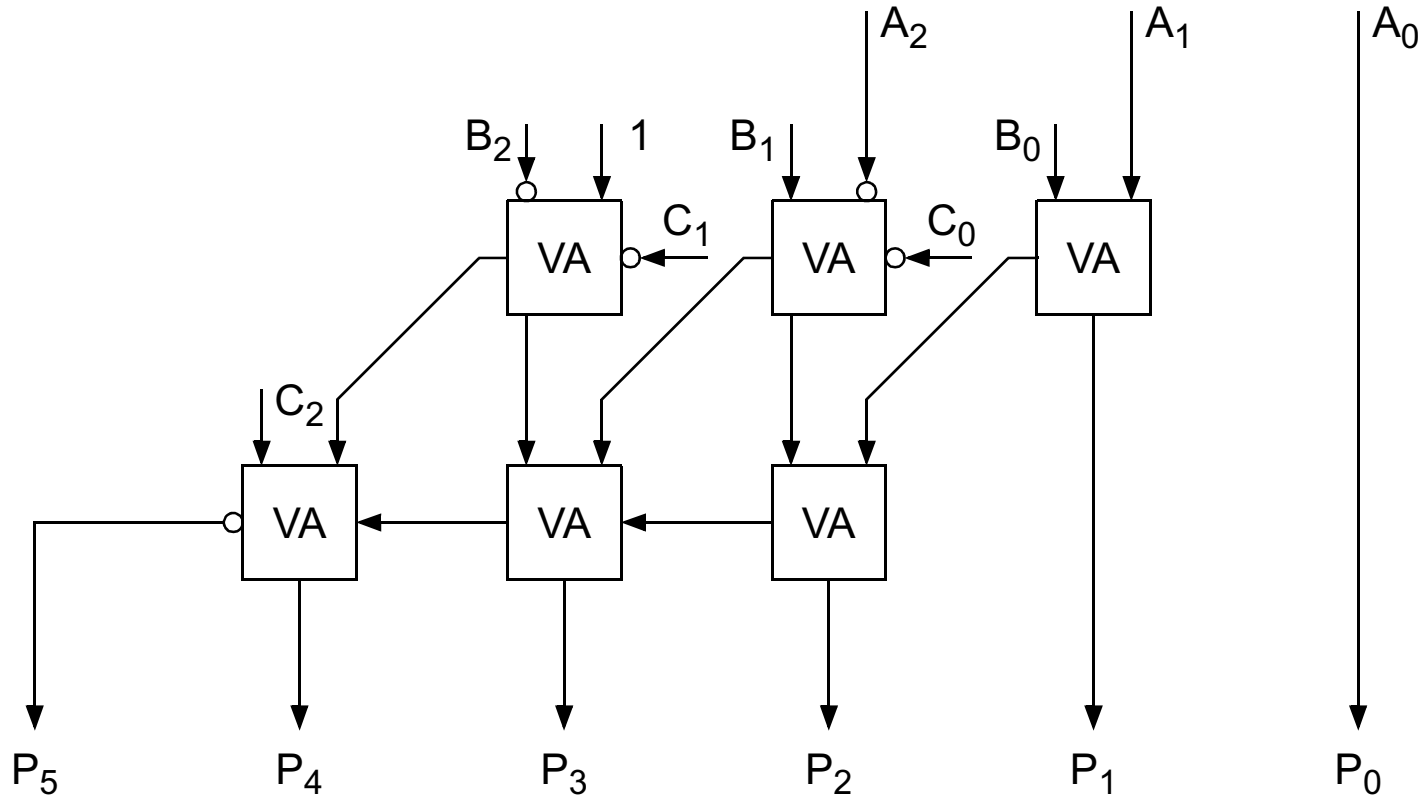
Schritt 2: Addition "geeigneter" Konstanten, die im Anschluss wieder subtrahiert werden

	x_2	x_1	x_0	.	y_2	y_1	y_0	
+		A_2	A_2	A_2	A_2	A_1	A_0	(1)
+					**1**			(2)
+		B_2	B_2	B_2	B_1	B_0		(3)
+				**1**				(4)
+		$\overline{C_2}$	$\overline{C_2}$	$\overline{C_1}$	$\overline{C_0}$			(5)
+			**1**					(6)
+					1			(7)
–			**1**	**1**	**1**			(8)
		P_5	P_4	P_3	P_2	P_1	P_0	

Schritt 3: Addition jeweils zweier aufeinanderfolgender Zeilen

	x_2	x_1	x_0	.	y_2	y_1	y_0	
+					$\overline{A_2}$	A_1	A_0	(1) + (2)
+				$\overline{B_2}$	B_1	B_0		(3) + (4)
+			C_2	$\overline{C_1}$	$\overline{C_0}$			(5) + (6)
+		1	0	1	0			(7) + (8)
		P_5	P_4	P_3	P_2	P_1	P_0	

Schritt 4: Schaltung (Carry Save)

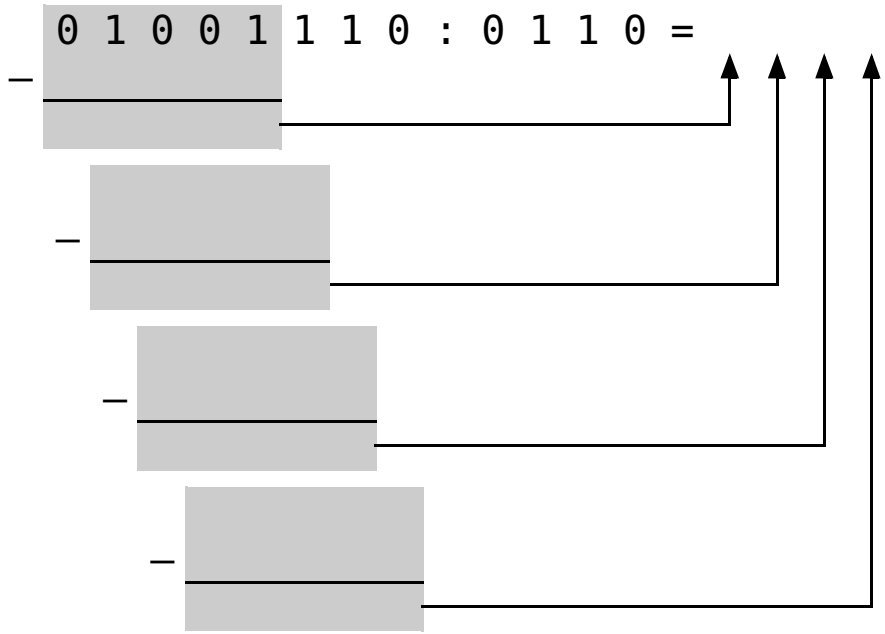


3.4.2 Dividierwerke

I. Vorzeichenlose Division

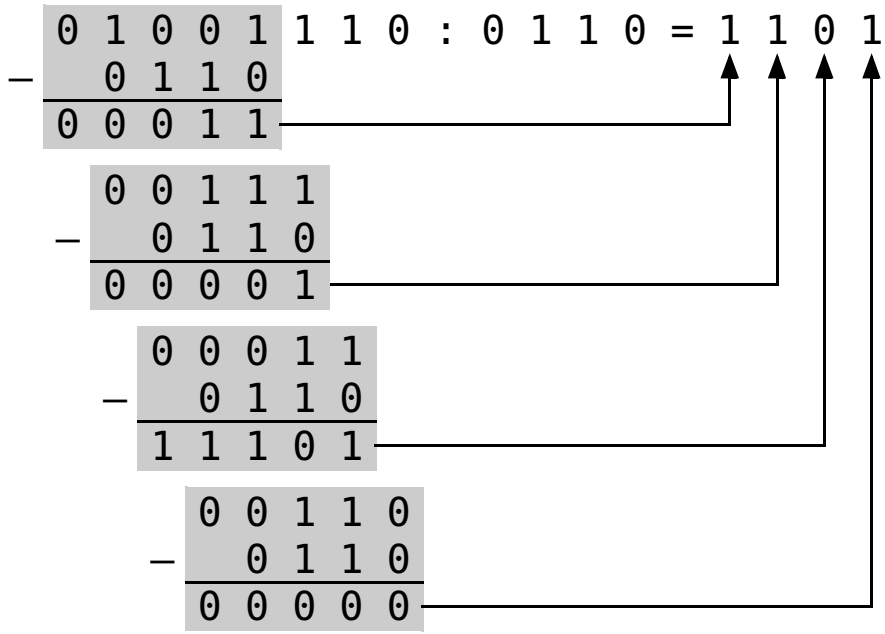
Beispiel

```
0 1 1 0 · 1 1 0 1
```



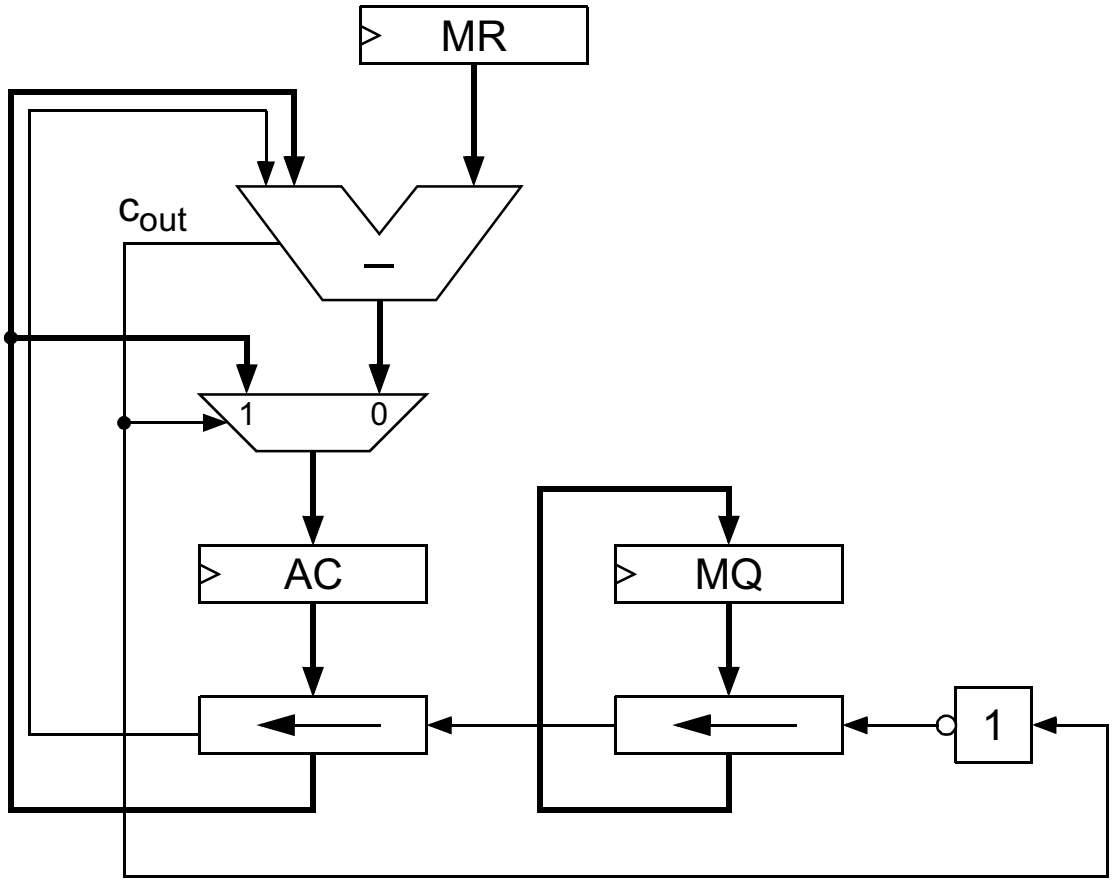
Beispiel

```
0 1 1 0 · 1 1 0 1
-----------------
          0 1 1 0
        0 0 0 0
      0 1 1 0
    0 1 1 0
-----------------
  0 1 0 0 1 1 1 0
```



II. *N*-schrittiges Dividierwerk für vorzeichenlose Zahlen

Schaltung



Bezeichnungen

MR Register für Divisor
MQ Register für Dividend und Quotient
AC Akkumulatorregister

Funktionsweise

- Inhalt der Register vor Beginn der Division

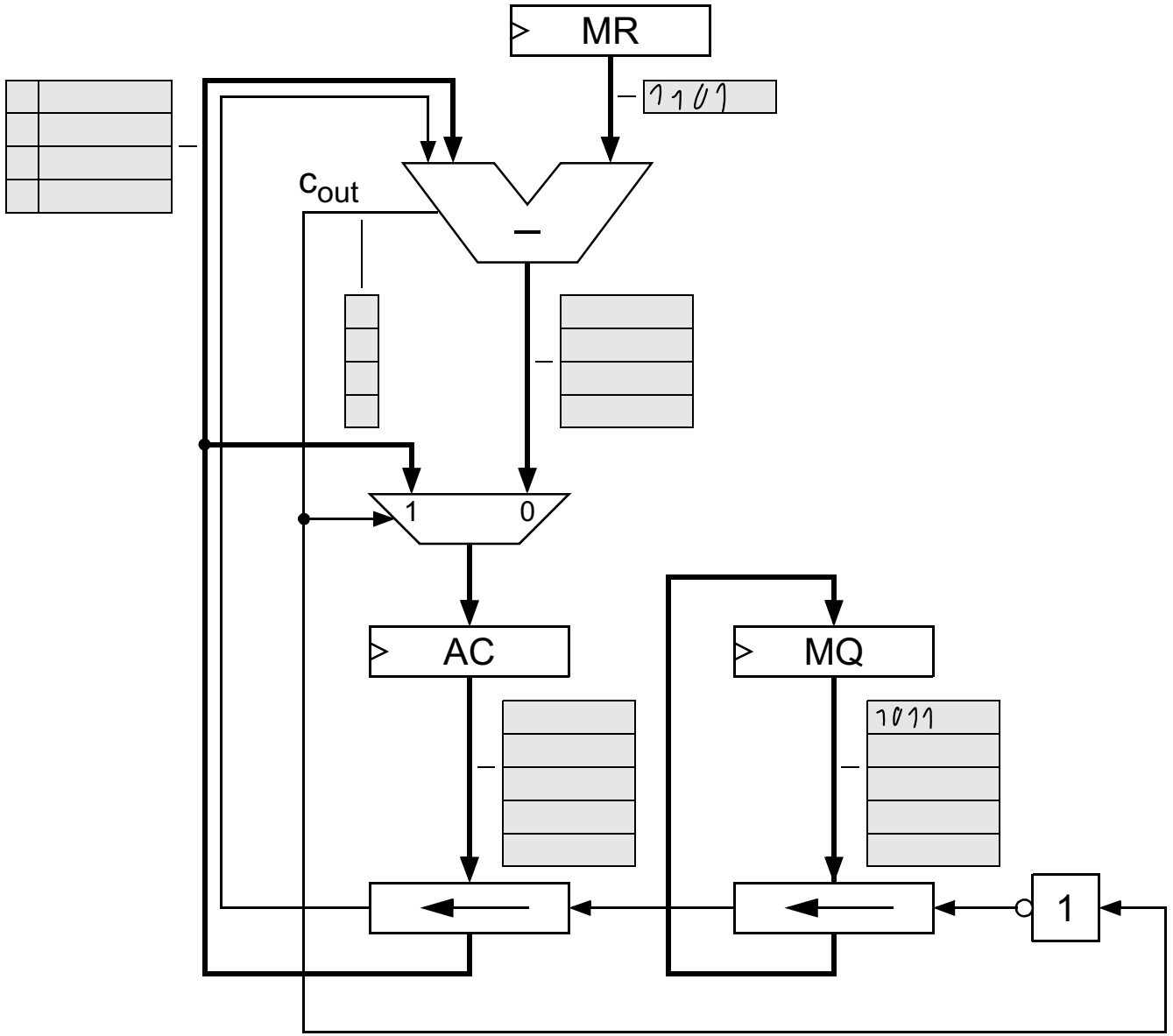
  MR Divisor
  MQ Dividend
  AC gelöscht

- Division erfolgt in *N* Schritten bzw. *N* Taktzyklen

- Inhalt der Register nach Ende der Division

  MR Divisor (unverändert)
  MQ Quotient
  AC Rest

Beispiel für $N = 4$



III. Vorzeichenbehaftete Division

nicht Prüfung relevant

Auswirkungen eines vorzeichenbehafteten Divisors

Positiver Divisor

- Annäherung an die Null durch sukzessive Subtraktionen "von oben"
- Quotientenstelle = 1, wenn Ergebnis der probeweisen Subtraktion ≥ 0
- erkennbar an: Vorzeichenbit der Differenz = 0
- verbleibende niederwertige Bit des Dividenden können unberücksicht bleiben
  (denn wenn *xxx* ≥ 0, dann ist für beliebige *yyy* auch *xxxyyy* ≥ 0)

Negativer Divisor

- Annäherung an die Null durch sukzessive Subtraktionen "von unten"
- Quotientenstelle = 1, wenn Ergebnis der probeweisen Subtraktion ≤ 0
- erkennbar an: Vorzeichenbit der Differenz = 1 (Differenz < 0) oder Differenz = 0
- verbleibende niederwertige Bit des Dividenden müssen jetzt berücksicht werden
  (denn wenn *xxx* ≤ 0, dann ist nicht immer auch *xxxyyy* ≤ 0)

Auswirkungen eines vorzeichenbehafteten Quotienten

Division Dividend : Divisor = **Quotient**

Entsprechende Multiplikation Dividend = Divisor · **Quotient**

Divison	Dividend	Divisor	Quotient
	+	+	+
	–	+	–
	–	–	+
	+	–	–
Multiplikation	Produkt	Multiplikand	Multiplikator

Multiplikation

- Multiplikator positiv: höchstwertiges Teilprodukt im Produkt ist 0
- Multiplikator negativ: höchstwertiges Teilprodukt im Produkt ist negiert

Entsprechende Division

- Quotient positiv: höchstwertiges Teilprodukt im Dividenden ist 0
- Quotient negativ: höchstwertiges Teilprodukt im Dividenden ist negiert

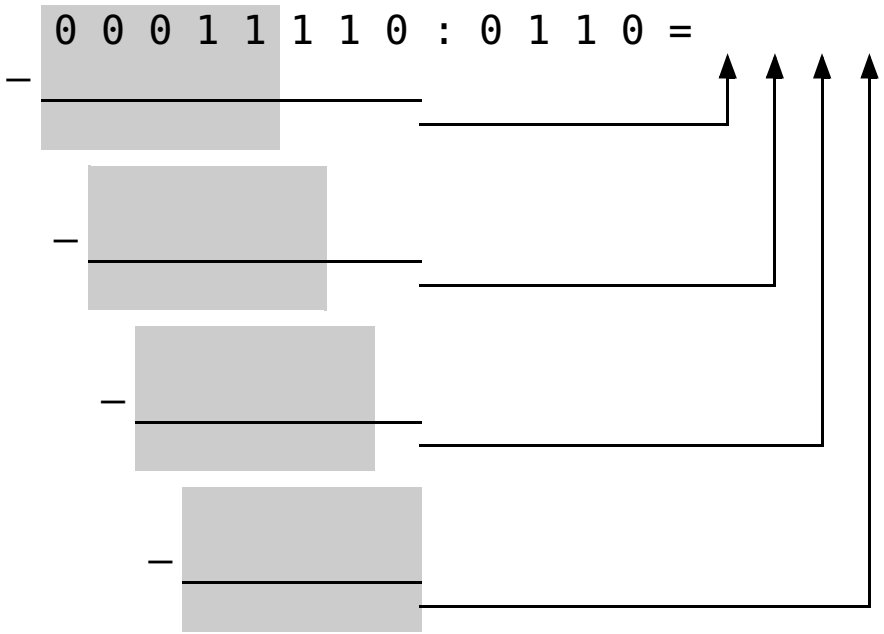
Der Quotient ist genau dann negativ, wenn Dividend und Divisor verschiedene Vorzeichen haben.

Beispiele (1)

Positiv : Positiv

- Divisor positiv: probeweise Subtraktion erfolgreich für Differenz ≥ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
0 1 1 0 · 0 1 0 1
-----------------
  0 0 0 0 0 1 1 0
  0 0 0 0 0 0 0
  0 0 0 1 1 0
  0 0 0 0 0
-----------------
  0 0 0 1 1 1 1 0
```



Beispiele (1)

Positiv : Positiv

- Divisor positiv: probeweise Subtraktion erfolgreich für Differenz ≥ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
 0 1 1 0 · 0 1 0 1
-----------------
   0 0 0 0 0 1 1 0
   0 0 0 0 0 0 0
   0 0 0 1 1 0
   0 0 0 0 0
-----------------
   0 0 0 1 1 1 1 0
```

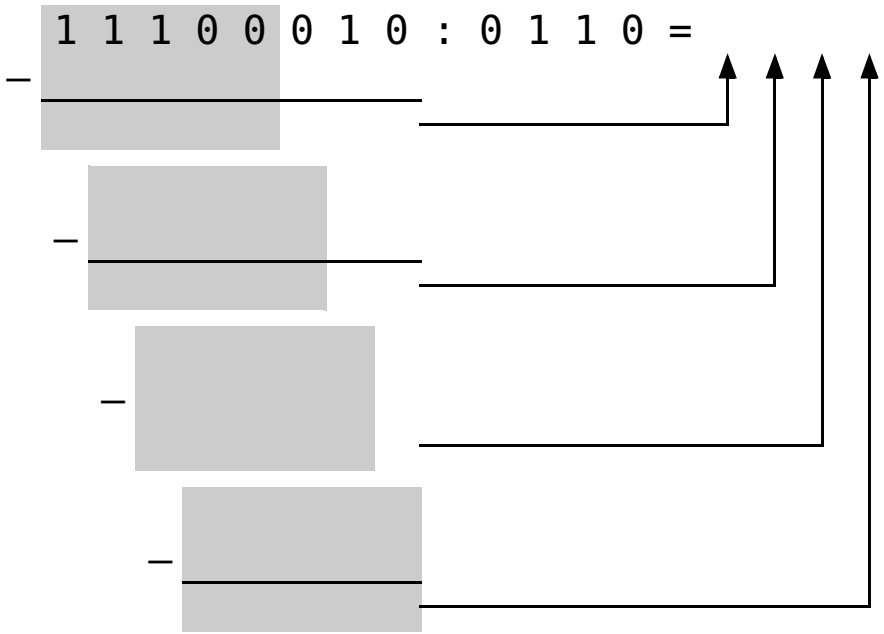
```
  0 0 0 1 1 1 1 0 : 0 1 1 0 = 0 1 0 1
- 0 0 1 1 0
-----------
  1 1 1 0 1 1 1 0
    0 0 1 1 1 1 0
  - 0 0 1 1 0
    -----------
    0 0 0 0 1 1 0
      0 0 0 1 1 0
    - 0 0 1 1 0
      -----------
      1 1 1 0 1 0
        0 0 1 1 0
      - 0 0 1 1 0
        -----------
        0 0 0 0 0
```

Beispiele (2)

Negativ : Positiv

- Divisor positiv: probeweise Subtraktion erfolgreich für Differenz ≥ 0
- Quotient negativ: erste probeweise Subtraktion des negierten Divisors erfolgreich (sonst Überlauf)

```
0 1 1 0 · 1 0 1 1
-----------------
  0 0 0 0 0 1 1 0
  0 0 0 0 1 1 0
  0 0 0 0 0 0
  1 1 0 1 0
-----------------
  1 1 1 0 0 0 1 0
```



Beispiele (2)

Negativ : Positiv

- Divisor positiv: probeweise Subtraktion erfolgreich für Differenz ≥ 0
- Quotient negativ: erste probeweise Subtraktion des negierten Divisors erfolgreich (sonst Überlauf)

```
 0 1 1 0 · 1 0 1 1
-----------------
   0 0 0 0 0 1 1 0
   0 0 0 0 1 1 0
   0 0 0 0 0 0
   1 1 0 1 0
-----------------
   1 1 1 0 0 0 1 0
```

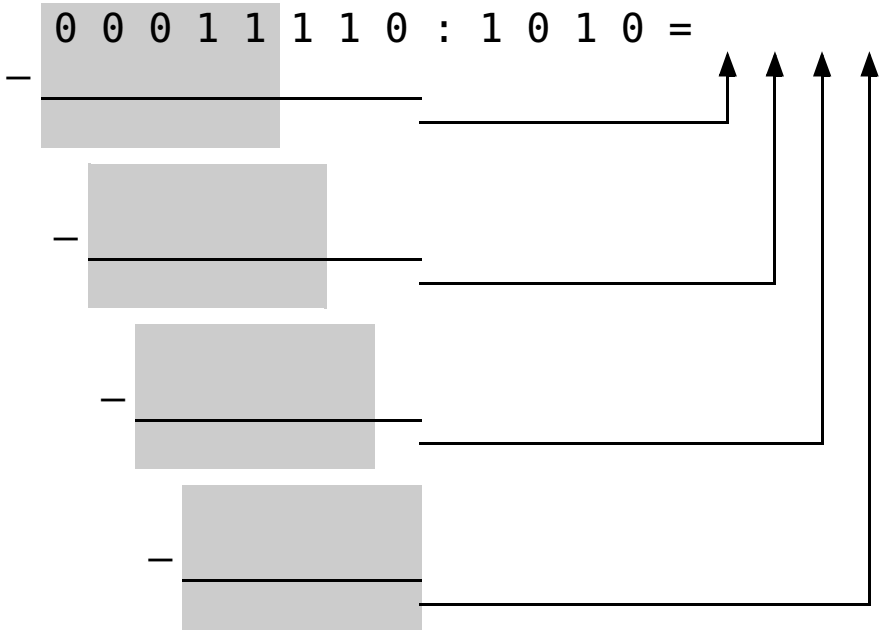
```
  1 1 1 0 0 0 1 0 : 0 1 1 0 = 1 0 1 1
- 1 1 0 1 0
-----------
  0 0 0 1 0 0 1 0
    0 0 1 0 0 1 0
  - 0 0 1 1 0
    -----------
    1 1 1 1 0 1 0
      0 1 0 0 1 0
    - 0 0 1 1 0
      -----------
      0 0 0 1 1 0
        0 0 1 1 0
      - 0 0 1 1 0
        -----------
        0 0 0 0 0
```

Beispiele (3)

Positiv : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient negativ: erste probeweise Subtraktion des negierten Divisors erfolgreich (sonst Überlauf)

```
 1 0 1 0 · 1 0 1 1
-----------------
   1 1 1 1 1 0 1 0
   1 1 1 1 0 1 0
   0 0 0 0 0 0
   0 0 1 1 0
-----------------
   0 0 0 1 1 1 1 0
```



Beispiele (3)

Positiv : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient negativ: erste probeweise Subtraktion des negierten Divisors erfolgreich (sonst Überlauf)

```
 1 0 1 0 · 1 0 1 1
-----------------
   1 1 1 1 1 0 1 0
   1 1 1 1 0 1 0
   0 0 0 0 0 0
   0 0 1 1 0
-----------------
   0 0 0 1 1 1 1 0
```

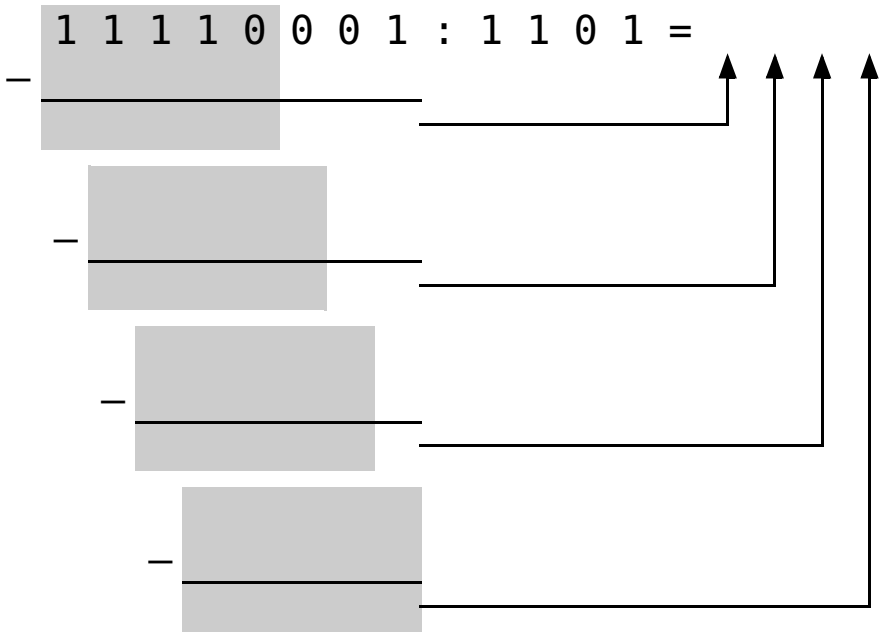
```
  0 0 0 1 1 1 1 0 : 1 0 1 0 = 1 0 1 1
- 0 0 1 1 0
-----------
  1 1 1 0 1 1 1 0
    1 1 0 1 1 1 0
  - 1 1 0 1 0
    -----------
    0 0 0 0 1 1 0
      1 0 1 1 1 0
    - 1 1 0 1 0
      -----------
      1 1 1 0 1 0
        1 1 0 1 0
      - 1 1 0 1 0
        -----------
        0 0 0 0 0
```

Beispiele (4)

Negativ : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
 1 1 0 1 · 0 1 0 1
-----------------
   1 1 1 1 1 1 0 1
   0 0 0 0 0 0 0
   1 1 1 1 0 1
   0 0 0 0 0
-----------------
   1 1 1 1 0 0 0 1
```



Beispiele (4)

Negativ : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
 1 1 0 1 · 0 1 0 1
-----------------
   1 1 1 1 1 1 0 1
   0 0 0 0 0 0 0
   1 1 1 1 0 1
   0 0 0 0 0
-----------------
   1 1 1 1 0 0 0 1
```

```
  1 1 1 1 0 0 0 1 : 1 1 0 1 = 0 1 0 1
- 1 1 1 0 1
-----------
  0 0 0 0 1 0 0 1
    1 1 1 0 0 0 1
  - 1 1 1 0 1
  -------------
    1 1 1 1 1 0 1
      1 1 1 1 0 1
    - 1 1 1 0 1
    -----------
      0 0 0 0 1 1
        1 1 1 0 1
      - 1 1 1 0 1
      -----------
        0 0 0 0 0
```

Beispiele für Relevanz der niederwertigen Bits des Dividenden (1)

Positiv : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient negativ: erste probeweise Subtraktion des negierten Divisors erfolgreich (sonst Überlauf)

```
 1 1 0 1 · 1 0 1 1
-----------------
   1 1 1 1 1 1 0 1
   1 1 1 1 1 0 1
   0 0 0 0 0 0
   0 0 0 1 1
-----------------
   0 0 0 0 1 1 1 1
```

```
  0 0 0 0 1 1 1 1 : 1 1 0 1 = 1 0 1 1
- 0 0 0 1 1
-----------
  1 1 1 1 0 1 1 1
    1 1 1 0 1 1 1
  - 1 1 1 0 1
    -----------
    0 0 0 0 0 1 1
      1 1 0 1 1 1
    - 1 1 1 0 1
      -----------
      1 1 1 1 0 1
        1 1 1 0 1
      - 1 1 1 0 1
        -----------
        0 0 0 0 0
```

Beispiele für Relevanz der niederwertigen Bits des Dividenden (2)

Negativ : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
1 1 0 1 · 0 0 1 1
-----------------
  1 1 1 1 1 1 0 1
  1 1 1 1 1 0 1
  0 0 0 0 0 0
  0 0 0 0 0
-----------------
  1 1 1 1 0 1 1 1
```

```
  1 1 1 1 0 1 1 1 : 1 1 0 1 = 0 0 1 1
- 1 1 1 0 1
-----------
  0 0 0 0 1 1 1 1

    1 1 1 0 1 1 1
  - 1 1 1 0 1
  -------------
    0 0 0 0 0 1 1

      1 1 0 1 1 1
    - 1 1 1 0 1
    -----------
      1 1 1 1 0 1

        1 1 1 0 1
      - 1 1 1 0 1
      -----------
        0 0 0 0 0
```

Beispiele für Relevanz der niederwertigen Bits des Dividenden (3)

Negativ : Negativ

- Divisor negativ: probeweise Subtraktion erfolgreich für Differenz ≤ 0
- Quotient positiv: erste probeweise Subtraktion des nicht negierten Divisors erfolglos (sonst Überlauf)

```
 1 1 1 1 · 0 0 0 1
-----------------
   1 1 1 1 1 1 1 1
   0 0 0 0 0 0 0
   0 0 0 0 0 0
   0 0 0 0 0
-----------------
   1 1 1 1 1 1 1 1
```

```
  1 1 1 1 1 1 1 1 : 1 1 1 1 = 0 0 0 1
- 1 1 1 1 1
-----------
  0 0 0 0 0 1 1 1

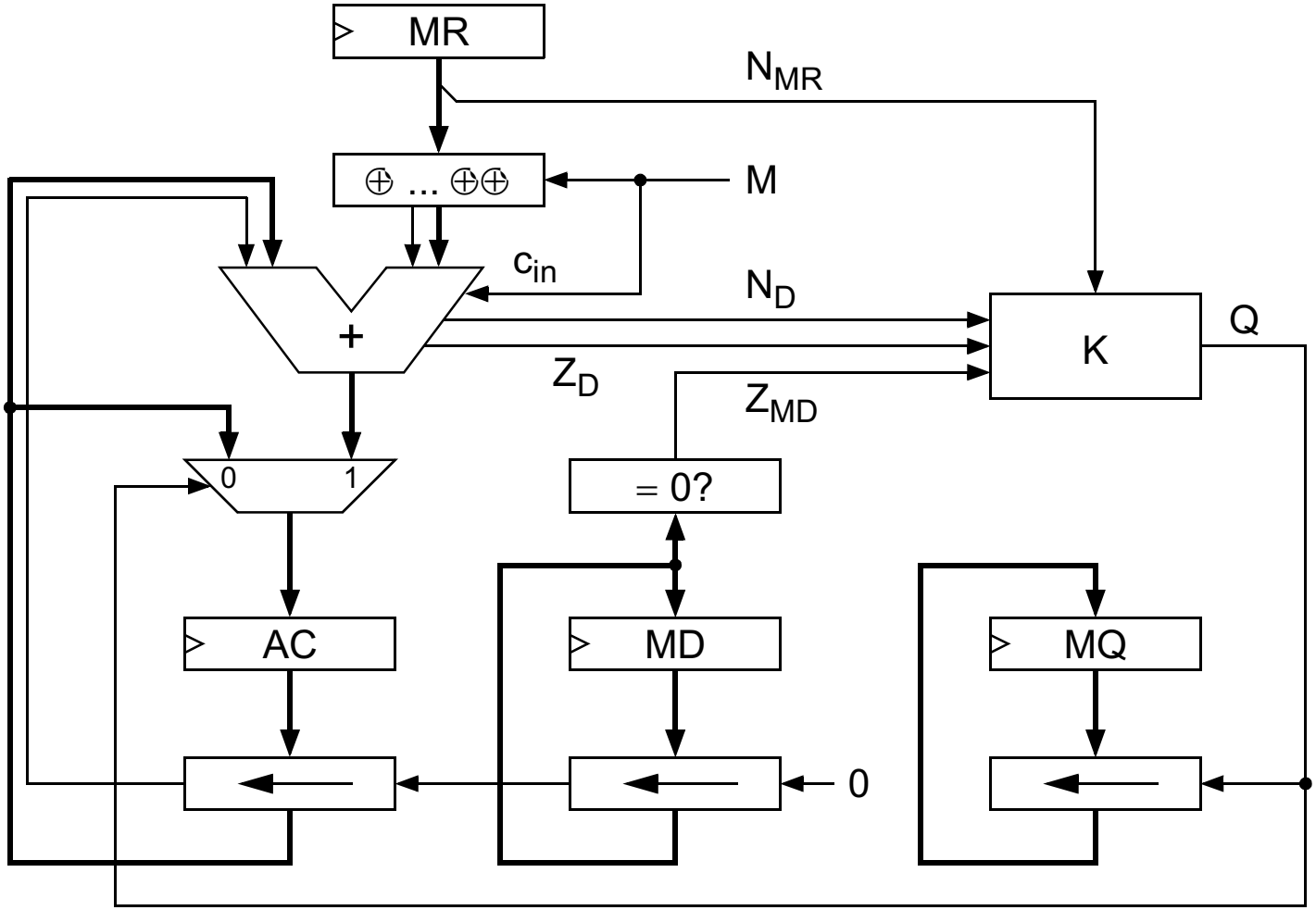
    1 1 1 1 1 1 1
  - 1 1 1 1 1
  -------------
    0 0 0 0 0 1 1

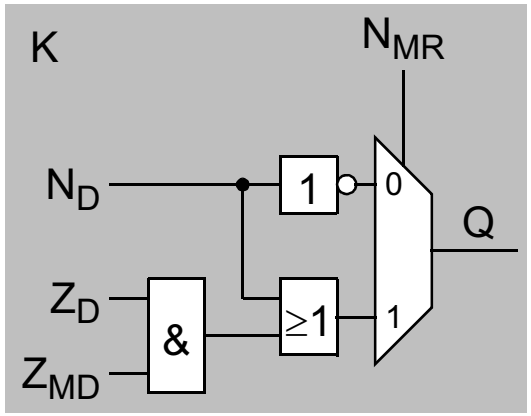
      1 1 1 1 1 1
    - 1 1 1 1 1
    -----------
      0 0 0 0 0 1

        1 1 1 1 1
      - 1 1 1 1 1
      -----------
        0 0 0 0 0
```

IV. *N*-schrittiges Dividierwerk für vorzeichenbehaftete Zahlen

Schaltung





Bezeichnungen

MD	Register für Dividend
MR	Register für Divisor
MQ	Register für Quotient
AC	Akkumulatorregister
M	*M* = 1: subtrahieren, *M* = 0: addieren (= negiert subtrahieren)
N_{MR}	Vorzeichen von *MR*
N_D	Vorzeichen der Differenz
Z_{MD}	1 wenn *MD* = 0
Z_D	1 wenn Differenz = 0
Q	Quotientenbit, zeigt erfolgreiche probeweise Subtraktion an

Funktionsweise

- Inhalt der Register vor Beginn der Division

  MD Dividend
  MR Divisor
  MQ uninitialisiert
  AC Vorzeichen des Dividenden

- Division erfolgt in *N* Schritten bzw. *N* Taktzyklen

- Inhalt der Register nach Ende der Division

  MD Null
  MR Divisor
  MQ Quotient
  AC Rest

Unterschiede zum Dividierwerk für vorzeichenlose Zahlen

- *AC* mit Vorzeichen von *MD* initialisieren (vorzeichenrichtige Erweiterug des Dividenden auf 2*N* Bit)
- *MR* auf *N* + 1 Bit vorzeichenrichtig erweitern
- Steuerleitung *M* für Sonderbehandlung des höchstwertigen Teilprodukts
  - erster Schritt M = Vorzeichen(MD) $\equiv$ Vorzeichen(MR)
  - folgende Schritte $M = 1$
- Funktion des Addierers und der XOR-Gatter
  - $M = 1$: $AC - MR$ (probeweise Subtraktion von MR)
  - $M = 0$: $AC + MR = AC - (-MR)$ (probeweise Subtraktion von –MR)
- niederwertige Bit des Dividenden in *MD* mit Null vergleichen, $Z_{MD} = 1$ wenn $MD = 0$
- separates Register *MQ* für Quotient
- *K* bestimmt, ob probeweise Subtraktion erfolgreich
  - Divisor positiv ($N_{MR} = 0$): $Q = \overline{N}_D$ (erfolgreich wenn Differenz ≥ 0)
  - Divisor negativ ($N_{MR} = 1$): $Q = N_D + Z_D Z_{MD}$ (erfolgreich wenn Differenz ≤ 0)

  damit $Q = \overline{N}_{MR}\overline{N}_D + N_{MR}(N_D + Z_D Z_{MD})$

Notizen

- Quotient wird stets nach unten gerundet, Rest hat stets das Vorzeichen des Divisors

  (+13) : (+5) = (+2) Rest (+3), denn (+5) · (+2) + (+3) = (+13)

  (–13) : (+5) = (–3) Rest (+2), denn (+5) · (–3) + (+2) = (–13)

  (+13) : (–5) = (–3) Rest (–2), denn (–5) · (–3) + (–2) = (+13)

  (–13) : (–5) = (+2) Rest (–3), denn (–5) · (+2) + (–3) = (–13)

- Division durch Null: $x : 0 = -1$ Rest x für $x \geq 0$, $x : 0 = 0$ Rest x für $x < 0$
- Sonderfall $-2^{N-1} : -1$ ergibt falsches Ergebnis -2^{N-1}, da $+2^{N-1}$ nicht mit N Bit darstellbar (Überlauf)
- Sonderfall $MR = -2^{N-1}$ kein Problem, denn für Zweierkomplementbildung von MR erforderliche Operation "+1" (über $c_{in} = M = 1$) erfolgt erst nach vorzeichenrichtiger Erweiterung von $\overline{MR}$
- Dividend darf auch $2N$ Bit umfassen, wenn der Quotient mit N Bit darstellbar ist (dafür MD initialisieren mit niederwertiger Hälfte des Dividenden, AC initialisieren mit höherwertiger Hälfte), Überlauf erkannt falls erwartetes Vorzeichen und errechnetes Vorzeichen verschieden
- vorzeichenbehaftete Division vergleichsweise aufwändig, deshalb stattdessen häufig separate Vorzeichen- und Betragsrechnung

Rechnerarchitektur

Matthias Meyer

Kapitel 4 Entwurf eines Minimalprozessors

Inhaltsübersicht

4.1 Mikroprozessor

4.2 Entwurfsebenen

4.3 Programmiermodell (Instruction Set Architecture)

4.4 Register-Transfer-Ebene

4.5 Implementierung

4.1 Mikroprozessor

hier

Ein *Prozessor* ist ein programmierbares Rechenwerk, das Operationen programmgesteuert ausführt. Ein *Mikroprozessor* ist ein Prozessor, der vollständig auf einem (Mikro-) Chip untergebracht ist.

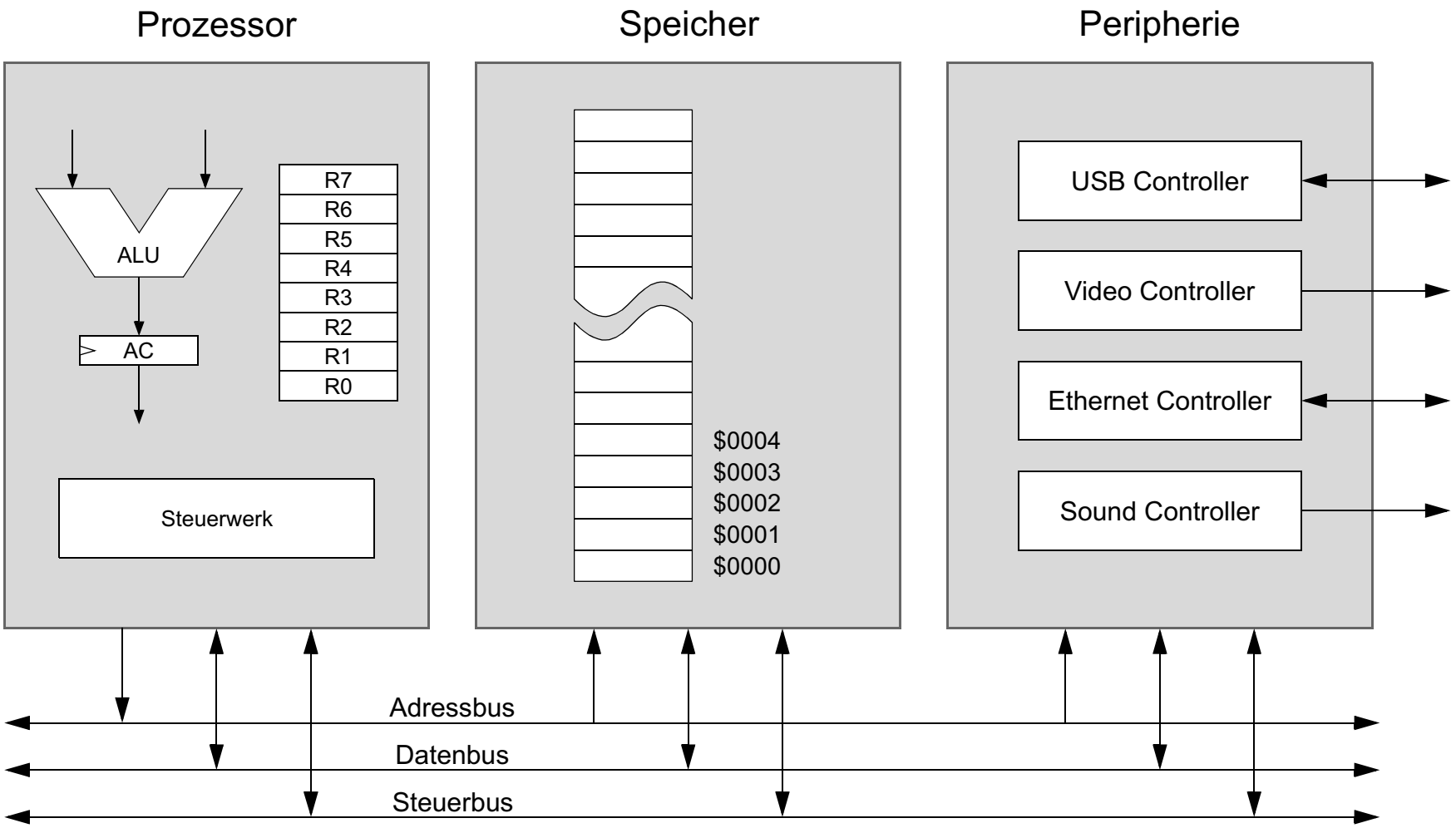
Inhaltsübersicht

4.1.1 Aufbau eines einfachen Rechners (Beispiel)

4.1.2 Klassifikationskriterien

4.1.3 Programmiermodell (Instruction Set Architecture)

4.1.1 Aufbau eines einfachen Rechners (Beispiel)



4.1.2 Klassifikationskriterien

I. Klassifikation nach (typischer) Wortbreite

8-Bit-Prozessoren

Merkmale

- Datenbreite 8 Bit (ein Byte)
- Adressbreite 16 Bit (zwei Bytes)
- adressierbarer Speicher 2^{16} * 8 Bit = 65536 Byte = 64 KByte
- festverdrahtetes Steuerwerk

Beispiele

- Intel 8008 (1972)
- Intel 8080 (1974)
- Motorola 6800 (1974)
- MOS Technology 6502 (1975)
- Zilog Z80 (1976)

16-Bit-Prozessoren

Beispiele

- Intel 8086 (1978)
  - segmentierter Speicher (Segmentgröße 64 kByte)
  - externe Datenbreite 16 Bit, externe Adressbreite 20 Bit (gemultiplext)
  - adressierbarer Speicher 2^{20} * 8 Bit = 16 * 64 KByte = 1 MByte
- Motorola 68000 (1979)
  - interne Daten- und Adressbreite jeweils 32 Bit
  - externe Datenbreite 16 Bit, externe Adressbreite 24 Bit
  - adressierbarer Speicher 2^{24} * 8 Bit = 16 MByte

32-Bit-Prozessoren

Beispiele

- Motorola 68020 (1984), Motorola 68030 (1987), Motorola 68040 (1990), Motorola 68060 (1994)
- Intel 80386 (1985), Intel 80486 (1989), Intel Pentium (1993)
- PowerPC 601 (1991), 603 (1994), 604 (1994)

64-Bit-Prozessoren

Beispiele

- DEC Alpha (1992)
- PowerPC 620 (1997)
- Intel Itanium (2001)
- AMD64 (2003)
- Intel 64 (2005)

II. Klassifikation nach Art der Befehlssatzarchitektur (Instruction Set Architecture)

CISC (Complex Instruction Set Computer)

Merkmale

- viele und teilweise sehr mächtige Befehle
- viele und teilweise sehr mächtige Adressierungsarten
- Anzahl der Takte und Speicherzugriffe zur Ausführung abhängig von Befehl und Adressierungsart
- Operanden für arithmetische Operationen im Speicher oder in Registern
- nur wenige Register

RISC (Reduced Instruction Set Computer)

Merkmale

- einfache Befehle
- einfache Adressierungsarten
- Ziel: jeden Befehl effektiv in einem Takt ausführen
- Operanden für arithmetische Operationen ausschließlich in Registern
- viele Register

III. Flynnsche Klassifikation

fragt er nie

SISD (Single Instruction, Single Data)

Ein Befehl bearbeitet ein einziges Datenwort (z. B. traditioneller Einkern-Prozessor).

SIMD (Single Instruction, Multiple Data)

Ein Befehl bearbeitet mehrere Datenworte (z. B. Vektor-Prozessoren), auch als Erweiterung für traditionelle SISD-Architekturen, z. B. Intel SSE (Streaming SIMD Extensions).

MISD (Multiple Instruction, Single Data)

Umstritten, ob sinnvoll.

MIMD (Multiple Instruction, Multiple Data)

Multiprozessorsysteme.

IV. Klassifikation nach der Anzahl möglicher Operanden

Null-Adress-Maschine	Befehl enthält keine Information über die Operanden oder die Adresse der Operanden (Implizite Adressierung). Beispiel: Kellermaschine: Alle Operationen beziehen sich implizit auf einen Kellerspeicher (Stapel, LIFO)
Einadress-Maschine	Befehl enthält Information über einen Operanden bzw. über die Adresse eines Operanden. Der zweite Operand liegt implizit fest (z.B. ein Akkumulator-Register).
Zweiadress-Maschine	Befehl enthält Information über zwei Operanden bzw. über die Adresse zweier Operanden. In der Regel fungiert einer der Operanden auch als Zieloperand.
Dreiadress-Maschine	Befehl enthält Information über drei Operanden bzw. über die Adresse dreier Operanden. Auf diese Weise lassen sich zwei Quelloperanden und ein davon unabhängiger Zieloperand angeben.

V. Klassifikation nach Speicheranbindung (von Neumann, Harvard)

Von-Neumann-Architektur

Harvard-Architektur

4.1.3 Programmiermodell (Instruction Set Architecture)

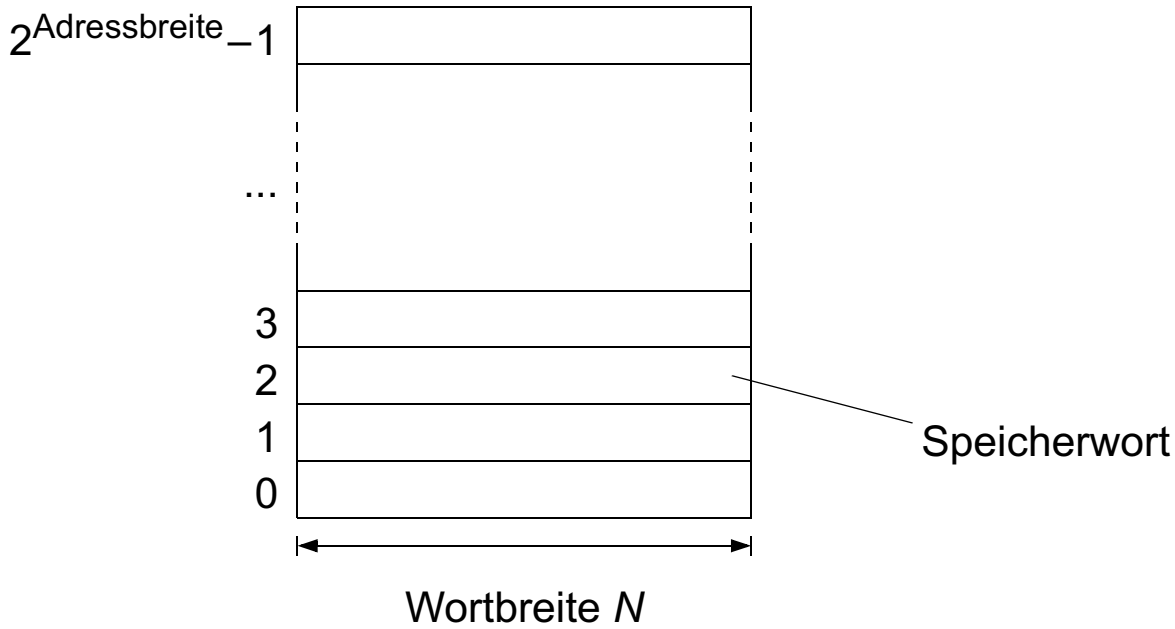
Die Schnittstelle eines Prozessors zum Anwender, gleichbedeutend mit der Schnittstelle zwischen Hardware und Software, wird als *Programmiermodell* oder *Befehlssatzarchitektur* (*Instruction Set Architecture*, *ISA*) bezeichnet. Dazu gehören Registermodell, Speichermodell, Befehlssatz, Adressierungsverfahren sowie eine Codierungsvorschrift für Befehle und Adressierungsverfahren.

Die tatsächliche Implementierung eines Prozessors in Hardware (*Mikroarchitektur, Micro Architecture*) spielt auf der Ebene der Instruction Set Architecture keine Rolle. Deshalb sind Programme, die für eine gegebene Instruction Set Architecture geschrieben (bzw. compiliert) werden, auf allen Prozessoren lauffähig, die auf derselben Instruction Set Architecture basieren.

Ein Prozessor führt ausschließlich Programme in der ihm eigenen *Maschinensprache* aus. Diese Maschinenprogramme bestehen aus einer Folge von Maschinenworten, die für Menschen schwer lesbar sind. Aus diesem Grund führt man Assemblersprachen ein, die Befehle und Adressierungsverfahren mit Hilfe von Befehlskürzeln (*Mnemonics*) und Symbolen in menschenlesbarer Form darstellen. Diese Assemblerprogramme werden von einem *Assembler* in Maschinenprogramme übersetzt.

I. Speichermodell

Das Speichermodell eines typischen Prozessors besteht aus einer einfachen durchnummerierten Folge von Speicherworten. Dabei wird die Nummer eines Speicherworts als die *Adresse* des Speicherworts bezeichnet. Ein Speicherwort selbst besteht aus *N* Bit, wobei man *N* die *Wortbreite* des Prozessors nennt. Die maximal mögliche Anzahl adressierbarer Speicherworte wird durch die *Adressbreite* des Prozessors bestimmt.

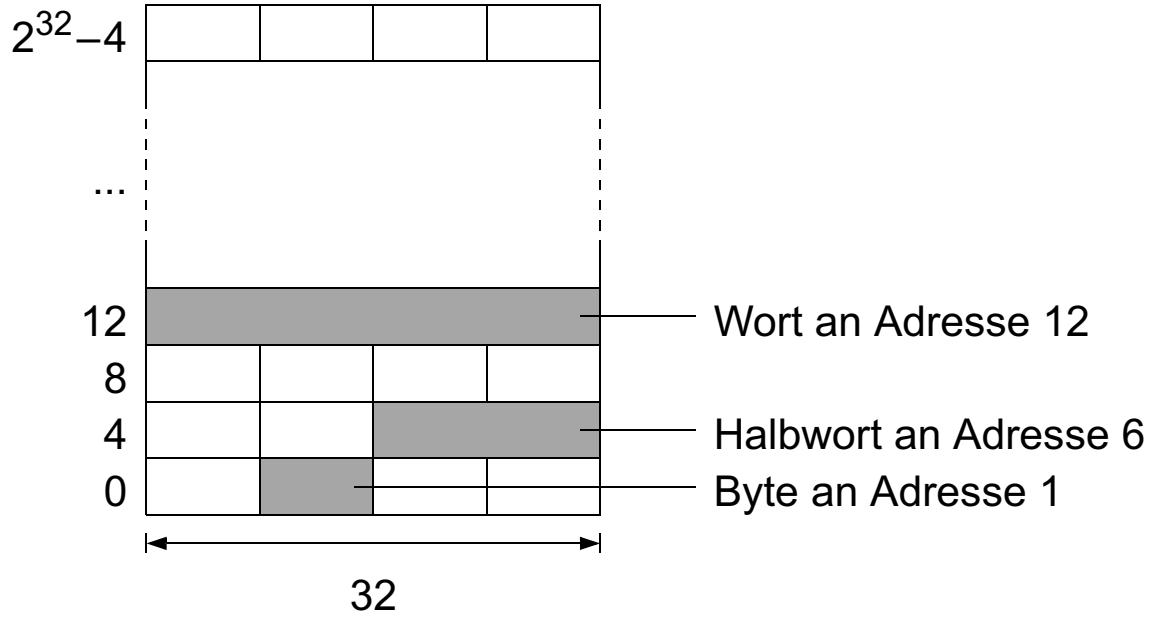


Häufig stimmen Wort- und Adressbreite überein, so hat ein typischer 32-Bit-Prozessor eine Wortbreite von 32 Bit und eine Adressbreite von 32 Bit.

Prozessoren erlauben meist auch den Zugriff auf Teile eines Wortes, d.h. die kleinste adressierbare Einheit ist dann kleiner als ein Wort. So kann ein 32-Bit-Prozessor in der Regel nicht nur auf ganze Worte (32 Bit) zugreifen, sondern auch auf einzelne Bytes (8 Bit) oder Halbworte (16 Bit). Dafür ist es erforderlich, dass man nicht jedem Wort, sondern jeder kleinsten adressierbaren Einheit eine eigene Adresse zuweist.

Beispiel: 32-Bit-Prozessor

- Wortbreite: 32 Bit, kleinste adressierbare Einheit: ein Byte
- Zugriff möglich auf einzelne Bytes (8 Bit), Halbworte (16 Bit), ganze Worte (32 Bit)
- jedes Byte: eigene Adresse, Halbworte: gerade Adressen*, Worte: durch 4 teilbare Adressen*



*_Aligned Memory Access_ (manche Prozessoren erlauben auch _Unaligned Memory Access_)

Problem **Byte Ordering**

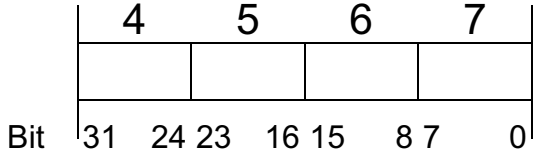
Welche Reihenfolge haben die Bytes in einem Wort?

Beispiel

Schreibe Byte an Adresse 4: CA, an Adresse 5: FE, an Adresse 6: BA, an Adresse 7: BE

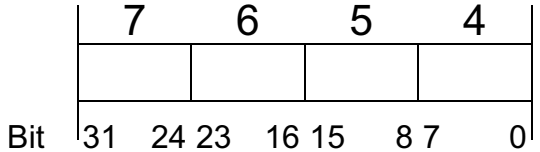
Lese Wort von Adresse 4

Möglichkeit 1: **Big Endian** (höchstwertiges Byte kommt zuerst, d.h. hat die kleinste Adresse)



Wort an Adresse 4: CAFEBABE

Möglichkeit 2: **Little Endian** (niederwertigstes Byte kommt zuerst, d.h. hat die kleinste Adresse)



Wort an Adresse 4: BEBAFECA

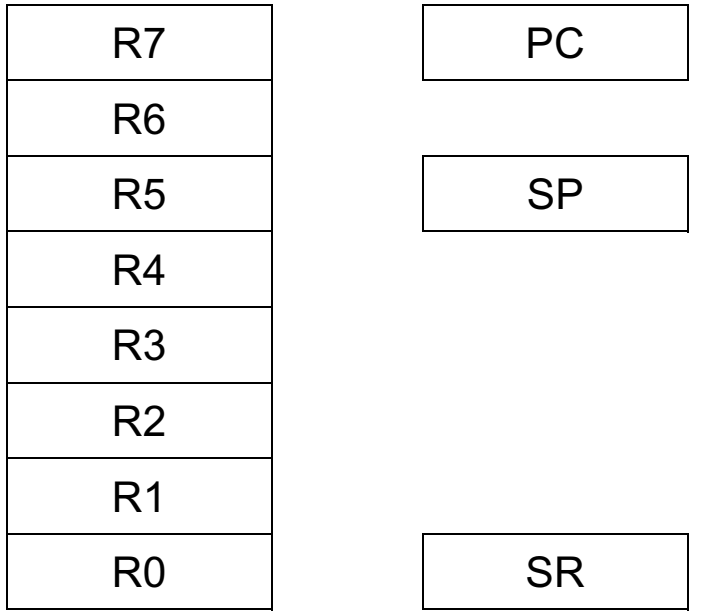
Beispiele für "Big Endian"-Architekturen: Motorola 6800, Motorola 68000, IBM PowerPC, Sun SPARC

für "Little Endian"-Architekturen: Intel x86, DEC VAX, DEC Alpha

II. Registermodell

Die meisten Prozessoren verfügen über eine begrenzte Anzahl interner Register, auf die Maschinenbefehle effizient zugreifen können, weil nur wenige Bits erforderlich sind, um die Register voneinander zu unterscheiden und weil kein zeitaufwändiger Speicherzugriff erforderlich ist.

Beispiel für ein einfaches Registermodell



R0 – R7	Allzweckregister (General Purpose Register)
PC	Befehlsfolgezähler (Program Counter)
SP	Stapelzeiger (Stack Pointer)
SR	Status-Register (Status Register)

III. Befehlssatz

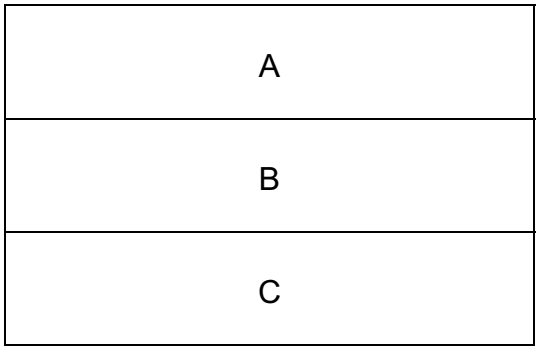
Die wichtigsten Befehlstypen eines Mikroprozessors sind Datenübertragungsbefehle, Datenbearbeitungsbefehle und Programmsteuerbefehle.

Datenübertragungsbefehle	Speicher ↔ Register	z.B. `MOVE`
	Register ↔ Register	
	Speicher ↔ Speicher	
Datenbearbeitungsbefehle	arithmetische Befehle	z.B. `ADD`
	logische Befehle	z.B. `AND`
	Schiebe- u. Rotierbefehle	z.B. `LSL`
Programmsteuerbefehle	bedingte Verzweigung	z.B. `BEQ, BNE`
	unbedingte Verzweigung	z.B. `BRA`
	Unterprogrammaufruf	z.B. `BSR`
	Unterprogrammende	z.B. `RTS`

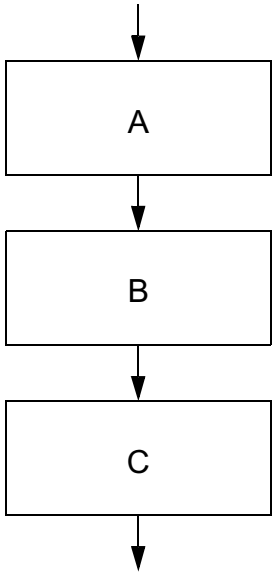
Programmsteuerbefehle und strukturierte Programmierung

Sequenz

Struktogramm



Flussdiagramm

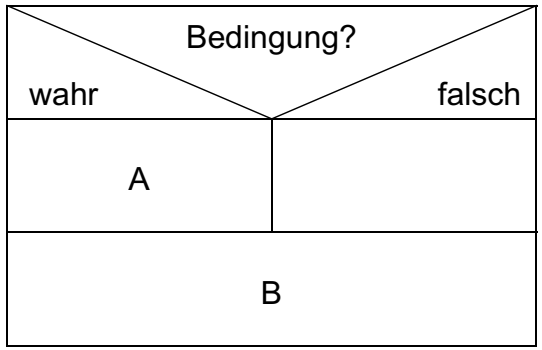


Pseudo-Assemblercode

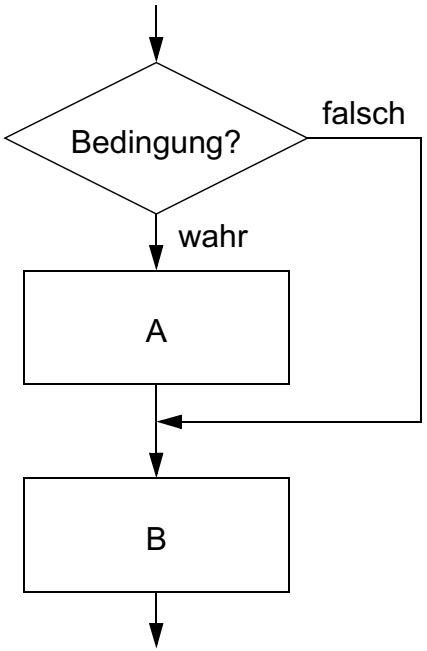
```
A:  ...
    ...
B:  ...
    ...
C:  ...
    ...
```

Einseitige Auswahl (if)

Struktogramm



Flussdiagramm

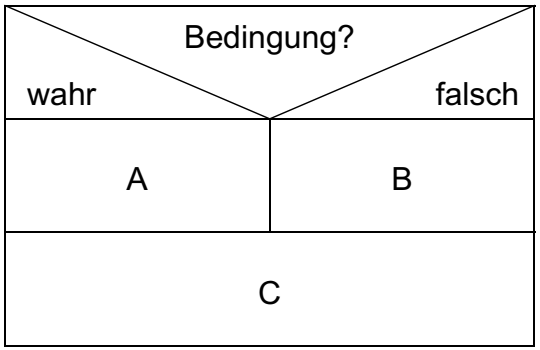


Pseudo-Assemblercode

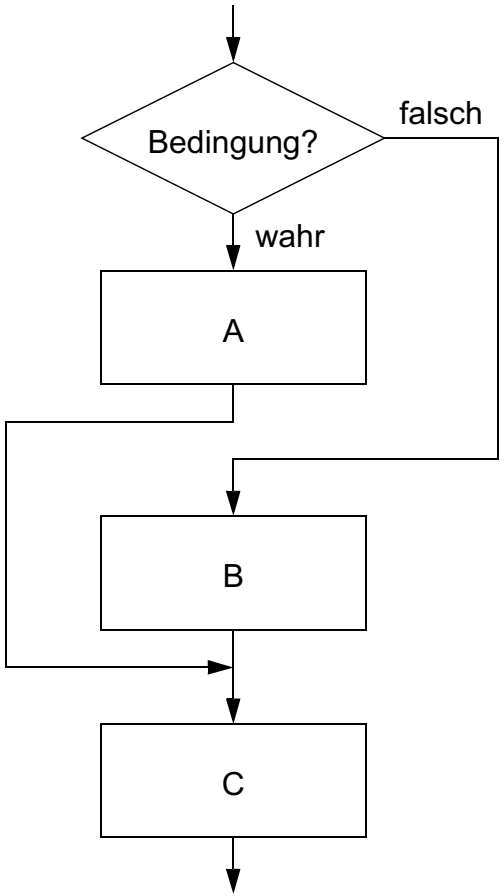
```
     CHECK_COND
     BNE B
A:   ...
     ...
B:   ...
     ...
```

Zweiseitige Auswahl (if-else)

Struktogramm



Flussdiagramm

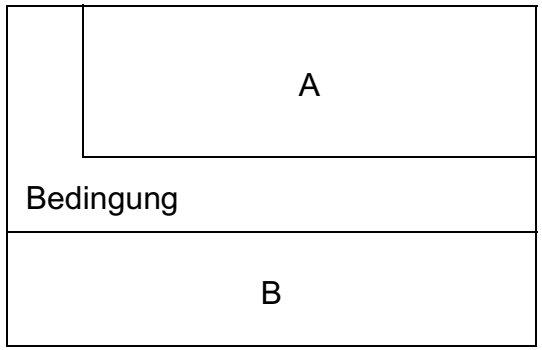


Pseudo-Assemblercode

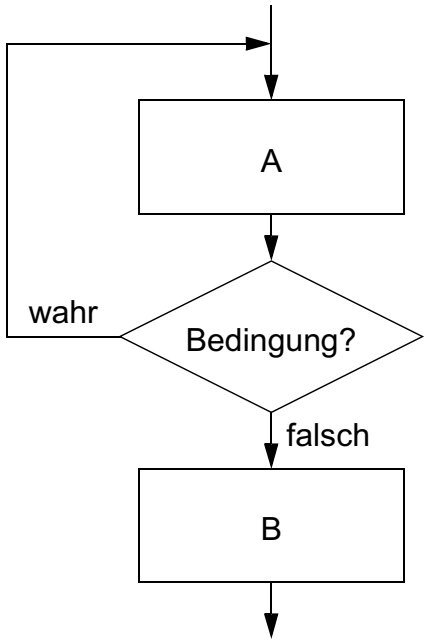
```
     CHECK_COND
     BNE B
A:   ...
     ...
     BRA C
B:   ...
     ...
C:   ...
     ...
```

Fußgesteuerte Schleife (do-while)

Struktogramm



Flussdiagramm

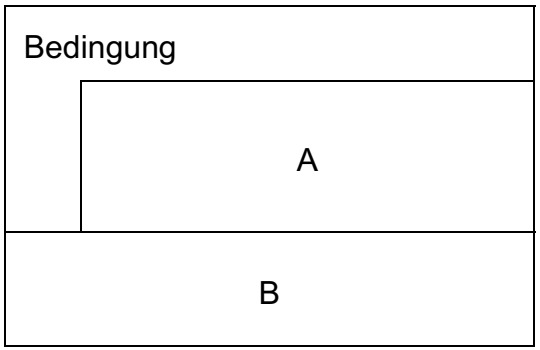


Pseudo-Assemblercode

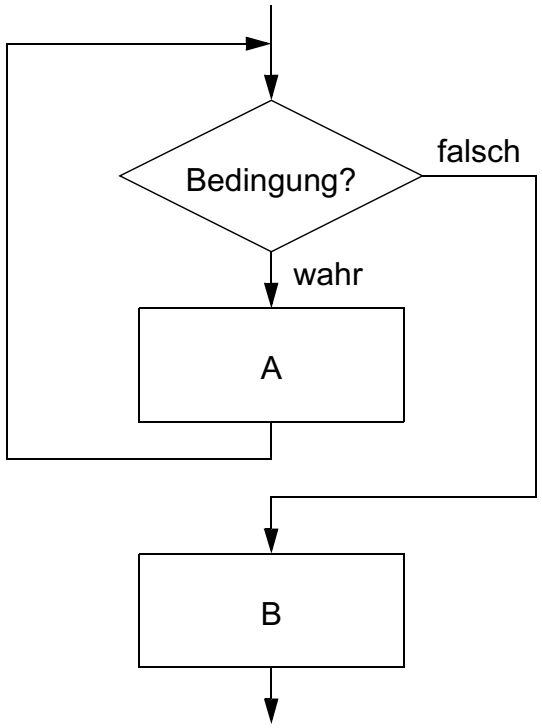
```
A:  ...
    ...
    CHECK_COND
    BEQ A
B:  ...
    ...
```

Kopfgesteuerte Schleife (while, for)

Struktogramm



Flussdiagramm



Pseudo-Assemblercode

```
L:  CHECK_COND
    BNE B
A:  ...
    ...
    BRA L
B:  ...
    ...
```

IV. Grundlegende Adressierungsverfahren

Adressierungsverfahren (Adressierungsarten) legen die Quell- und die Zieloperanden eines Befehls fest. Mögliche Operanden sind

- Konstanten (nur als Quelloperanden)
- Registerinhalte
- Inhalt von Speicherworten

Die wichtigsten Grundtypen von Adressierungsverfahren sind nachfolgend zusammengestellt.

		Beispiel	
unmittelbar	Der Operand ist eine Konstante, die im Befehl enthalten ist.	`ADD **#5,**R0`	Addiere 5 zum Register R0.
direkt	Der Befehl enthält die Adresse der Speicherzelle des Operanden.	`ADD **($100),**R0`	Addiere den Inhalt der Speicherzelle 100_{16} zum Register R0.
indirekt	Die Adresse des Operanden wird indirekt ermittelt (z. B. über ein Register oder eine Adressrechnung).	`ADD **(R1),**R0`	Addiere den Inhalt der Speicherzelle zu R0, deren Adresse durch R1 gegeben ist.

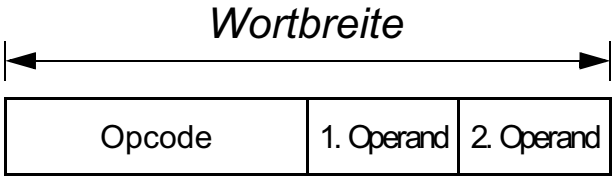
Kommerzielle Prozessoren bieten verschiedene Varianten und Kombinationen dieser grundlegenden Adressierungsverfahren.

V. Codierung von Befehlen am Beispiel einer Zweiadress-Maschine

Die folgenden Bilder zeigen Codierung und Format von 1-Wort, 2-Wort und 3-Wort-Befehlen am Beispiel einer Zweiadress-Maschine.

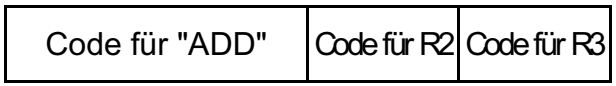
Ist die Wortbreite ausreichend groß, so können in einem Befehlswort (Instruction Word) neben dem eigentlichen Operationscode (kurz "Opcode") Registeradressen, Konstanten oder Offsets abgelegt werden. Reicht die Wortbreite nicht für die Operanden aus, so schließen sich dem Befehlswort sogenannte Erweiterungsworte an, die typischerweise Konstanten, Adressen oder Offsets enthalten.

1-Wort-Befehle

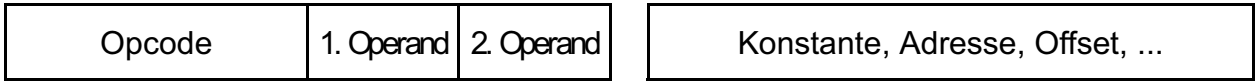


Befehlswort

Beispiel: `ADD R2,R3`

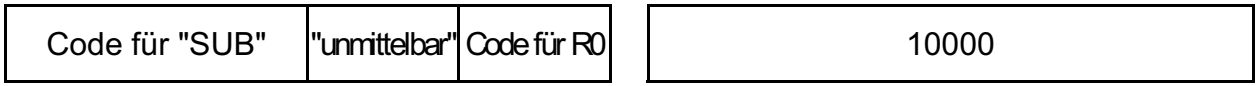


2-Wort-Befehle



Befehlswort Erweiterungswort

Beispiel: `SUB #10000,R0`

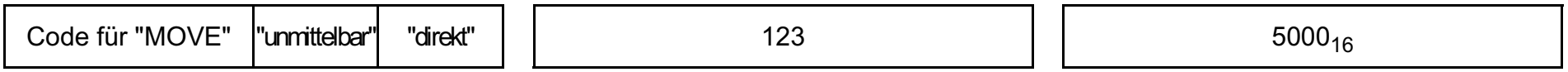


3-Wort-Befehle



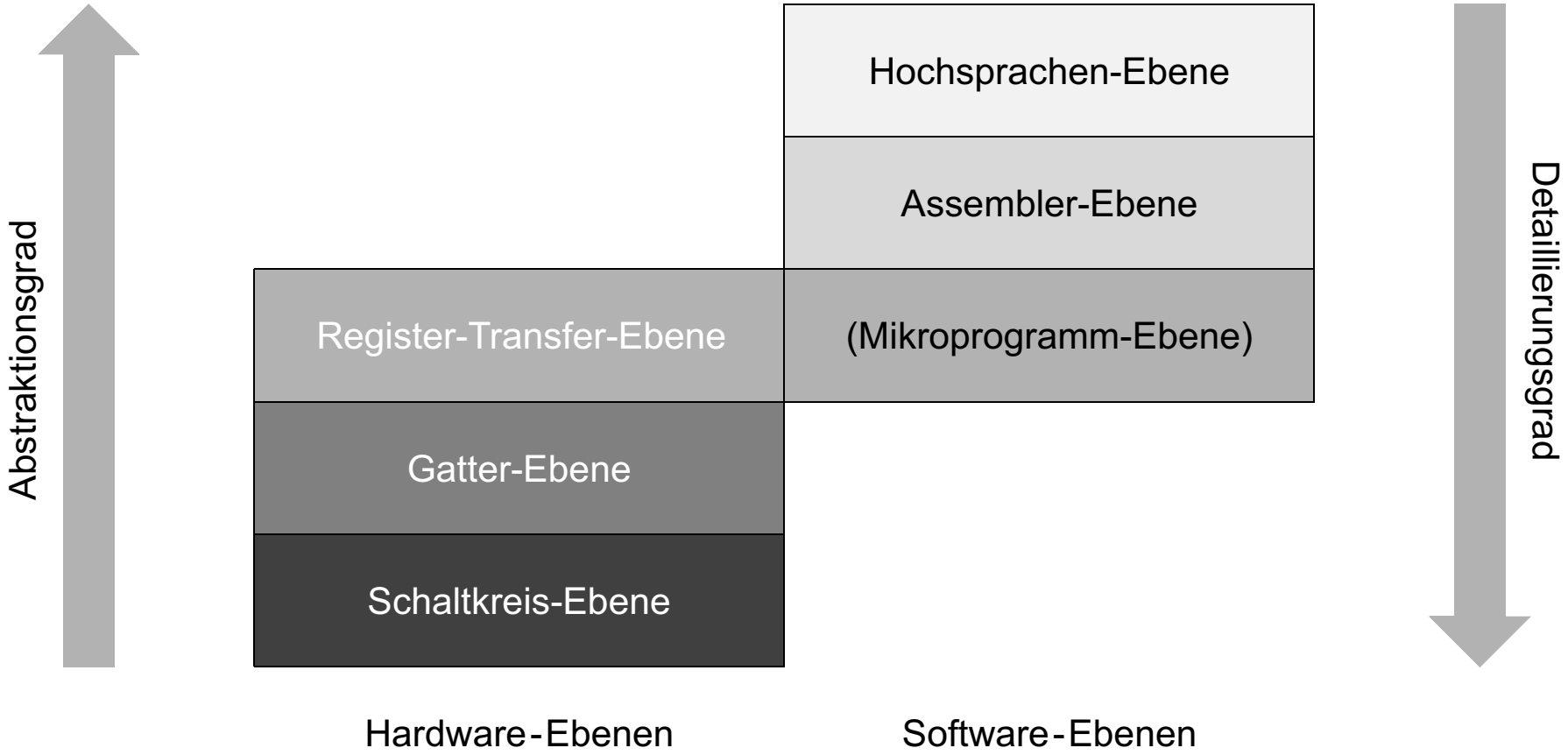
Befehlswort 1. Erweiterungswort 2. Erweiterungswort

Beispiel: `MOVE #123,($5000)`



4.2 Entwurfsebenen

Die Entwicklung digitaler Systeme ist bei der heute erreichbaren Komplexität nur durch strukturierte Verfahren beherrschbar. In diesem Zusammenhang spielt die Abstraktionsebene des Entwurfs (Entwurfsebene) eine zentrale Rolle.



Inhaltsübersicht

4.2.1 Die Schaltkreis-Ebene

4.2.2 Die Gatter-Ebene (Logik-Ebene)

4.2.3 Die Register-Transfer-Ebene

4.2.4 Die Mikroprogramm-Ebene

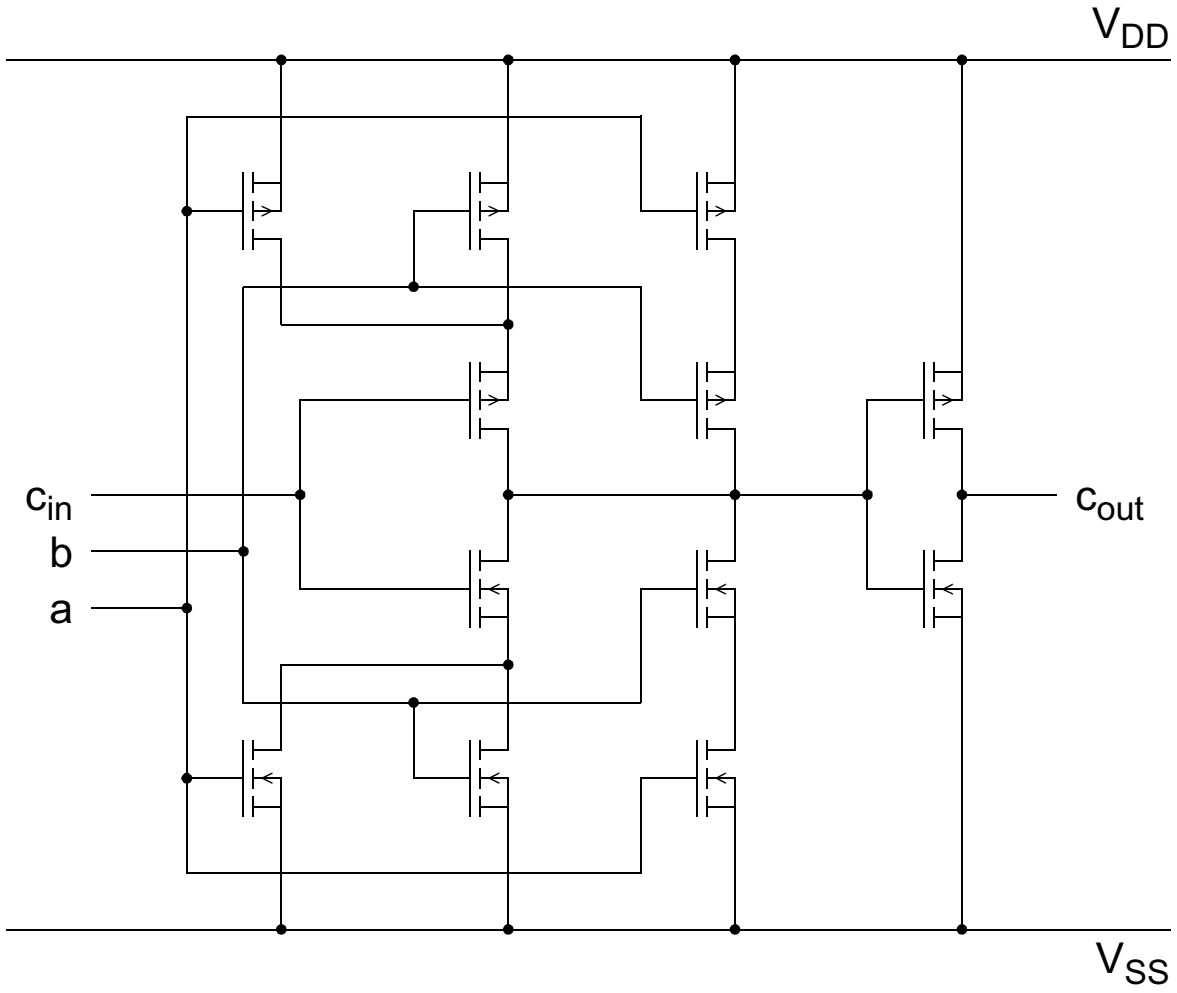
4.2.5 Die Assembler-Ebene (Maschinen-Ebene)

4.2.1 Die Schaltkreis-Ebene

Auf der Schaltkreis-Ebene wird die Struktur eines Systems durch eine Netzliste elektronischer Bauelemente („Schaltplan“) beschrieben. Bei den Bauelementen handelt es sich in Abhängigkeit von der verwendeten Technologie (TTL, CMOS, ECL, ...) typischerweise um Transistoren, Dioden und Widerstände.

Schaltkreis-Ebene	
Elemente	Transistoren, Dioden, Widerstände
Zeiteinheiten	ns, ps
Informationseinheiten	repräsentiert durch Spannung bzw. Strom
Strukturbeschreibung	Schaltplan, Netzlisten, Chiplayout (Geometrie)
Verhaltensbeschreibung	mathematische Beschreibungsformen (z. B. Differenzialgleichungen)
Methoden	Maschen- bzw. Knotenanalyse, mathematische Methoden, Graphentheorie

Beispiel



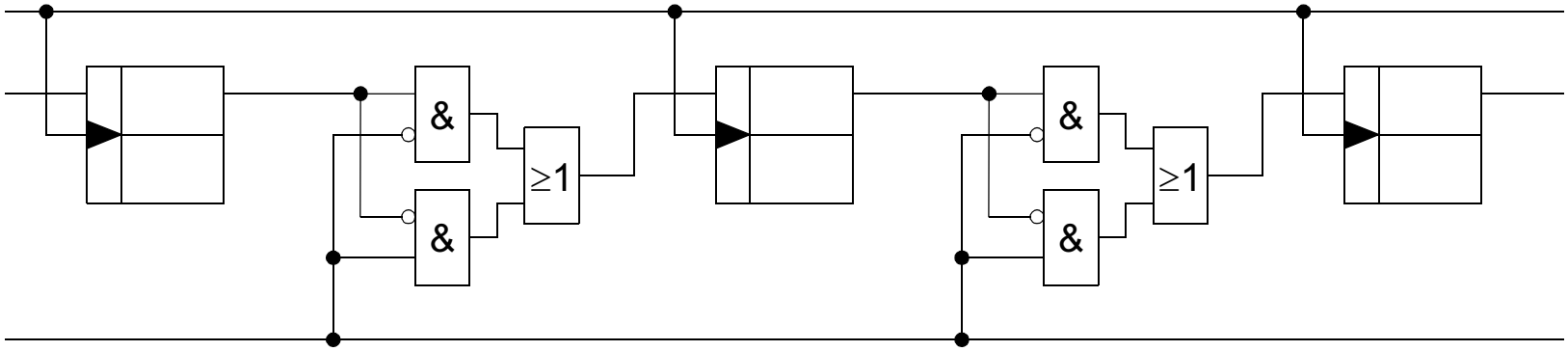
4.2.2 Die Gatter-Ebene (Logik-Ebene)

Auf der Gatter-Ebene (bzw. Logik-Ebene) werden die Eigenschaften eines digitalen Systems durch logische Gleichungen beschrieben. Als Elemente stehen logische Gatter und elementare Speicherglieder (Flipflops) zur Verfügung.

Auf der Gatter-Ebene arbeitet man ausschließlich mit digitalen Signalen, die durch Boolesche Größen beschrieben werden. Effekte wie z. B. Einschwingvorgänge und Reflexionen werden durch Verzögerungszeiten (Gatterlaufzeiten) abstrahiert. Realisierungsaspekte wie z. B. die verwendete Technologie und die Definition der Logikpegel (d. h. die Umsetzung der analogen Größen Strom bzw. Spannung in die digitalen Größen „0“ und „1“ bzw. „true“ und „false“) bleiben auf der Gatter-Ebene verborgen.

Gatter-Ebene (Logik-Ebene)	
Elemente	Gatter, Flipflops
Zeiteinheiten	Gatterlaufzeiten
Informationseinheiten	Bits
Strukturbeschreibung	Schaltplan, Netzlisten
Verhaltensbeschreibung	Schaltalgebraische Gleichungen
Methoden	Anwendung der Axiome und Sätze der Schaltalgebra, rechnergestützte Minimierungs- und Optimierungsverfahren

Beispiel



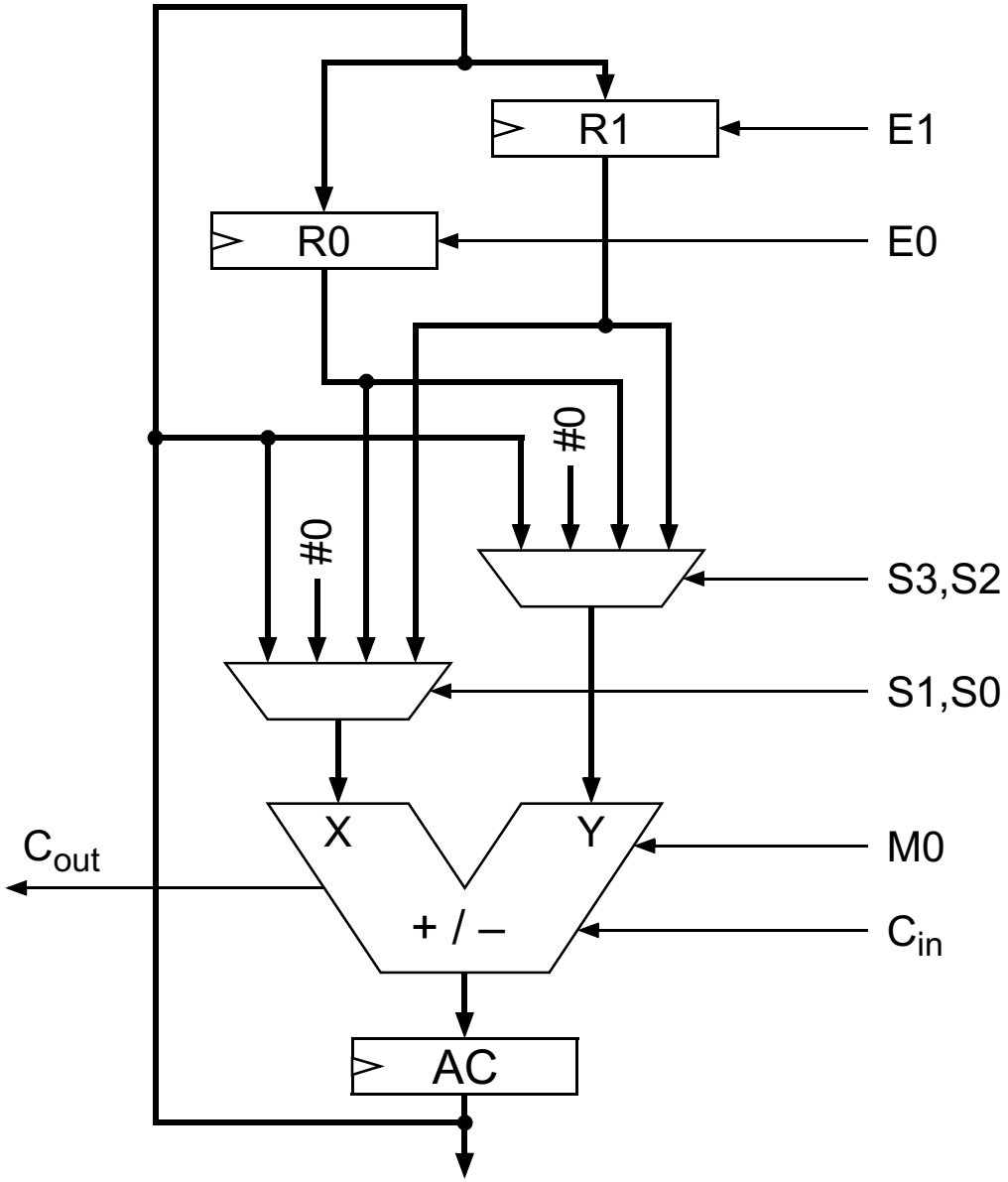
4.2.3 Die Register-Transfer-Ebene

Mit Hilfe logischer Gatter werden komplexere kombinatorische Elemente wie Multiplexer, Addierer und ALUs aufgebaut. Mehrere Flipflops werden zu Registern zusammengefasst. Steuerwerke, die mikroprogrammiert oder festverdrahtet als sequenzielle Netzwerke ausgeführt werden können, erlauben die Steuerung der kombinatorischen Elemente und der Register. Baugruppen und Elemente der Register-Transfer-Ebene werden über Bussysteme miteinander verbunden.

Register-Transfer-Ebene	
Elemente	Datentore, Multiplexer, Register, Addierer, Komparatoren, Arithmetische Einheiten, ALUs, Schiebe- und Rotierwerke, Steuerwerke, Speicherblöcke
Zeiteinheiten	eine Taktperiode
Informationseinheiten	Worte
Strukturbeschreibung	Blockschaltbild, VHDL[1]
Verhaltensbeschreibung	Endliche Automaten, Ablaufdiagramme, Mikroprogramme, VHDL[1]
Methoden	Entwurf digitaler Systeme, VHDL[1]

[1] Hardware-Beschreibungssprache, VHDL = VHSIC Hardware Description Language, VHSIC = Very High Speed Integrated Circuit

Beispiel



4.2.4 Die Mikroprogramm-Ebene

Auf der Mikroprogramm-Ebene werden mit Hilfe elementarer Operationen komplexere Abläufe realisiert. Die elementaren Operationen werden jeweils während eines einzelnen Taktzyklus ausgeführt. Beispiele für solche elementare Operationen sind Registertransfers, einzelne Lese- oder Schreibzyklen und arithmetische Operationen, die sich direkt auf Register beziehen. Der Satz möglicher elementarer Operationen spiegelt die Struktur des Systems auf der Register-Transfer-Ebene wider. Elementare Operationen können in der Regel parallel ausgeführt werden, wenn sie sich auf separate Baugruppen bzw. Elemente des Systems beziehen. Die parallele Ausführung mehrerer elementarer Operationen wird durch ein Mikrocodesteuerwort ausgelöst. Eine Folge von Mikrocodesteuerworten (Mikroprogramm) wird in einem Mikrocodespeicher abgelegt, der meist als interner Nur-Lese-Speicher (ROM) ausgeführt ist.

Die Spezifikation der Abläufe erfolgt in der Regel durch eine Mikrocode-Assemblersprache. Dabei wird jeder elementaren Operation eine lesbare Syntax zugeordnet, die von einem Mikrocode-Assembler direkt in Mikrocodesteuerworte umgesetzt wird.

Mikroprogramm-Ebene	
Elemente	Register, Datenpfade, Operatoren
Zeiteinheiten	eine Taktperiode
Informationseinheiten	Worte
Strukturbeschreibung	Blockschaltbild, VHDL
Verhaltensbeschreibung	Mikrocode-Assemblersprachen
Methoden	Mikrocode-Programmierung

Beispiel

```
                                    MY:=[ PC] ;        PC:=++PC.
                          AA:=Aq+Dj;      MD:=[ PC] ;        PC:=++PC.
                          AA:=AA+MD.
                          AA:=Aq+Dj;      MX:=(AA).
AC:=MX-MY;        SCC;    AA:=AA+MD;      IR:=[ PC] ;        PC:=++PC.
                                    (AA):=AC;                          END.
```

4.2.5 Die Assembler-Ebene (Maschinen-Ebene)

Die Maschinen-Ebene wird durch das Programmiermodell eines Prozessors charakterisiert. Es beinhaltet einen Satz von Maschinenbefehlen und all die Elemente, auf die Maschinenbefehle zugreifen können. Man spricht auch von der Instruction Set Architecture (ISA) eines Prozessors.

Ein Maschinenbefehl repräsentiert einen komplexen Ablauf, der gegebenenfalls in mehreren Taktzyklen bzw. Mikroprogrammschritten ausgeführt wird und in der Regel mehrere Speicherzugriffe und mehrere interne Operationen umfasst.

Maschinenbefehle werden mit Hilfe eines oder mehrerer Maschinenworte codiert (Operationscode + evtl. Erweiterungsworte) und als Maschinenprogramm in einem meist externen (Haupt-)Speicher abgelegt.

Zur leichteren Lesbarkeit von Maschinenprogrammen wird die Assemblersprache eingeführt. Dabei ist ein Assemblerbefehl die symbolische Darstellung des entsprechenden Maschinenbefehls und seiner Operanden. Assemblerprogramme werden von einem Übersetzungswerkzeug (Assembler) direkt in ausführbaren Maschinencode umgesetzt.

Assembler-Ebene (Maschinen-Ebene)	
Elemente	Register, Speicherzellen
Zeiteinheiten	mehrere Taktperioden (diskret)
Informationseinheiten	Worte
Strukturbeschreibung	Programmiermodell, Speichermodell
Verhaltensbeschreibung	Assemblersprachen
Methoden	Assembler-Programmierung

Beispiel

```
$F881                          CLR    D1
$F882                          CLR    D2
$D100 $00FF                    AND    #$00FF,D0
$8003                          MOVE   D0,D3
$5224              GEN_LOOP:   AND.B  (A0,D1),D0
$8053                          MOVE   D1,-(SP)
$7EC1 $0016                    BSR    PARITY
$F902                          LSL    D2
$D442                          OR     D1,D2
$83C1                          MOVE   (SP)+,D1
$80C0                          MOVE   D3,D0
$A7C1 $0001                    ADD    #1,D1
$97C1 $0004                    CMP    #4,D1
$7EC6 $FFE4                    BNE    GEN_LOOP
```

Maschinencode Assemblercode

4.3 Programmiermodell (Instruction Set Architecture)

Inhaltsübersicht

4.3.1 Motivation und Übersicht

4.3.2 Programmiermodell

4.3.3 Beispielprogramme

4.3.4 Befehlscodierung

4.3.1 Motivation und Übersicht

"Minimalprozessor"

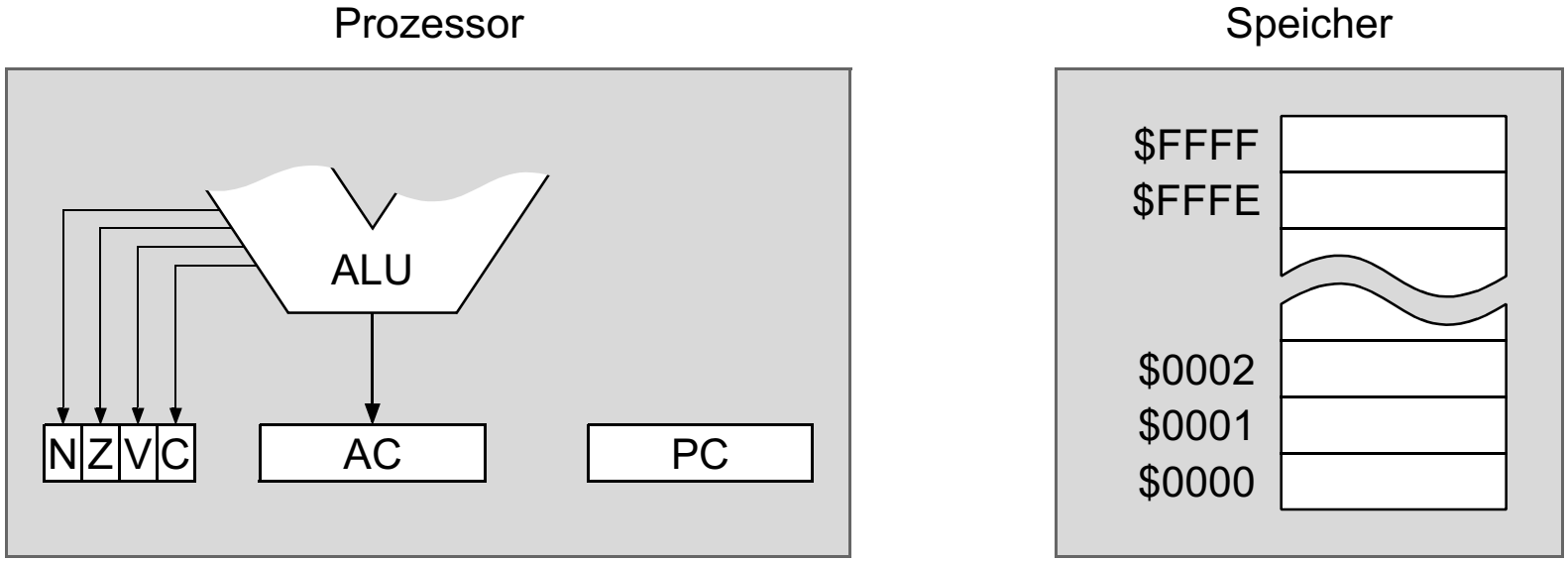
- Beschränkung auf die absolut notwendigen Konzepte
- einfach zu verstehen
- einfach zu implementieren
- prinzipiell praxistauglich
- Leistungsfähigkeit von untergeordnetem Interesse

Ansatz

- akkumulatorbasierte Einadressmaschine
- Wortbreite 16 Bit, Adressraum 2^{16} = 65536 Worte ($0000 ... $FFFF)
- Adressierungsarten implizit, unmittelbar, direkt und indirekt
- festverdrahtetes Steuerwerk (Automat)

4.3.2 Programmiermodell

I. Registermodell und Speichermodell



- Akkumulator-Register AC
- Statusregister mit Statusbits (Flags) N (Negative), Z (Zero), V (oVerflow), C (Carry)
- Befehlsfolgezähler PC
- Quelloperanden: Konstanten, Speicherworte, AC (implizit)
- Zieloperand: AC (implizit, Ablage von AC im Speicher mit STORE Befehl)

II. Befehlssatz

Erforderliche Befehle

- AC mit Operand laden $op \rightarrow AC$
- Operation auf AC ausführen $f(AC) \rightarrow AC$
- Operand mit AC verknüpfen $f(op, AC) \rightarrow AC$
- Operation auf Operand ausführen $f(op) \rightarrow AC$
- AC im Speicher ablegen $AC \rightarrow op$
- unbedingt verzweigen
- bedingt verzweigen
- in Unterprogramm verzweigen (d.h. verzweigen und Rücksprungadresse sichern)
- an dynamische Adresse verzweigen (d.h. an zur Laufzeit ermittelte Adresse)

Erforderliche Adressierungsarten

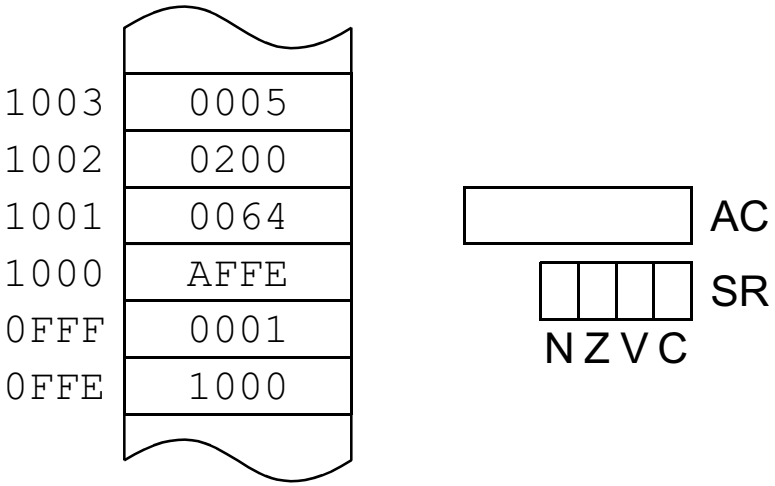
Adressierungsart: bestimmt Art bzw. Ort des Operanden

Mögliche Operanden: AC-Register, Konstante, Speicherwort

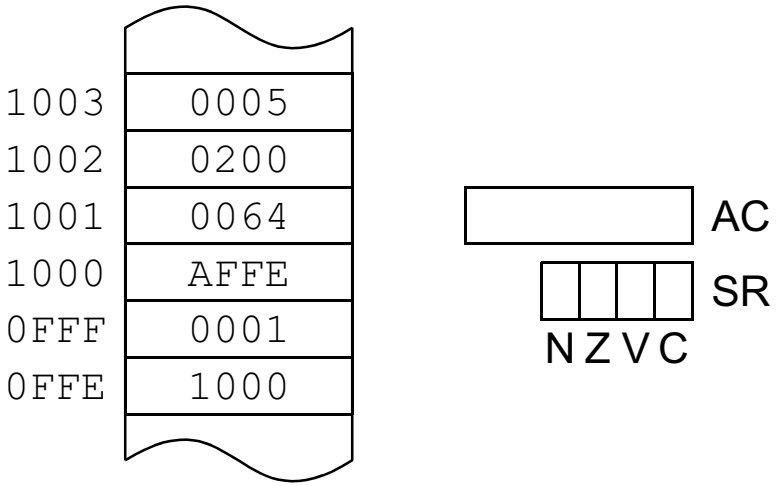
- implizite Adressierung: Operand ist Register AC
- unmittelbare Adressierung: Operand ist Konstante
- direkte Adressierung: Operand im Speicher, Befehl gibt Adresse des Operanden direkt an
- indirekte Adressierung: Operand im Speicher, Befehl gibt Adresse eines Speicherworts an, das die Adresse des Operanden enthält

Beispiele

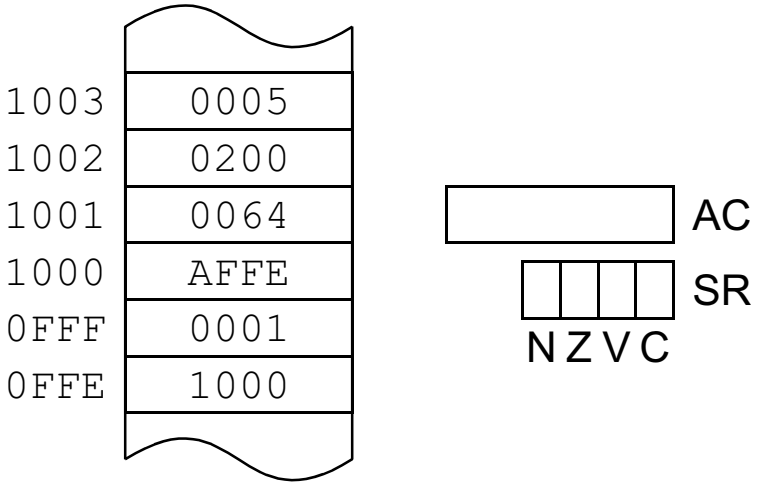
LOAD #100



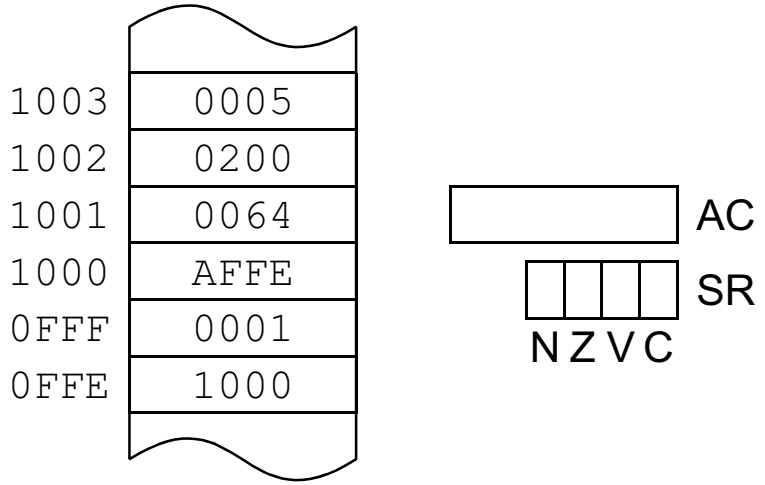
ADD ($1003)



INC



STORE (($0FFE))



Datenübertragungsbefehle

	–	#i	(m)	((m))	
`LOAD`		•	•	•	lade AC
`LOADI`	•				lade AC indirekt, Adresse des Operanden in AC
`STORE`			•	•	speichere AC im Operanden

Datenübertragungsbefehle lassen die Flags im Statusregister unverändert.

Datenverarbeitungsbefehle

	–	#i	(m)	((m))	
AND,OR,XOR		•	•	•	verknüpfe logisch Und, Oder, Exklusiv-Oder
ADD,ADDC		•	•	•	addiere, addiere mit Übertrag
SUB,SUBC		•	•	•	subtrahiere, subtrahiere mit Übertrag
LSL,LSR	•		•	•	schiebe logisch links, rechts
ASL,ASR	•		•	•	schiebe arithmetisch links, rechts
ROL,ROR	•		•	•	rotiere links, rechts
RCL,RCR	•	•	•	•	rotiere durch das Carry-Flag links, rechts
NOT	•		•	•	invertiere (logisches Nicht)
NEG			•	•	negiere
CLR	•				lösche
INC,DEC	•		•	•	inkrementiere, dekrementiere

Datenverarbeitungsbefehle setzen die Flags im Statusregister entsprechend der ausgeführten Operation.

Programmsteuerbefehle

	–	d	
`BRA`		•	verzweige unbedingt
`Bcc`		•	verzweige bedingt, cc = EQ, NE, CC, CS, ... (siehe nächste Seite)
`BSR`		•	verzweige in Unterprogramm (schreibe Folgeadresse nach AC)
`JMP`	•		springe an die Adresse in AC

Programmsteuerbefehle lassen die Flags im Statusregister unverändert.

Bedingte Verzweigungsbefehle werten die Flags aus, um zu entscheiden, ob sie verzweigen oder nicht.

Realisierung von Vergleichsoperationen mit Hilfe bedingter Verzweigungsbefehle

zu vergleichende Operanden: Akkumulator AC, beliebiger Operand X

```
SUB    X        ;subtrahiere X von AC
Bcc    LABEL    ;verzweige an Marke LABEL, wenn Bedingung "cc" zutrifft
```

Gleichheit, Ungleichheit (relevant: Flag Z)

Z = 1	AC = X	**BEQ**	Branch if EQual
Z = 0	AC ≠ X	**BNE**	Branch if Not Equal

Vorzeichenloser Vergleich (relevant: Flags Z und C)

C = 1	AC < X	**BCS**	Branch if Carry Set (Branch if Lower)
C = 0	AC ≥ X	**BCC**	Branch if Carry Clear (Branch if Higher or Same)
C + Z = 1	AC ≤ X	**BLS**	Branch if Lower or Same
C + Z = 0	AC > X	**BHI**	Branch if HIgher

Vorzeichenbehafteter Vergleich (relevant: Flags Z, N und V)

$N\overline{V} + \overline{N}V = 1$	AC < X	**BLT**	Branch if Less Than
$N\overline{V} + \overline{N}V = 0$	AC ≥ X	**BGE**	Branch if Greater or Equal
$N\overline{V} + \overline{N}V + Z = 1$	AC ≤ X	**BLE**	Branch if Less or Equal
$N\overline{V} + \overline{N}V + Z = 0$	AC > X	**BGT**	Branch if Greater Than

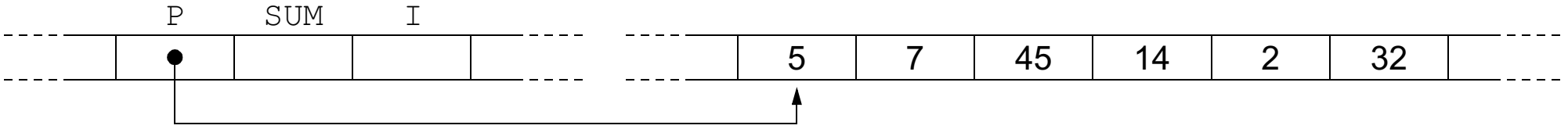
4.3.3 Beispielprogramme

I. Addition der Zahlen von 1 bis 100

II. Addition der Zahlen in einem Array

Speicherstelle P zeigt auf das Array, erstes Element enthält Array-Länge

Beispiel



III. Multiplikation

Multiplikator in MR, Multiplikand in MD

```
MULT:     CLR
          STORE  (PRODUCT)
          LOAD   #16
LOOP:     STORE  (COUNT)
          LSL    (PRODUCT)
          STORE  (PRODUCT)
          LSL    (MR)
          STORE  (MR)
          BCC    NOADD
          LOAD   (MD)
          ADD    (PRODUCT)
          STORE  (PRODUCT)
NOADD:    DEC    (COUNT)
          BNE    LOOP
```

IV. Unterprogrammaufruf

Verwendung einer vordefinierten Speicherstelle SP als Stapelzeiger

```
            LOAD   #17
            STORE  (MD)
            LOAD   #33
            STORE  (MR)
            BSR    MULT
            ...

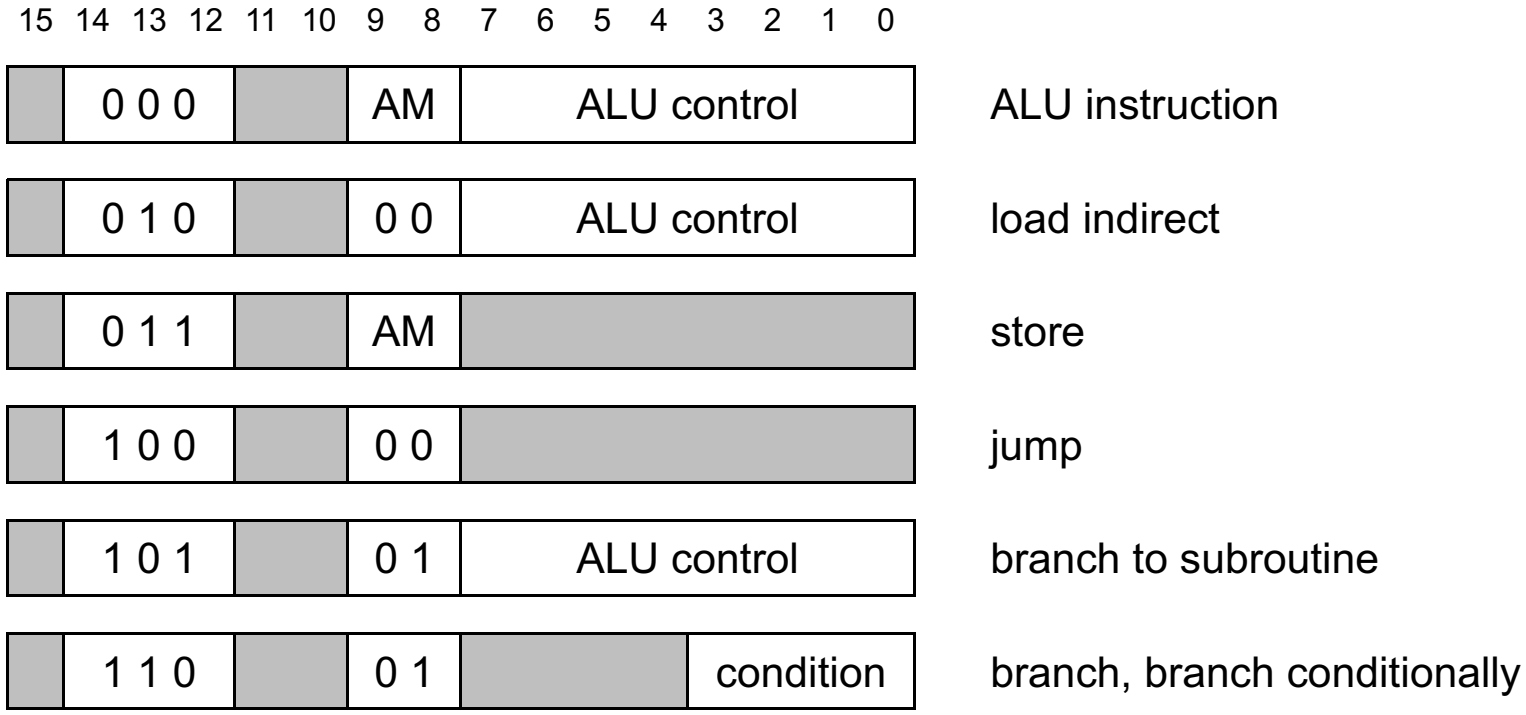
MULT:       STORE  ((SP))
            DEC    (SP)
            STORE  (SP)

            CLR
            STORE  (PRODUCT)
            ...
            BNE    LOOP

            INC    (SP)
            STORE  (SP)
            LOAD   ((SP))
            JMP
```

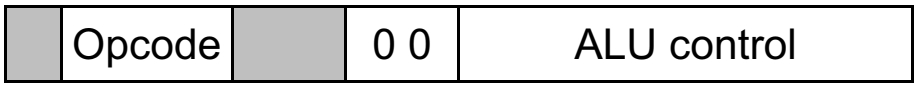
4.3.4 Befehlscodierung

Übersicht

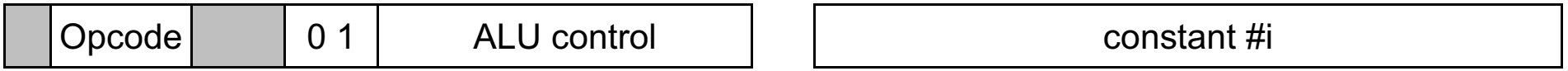


Adressierungsarten und Erweiterungsworte

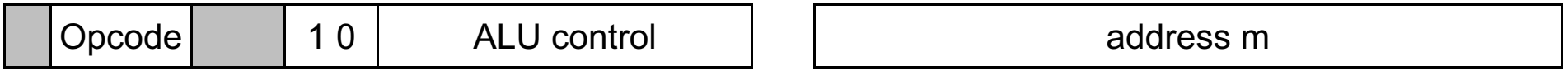
implizite Adressierung (kein expliziter Operand)



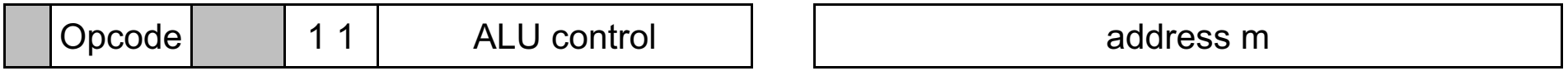
unmittelbare Adressierung



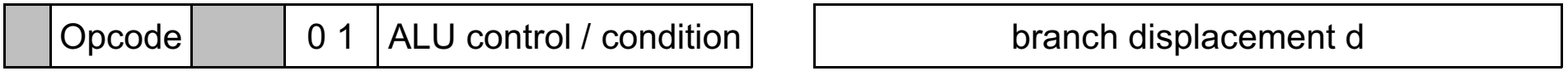
direkte Adressierung (Speicher-direkt)



indirekte Adressierung (Speicher-indirekt)



programmzählerrelativ (relative Sprünge)



Befehlswort Erweiterungswort

Vollständiger Befehlssatz mit Codierung

Mnemonic	impl.	#i/d	(m)	((m))	Beschreibung
LOAD	-	$010C	$020C	$030C	lade AC
OR	-	$0188	$0288	$0388	verknüpfe logisch Oder
XOR	-	$0189	$0289	$0389	verknüpfe logisch Exklusiv-Oder
AND	-	$018A	$028A	$038A	verknüpfe logisch Und
ADD	-	$018D	$028D	$038D	addiere
ADDC	-	$01AD	$02AD	$03AD	addiere mit Übertrag
SUB	-	$018E	$028E	$038E	subtrahiere
SUBC	-	$01AE	$02AE	$03AE	subtrahiere mit Übertrag

Mnemonic	impl.	#i/d	(m)	((m))	Beschreibung
LSL	$00A0		$0220	$0320	schiebe logisch links
LSR	$00A1		$0221	$0321	schiebe logisch rechts
ROL	$00A2		$0222	$0322	rotiere links
ROR	$00A3		$0223	$0323	rotiere rechts
ASL	$00A4		$0224	$0324	schiebe arithmetisch links
ASR	$00A5		$0225	$0325	schiebe arithmetisch rechts
RCL	$00A6	$0126	$0226	$0326	rotiere durch das C-Flag links
RCR	$00A7	$0127	$0227	$0327	rotiere durch das C-Flag rechts
NOT	$008B		$020B	$030B	invertiere (logisches Nicht)
NEG			$024E	$034E	negiere
CLR	$004C				lösche
INC	$009C		$021C	$031C	inkrementiere (addiere eins)
DEC	$009F		$021F	$031F	dekrementiere (subtrahiere eins)
LOADI	$200C				lade indirekt
STORE			$3200	$3300	speichere AC im Operanden
JMP	$4000				springe an die Adresse in AC

Mnemonic	impl.	#i/d	(m)	((m))	Beschreibung
BSR		$510C			verzweige, schreibe Folgeadresse nach AC
BRA		$6101			verzweige
BHI		$6102			verzweige, wenn "HIgher"
BLS		$6103			verzweige, wenn "Lower or Same"
BCC		$6104			verzweige, wenn "Carry Clear"
BCS		$6105			verzweige, wenn "Carry Set"
BNE		$6106			verzweige, wenn "Not Equal"
BEQ		$6107			verzweige, wenn "EQual"
BVC		$6108			verzweige, wenn "oVerflow Clear"
BVS		$6109			verzweige, wenn "oVerflow Set"
BPL		$610A			verzweige, wenn "PLus"
BMI		$610B			verzweige, wenn "MInus"
BGE		$610C			verzweige, wenn "Greater or Equal"
BLT		$610D			verzweige, wenn "Less Than"
BGT		$610E			verzweige, wenn "Greater Than"
BLE		$610F			verzweige, wenn "Less or Equal"

4.4 Register-Transfer-Ebene

Auf der Register-Transfer-Ebene werden mehrere Boolesche Größen zu Booleschen Vektoren oder Worten zusammengefasst, entsprechend mehrere Signale zu Bussen ("Signalsammelschienen").

$A = (A_{n-1}, A_{n-2}, \ldots, A_2, A_1, A_0)$, mit Wortbreite n

In der Regel findet man in komplexeren Systemen in Abhängigkeit vom Typ der darzustellenden Information (Datenleitungen, Adressleitungen, Steuerleitungen) mehrere Busse mit verschiedenen Wortbreiten. Die auf diesen Bussen in Form Boolescher Vektoren übertragene Information wird in Abhängigkeit vom Anwendungsfall und Kontext auf verschiedene Weisen interpretiert, z. B.

- als Darstellung einer ganzen Zahl (vorzeichenbehaftet oder vorzeichenlos)
- als Darstellung einer gebrochenen Zahl (Festkomma oder Fließkomma)
- als Code für ein Zeichen (z. B. ASCII-Code, Unicode)
- als Code für einen Befehl (Operationscode)
- als Steuerwort
- ...

Die Ein- und Ausgangssignale von Elementen der Register-Transfer-Ebene werden in Datensignale und in Steuersignale gegliedert.

Für die Erzeugung der Steuersignale gibt es prinzipiell zwei Möglichkeiten

- **Festverdrahtete Steuerwerke** werden durch Endliche Automaten (Finite State Machines, FSMs) bzw. durch Systeme kommunizierender Endlicher Automaten realisiert.
- **Mikroprogrammierte Steuerwerke** arbeiten ein in einem Speicher (Mikroprogrammspeicher) abgelegtes Steuerprogramm (Mikroprogramm) ab.

Die Elemente der Register-Transfer-Ebene werden über Freigabe- und/oder Auswahlsignale gesteuert und verarbeiten Boolesche Vektoren. Sie werden über Busse und Steuerleitungssysteme miteinander verbunden.

Inhaltsübersicht

4.4.1 Datenselektoren (Multiplexer)

4.4.2 Datentore (Tristate-Buffer)

4.4.3 Register

4.4.4 Elementare Speicherblöcke

4.4.5 Kombinatorische Schiebe- und Rotierwerke

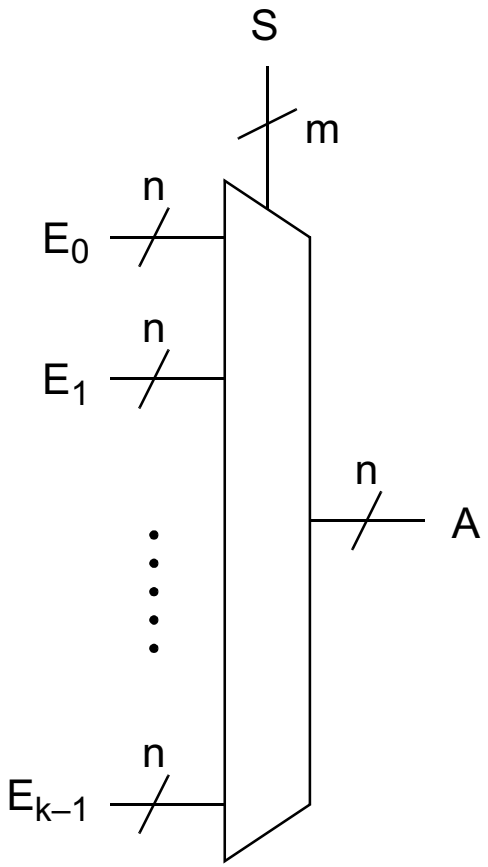
4.4.6 Arithmetische Elemente

4.4.7 Dekoder und Enkoder

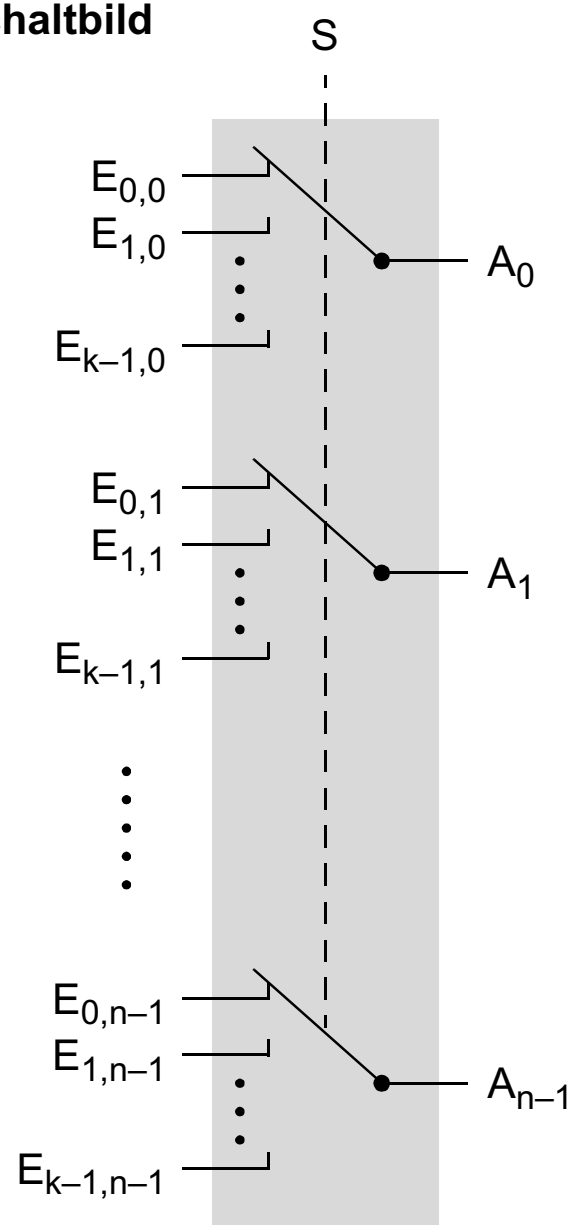
4.4.1 Datenselektoren (Multiplexer)

Datenselektoren (Multiplexer) dienen dazu, in Abhängigkeit von einem oder mehreren Auswahleingängen (Select) einen von mehreren Eingangsbussen auf den Ausgangsbus durchzuschalten. Die Anzahl notwendiger Auswahleingänge m hängt von der Anzahl möglicher Eingangsbusse k ab ($m \geq \mathrm{ld}\ k$).

Schaltzeichen

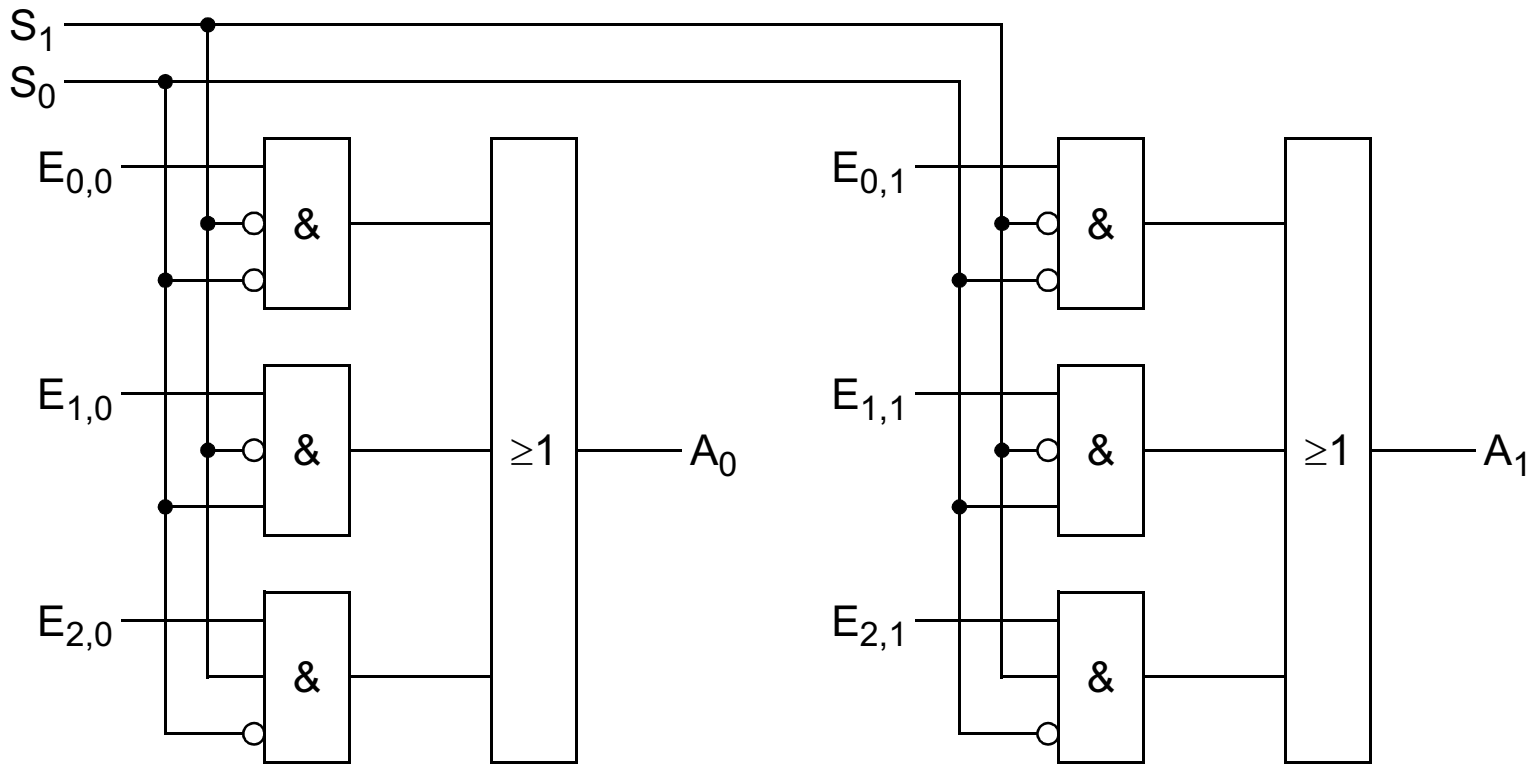


Ersatzschaltbild



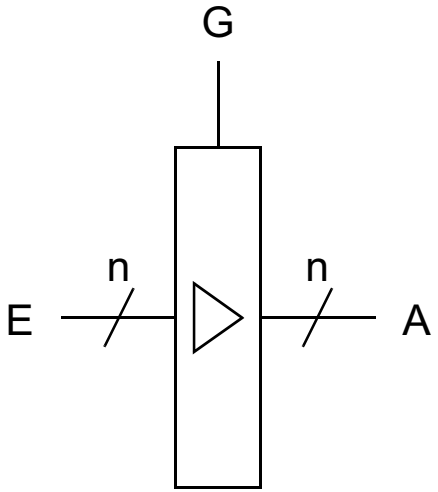
Realisierung auf Gatterebene

Wortbreite n = 2, Anzahl Eingangsbusse k = 3 → Anzahl Steuerleitungen m = 2

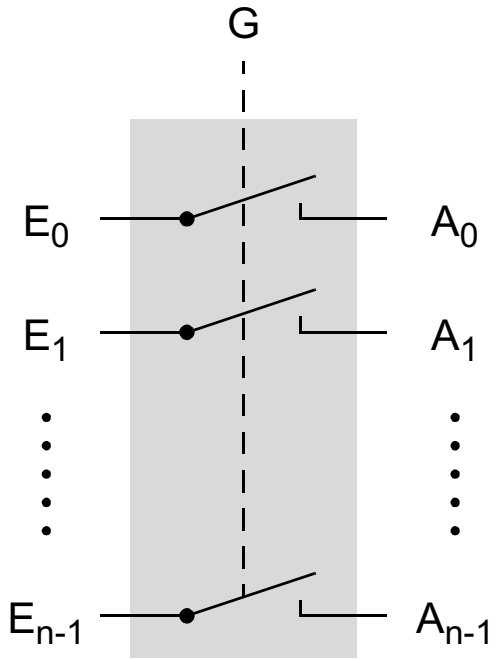


4.4.2 Datentore (Tristate-Buffer)

Datentore (oder Tristate-Buffer) verhalten sich wie Schalter, die nur in einer Richtung betrieben werden. Wenn der Steuereingang G (Gate) aktiviert ist (logisch „1“), dann werden die Eingänge mit den Ausgängen verbunden. Ist G inaktiv (logisch „0“), dann werden die Ausgänge von den Eingängen abgetrennt und gehen in einen hochohmigen Zustand über. Dieser Zustand wird manchmal als "3. Zustand", "High Impedance", "High-Z" oder nur mit "Z" bezeichnet. Aus diesem Grund nennt man das Element auch „Tristate-Buffer“.



Schaltzeichen



Ersatzschaltbild

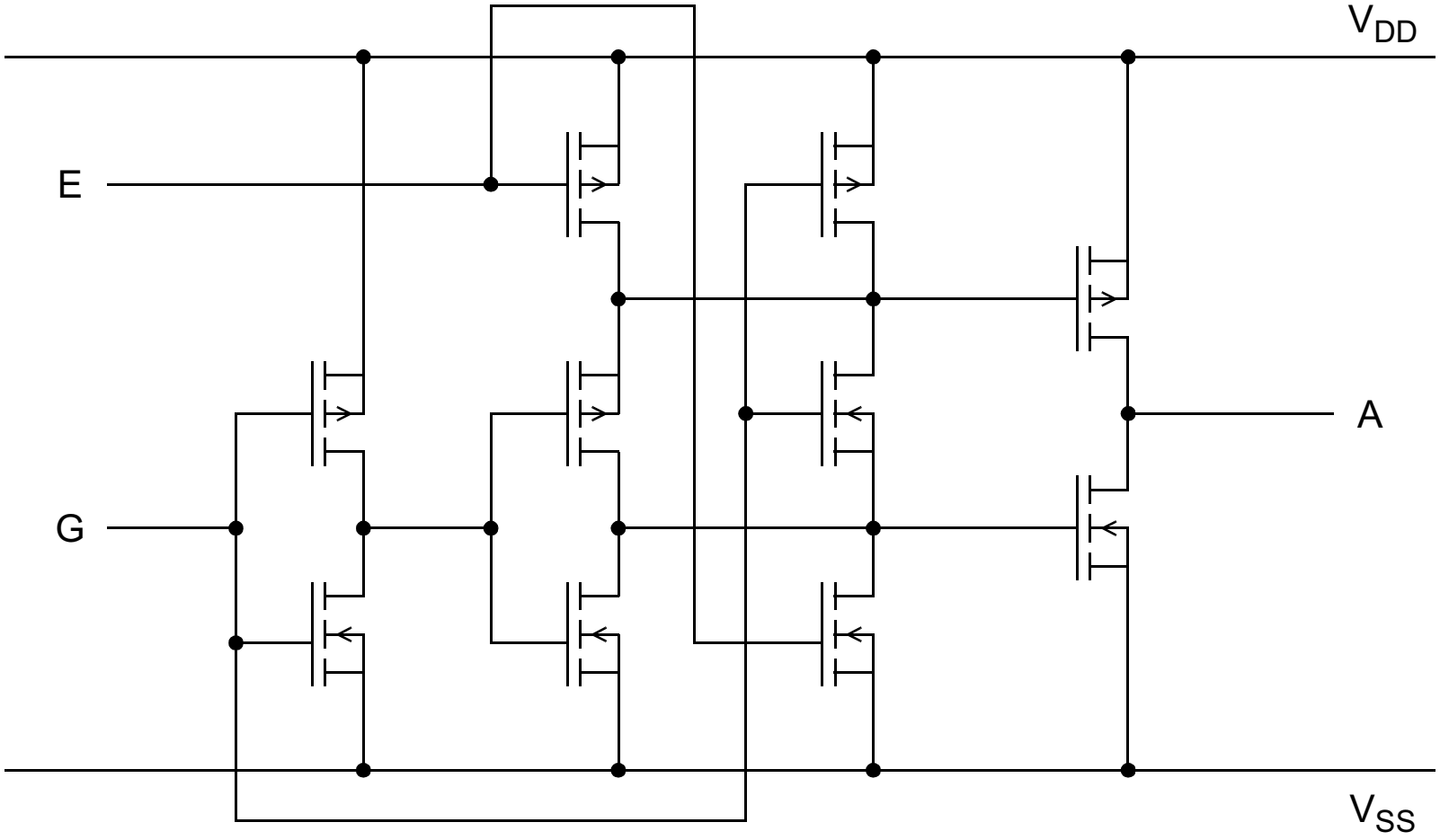
Wahrheitstabelle

G	E_i	A_i	
0	0	Z	hochohmig, „3. Zustand“
0	1	Z	
1	0	0	logische "0"
1	1	1	logische "1"

Datentore haben keine direkte Entsprechung auf der rein logischen Gatterebene. Sie werden benötigt, um verteilte und erweiterbare logische Strukturen zu realisieren (z.B. verteilte Multiplexer).

Realisierung auf Schaltkreisebene

Im folgenden Bild ist die Realisierung eines Datentors für eine Leitung (n = 1) in CMOS-Technik dargestellt. Wenn das Gate G = 0 ist, so sperren beide Ausgangstransistoren und der Ausgang A ist in einem hochohmigen Zustand. Für G = 1 leitet in Abhängigkeit vom Eingang E jeweils einer der beiden Ausgangstransistoren.



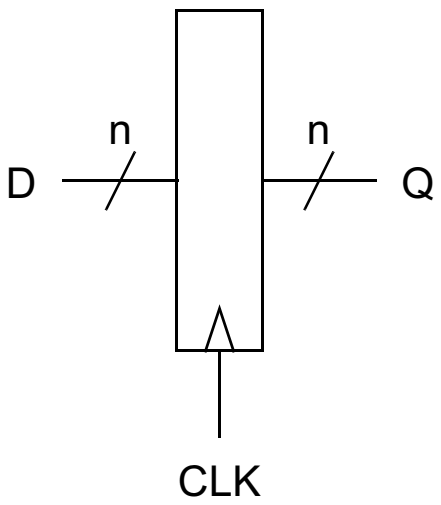
4.4.3 Register

Register werden verwendet, um Informationen einen oder mehrere Takte lang zwischenzuspeichern. Man unterscheidet

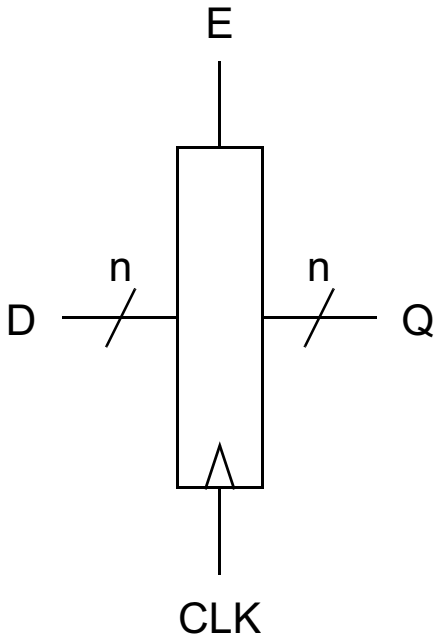
- **steuerbare Register**, die den Wert an den Eingängen bei einer aktiven Taktflanke des Systemtaktes nur dann übernehmen, wenn ein Freigabeeingang aktiviert ist
- **nicht steuerbare Register**, die den Wert an den Eingängen bei jeder aktiven Systemtaktflanke übernehmen

In synchronen Systemen werden die Takteingänge aller getakteten Elemente stets mit einem zentralen Systemtakt versorgt, d.h. der Takteingang eines Registers wird nicht als Steuereingang betrachtet. Aus diesem Grund können nicht steuerbare Register Informationen nur eine Taktperiode lang speichern.

Schaltzeichen

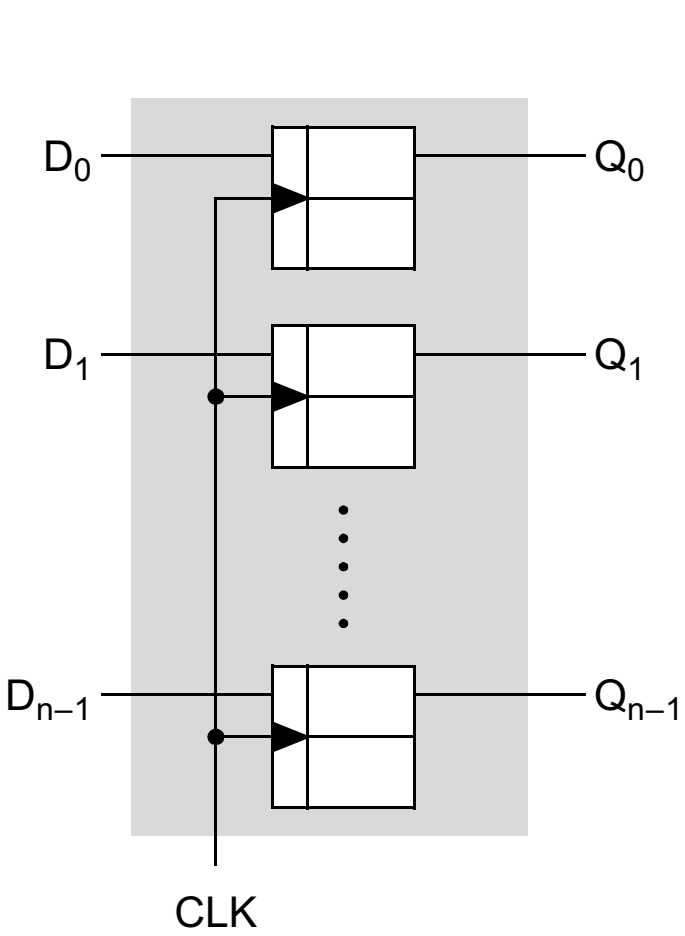


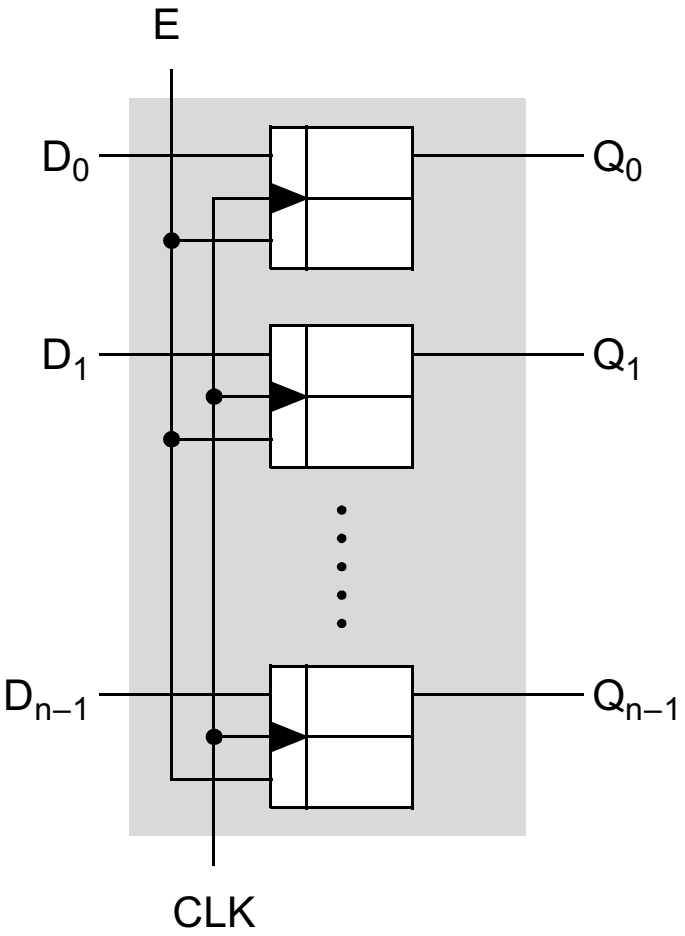
nicht steuerbares Register



steuerbares Register

Realisierung auf Gatterebene





Zur Realisierung von steuerbaren Registern auf Gatterebene verwendet man D-Flipflops mit Freigabeeingang (Enable-D-Flipflops).

E^n	D^n	Q^{n+1}	
0	0	Q^n	Registerinhalt beibehalten (disabled)
0	1	Q^n	
1	0	0	Eingangsdaten übernehmen (enabled)
1	1	1	

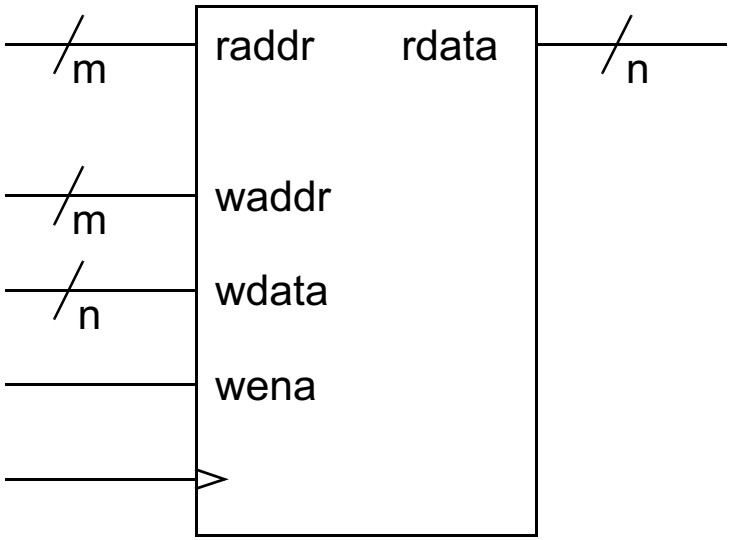
Wahrheitstabelle des Enable-D-Flipflops

Aus der Wahrheitstabelle lässt sich auf einfache Weise die charakteristische Schaltfunktion des Enable-D-Flipflops ableiten:

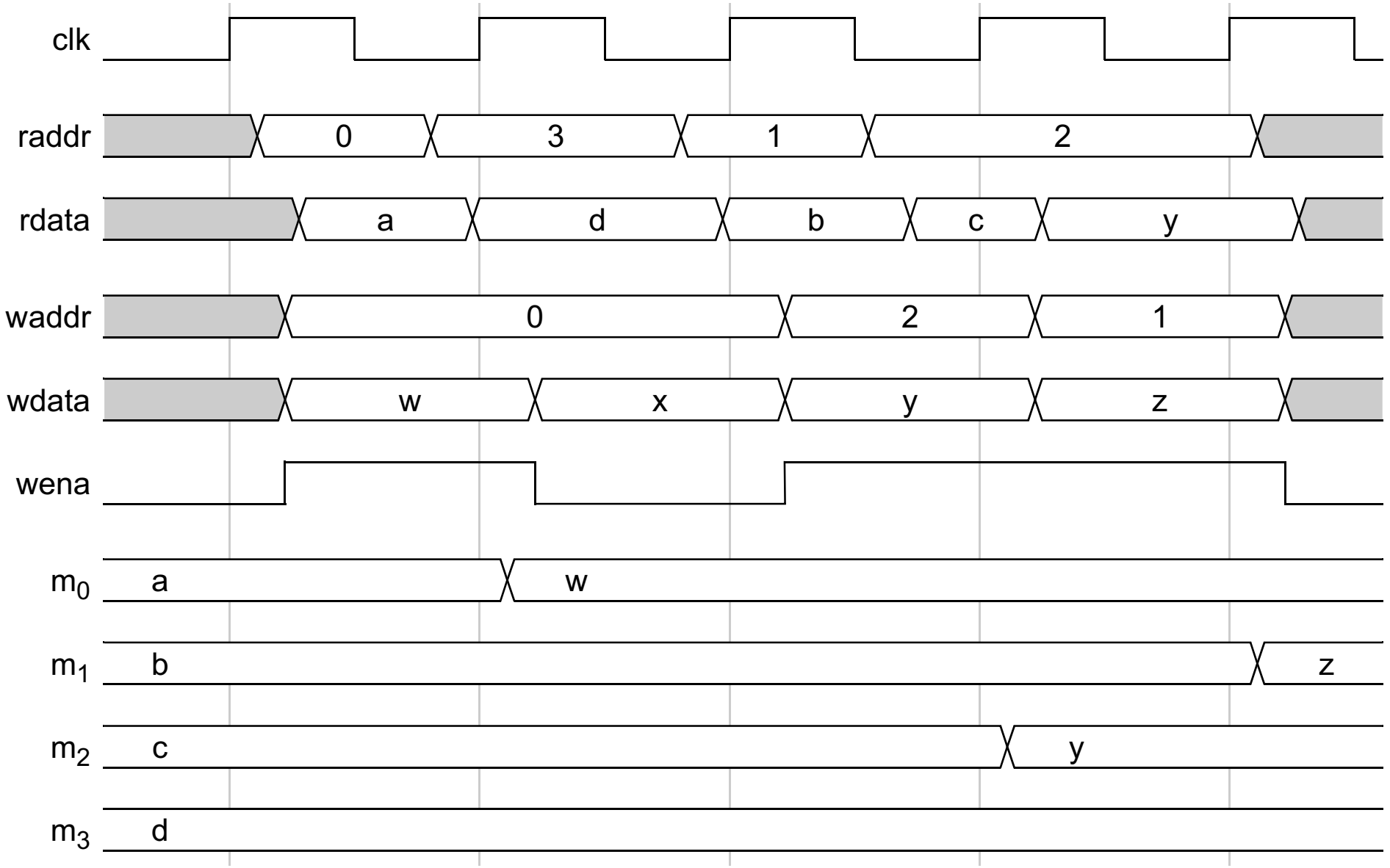
$$Q^{n+1} = (\overline{E}Q + ED)^n$$

4.4.4 Elementare Speicherblöcke

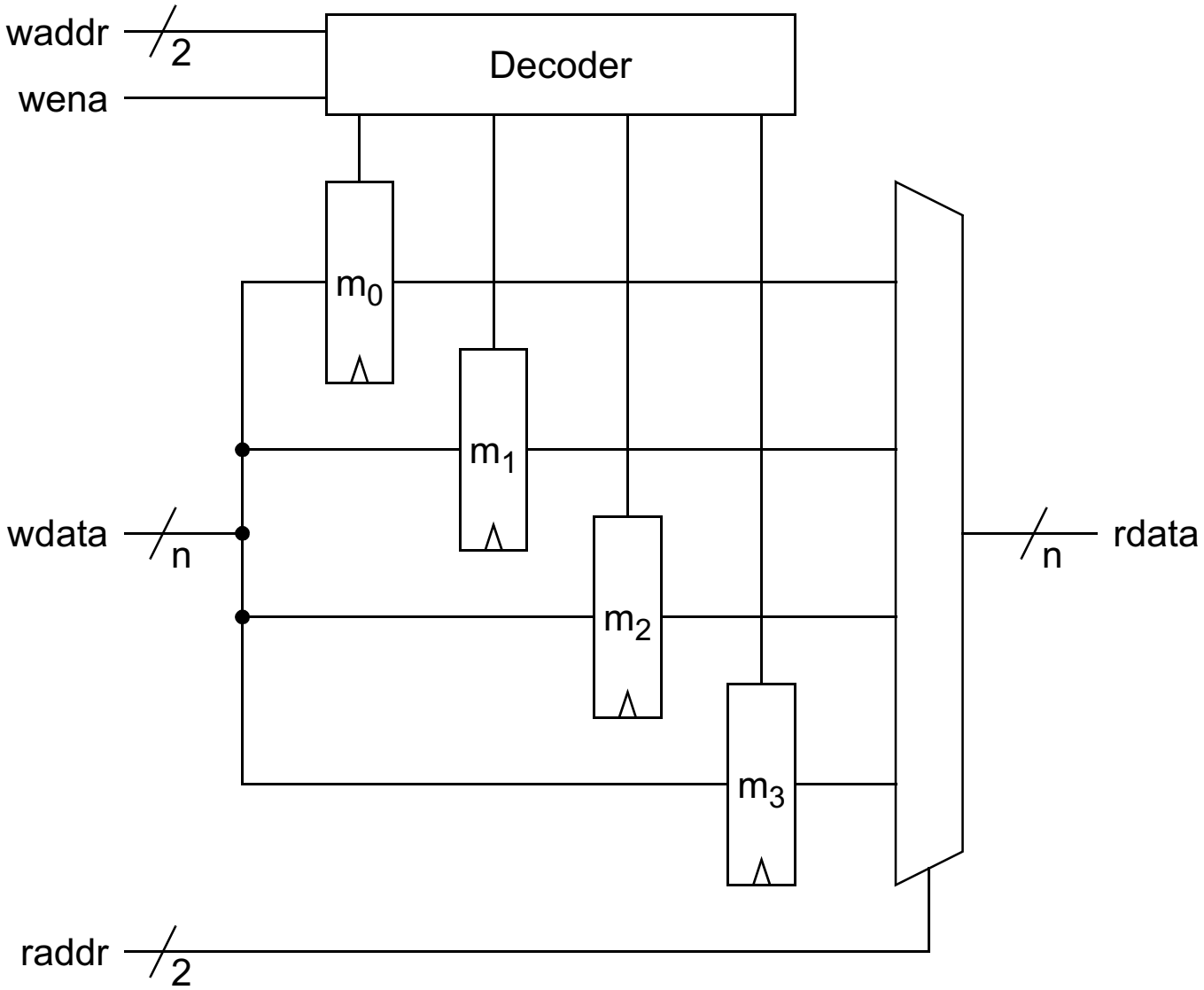
Beispiel: asynchrones Lesen, synchrones Schreiben, getrennte Schreib- und Leseports.



Signal-Zeit-Diagramm



Realisierung auf RT-Ebene (für kleine m, Beispiel m = 2)



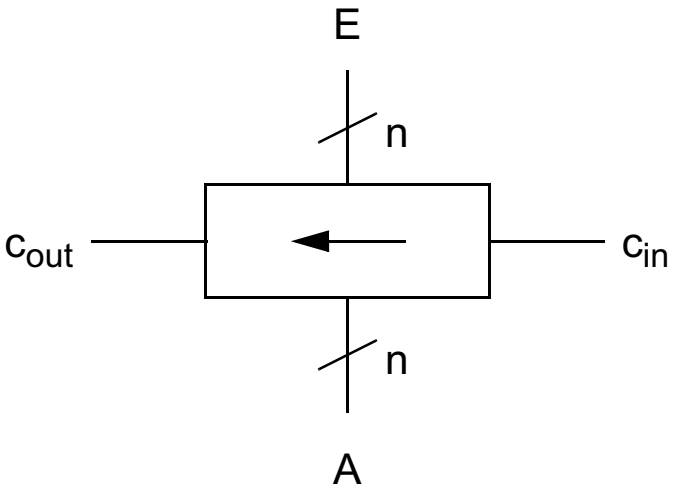
4.4.5 Kombinatorische Schiebe- und Rotierwerke

I. Festverdrahtete Schiebewerke

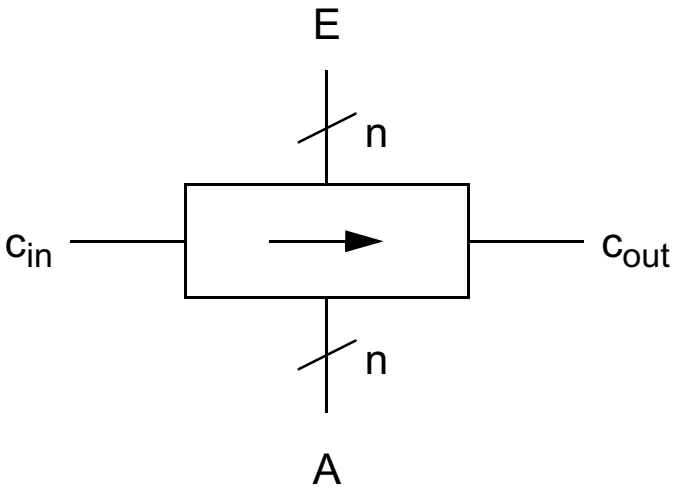
Festverdrahtete Schiebewerke kommen ohne Bauelemente aus. Sie stellen lediglich eine Abstraktion für bestimmte Verdrahtungsschemata dar.

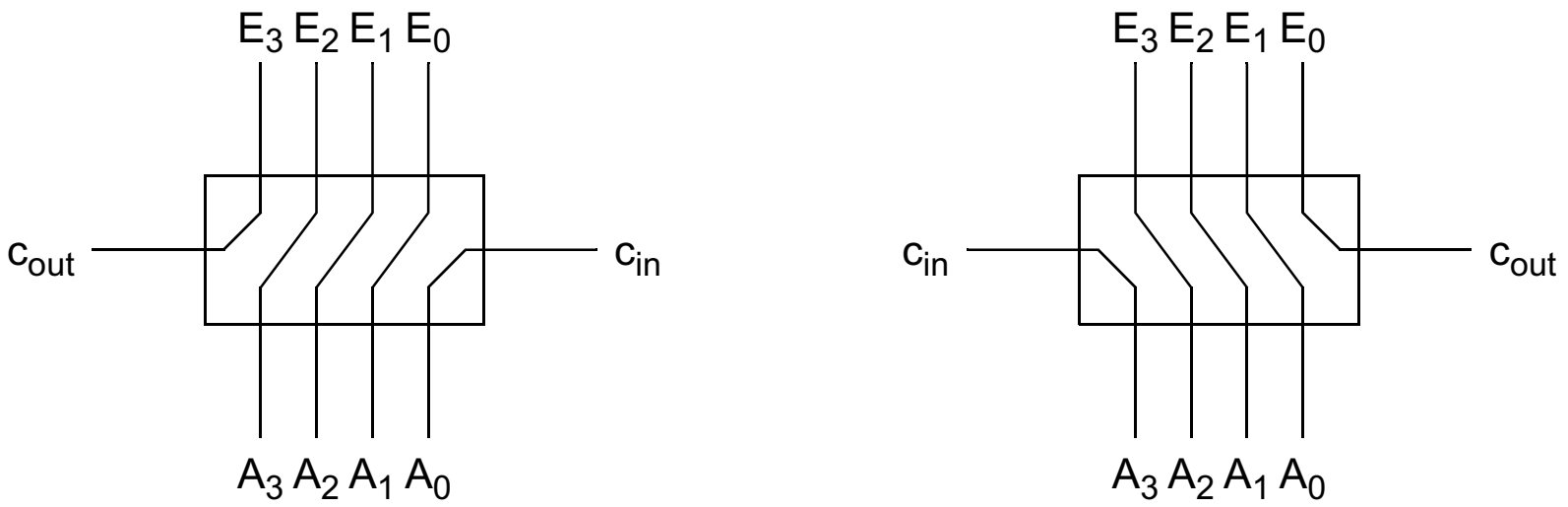
Schaltzeichen

festverdrahtetes Linksschiebewerk



festverdrahtetes Rechtsschiebewerk





Realisierung (Beispiel n = 4)

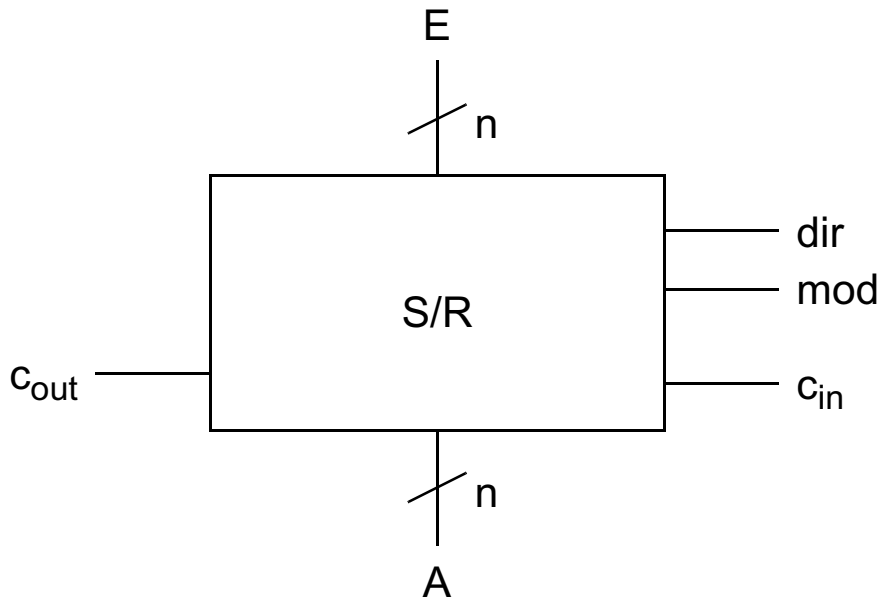
Entsprechend lassen sich festverdrahtete Schiebe- bzw. Rotierwerke realisieren, die um mehr als eine Stelle schieben bzw. rotieren. Dabei muss festgelegt werden, auf welche Weise der bzw. die Übertragseingänge interpretiert werden und auf welche Weise der bzw. die Übertragsausgänge erzeugt werden.

II. Einstellbare Schiebe- und Rotierwerke

Einstellbare Schiebe- und Rotierwerke verfügen neben den Ein- und Ausgängen für Daten und Überträge einen oder mehrere Steuereingänge, mit deren Hilfe die Betriebsart (Schieben oder Rotieren, links oder rechts, Anzahl der Stellen) des Schiebe- und Rotierwerks festgelegt wird.

Beispiel 1

Schiebe- und Rotierwerk, das um jeweils eine Stelle nach links oder nach rechts schieben bzw. rotieren kann.



Schaltzeichen

Für die Steuereingänge dir und mod (Auswahlsignale, Typ Select) sollen die Festlegungen gemäß nachfolgend angegebener Tabelle gelten:

mod	dir	Funktion
0	0	links schieben
0	1	rechts schieben
1	0	links rotieren (n Bit ohne Carry)
1	1	rechts rotieren (n Bit ohne Carry)

Die Realisierung dieses Schiebe- und Rotierwerks auf Gatterebene kann auf einfache Weise durch Umsetzung der folgenden Booleschen Gleichungen erfolgen

$$A_0 = c_{in}\,\overline{dir}\,\overline{mod} + E_3\,\overline{dir}\,mod + E_1\,dir$$

$$A_1 = E_0\,\overline{dir} + E_2\,dir$$

$$A_2 = E_1\,\overline{dir} + E_3\,dir$$

$$A_3 = E_2\,\overline{dir} + c_{in}\,dir\,\overline{mod} + E_0\,dir\,mod$$

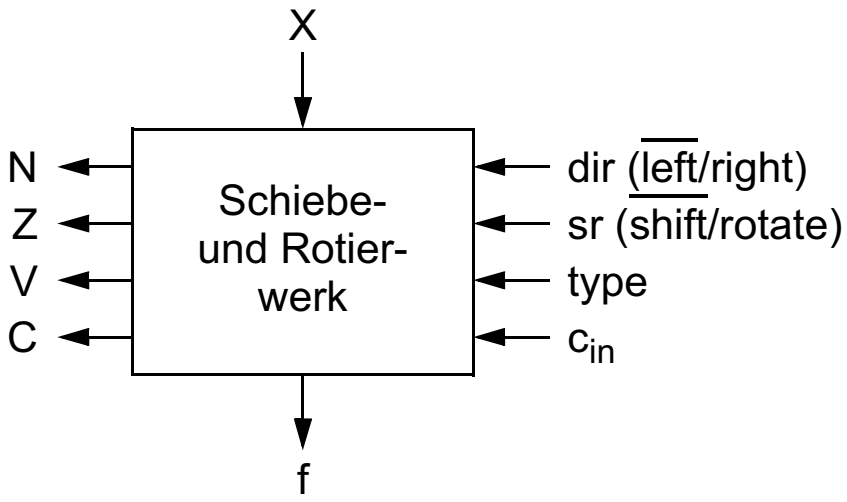
$$c_{out} = E_3\,\overline{dir} + E_0\,dir$$

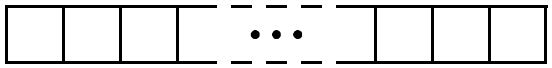
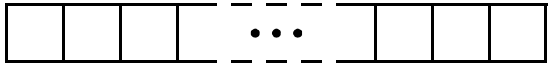
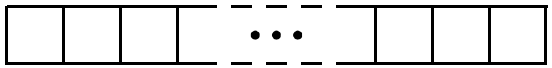
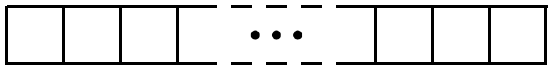
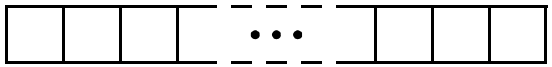
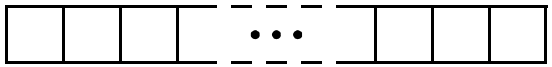
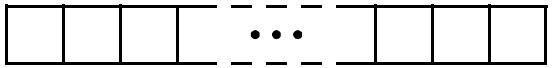
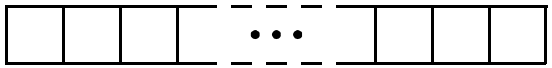
Verfügt ein rein kombinatorisches (d.h. ungetaktetes) Schiebe- und Rotierwerk über Steuereingänge, mit deren Hilfe die Anzahl der Stellen, um die geschoben werden soll, beliebig festgelegt werden, so spricht man von einem **Barrel Shifter**. Barrel Shifter sind insbesondere für große n sehr aufwändig.

Beispiel 2

Schiebe- und Rotierwerk mit folgenden Optionen

- links oder rechts
  - schieben
    - logisch
    - arithmetisch
  - rotieren
    - normal
    - mit Carry



type	s/r	dir	Operation	Illustration
0	0	0	logical shift left	
0	0	1	logical shift right	
0	1	0	rotate left	
0	1	1	rotate right	
1	0	0	arithmetic shift left	
1	0	1	arithmetic shift right	
1	1	0	rotate through carry left	
1	1	1	rotate through carry right	

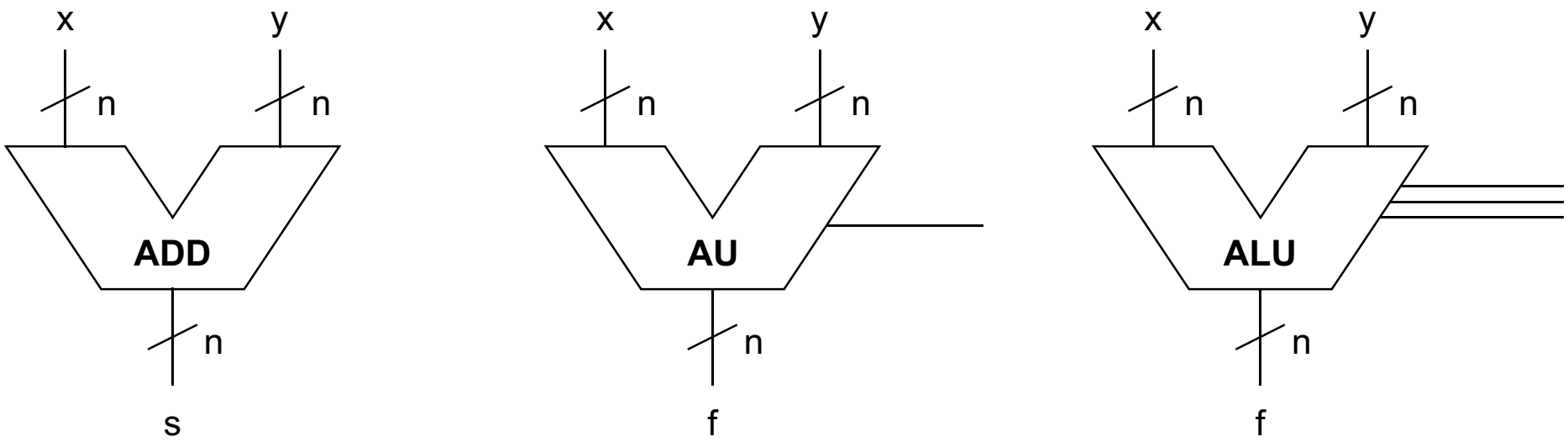
4.4.6 Arithmetische Elemente

Beispiele für arithmetische Elemente auf der Register-Transfer-Ebene

- Addierer, Subtrahierer, Komparatoren
- Arithmetische Einheiten, Arithmetisch-Logische Einheiten
- Einschrittige Multiplizierwerke

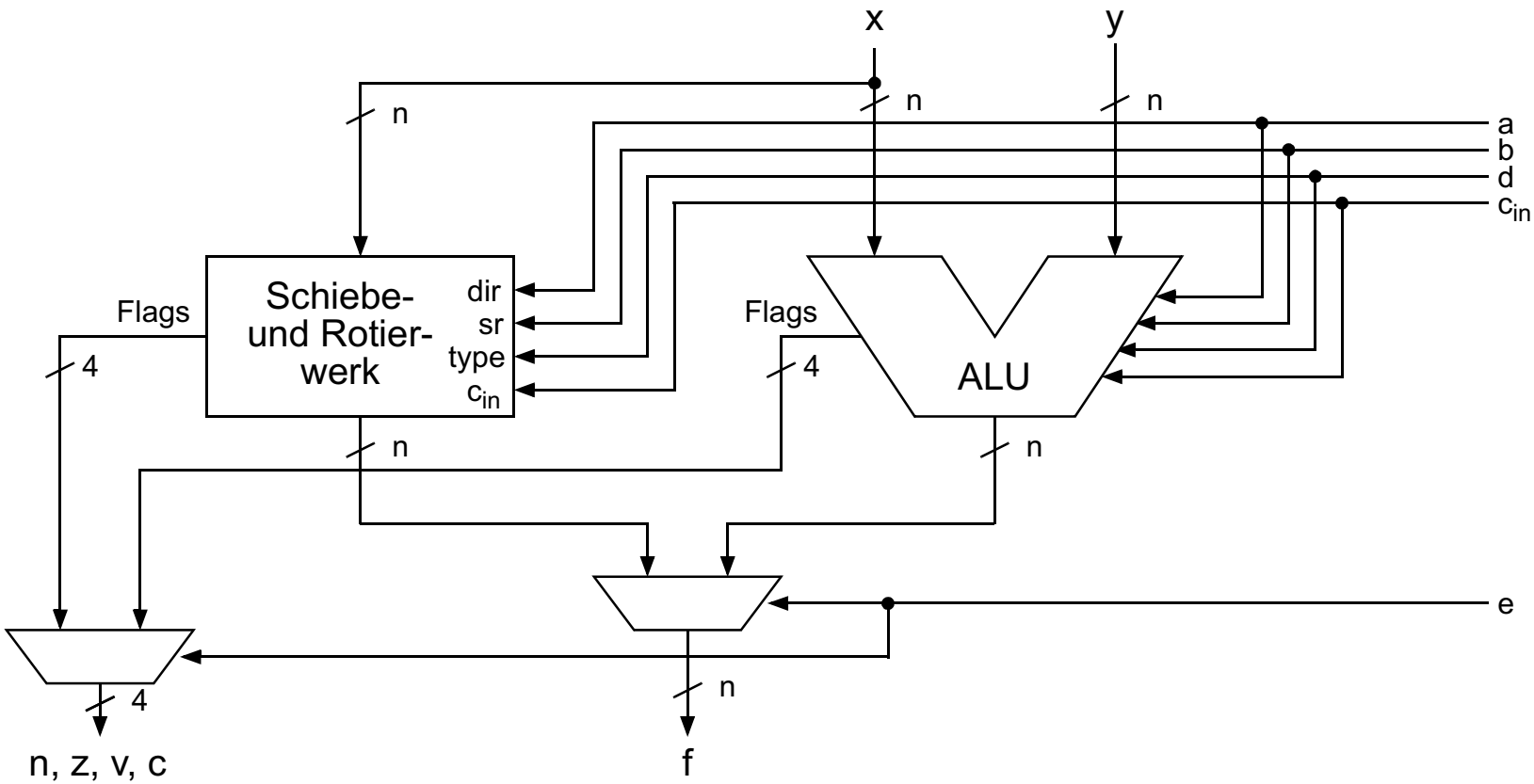
Arithmetische Elemente werden in Blockschaltbildern der Register-Transfer-Ebene in der Regel durch das V-Symbol repräsentiert, das entsprechend beschriftet und mit Steuereingängen versehen wird.

Beispiele



Realisierung einer ALU mit Schiebe- und Rotierwerk

Parallelschaltung einer Standard-ALU und eines Schiebe-/Rotierwerks

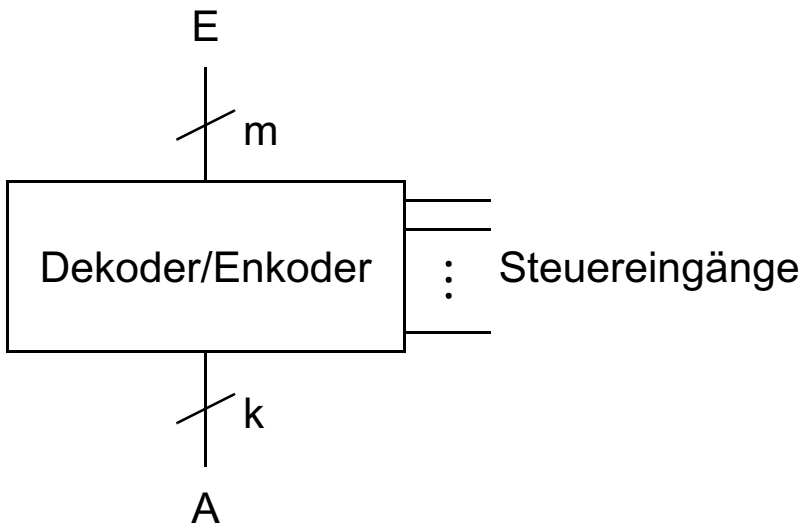


Funktionen der ALU mit Schiebe- und Rotierwerk (Übersicht)

	e,d,b,a	operation
lsl	0000	logic shift left x
lsr	0001	logic shift right x
rol	0010	rotate left x
ror	0011	rotate right x
asl	0100	arithmetic shift left x
asr	0101	arithmetic shift right x
rcl	0110	rotate through carry left x
rcr	0111	rotate through carry right x
or	1000	x or y
xor	1001	x xor y
and	1010	x and y
not	1011	not x
axc	1100	$x + c_{in}$
axyc	1101	$x + y + c_{in}$
sxyc	1110	$x - y - c_{in}$
sxc	1111	$x - c_{in}$

4.4.7 Dekoder und Enkoder

Kombinatorische Netzwerke zur Umsetzung eines Zeichenalphabets in ein anderes Zeichenalphabet nennt man Dekoder bzw. Enkoder.



Schaltzeichen

In der Regel spricht man von einem Dekoder, wenn $k \geq m$ ist und von einem Enkoder, wenn $k < m$ ist.

4.5 Implementierung

Inhaltsübersicht

4.5.1 Rechenwerk

4.5.2 Befehlsfolgezähler

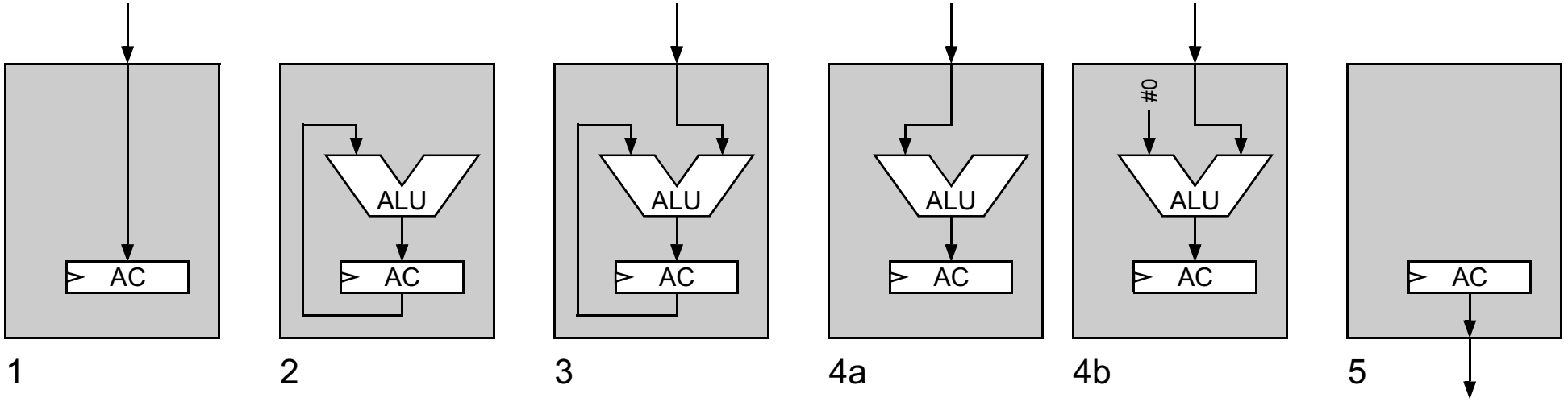
4.5.3 Speicheranbindung

4.5.4 Steuerwerk

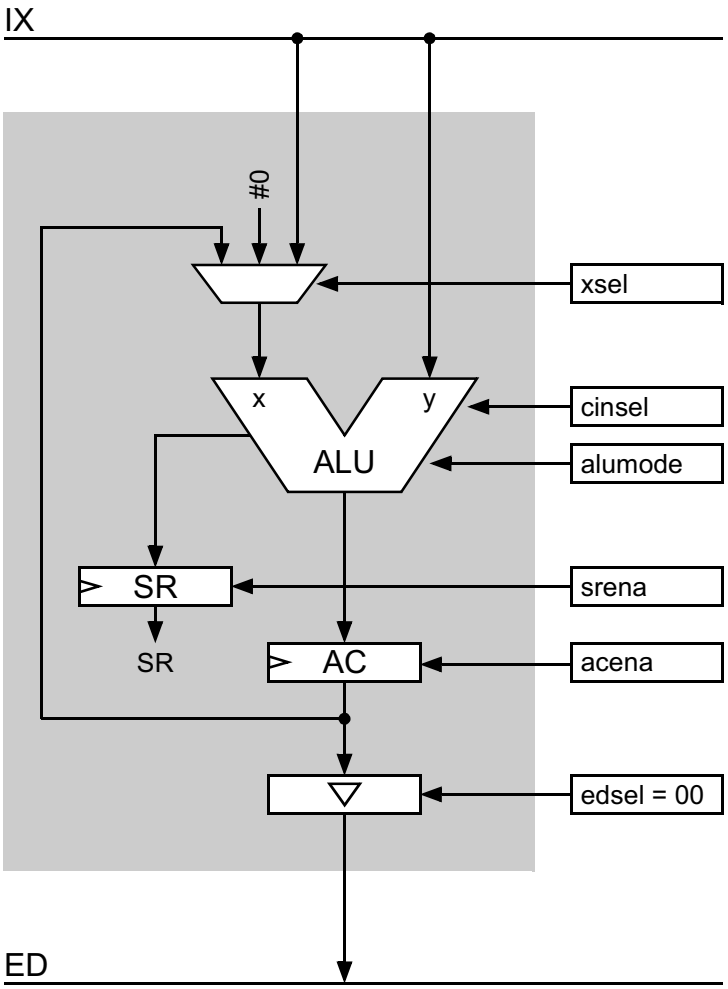
4.5.1 Rechenwerk

Anforderungen (vgl. Kapitel 4.3.2)

1)	AC mit Operand laden	op → AC	(z. B. LOAD, LOADI, BSR)
2)	Operation auf AC ausführen	f(AC) → AC	(z. B. INC, DEC, NOT, LSL, LSR, ...)
3)	Operand mit AC verknüpfen	f(op, AC) → AC	(z. B. ADD, SUB, AND, OR, ...)
4a)	Operation auf Operand ausführen	f(op) → AC	(z. B. INC, DEC, NOT, LSL, LSR, ...)
4b)	Operation auf Operand ausführen	f(#0, op) → AC	(z. B. NEG)
5)	AC speichern; AC als Sprungziel, Adresse	AC → ...	(z. B. STORE, JMP, LOADI)



Implementierung



alumode		operation
lsl	0000	log. shift left x
lsr	0001	log. shift right x
rol	0010	rotate left x
ror	0011	rotate right x
asl	0100	arith. shift left x
asr	0101	arith. shift right x
rcl	0110	rot. thru C left x
rcr	0111	rot. thru C right x
or	1000	x or y
xor	1001	x xor y
and	1010	x and y
not	1011	not x
axc	1100	$x + c_{in}$
axyc	1101	$x + y + c_{in}$
sxyc	1110	$x - y - c_{in}$
sxc	1111	$x - c_{in}$

xsel		ALU x input
IX	00	bus IX
#0	01	constant #0
AC	1X	AC feedback

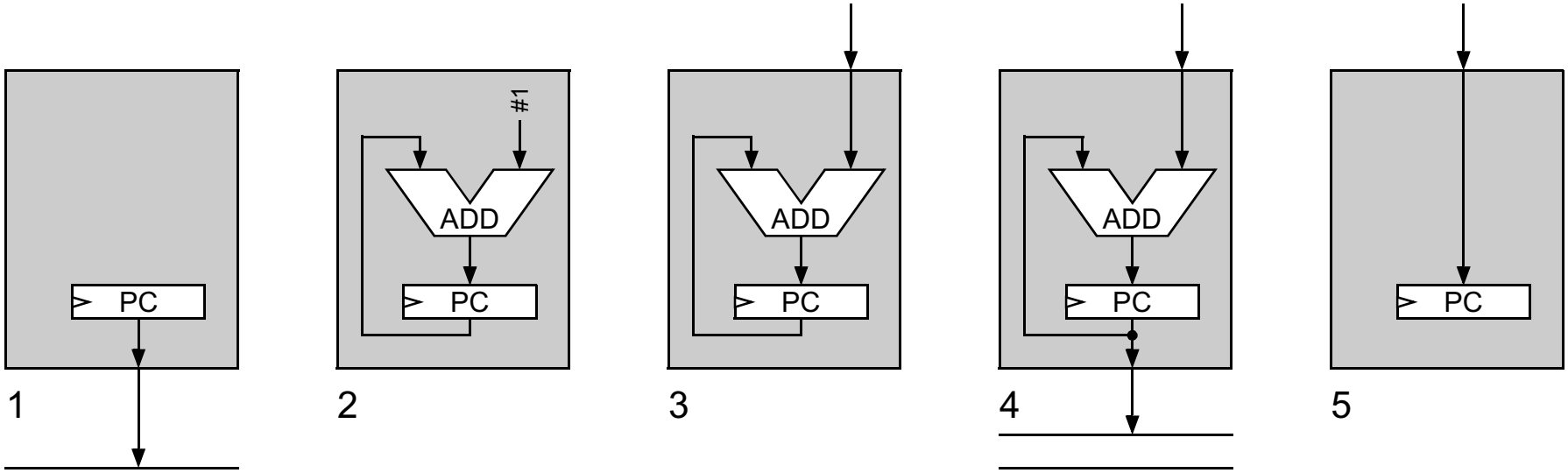
cinsel		ALU carry input
0	00	0
1	01	1
CF	1X	carry flag

edsel		ED select
AC	00	AC
PC	01	PC
d_in	1X	data from memory

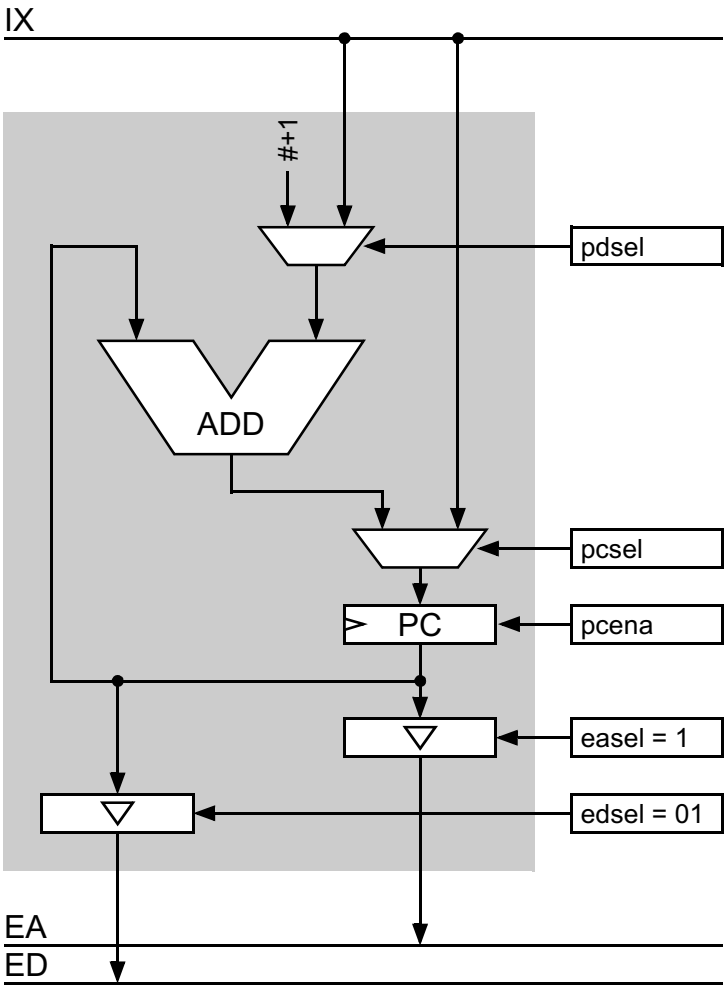
4.5.2 Befehlsfolgezähler

Anforderungen (vgl. Kapitel 4.3.2)

1)	Adressquelle für Programmzugriffe	PC → Adressbus	
2)	nächstes Befehlswort	PC + #1 → PC	(nächstes Befehlswort)
3)	bedingt, unbedingt verzweigen	PC + op → PC	(BRA, Bcc)
4)	in Unterprogramm verzweigen	PC + op → PC, PC → AC	(BSR)
5)	an dynamische Adresse verzweigen	op → PC	(JMP)



Implementierung



pdsel		PC displacement
IX	0	bus IX
#+1	1	constant +1

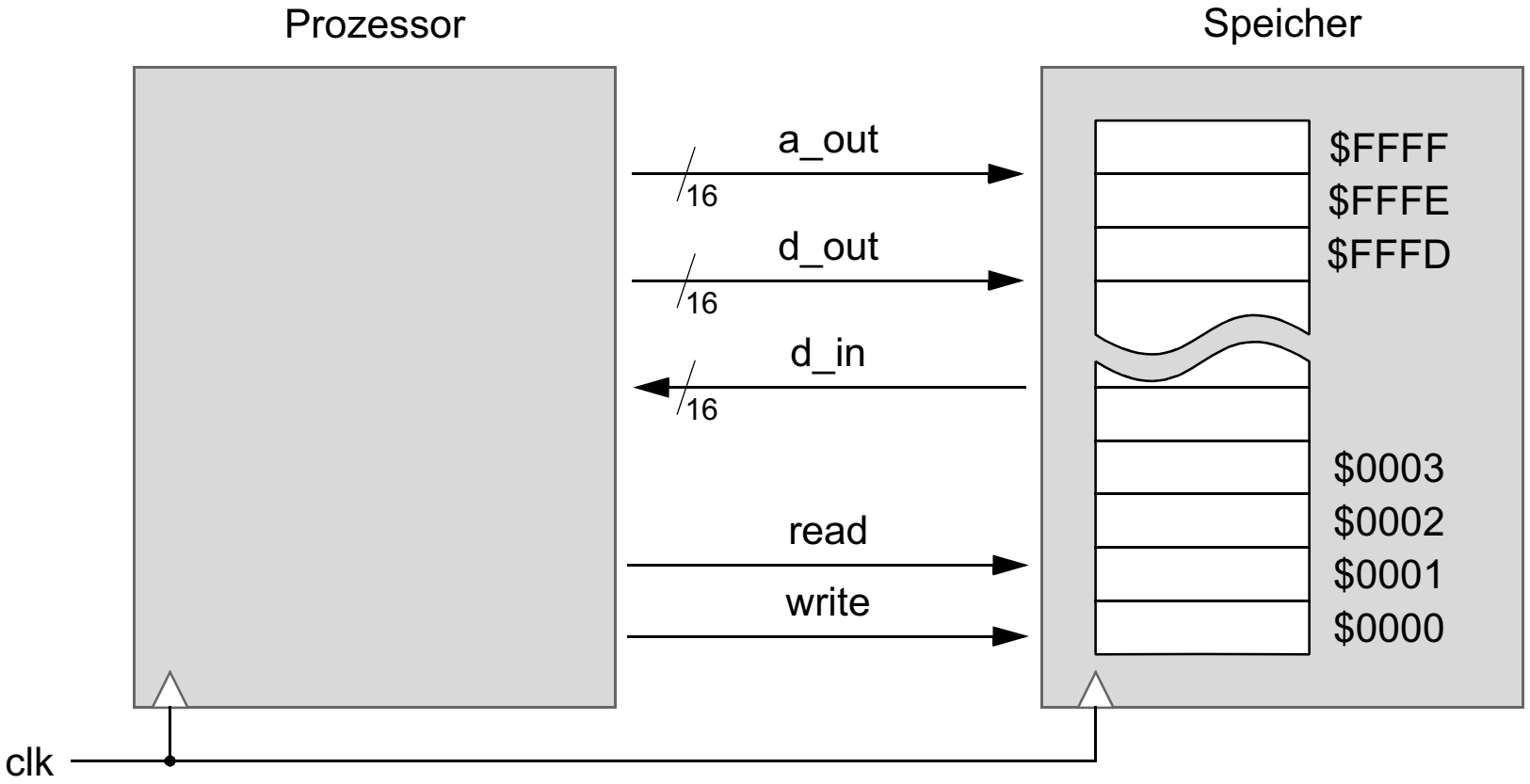
pcsel		PC input
adder	0	adder
IX	1	bus IX

easel		EA select
AA	0	AA
PC	1	PC

edsel		ED select
AC	00	AC
PC	01	PC
d_in	1X	data from memory

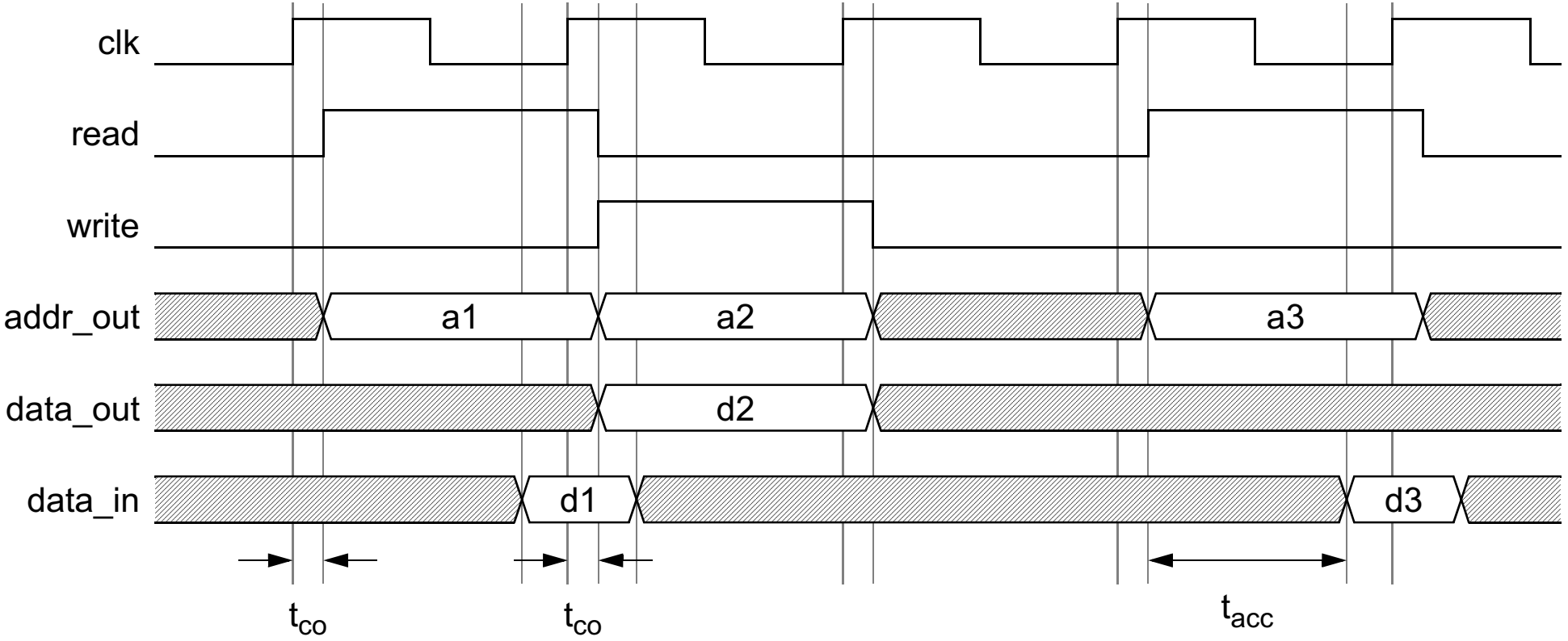
4.5.3 Speicheranbindung

Schnittstelle zum Speicher



a_out	Adresse für Lese- und Schreibzugriffe	read	signalisiert einen Lesezugriff
d_out	zu schreibende Daten bei einem Schreibzugriff	write	signalisiert einen Schreibzugriff
d_in	gelesene Daten bei einem Lesezugriff		

Signal-Zeitdiagramm von Lese- und Schreibzugriffen



t_{co} Clock-to-Output-Zeit (Prozessor)

t_{acc} Zugriffszeit (Speicher)

Anforderungen

- Daten und Programmcode aus dem Speicher laden
- Daten im Speicher ablegen

Geladen werden

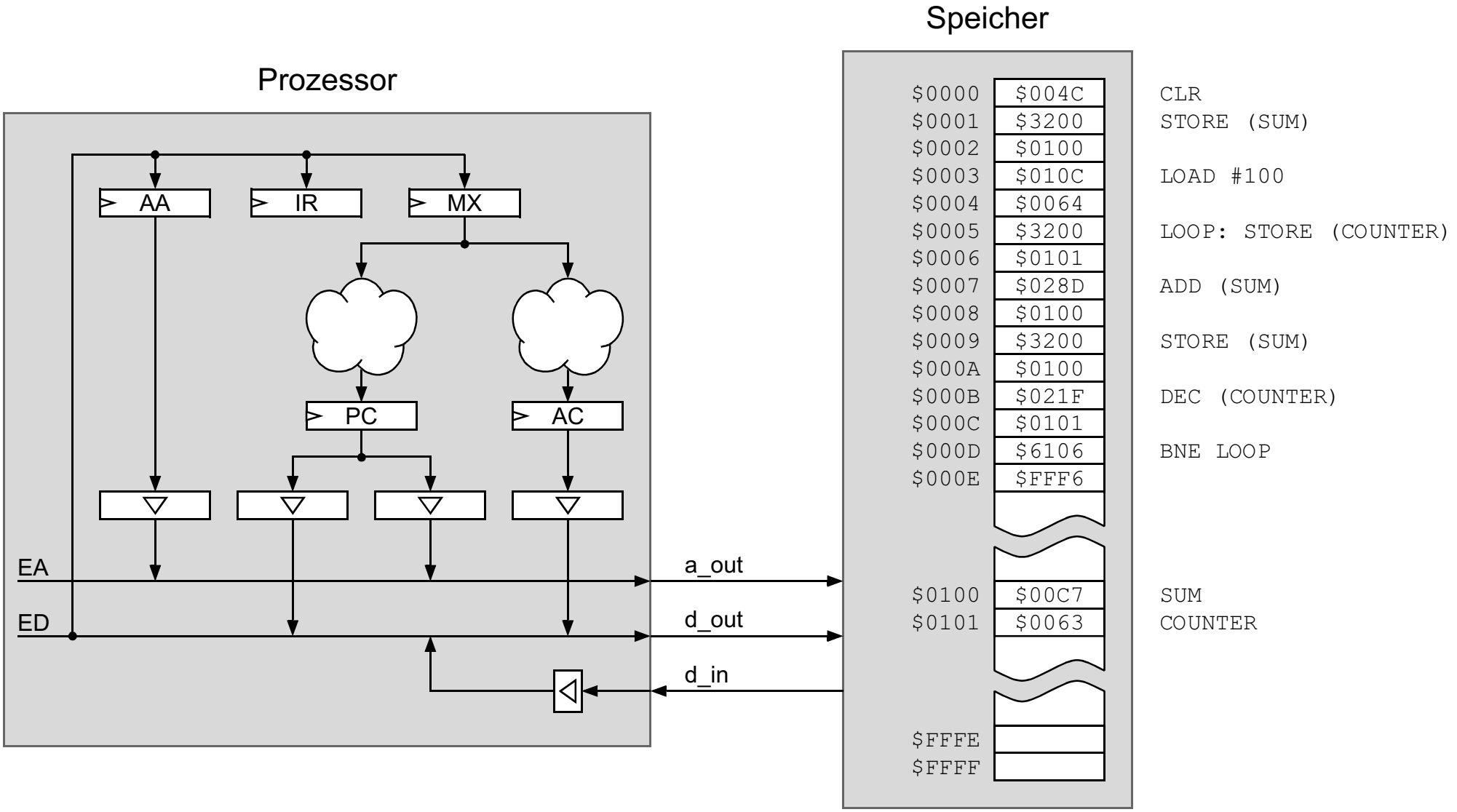
1) Befehle für das Steuerwerk → *Instruktionsregister IR*
2) Operanden für das Rechenwerk und den Befehlsfolgezähler → *Register MX*
   a) bei einem Zugriff auf Daten, z.B. LOAD (m) oder LOAD ((m))
   b) bei einem Zugriff auf Programmcode (Erweiterungswort für Konstante #i oder Displacement d)
3) Operanden für Speicheradressierung → *Register AA*
   a) bei einem Zugriff auf Daten, die als Pointer verwendet werden, z.B. LOAD ((m))
   b) bei einem Zugriff auf Programmcode (Erweiterungswort für Adresse m)

Gespeichert wird

ausschließlich der Inhalt des Akkumulatorregisters AC

Implementierung

Übersicht und Beispiel



Zusammenfassung

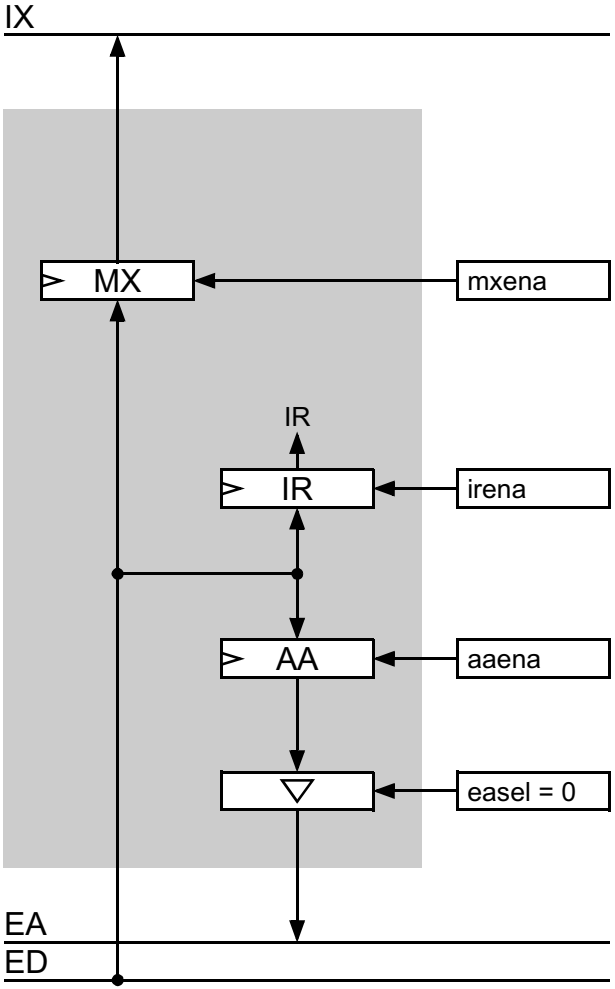
Lesezugriffe

1) Befehle für das Steuerwerk **(PC) → IR**
2) Operanden für das Rechenwerk und den Befehlsfolgezähler
   a) bei einem Zugriff auf Daten, z.B. LOAD (m) oder LOAD ((m)) **(AA) → MX**
   b) bei einem Zugriff auf Programmcode (Erweiterungswort für Konstante #i) **(PC) → MX**
3) Operanden für Speicheradressierung
   a) bei einem Zugriff auf Daten, die als Pointer verwendet werden, z.B. LOAD ((m)) **(AA) → AA**
   b) bei einem Zugriff auf Programmcode (Erweiterungswort für Adresse m) **(PC) → AA**

Schreibzugriffe **AC → (AA)**

Implementierung

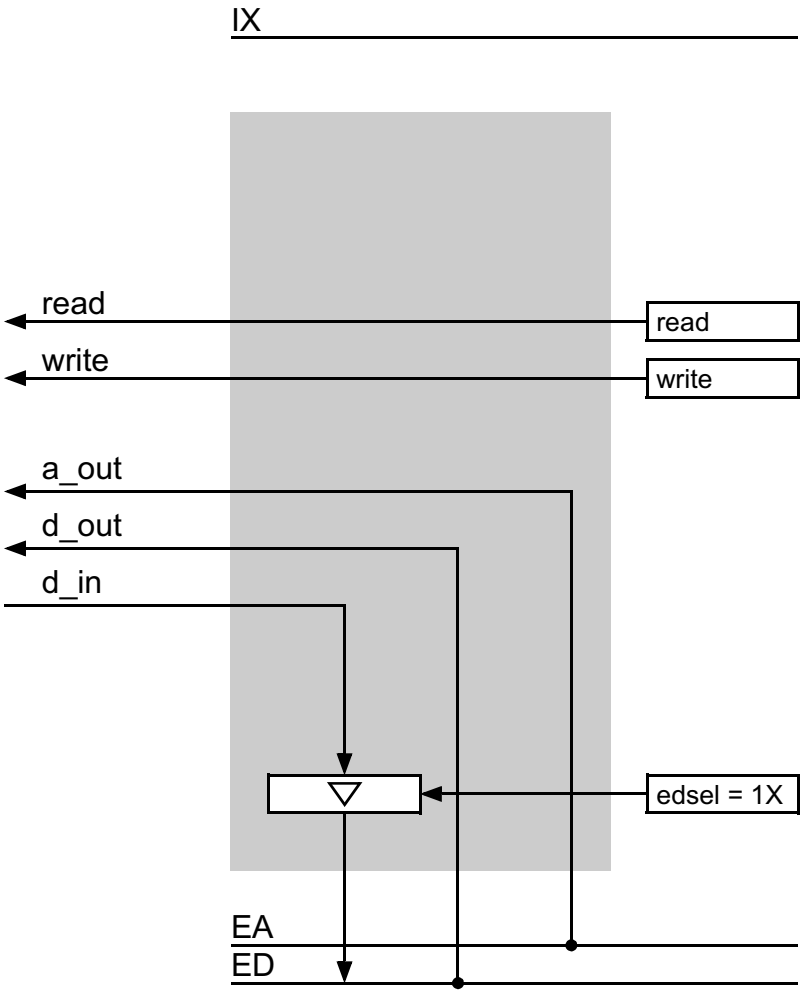
Im Detail: Speicherregister



easel		EA select
AA	0	AA
PC	1	PC

Implementierung

Im Detail: Businterface

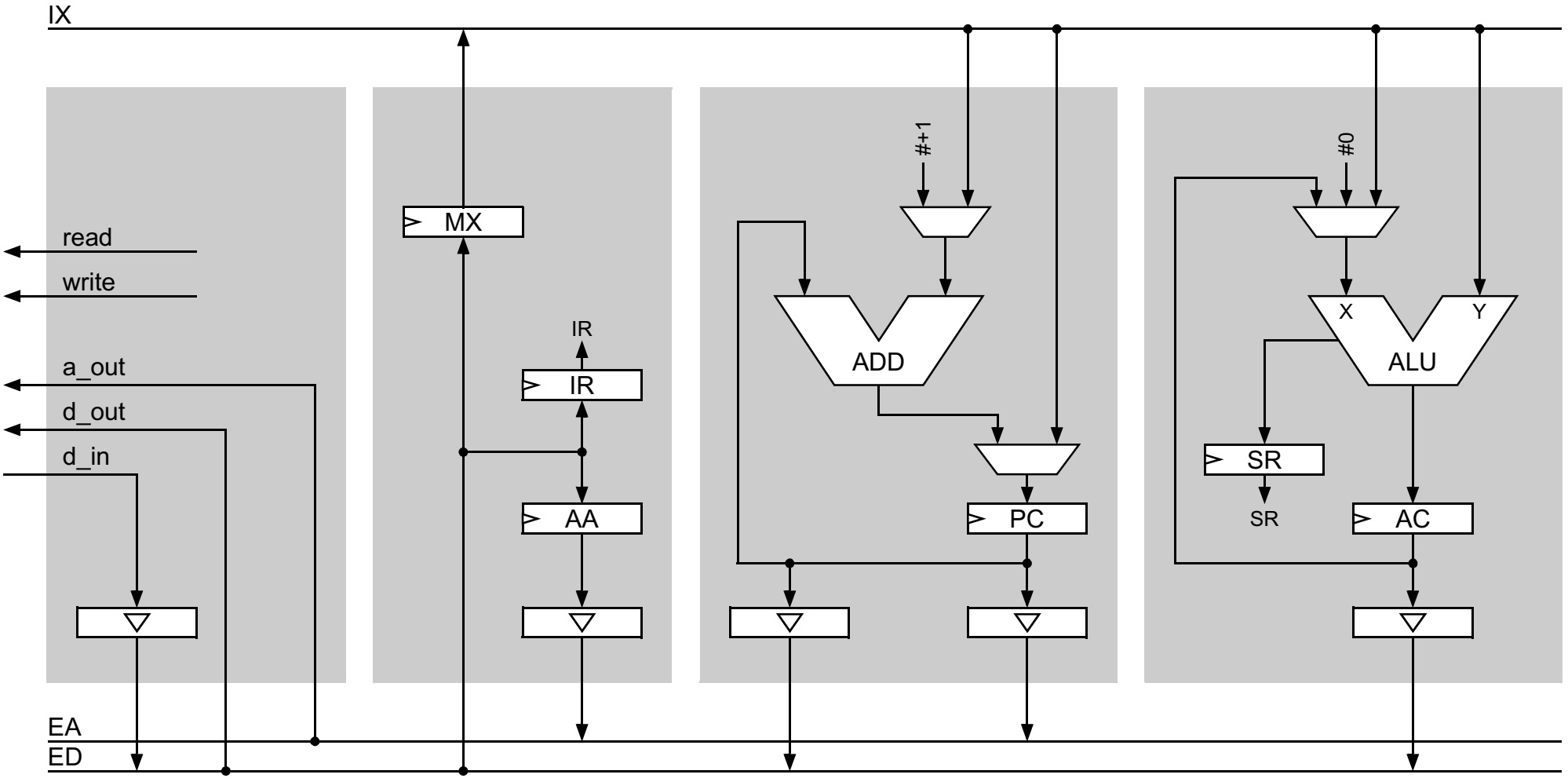


edsel		ED select
AC	00	AC
PC	01	PC
d_in	1X	data from memory

read	write	bus activity
0	0	bus idle
0	1	memory write cycle, memory(a_out) = d_out
1	0	memory read cycle, d_in = memory(a_out)
1	1	illegal

4.5.4 Steuerwerk

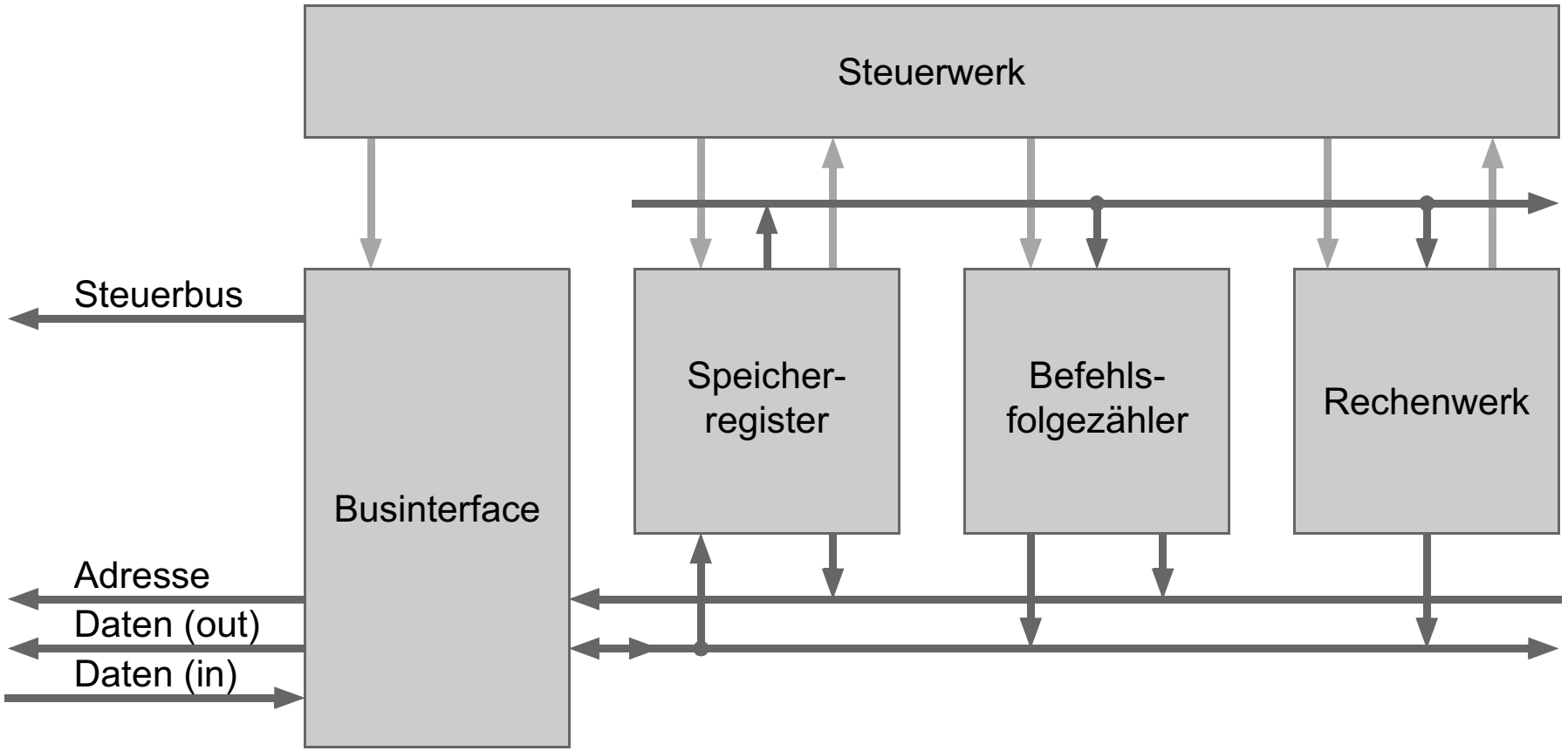
Zusammenfassung: Übersicht über die Ausführungseinheiten



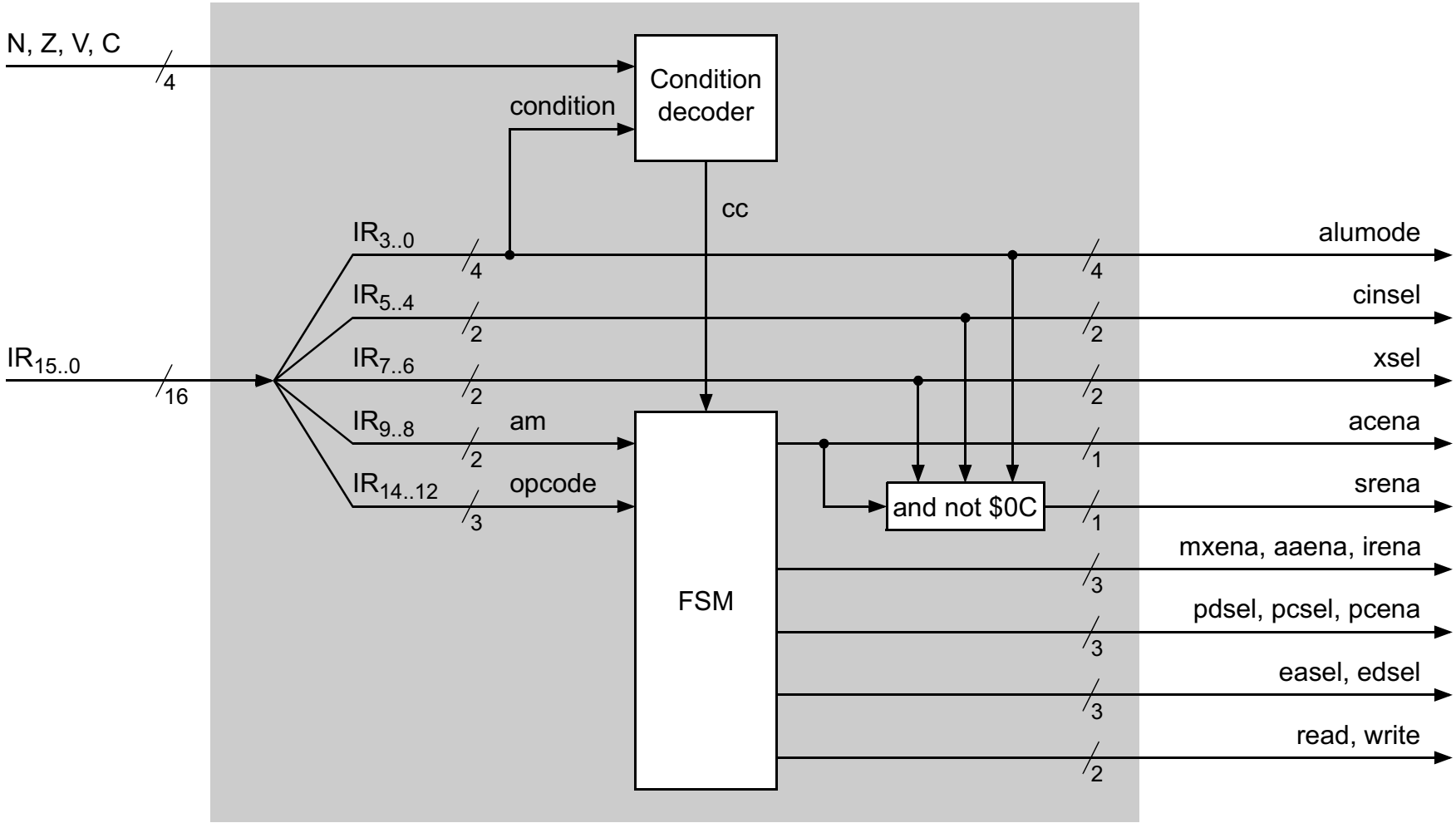
Zusammenfassung: Übersicht über die Steuerleitungen

Einheit	Steuerleitung		Codierung		
ALU	xsel	ALU x-operand	00: IX	01: #0	1X: AC
	cinsel	ALU carry input	00: $c_{in} = 0$	01: $c_{in} = 1$	1X: $c_{in} = C$
	alumode	ALU mode	0000: lsl	0001: lsr	...
	acena	AC register enable			
	srena	SR enable			
MREG	mxena	MX register enable			
	aaena	AA register enable			
	irena	IR enable			
PC	pdsel	PC displacement select	0: IX	1: #1	
	pcsel	PC select	0: Adder	1: IX	
	pcena	PC register enable			
EA, ED	easel	EA bus select	0: AA	1: PC	
	edsel	ED bus select	00: AC	01: PC	1X: d_in
EBIF	read	memory read request			
	write	memory write request			

Integration des Steuerwerks



Implementierung des Steuerwerks



Realisierung des Bedingungsdekoders (condition decoder) als kombinatorisches Netzwerk

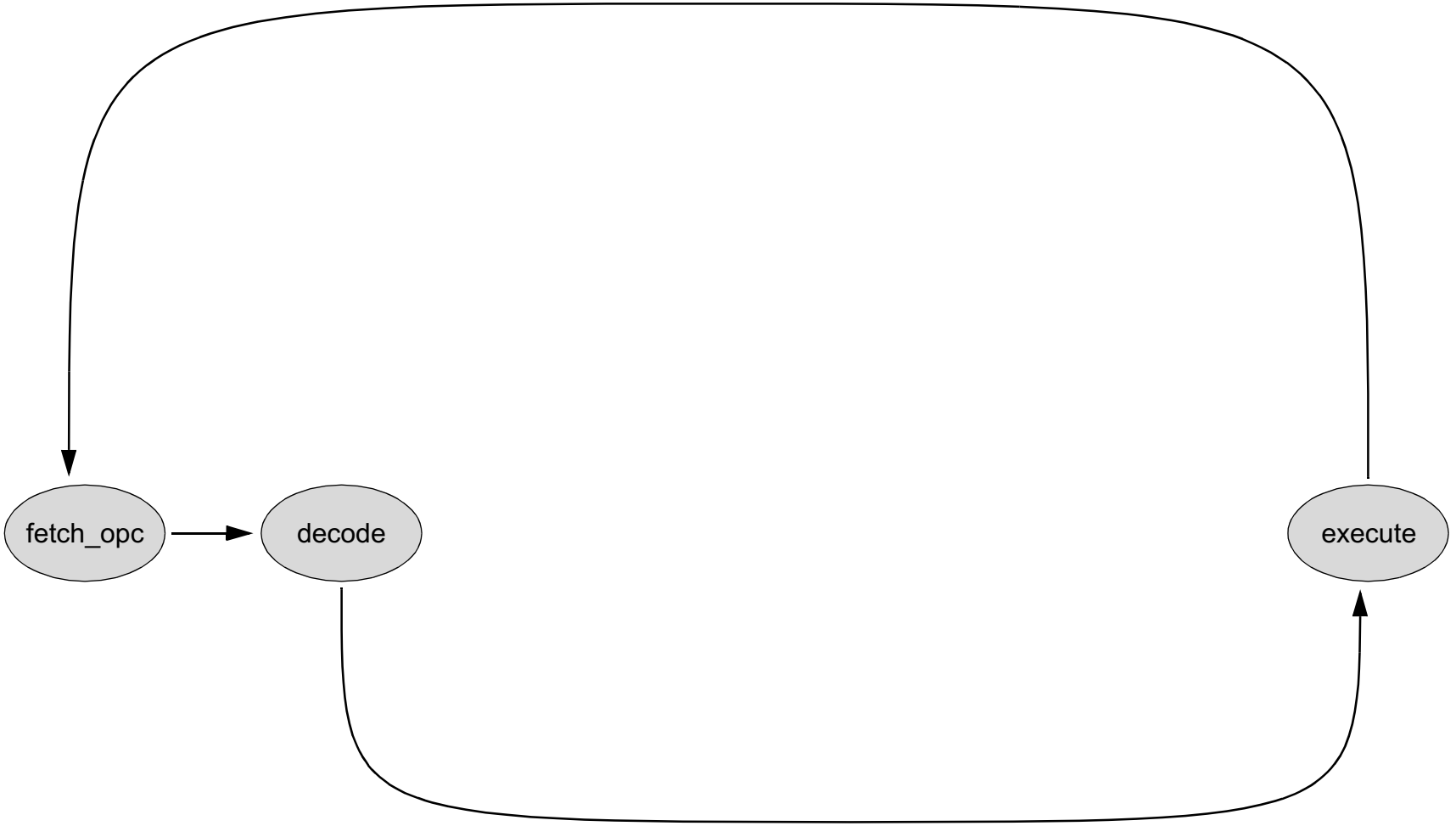
condition			cc = 1 if ...
F	0000	False	0
T	0001	True	1
HI	0010	HIgher	C + Z = 0
LS	0011	Lower or Same	C + Z = 1
CC	0100	Carry Clear	C = 0
CS	0101	Carry Set	C = 1
NE	0110	Not Equal	Z = 0
EQ	0111	EQual	Z = 1
VC	1000	oVerflow Clear	V = 0
VS	1001	oVerflow Set	V = 1
PL	1010	PLus	N = 0
MI	1011	MInus	N = 1
GE	1100	Greater or Equal	$N\overline{V} + \overline{N}V = 0$
LT	1101	Less Than	$N\overline{V} + \overline{N}V = 1$
GT	1110	Greater Than	$N\overline{V} + \overline{N}V + Z = 0$
LE	1111	Less or Equal	$N\overline{V} + \overline{N}V + Z = 1$

Implementierung des Automaten (Finite State Machine, FSM)

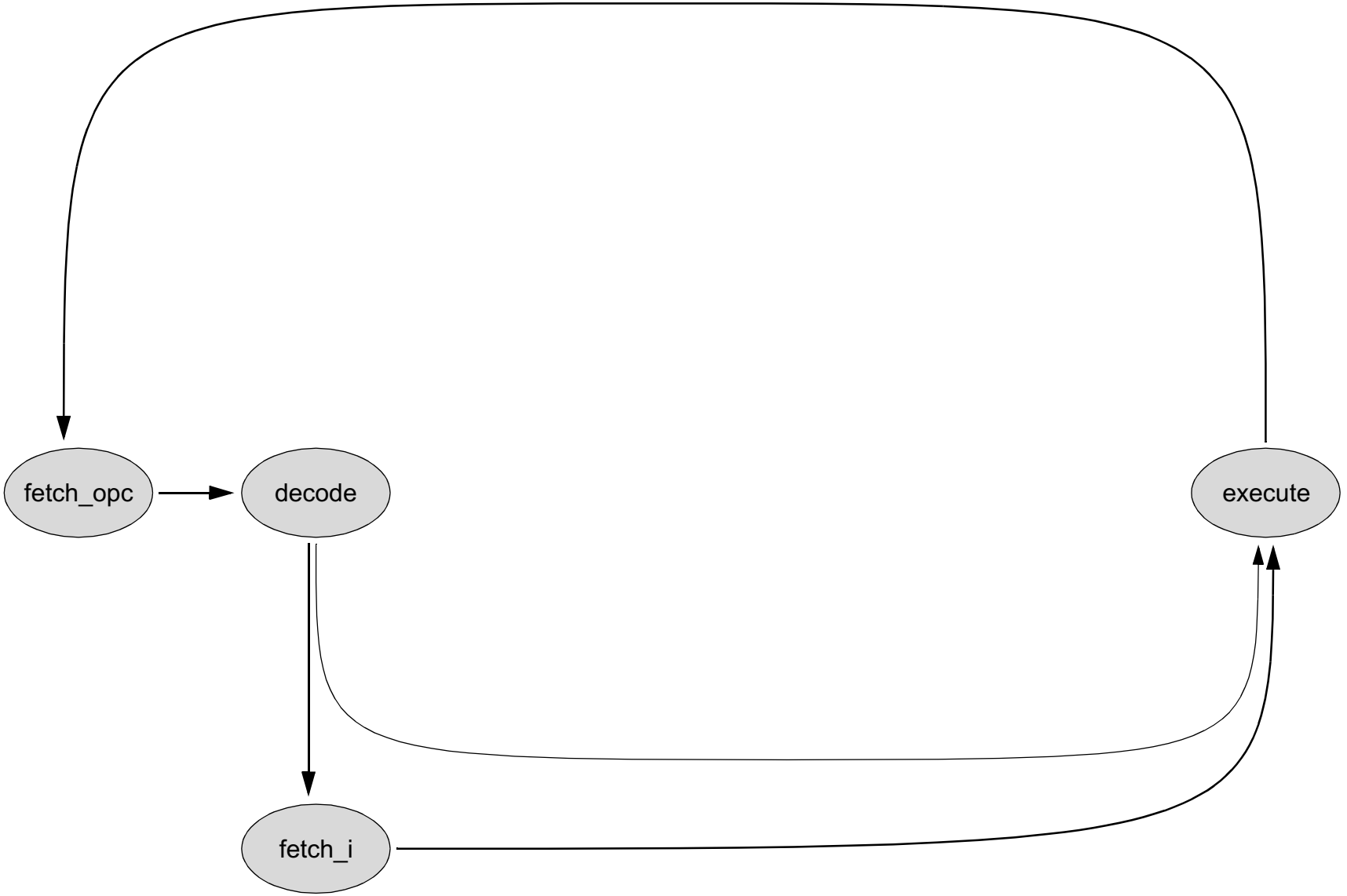
Entwurf in 11 Schritten

1) ALU-Befehle, Adressierungsart "implizit"
2) ALU-Befehle, Adressierungsart "unmittelbar"
3) ALU-Befehle, Adressierungsart "direkt"
4) ALU-Befehle, Adressierungsart "indirekt"
5) LOADI
6) STORE (m)
7) STORE ((m))
8) BRA, Bcc mit erfüllter Bedingung
9) Bcc mit nicht erfüllter Bedingung
10) BSR
11) JMP

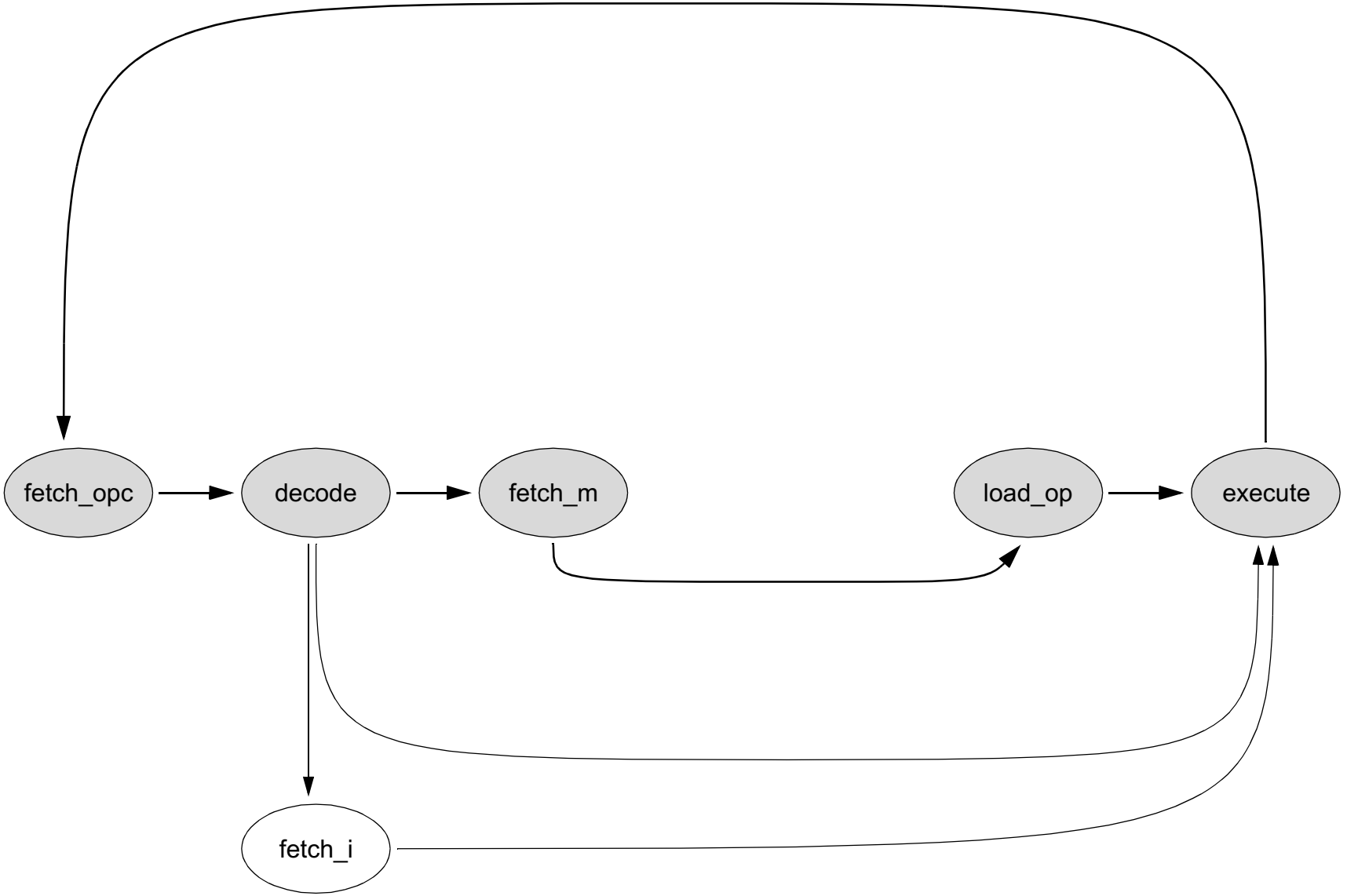
1) ALU-Befehle, Adressierungsart "implizit"



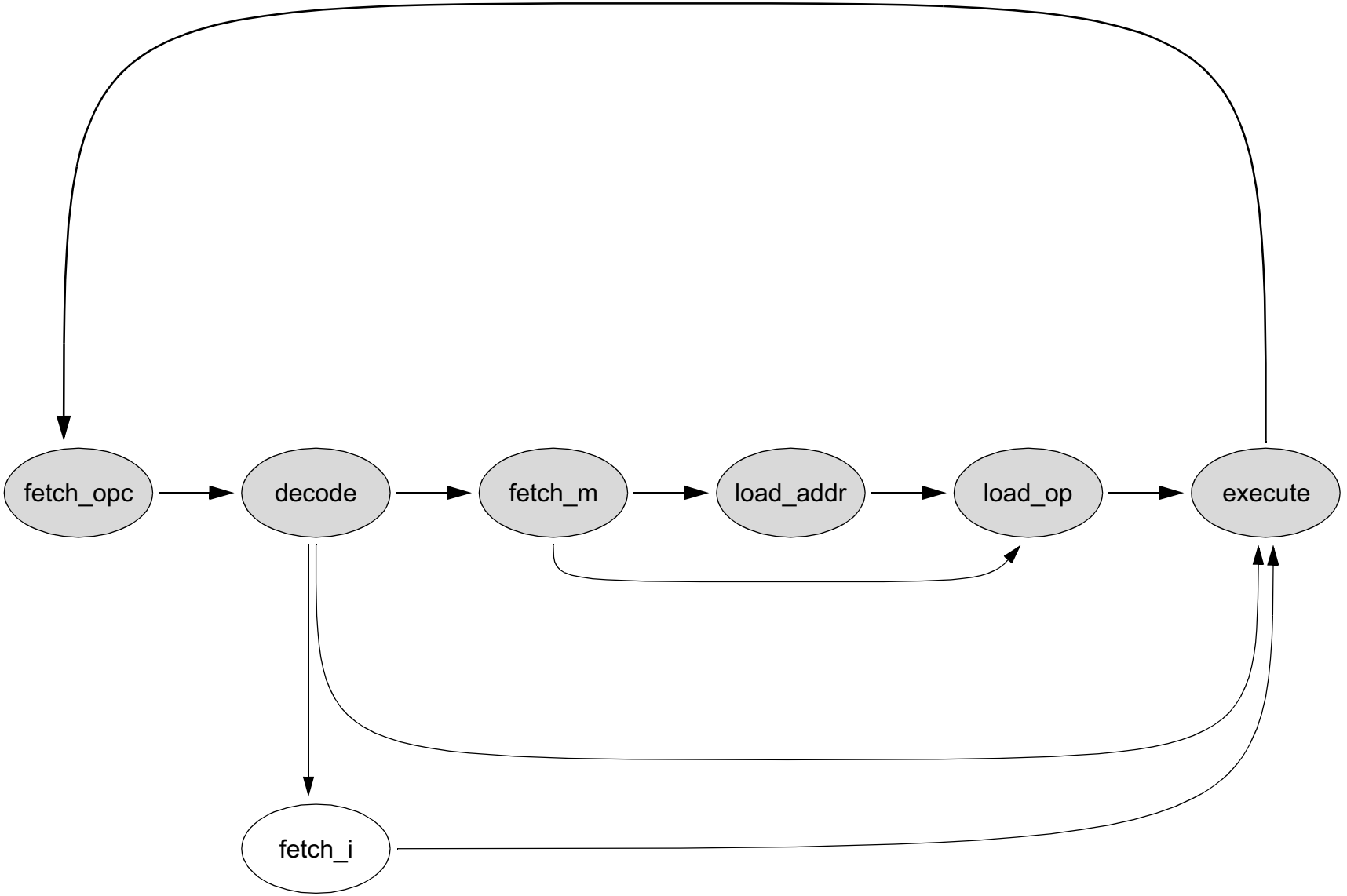
2) ALU-Befehle, Adressierungsart "unmittelbar"



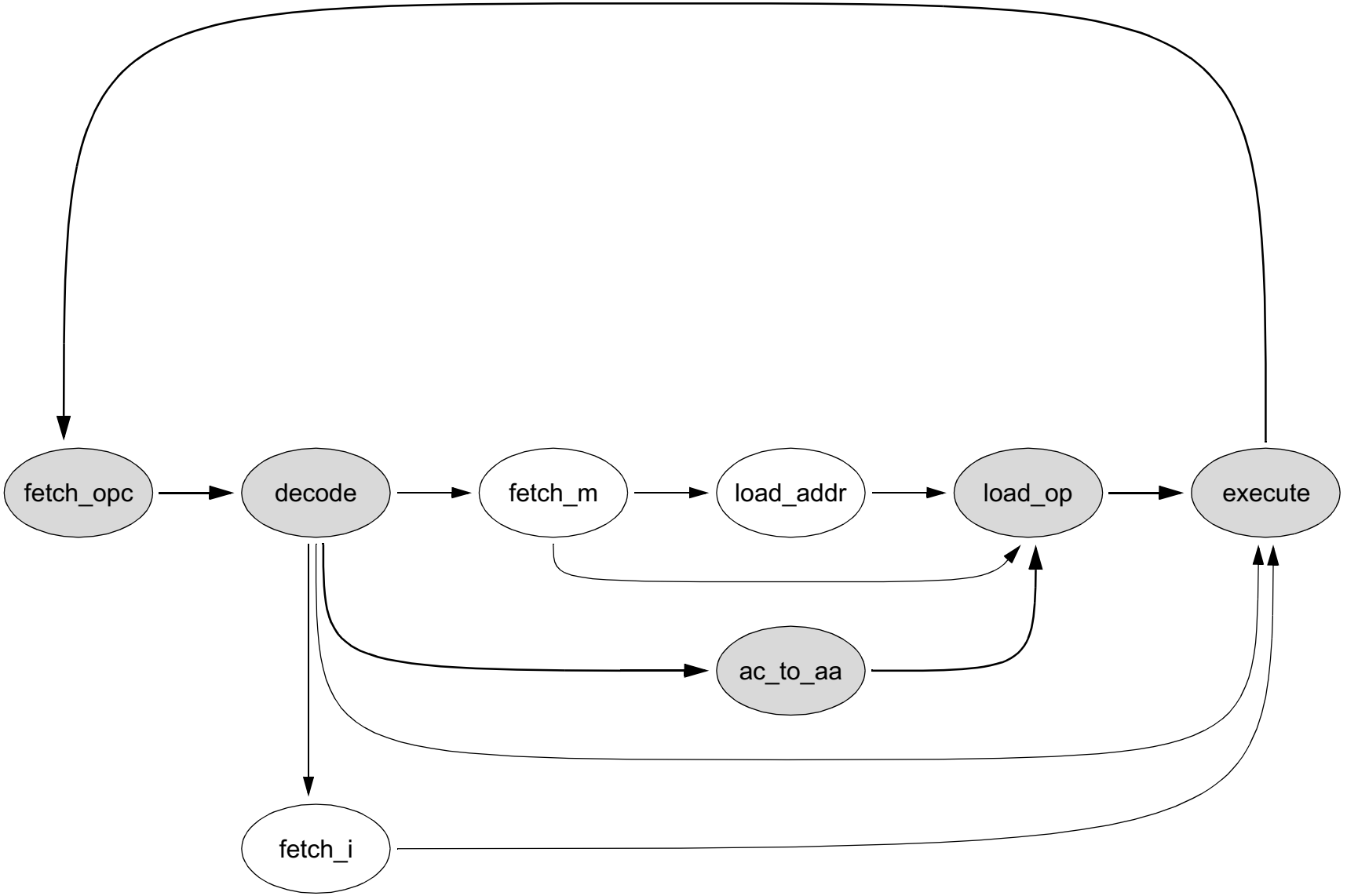
3) ALU-Befehle, Adressierungsart "direkt"



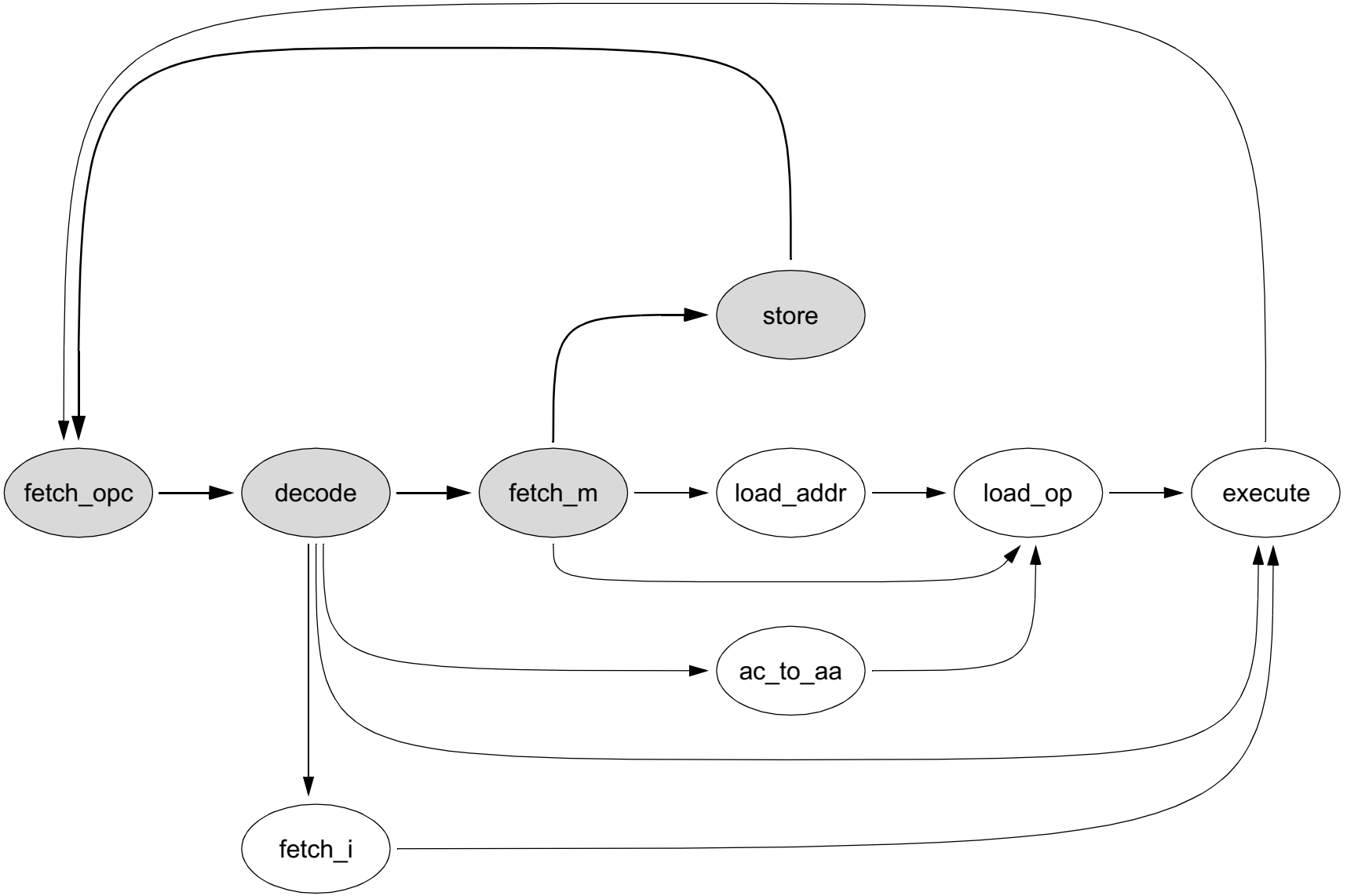
4) ALU-Befehle, Adressierungsart "indirekt"



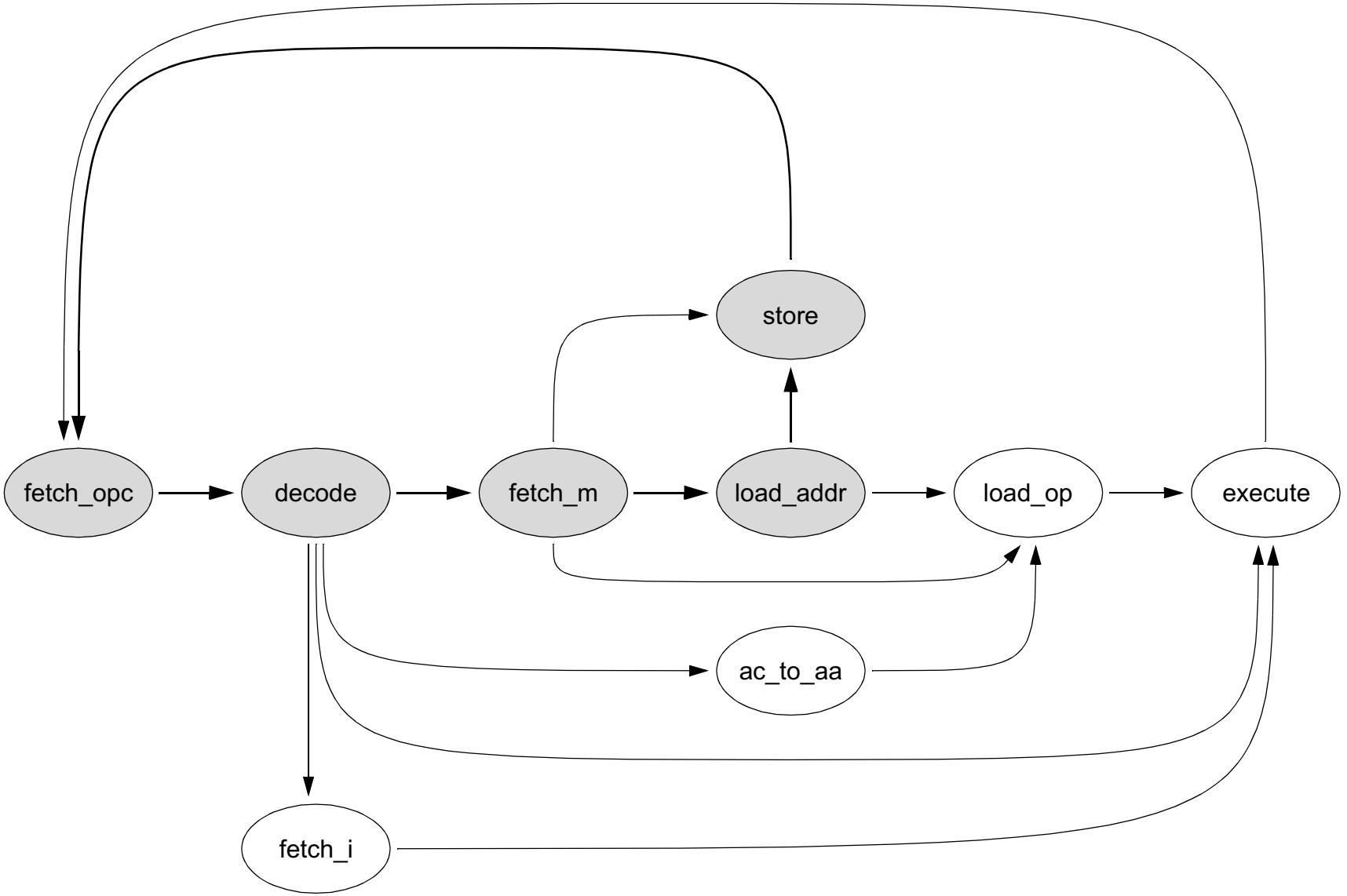
5) LOADI


fetch_opc
decode
fetch_m
load_addr
load_op
execute
ac_to_aa
fetch_i

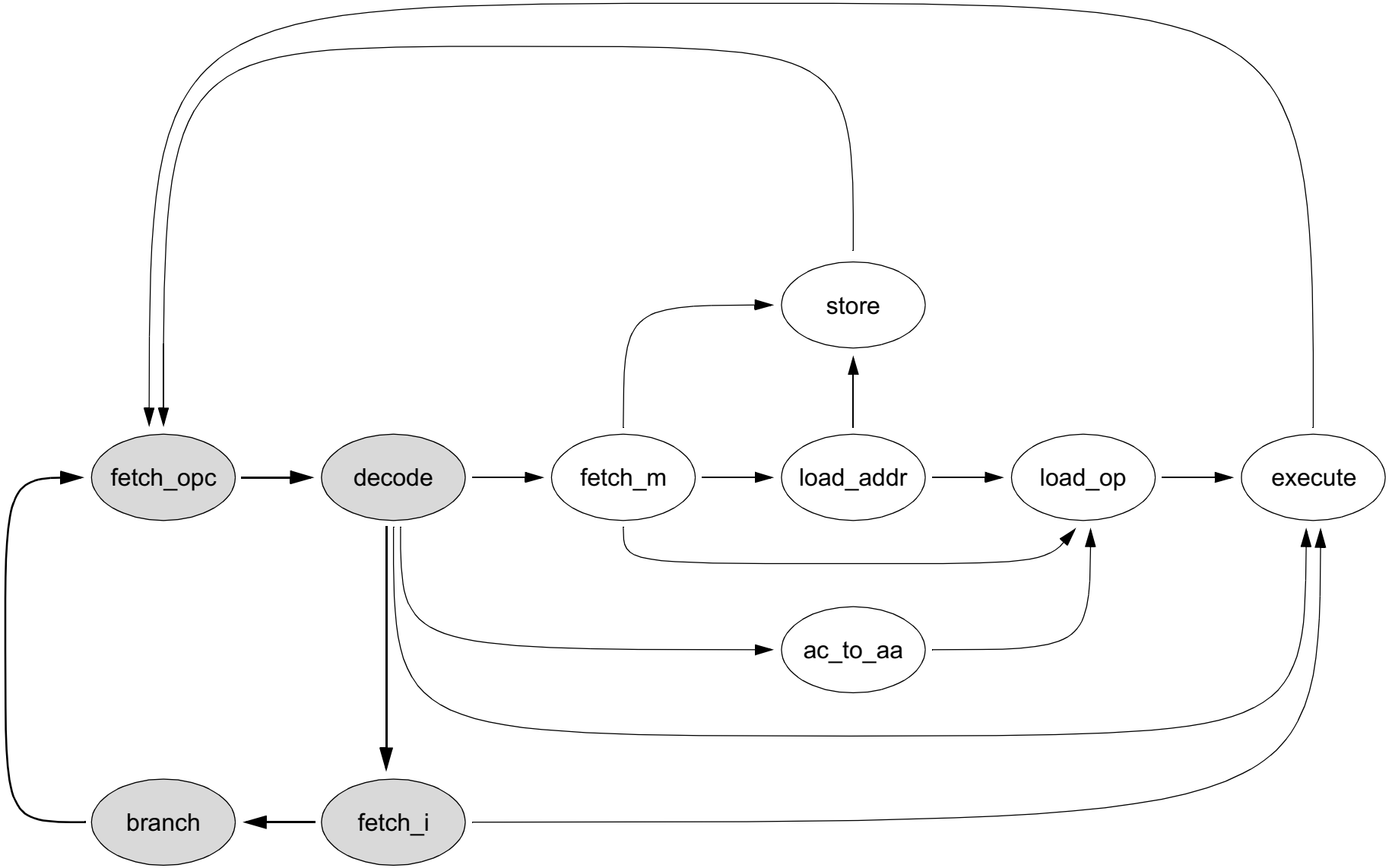
6) STORE (m)


store
fetch_opc
decode
fetch_m
load_addr
load_op
execute
ac_to_aa
fetch_i

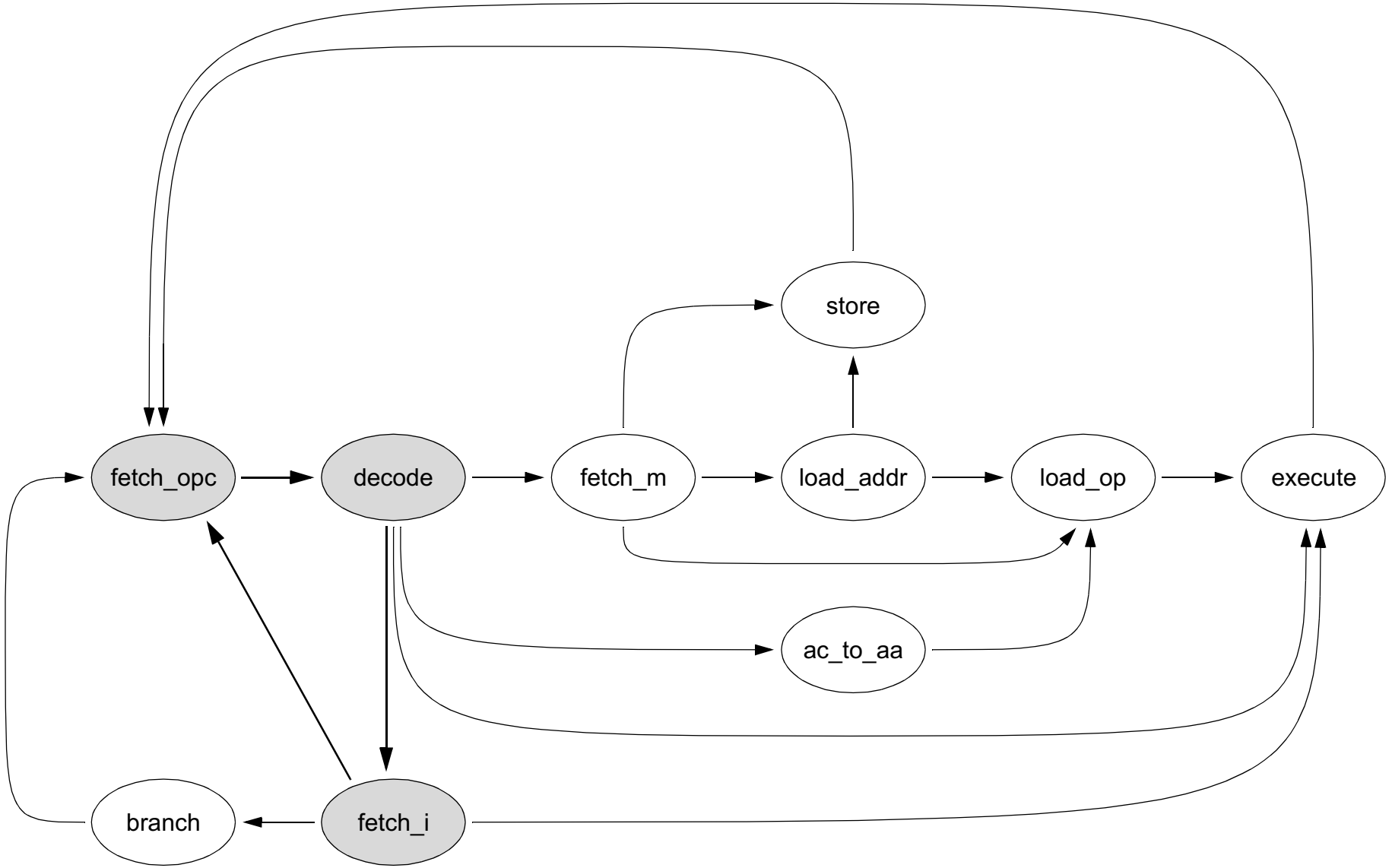
7) STORE ((m))


store
fetch_opc
decode
fetch_m
load_addr
load_op
execute
ac_to_aa
fetch_i

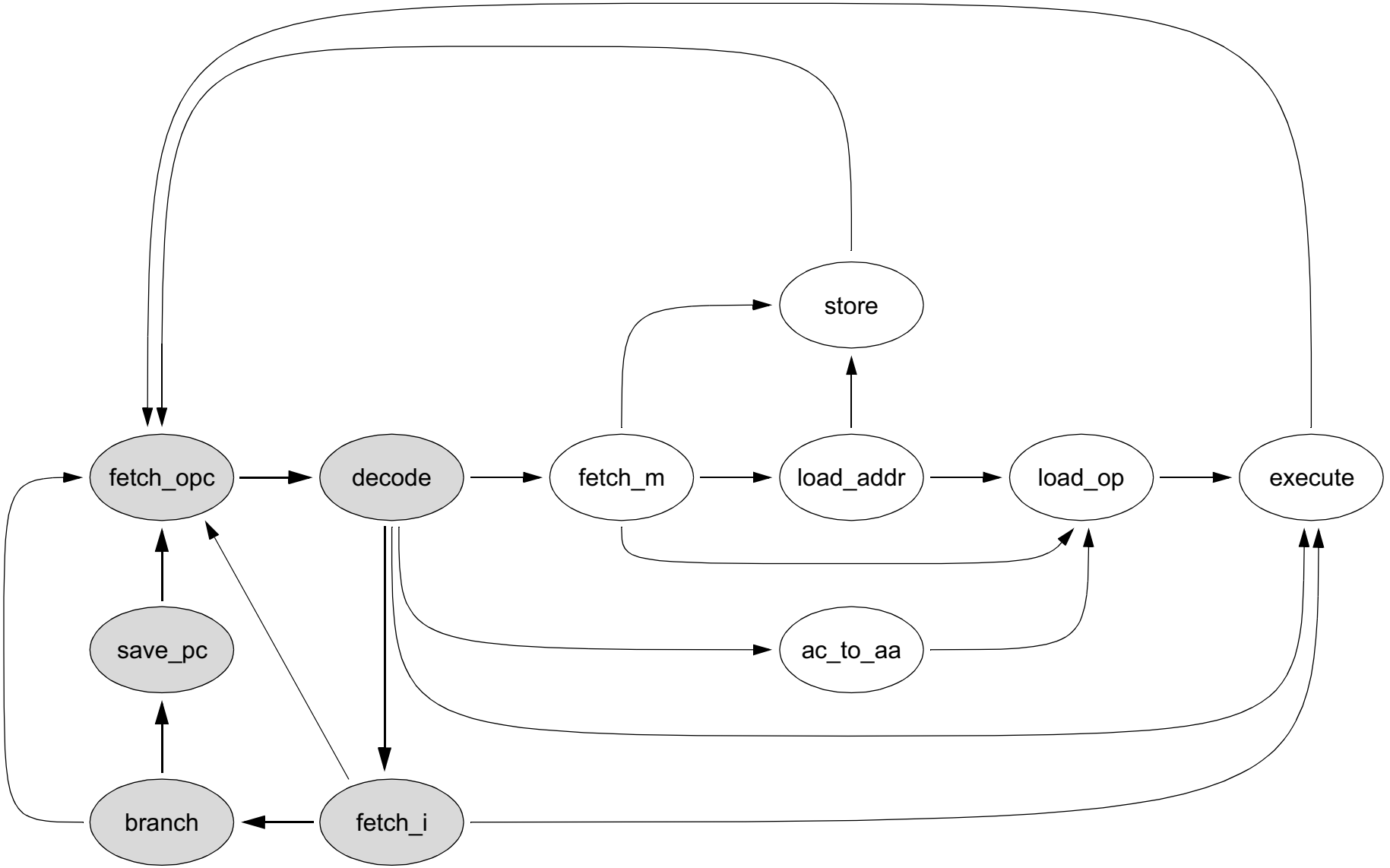
8) BRA, Bcc mit erfüllter Bedingung



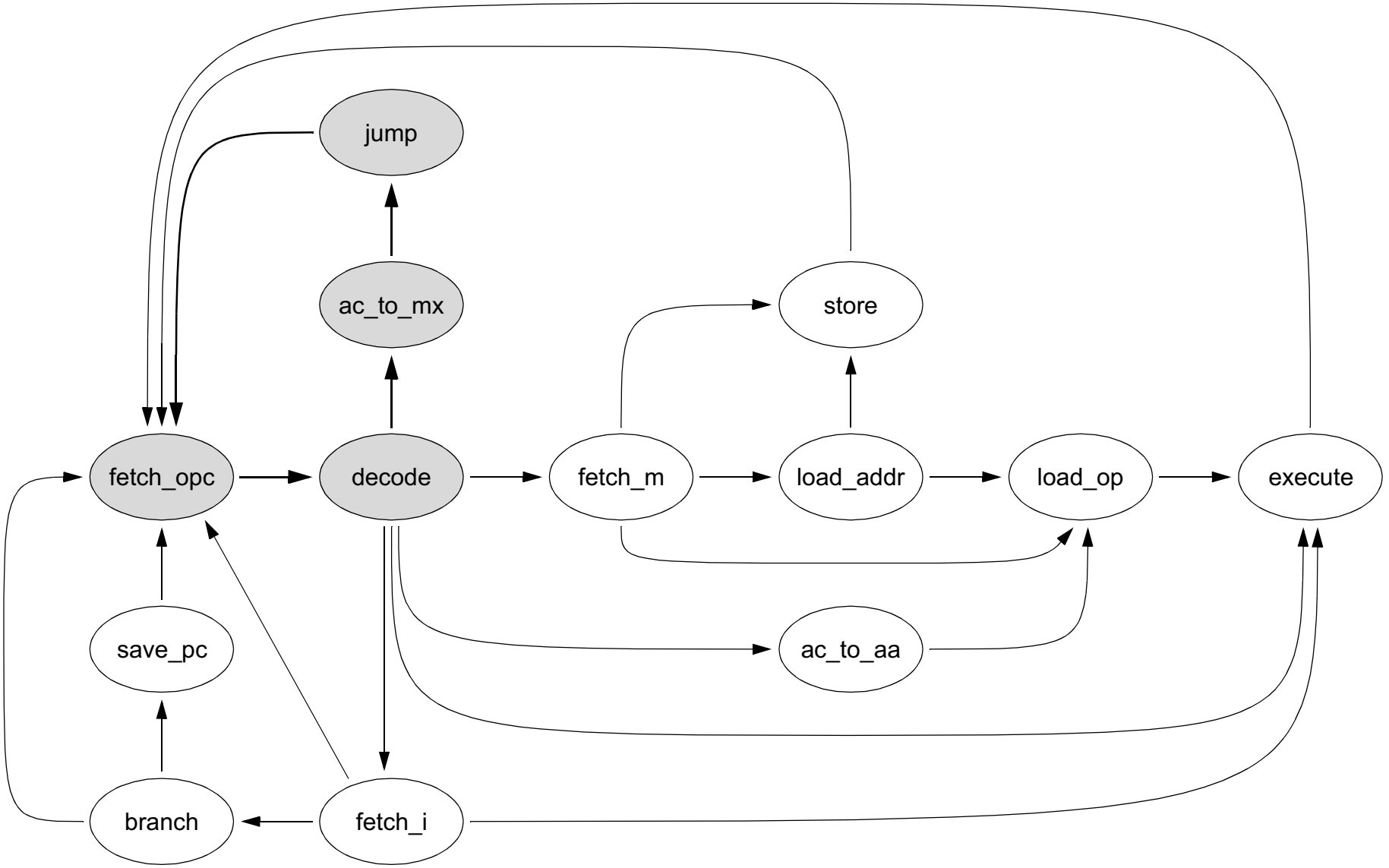
9) Bcc mit nicht erfüllter Bedingung



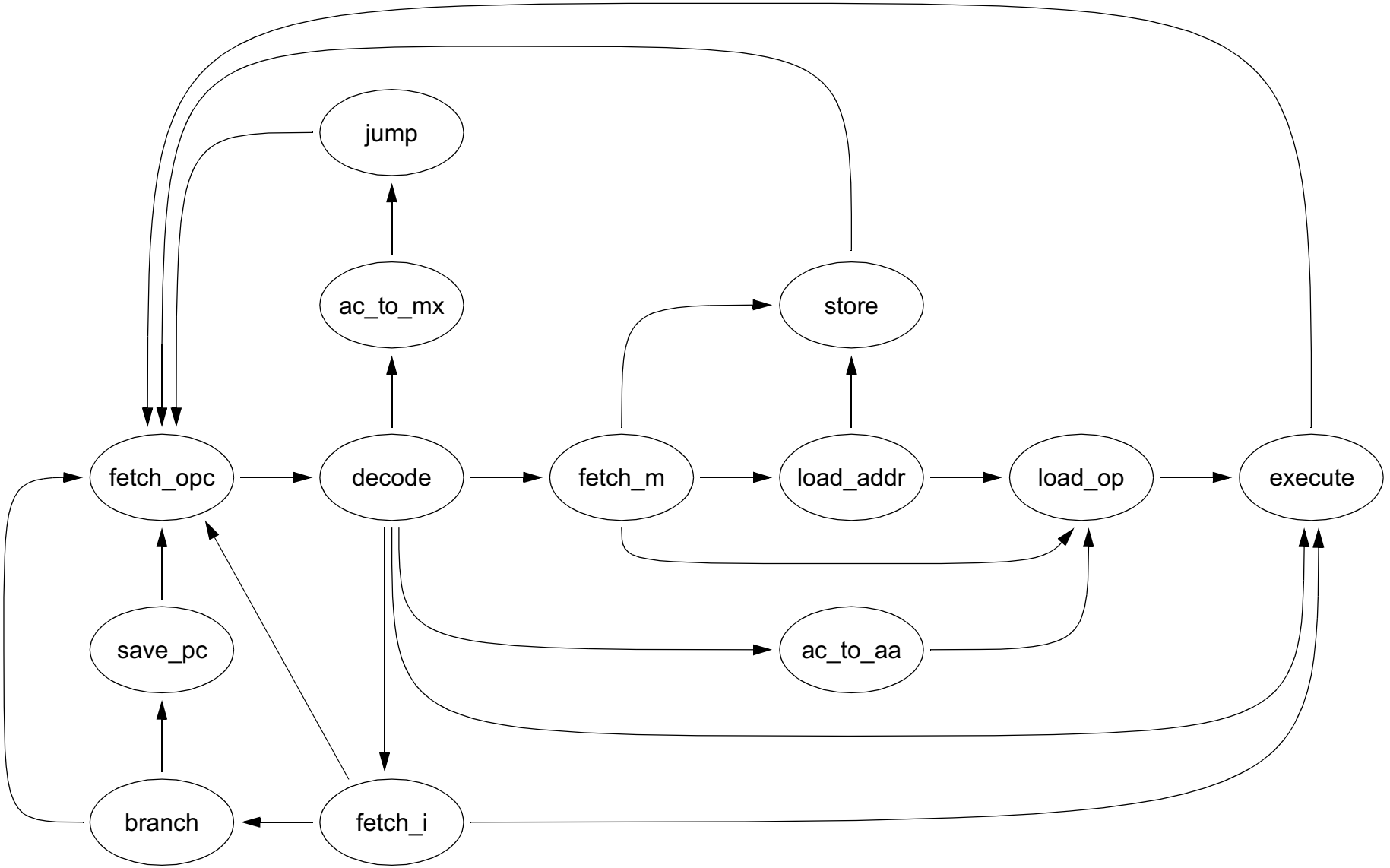
10) BSR


store
fetch_opc
decode
fetch_m
load_addr
load_op
execute
save_pc
ac_to_aa
branch
fetch_i

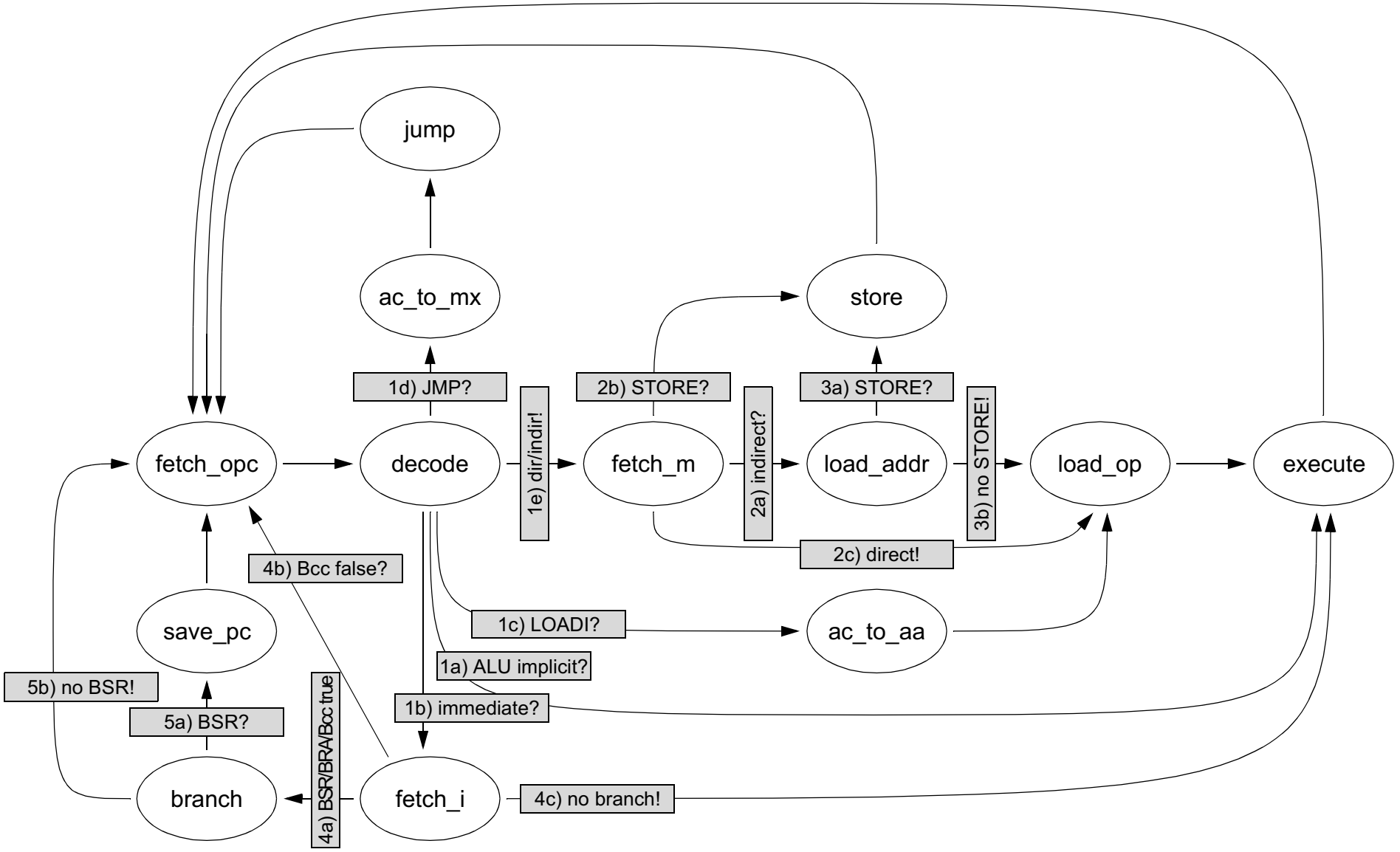
11) JMP


jump
ac_to_mx
store
fetch_opc
decode
fetch_m
load_addr
load_op
execute
save_pc
ac_to_aa
branch
fetch_i

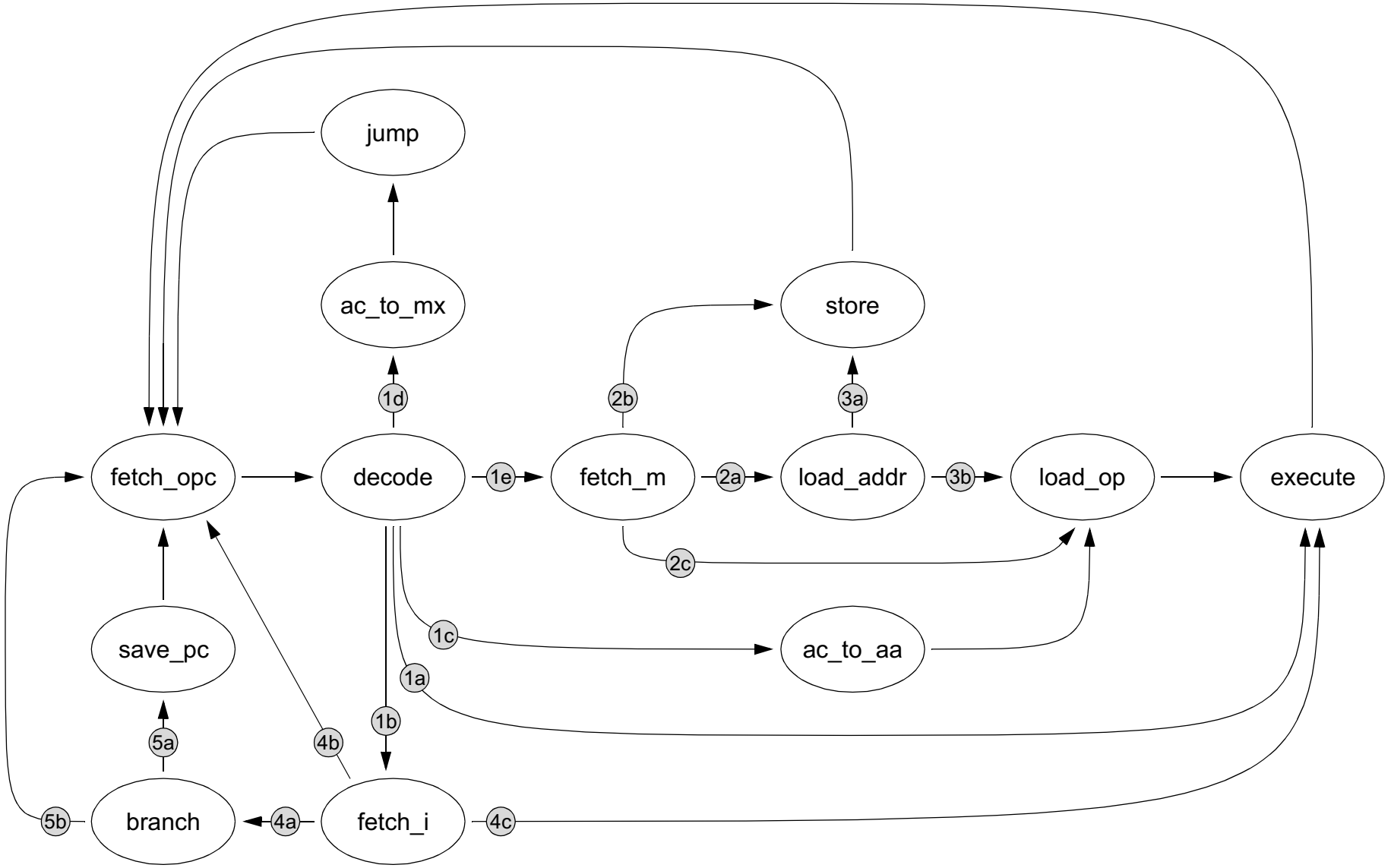
Ergebnis: Zustands-Übergangsdiagramm (noch ohne Bedingungen für Zustandsübergänge)



Bedingungen für Zustandsübergänge



Vollständiges Zustands-Übergangsdiagramm (Bedingungen abgekürzt)



Zusammenfassung der Zustandsübergänge

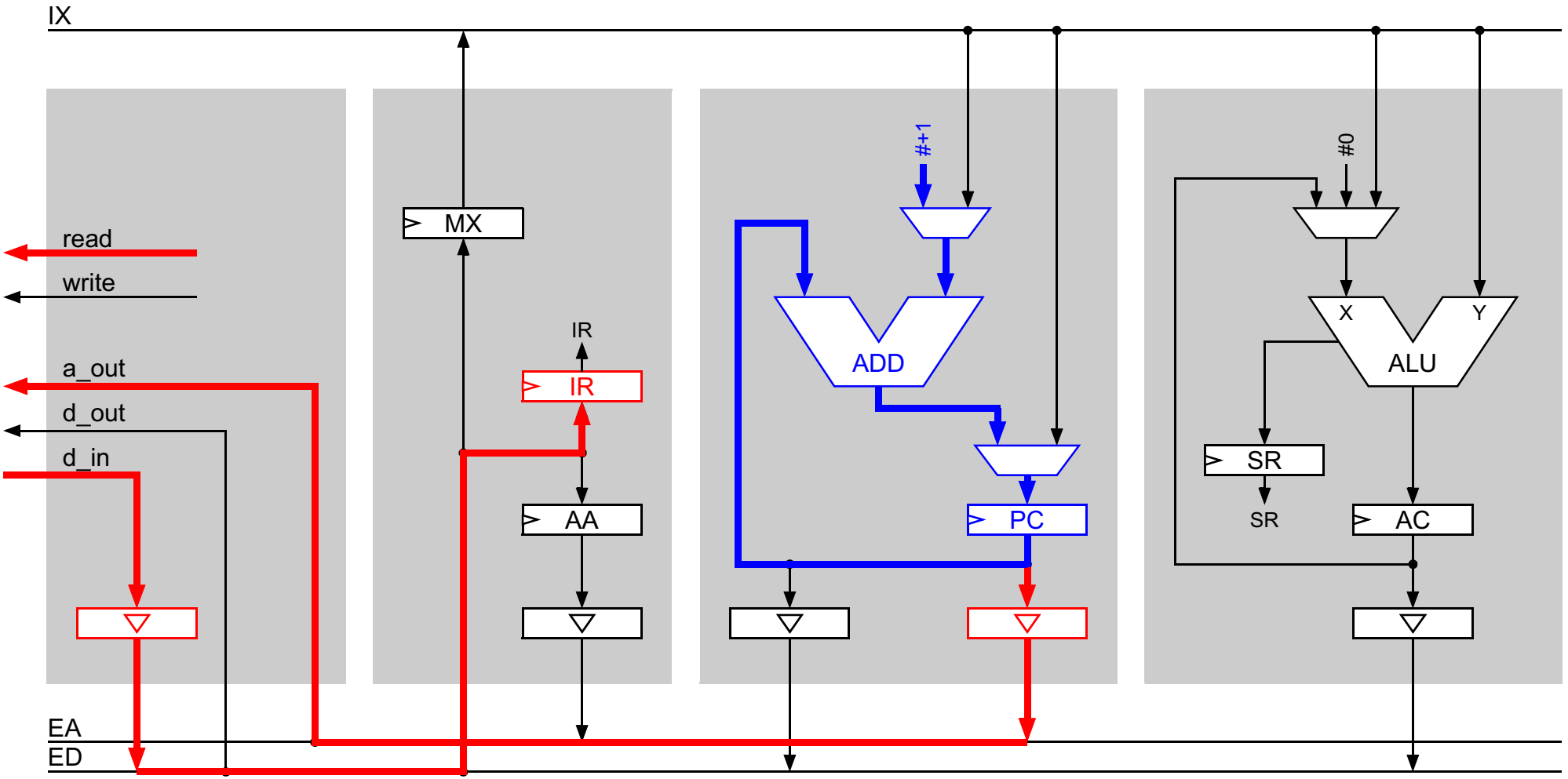
Istzustand Q^n	Bedingung für Zustandsübergang					Folgezustand Q^{n+1}
			$IR_{14..12}$	$IR_{9..8}$	cc	
decode	1a	ALU implicit?	000	00		execute
	1b	else immediate?		01		fetch_i
	1c	else LOADI?	010			ac_to_aa
	1d	else JMP?	100			ac_to_mx
	1e	else direct or indirect				fetch_m
fetch_m	2a	indirect?		11		load_addr
	2b	else STORE?	011			store
	2c	else direct!				load_op
load_addr	3a	STORE?	011			store
	3b	else no STORE!				load_op
fetch_i	4a	BSR BRA or Bcc true	101 110		1	branch
	4b	else BCC false	110		0	fetch_opc
	4c	else no branch!				execute
branch	5a	BSR?	101			save_pc
	5b	else no BSR!				fetch_opc

Zustandscodierung (Medwedew-Automat)

Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
fetch_opc	1	10	0	0	1	1	0	1	0	1	0
decode	-	-	0	0	0	-	-	0	0	0	0
fetch_i	1	10	1	0	0	1	0	1	0	1	0
fetch_m	1	10	0	1	0	1	0	1	0	1	0
load_addr	0	10	0	1	0	-	-	0	0	1	0
load_op	0	10	1	0	0	-	-	0	0	1	0
execute	-	-	0	0	0	-	-	0	1	0	0
store	0	00	0	0	0	-	-	0	0	0	1
ac_to_aa	-	00	0	1	0	-	-	0	0	0	0
ac_to_mx	-	00	1	0	0	-	-	0	0	0	0
jump	-	-	0	0	0	-	1	1	0	0	0
branch	-	01	1	0	0	0	0	1	0	0	0
save_pc	-	-	0	0	0	-	-	0	1	0	0

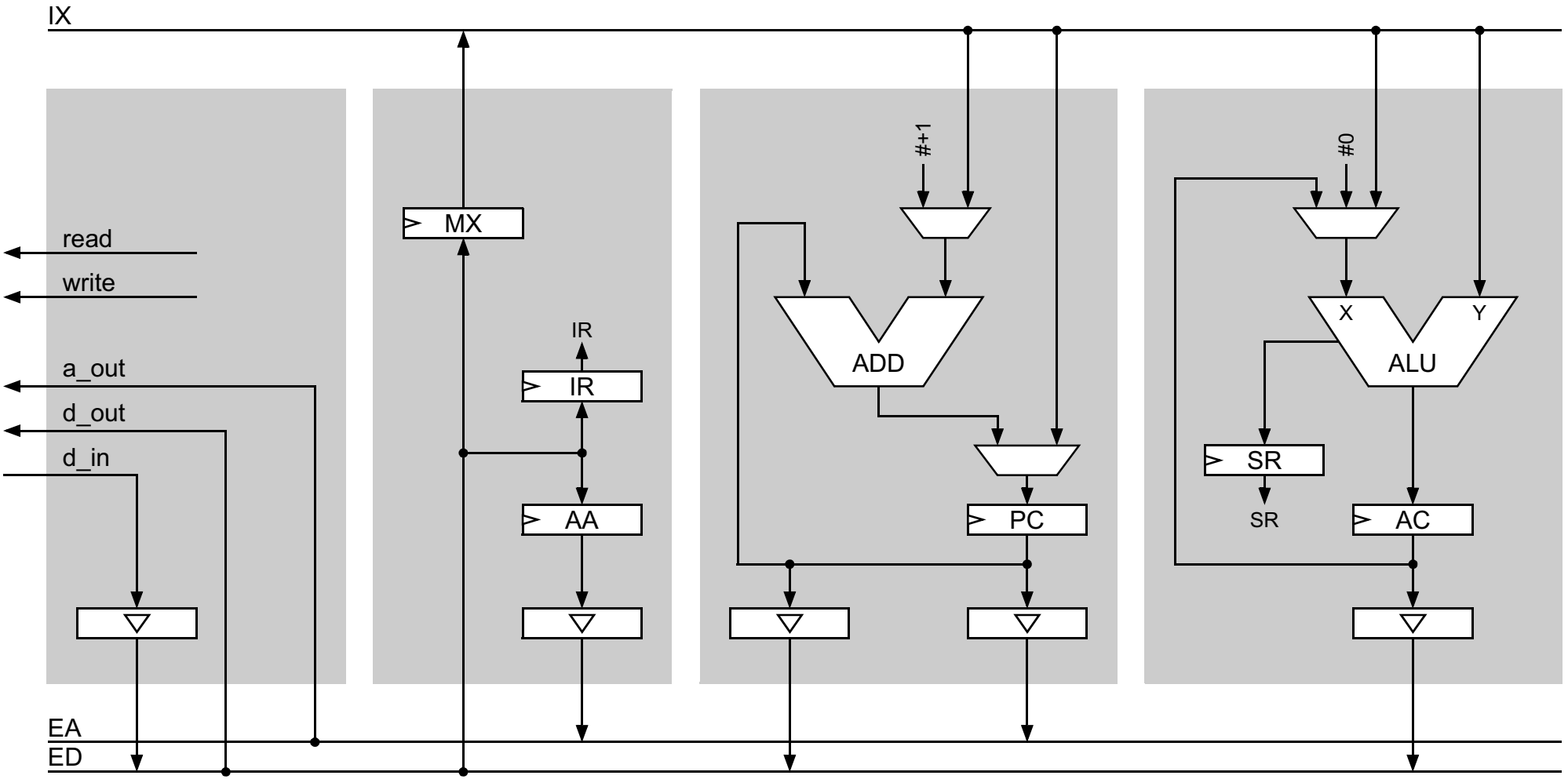
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
fetch_opc	1	10	0	0	1	1	0	1	0	1	0

(PC) → IR, PC + 1 → PC



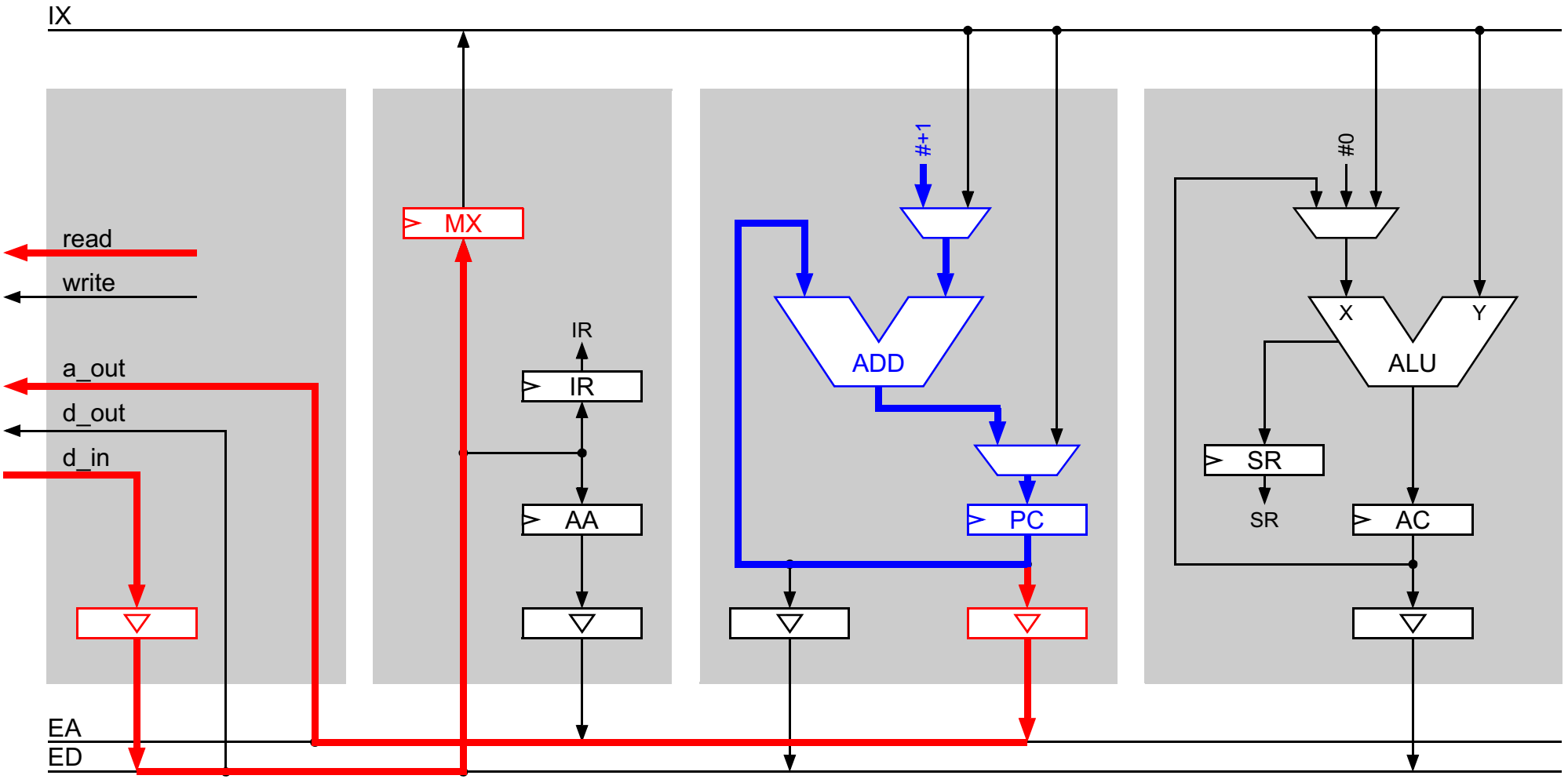
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
decode	-	-	0	0	0	-	-	0	0	0	0

– keine Operation in den Ausführungseinheiten –



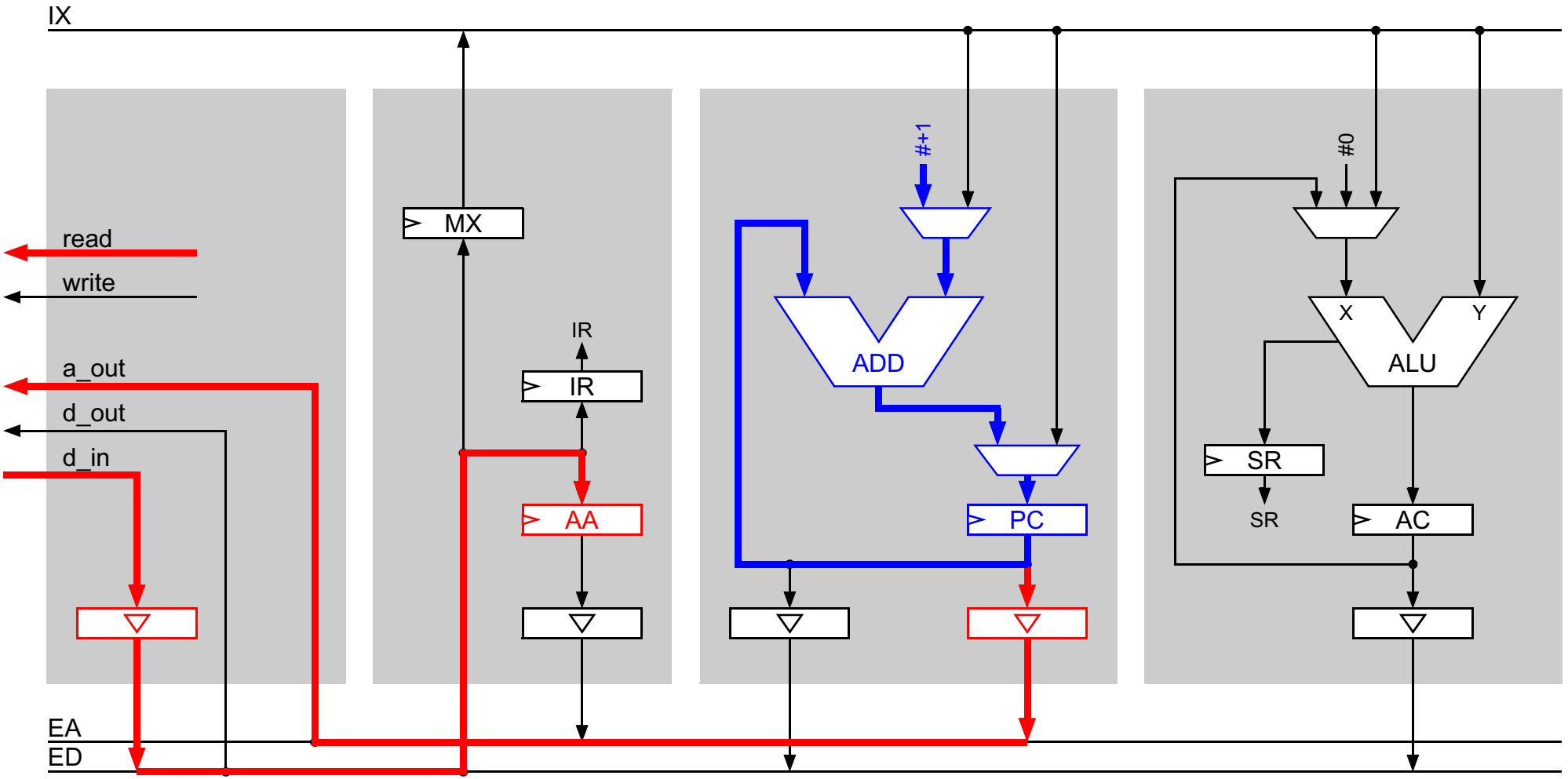
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
fetch_i	1	10	1	0	0	1	0	1	0	1	0

(PC) → MX, PC + 1 → PC



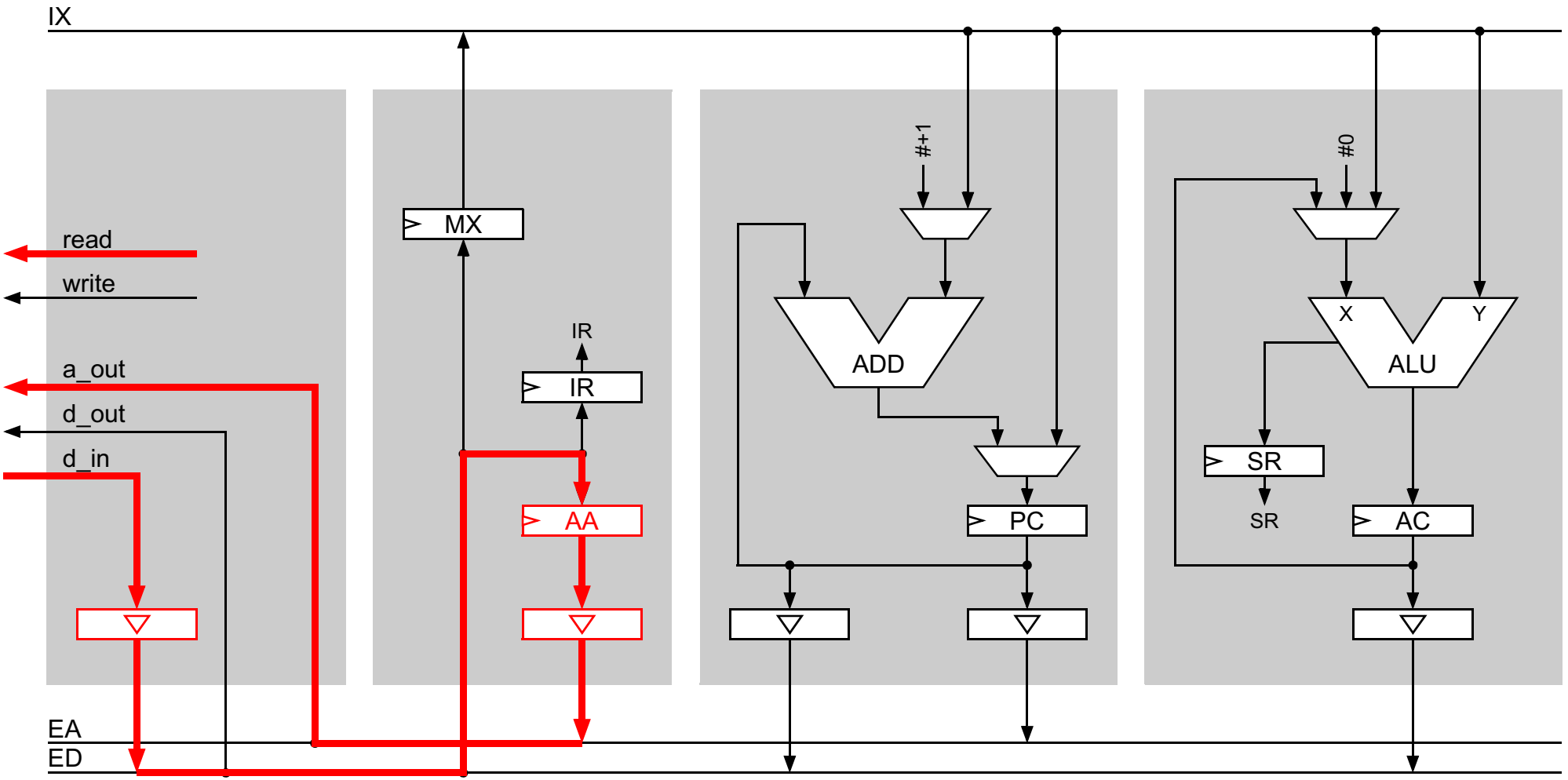
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
fetch_m	1	10	0	1	0	1	0	1	0	1	0

(PC) → AA, PC + 1 → PC



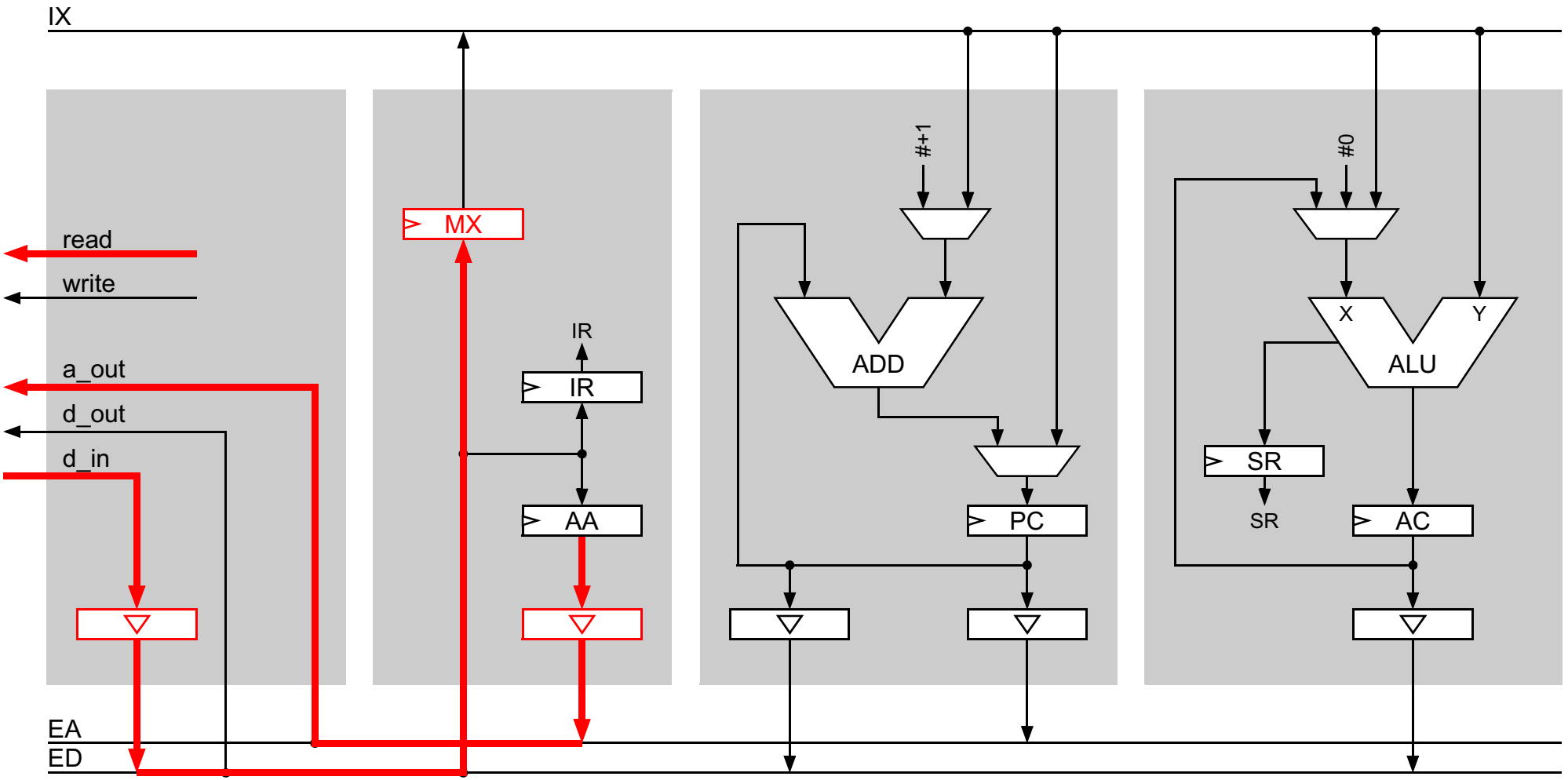
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
load_addr	0	10	0	1	0	-	-	0	0	1	0

(AA) → AA



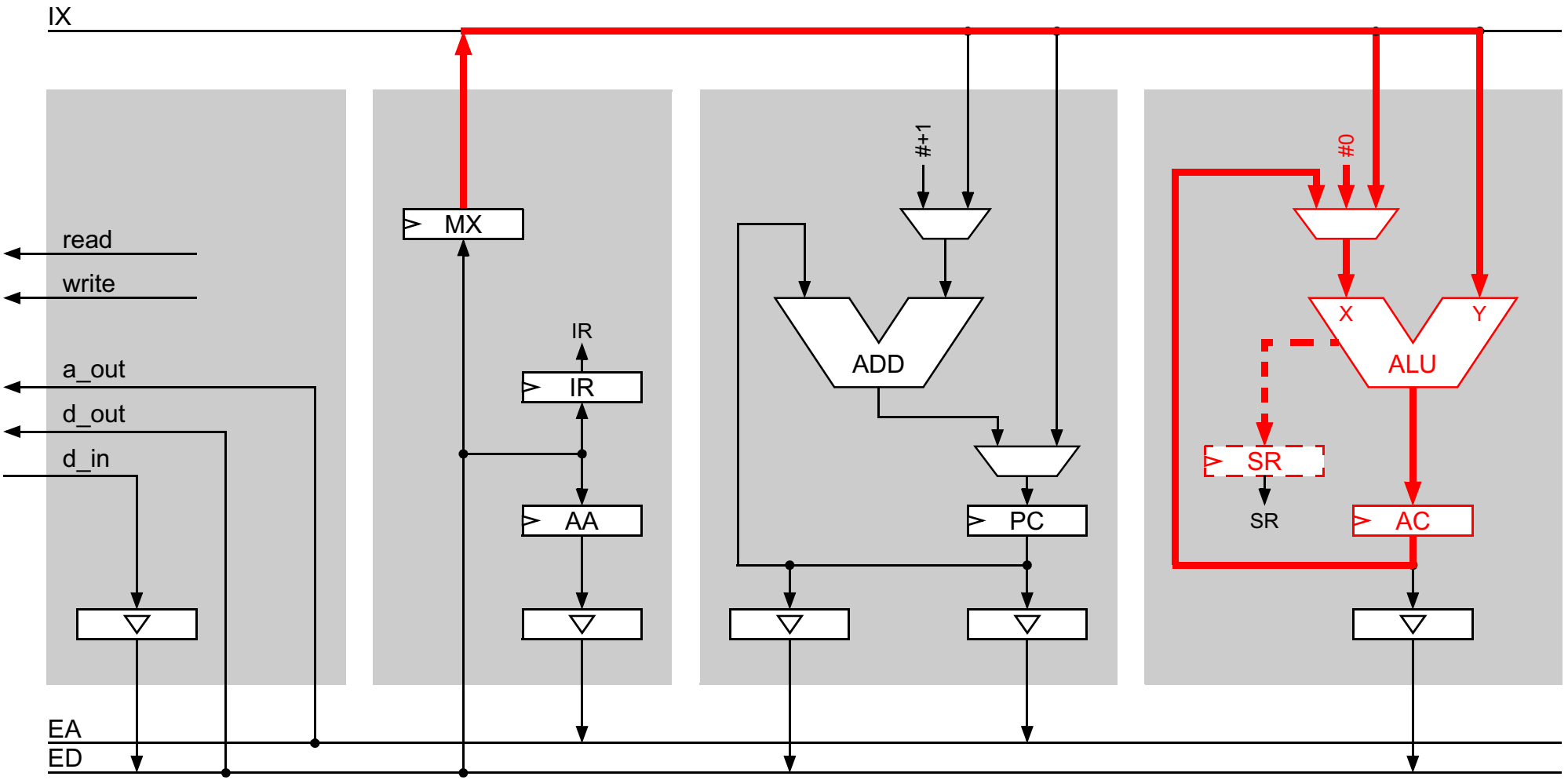
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
load_op	0	10	1	0	0	-	-	0	0	1	0

(AA) → MX



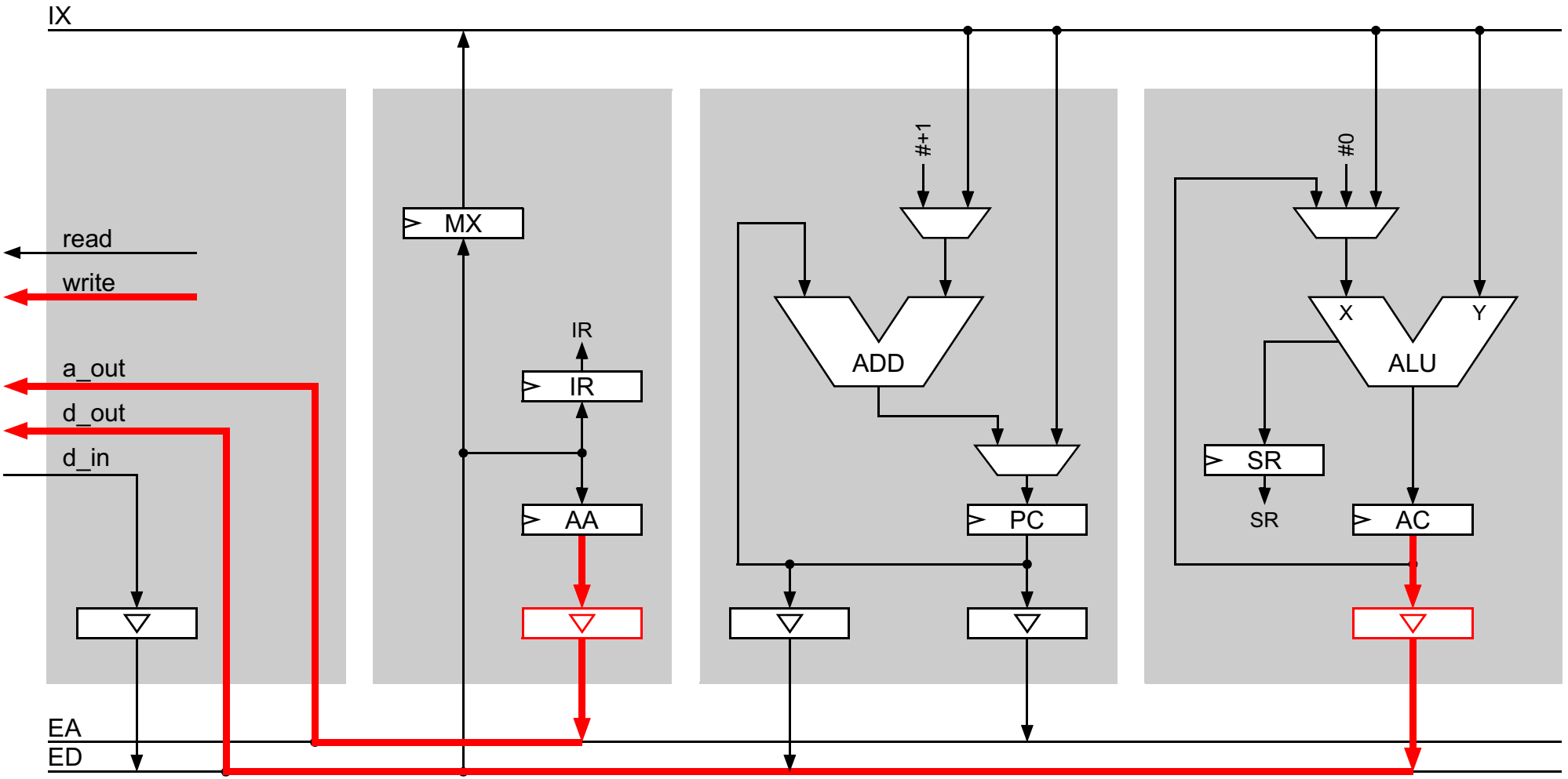
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
execute	-	-	0	0	0	-	-	0	1	0	0

f(AC,MX) → AC



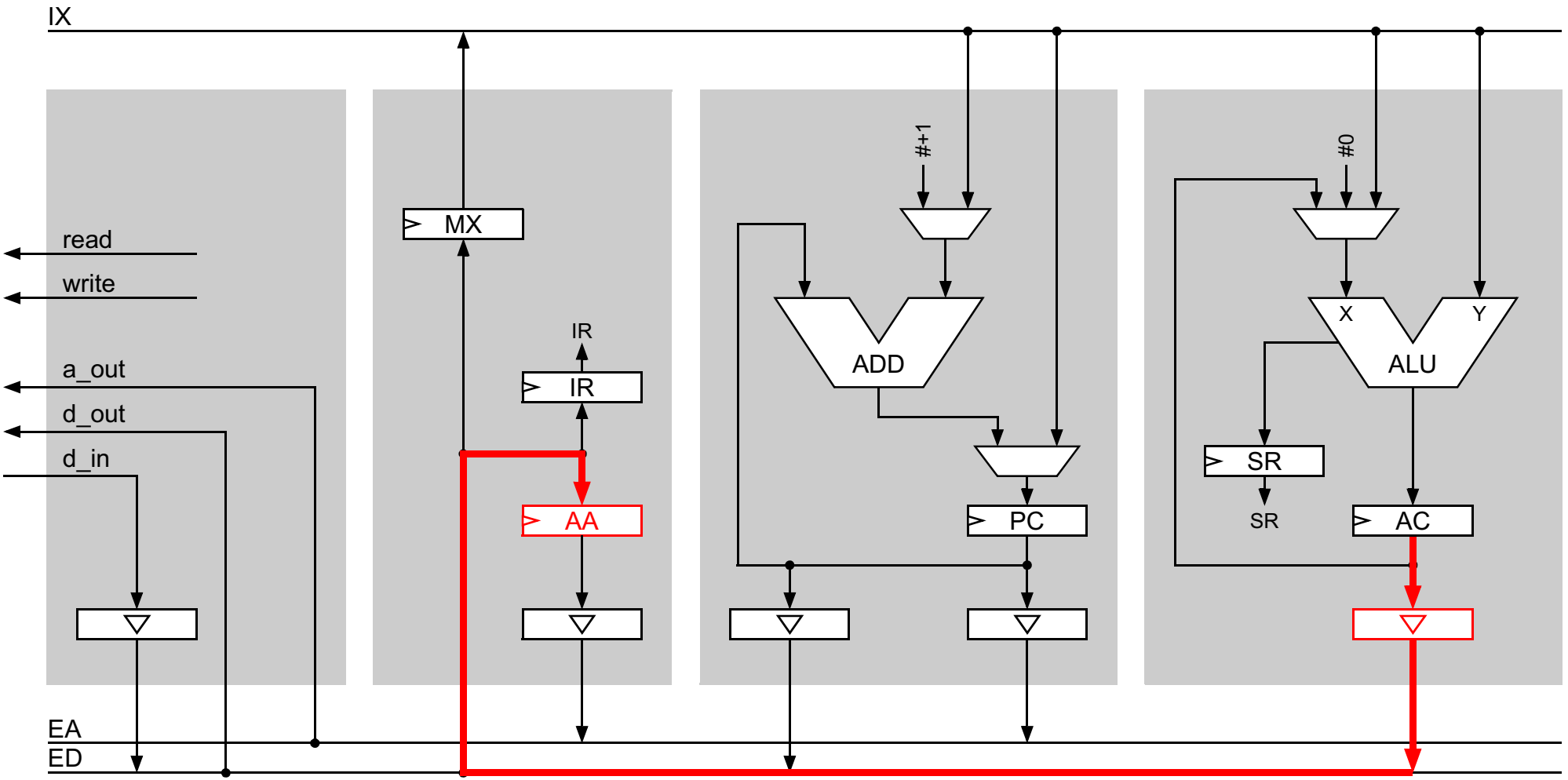
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
store	0	00	0	0	0	-	-	0	0	0	1

AC → (AA)



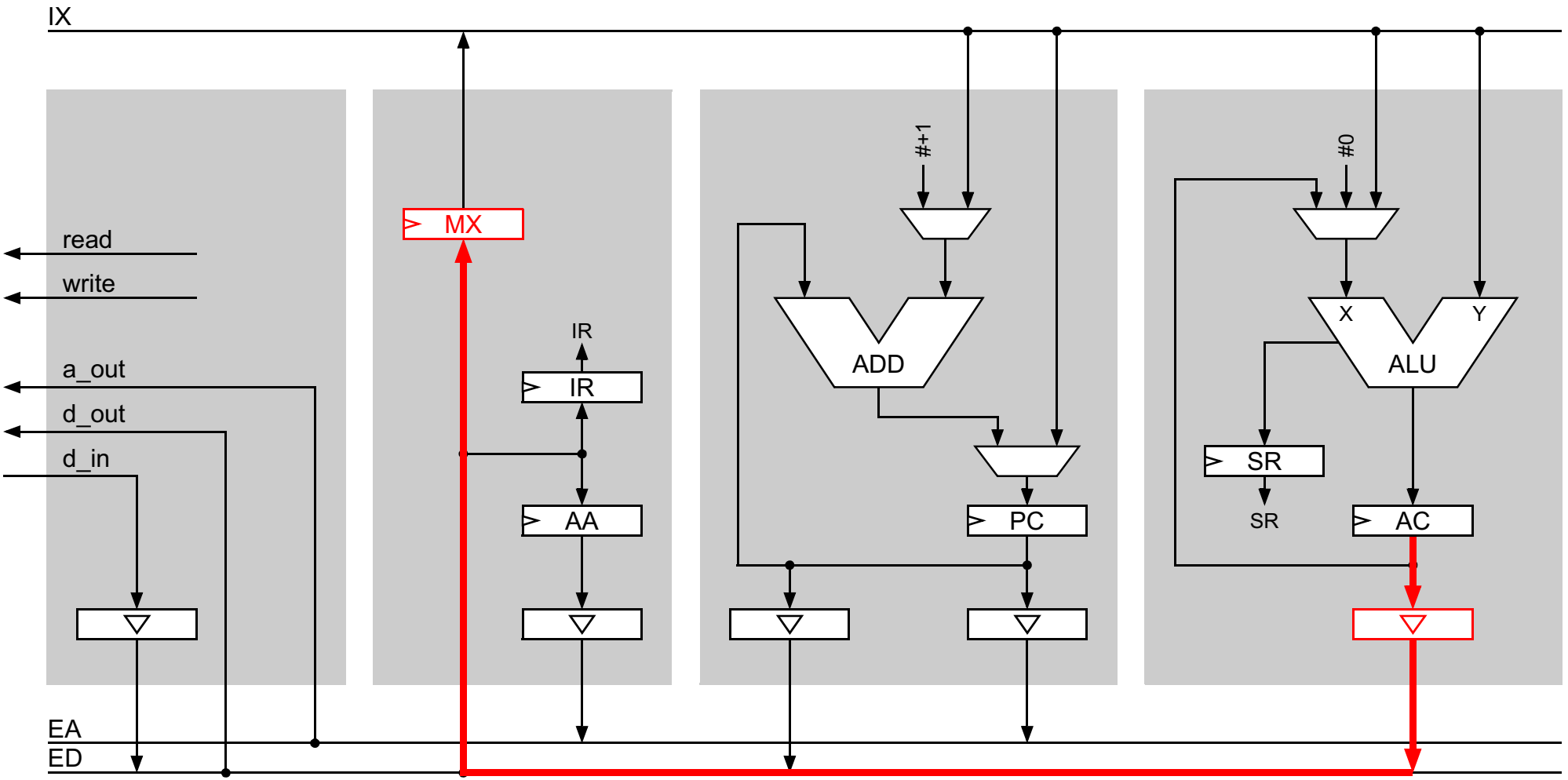
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
ac_to_aa	-	00	0	1	0	-	-	0	0	0	0

AC → AA



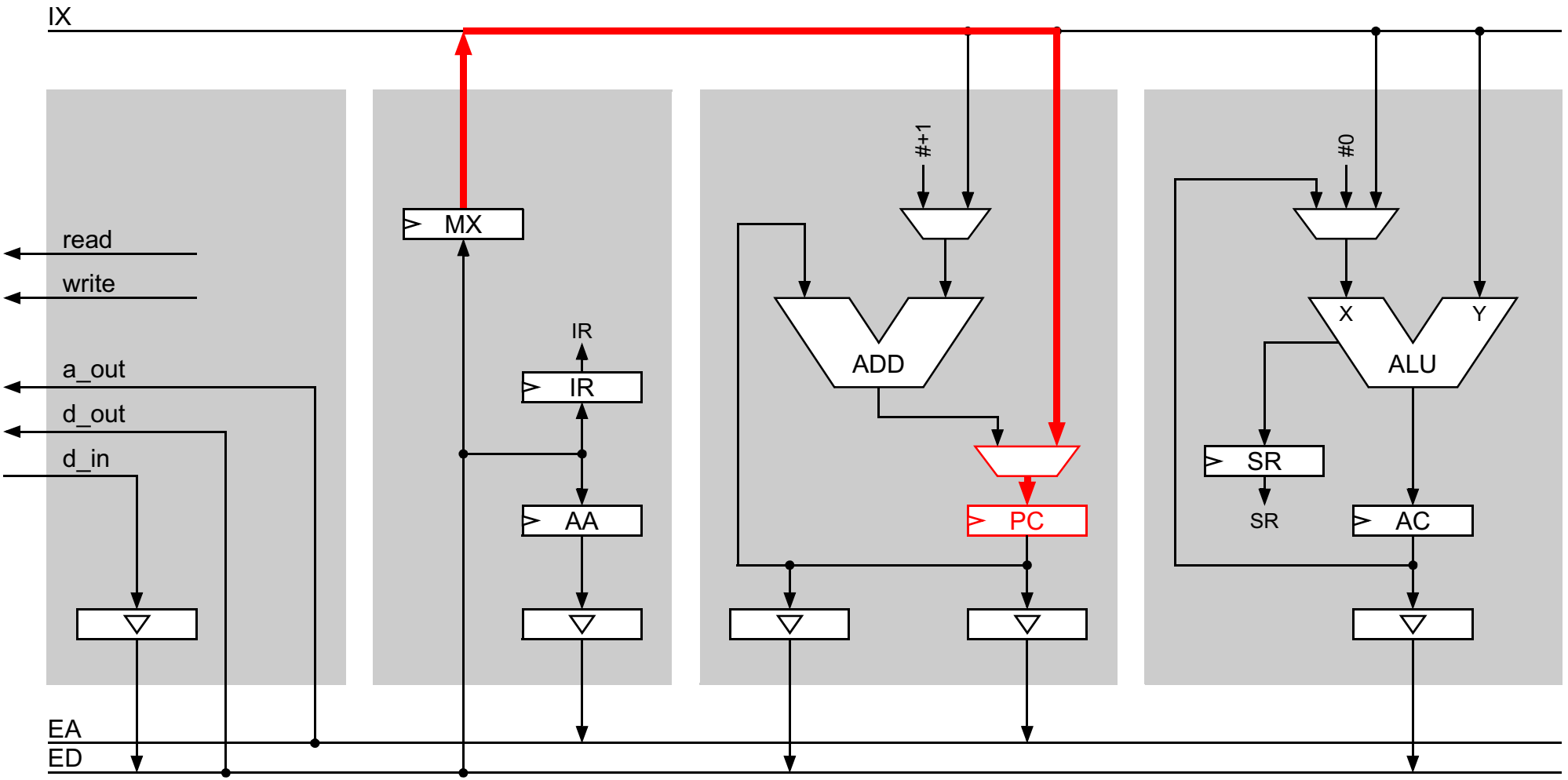
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
ac_to_mx	-	00	1	0	0	-	-	0	0	0	0

AC → MX



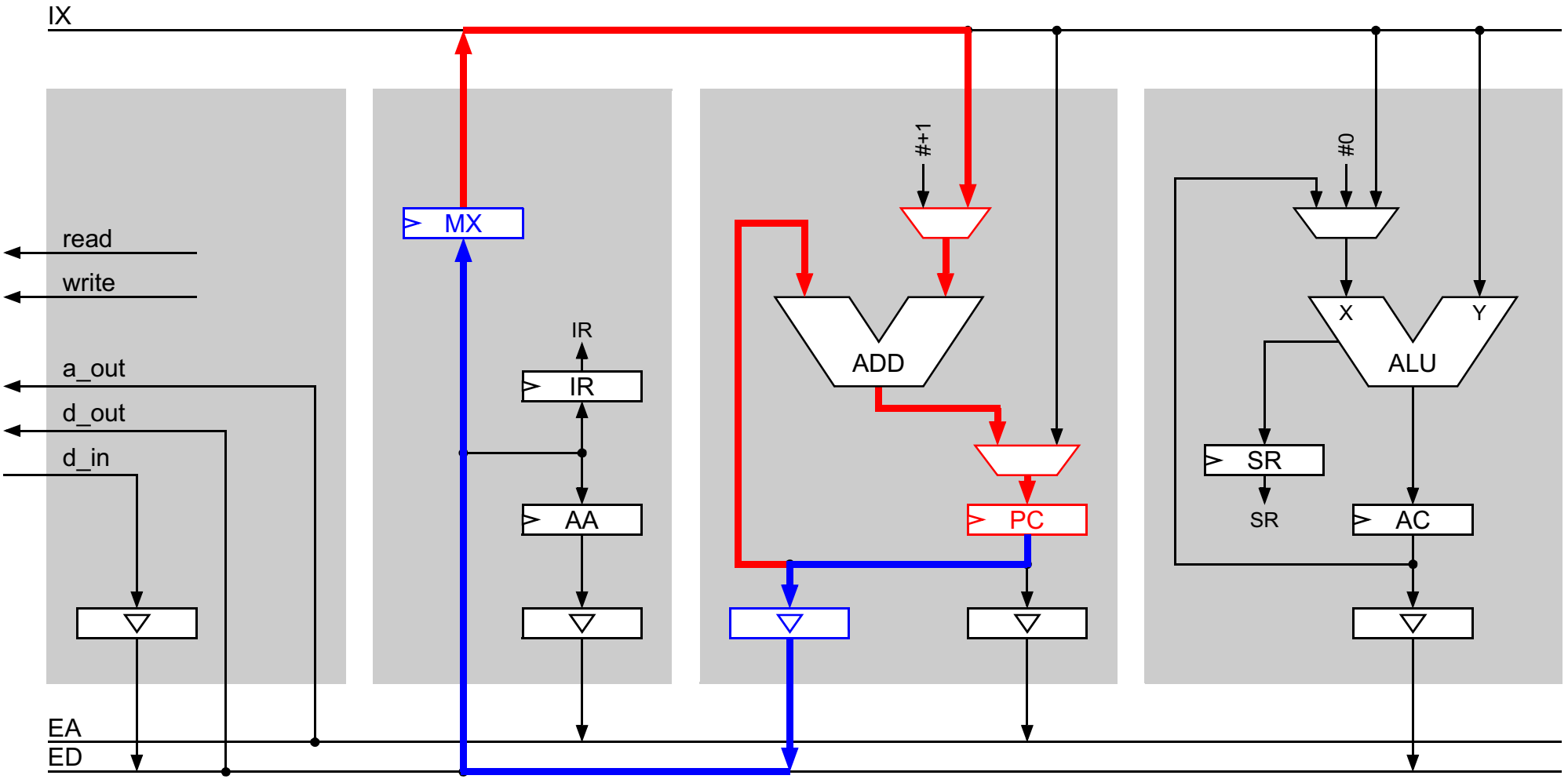
Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
jump	-	-	0	0	0	-	1	1	0	0	0

MX → PC



Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
branch	-	01	1	0	0	0	0	1	0	0	0

MX + PC → PC, PC → MX



Zustand	easel	edsel	mxena	aaena	irena	pdsel	pcsel	pcena	acena	read	write
save_pc	-	-	0	0	0	-	-	0	1	0	0

MX → AC

